

Agentic Agile 智能体敏捷：从碳基协作到硅基自治

构建 AI 时代的人机协同研发操作系统

人员、组织、流程与工具的系统重构

作者：王立杰（无敌哥）

适用人群：CTO、研发管理者、AI 转型负责人、产品与技术负责人、架构师、研发效能与敏捷负责人、FDE、AI 工程团队及一线开发者

核心方法与工具：Agentic Agile 3-4-3、SCOPE-V、验证工程与配套 Skill

书稿版本：v3.1 工作稿

本版本强化 AI First / AI Native 设计原则、组织级分形扩展。

前言 | 当 AI 成为执行者，研发组织必须重新设计

AI 带来的变化，已经不只是“写代码更快”。

今天，一个能够读取仓库、调用工具、修改文件、运行测试并持续修复的 Agent，正在从代码助手变成研发执行者。过去按天计算的分析、编码和验证，开始按小时甚至分钟完成；过去需要多个岗位接力的任务，也开始被一个人和一组 Agent 并行推进。

这是一场真实的生产力跃迁。但很多企业正在以采购 AI 工具开始，以组织效率没有明显改善结束。

代码生成得更多，需求仍在排队；Agent 执行得更快，决策仍在会议中等待；个人产出上升，测试、审核和返工却成为新的瓶颈；工具数量不断增加，目标、权限、责任和证据仍然散落在聊天、文档和个人经验里。

问题不只是模型偶尔出错，而是企业把新的智能生产力塞进了旧的岗位、旧的组织、旧的流程和旧的工具链。

三次“换操作系统”的生产力跃迁

历史上的通用技术，很少在“装上去”的那一刻兑现全部生产力。真正的跃迁，通常发生在组织围绕新能力重新设计生产方式之后。

第一次，是工厂电气化。早期工厂只是用大型电动机替换蒸汽机，仍然保留中央传动轴、皮带、楼层和机器布局。动力源变了，生产系统没有变，因此并未立即出现与新技术相称的生产率跃迁。后来，“单机驱动”让每台机器拥有独立电机，机器不再围绕传动轴摆放，而可以按照材料和工艺流重新布局。电力的价值由此从“更方便的动力”变成了“重新设计工厂的自由”。¹

第二次，是福特的移动装配线。美国工厂从 1880 年代开始通电。经济史学家 Paul David 在 1990 年的论文里指出过一个著名的悖论：此后三十多年，制造业生产率几乎没有起色。原因很朴素——工厂主干的事，是把蒸汽机换成电动机，厂房布局一寸没动。1913 年，福特并不是简单要求工人动作更快，他给每台机器配一个电机，机器不必再围着中央动力源摆，可以按工序排。于是有了海兰帕克的流水线，把可互换零件、任务分解和连续物料流组合成新的生产系统。到 1914 年初，生产一辆 Model T 所需的时间由固定工位方式下约 12.5 小时缩短到 93 分钟，接近八倍提升。² 效率跃迁来自工作、空间、角色和流动方式的整体重构。

福特赢，不是因为他用了电，是因为他为电重画了工厂。

第三次，是集装箱化。 集装箱本身只是一个标准化钢箱。真正改变全球物流的，是船舶、港口、起重机、卡车、铁路、装卸制度和跨组织协作围绕同一标准重新设计。没有这些配套变化，钢箱只是更大的包装；完成端到端重构后，它才成为全球化运输的基础设施。³

三次变化说明同一个规律：**替换工具，只会得到局部效率；重构生产方式，才会释放通用技术的系统价值。** 这才是“创造性破坏”真正可见的部分：旧系统不是被一个新部件替换，而是整套生产方式被重新组合。这也正是本书所说的“换操作系统”。

AI 时代同样如此。给每位工程师安装 Copilot，类似于用电动机替换蒸汽机却保留旧传动轴；让 Agent 在旧需求、旧审批和旧交接中跑得更快，也可能只是把堵点推向测试、审核和管理者。真正的转型，是围绕 Agent 的新能力重新设计人员责任、组织决策、研发流程和工程工具。

回到组织架构上，今天大多数公司的结构图：CEO → VP → 总监 → 经理 → 员工，这是从顶向下的组织设计，是围着“中央蒸汽机”设计的布局——信息和决策，沿一根传动轴上下传。AI 原生组织的布局可能是：1个CEO→10个核心员工→调度1000个 Agent→并行推进10000个任务，不再有过多的中间层，对应AI原生组织的产出公式：

$$\text{产出} = \text{碳基跟硅基有效人机协作时间}(\text{硅基时间} \times \text{并行系数})$$

在这个公式里面，决定一个组织效能的，一个是并行系数，另外一个是人跟Agent的有效协作时间。

再看传统研发组织的工作模式，默认人是主要执行者：岗位承载能力，团队承载协作，流程和会议控制节奏，工具辅助人完成局部任务。当 Agent 开始承接完整任务、跨边界调用工具并以远高于人类流程的速度持续行动，这些默认假设就需要重新审视。

所以，真正的 AI 研发转型必须同时回答四个问题：

1. **人员如何重构**：当大量执行交给 Agent，人还负责什么？谁定义意图、设计边界、处理异常并承担最终责任？
2. **组织如何重构**：如何从固定岗位和部门协作，走向“人类责任节点 + Agent 能力网络”？决策权和授权边界放在哪里？
3. **流程如何重构**：当机器高速并发执行，哪些 Sprint 仪式、审批和交接仍有价值，哪些应被事件、证据和实时反馈替代？
4. **工具如何重构**：如何从彼此孤立的 Copilot，走向具备上下文、编排、约束、验证、记忆、证据和遥测的 Agent 工程系统？

这就是本书重新定义 Agentic Agile 的起点。

Agentic Agile 不是传统敏捷加 AI，也不是一套更重的治理流程。它是面向智能体时代，对研发人员、组织、流程与工具进行系统重构的人机协同研发操作系统。

本书用“从碳基协作到硅基自治”概括这套操作系统的演进方向：从人类承担绝大多数理解、执行、检查和协调，逐步走向 Agent 在已签署意图、可执行约束和明确责任边界内，自主完成执行、验证、恢复与进化。硅基自治不是 AI 获得组织主权，更不是人类退出研发；执行权可以逐步交给硅基系统，价值定义权、不可逆决策权和责任承担仍由人类掌握。

这里所说的“操作系统”，不是一套软件平台，也不是试图替代企业战略、产品管理、HR、DevOps 或行业治理的万能框架。它描述的是 AI 进入研发执行系统后，价值、责任、协作、运行和反馈怎样共同工作。

AI First 与 AI Native：本书的两个设计原则

本书所说的 **AI First**，不是“所有事情先让 AI 做”，而是当新任务、新价值流或新组织设计出现时，先重新判断人和 Agent 的分工：哪些执行可以授权给机器，哪些价值判断、不可逆决策和最终责任必须由人掌握。

AI Native 也不是多采购一个模型或建立统一平台。它要求人员责任、组织边界、流程状态和工具基础从设计之初就适配 Agent 的持续执行、验证、恢复与反馈，而不是把 Agent 外接到旧岗位、旧工单和旧审批链上。

因此，三种状态需要区分：

状态	主要特征	不能误判成什么
AI Enabled	在既有岗位、流程或工具中加入 AI 能力	使用了 AI 就完成了转型
AI First	每次新设计先判断合理的人机分工与责任边界	凡事先让 AI 做
AI Native	生产系统从设计之初就适配 Agent 的执行、验证、恢复和反馈	所有旧方法都必须推倒

企业可以让低风险重复任务先进入 AI Native 运行方式，同时让高风险场景保持人主导。是否转型成功，最终由真实价值、工程证据、风险和长期遥测判断。

在这套操作系统中：

- 三大理论基石解释为什么需要外部化认知、建立反馈并治理复杂协作；
- 五条 Agentic Agile 宣言说明发生冲突时优先保护什么；
- 3-4-3 用三个超级角色、四大动态工件和三大自治机制建立运行架构；
- SCOPE-V 把意图、约束、编排、证明、进化与裁决连接成持续控制回路；
- Harness、验证工程与 Evidence Bundle 让 Agent 的完成声明变成可以复查的事实；
- Compute & Value Telemetry 判断 AI 是否真正创造价值、形成受控自治、提高人机效率并推动系统进化。

这套操作系统还必须能够从任务向组织扩展。其组织级结构不是传统汇报层级，而是三个分辨率：

战略意图层：企业要让 AI 放大什么价值，拒绝什么风险
↓ 约束继承、投资边界

价值网络层：多产品、多团队、多平台如何围绕价值流协同
↓ 能力编排、协议、证据聚合

任务运行层：每次 Agent 执行如何被授权、验证、恢复和裁决
↑ Evidence 向上聚合，Telemetry 进入组织慢外环

分形不是在每一层机械复制相同表格，而是让意图、约束、证据、裁决和反馈保持同一控制语法。任务层的一次 PASS 不能直接证明价值网络有效，更不能直接证明企业级 AI 投资应该扩大。

治理在这里不是目的，而是操作系统的控制机制。它不是为了减慢 AI，而是让机器的速度能够被组织安全地转化为业务结果。验证工程也不是全书的终点，而是这套操作系统不可缺少的可信执行内核。

提效则是 AI 转型必须兑现的结果。如果组织不能以更少投入、更短周期或更低边际成本创造更多有效价值，就没有理由把工具采用称为转型。但提效不能用代码量、Agent 调用次数或一场漂亮演示证明，它必须由真实业务结果、工程证据和持续遥测共同支持。

因此，本书不从遥远的“全自动研发”开始，而从一个今天就可以运行的最小闭环开始：

业务意图与人类责任

→ 可签署的契约与可执行的边界

→ 人与 Agent 的受控协作

→ 基于真实反馈的证明与修复

→ 独立证据与责任裁决

→ 价值与效能遥测

→ 下一轮人员、组织、流程与工具改进

读完本书，你应当能够回答七个问题：

1. 为什么 AI 工具提效没有自动变成组织提效？
2. 人员、组织、流程和工具分别需要重构什么，四者如何协同？
3. IO、OA、AS 如何重新分配价值、治理和执行责任？

4. 四大动态工件如何把战略意图、任务边界、组织约束和完成证据连接起来？
5. SCOPE-V 与 Harness 如何让 Agent 高速执行、自我纠偏并在越界时停止？
6. 如何用验证工程、Evidence 与 4 层 9 维遥测证明 AI 真正兑现了提效？
7. 如何从一个真实任务扩展到既有系统、多团队、多工具和研发组织转型？
8. 如何把任务运行层的闭环扩展到价值网络层和战略意图层，同时避免把 Agent 数量、Token 或工具激活率误当成组织转型？

本书负责建立完整认知与方法体系，Agentic Agile 3-4-3 Skill 提供跨 AI 工具的可运行能力，课程通过真实项目训练四维转型，而企业实践负责用事实检验这套操作系统是否成立。

不要只让 AI 生成更多。

让人与 Agent 共同交付更快、更可信、可以持续进化的价值。

作者简介

王立杰，AI治理架构师，资深研发效能顾问，工信部研发效能工程师认证专家讲师、PMI-ACP授权讲师、企业级规模化敏捷SAFe认证咨询师(SPC6)、华为云MVP（最有价值专家），曾任京东首席敏捷创新教练、IBM客户技术专家、DNV高级咨询师等，帮助小米、OPPO、海康威视、京东方、吉利亿咖通、晓羊教育集团、中远海运、京东购物APP、京东到家、招商银行、工商银行等组织实现研发效能提效。畅销书《敏捷无敌》、《京东敏捷实践指南》作者，江湖人称“无敌哥”。

教授经典课程：

- 《Agentic Agile（智能体敏捷）：从碳基协作到硅基自治——AI 时代研发组织重构与转型沙盘》
- 《Agentic AI 项目管理：企业项目管理的人机协同新范式》
- 《AI时代研发治理与领导力：从“驭人”到“驭智”高管研修班》
- 《乌托邦计划沙盘：跨部门协同与高效项目管理特训营》
- 《敏捷项目管理实战沙盘演练》
- 《DevOps黑客马拉松》
- 《创新设计思维Design Thinking》
- 北大光华管理学院/新华都商学院《创业机会分析与识别》
- 京东大学《京东创新之路》、《从0到1商业模式快速探索》

曾经提供培训与咨询的企业包括宝马、博世、吉利、上汽、广汽传祺等车企，百度、京东、小米、Oppo、微博、360、58同城、美团等互联网企业，中国移动、联通、Agilent、IBM、阿朗、爱立信、诺基亚、东软、华为等传统电信/IT/软件企业，招商银行、工商银行、中信银行、中国银行、山东城商行等金融企业，E人E本、长虹、海尔、美的等白电企业，海天建筑、同方威视、中大医疗等传统行业；曾经在“AgileChina敏捷中国、RSG、51CTO、MPD、质量竞争力大会/TiD”等大会做过多次演讲，被评为质量竞争力大会/TiD 2014最受欢迎10大讲师；**目前专注于企业 AI组织转型、研发效能提升、企业产品创新。**

全书结构

Agentic Agile 智能体敏捷：从碳基协作到硅基自治

构建 AI 时代的人机协同研发操作系统

前言 | 当 AI 成为执行者，研发组织必须重新设计

作者简介

全书结构

第一部分 | 为什么 AI 时代需要新的研发操作系统

第一章 | 新生产力进入旧组织：为什么局部提效没有变成组织提效

- 1.1 AI 改变的不只是工具，而是执行主体
- 1.2 局部提效为什么没有自动成为组织提效
- 1.3 从工具采用到智能体组织：四种不同变化
- 1.4 旧研发系统的四个默认假设
- 1.5 氛围编程的真正风险：探索模式进入正式交付
- 1.6 更快的执行，需要更快而不是更多的控制
- 1.7 同一个报销需求，为什么“全绿”仍然失败
- 1.8 判断转型是否发生，不能只看 AI 使用量
- 1.9 本章小结：AI 首先是一场生产方式变革

第二章 | 四维重构：人员、组织、流程与工具必须同时改变

- 2.1 一张图看懂四维重构
- 2.2 必须协同改变，但不能“大爆炸式转型”
- 2.3 人员重构：迁移任务，不是预测岗位消亡
- 2.4 组织重构：从岗位接力到人机责任网络
- 2.5 流程重构：让机器运行快内环，让人承担责任外环
- 2.6 工具重构：从 Copilot 孤岛到可替换的工程系统
- 2.7 治理与度量为什么不是第五、第六维
- 2.8 用报销价值流检验四维是否连通
- 2.9 四维转型的最小诊断法
- 2.10 本章小结：四维不是四条线，而是一套系统

第三章 | 理论基石：认知科学、控制论与系统论

- 3.1 认知科学：复杂度不能只放在人脑里
- 3.2 控制论：没有可靠反馈，速度只会放大偏差
- 3.3 系统论：以受治理的多样性应对复杂性
- 3.4 三大理论的融合：从理论启示到工程机制
- 3.5 本章实践：用三副镜子诊断一个 AI 工作流
- 3.6 本章小结

第四章 | Agentic Agile 五条宣言：先有价值取舍，再有方法

- 4.1 意图锚定，胜过详尽规约
- 4.2 多模态涌现协作，胜过仅依赖跨职能人类团队
- 4.3 算法规则与架构韧性，胜过仅依赖流程纪律与仪式
- 4.4 实时遥测与闭环对抗，胜过仅依赖阶段性复盘
- 4.5 基于验证的系统工程，胜过随性提示与氛围编程
- 4.6 五条宣言如何共同工作
- 4.7 组织级价值观冲突练习
- 4.8 本章小结

第二部分 | Agentic Agile：人机协同研发操作系统架构

第五章 | 系统总览：四维转型、3-4-3、SCOPE-V 与遥测如何组成整体

- 5.1 总装图：从价值意图到组织进化
- 5.2 各模型回答不同问题
- 5.3 组织级分形扩展：从任务运行到战略意图
- 5.4 3-4-3：相对稳定的运行架构
- 5.5 SCOPE-V：把架构变成持续控制回路
- 5.6 三种时间尺度：机器快内环、任务闭环、组织慢外环
- 5.7 Evidence 与 Telemetry：一次裁决和长期进化
- 5.8 四维重构怎样落到 3-4-3 与工程系统
- 5.9 治理、验证与 Dashboard 在系统中的位置
- 5.10 一个报销规则如何穿过整套操作系统
- 5.11 最小可运行操作系统
- 5.12 六个架构反模式
- 5.13 本章小结：从模型集合到运行系统

第六章 | 三个超级角色：从岗位分工到人机责任网络

- 6.1 为什么不是按传统岗位映射
- 6.2 IO：意图主理人
- 6.3 OA：编排架构师
- 6.4 AS：自治蜂群
- 6.5 分开执行、判断与责任
- 6.6 显式签署为什么重要
- 6.7 三个角色之间的制衡

- 6.8 授权矩阵
- 6.9 最小人机责任单元
- 6.10 一个任务中的角色协作
- 6.11 角色反模式
- 6.12 角色能力如何成长
- 6.13 本章小结

第七章 | 四大动态工件：让组织共同语言可以被机器执行

- 7.0 三层架构中的工件分辨率
- 7.1 四大动态工件的闭环关系
- 7.2 意图图谱：全局导航与组织记忆
- 7.3 意图契约：单次任务的执行基准
- 7.4 约束矩阵：可执行的安全边界
- 7.5 Evidence Bundle：支持裁决的证据包
- 7.6 涌现式增量与遥测的位置
- 7.7 三方与四方一致性
- 7.8 工件的生命周期
- 7.9 工件版本与时效
- 7.10 工件粒度
- 7.11 一份完整证据包如何被阅读
- 7.12 工件反模式
- 7.13 本章小结

第八章 | 三大自治机制与 Harness：让 Agent 获得有边界的执行权

- 8.1 三大自治机制
- 8.2 从三大自治机制到 Harness 工程承载
- 8.3 Harness 的六大能力支柱
- 8.4 六大支柱如何通过四类工程方法落地
- 8.5 一个任务中的四类工程接力
- 8.6 轻量模式与完整模式：按风险裁剪
- 8.7 风险驱动的最小建设路径：先闭环，再扩大自治
- 8.8 常见概念混淆与机制反模式
- 8.9 管理者如何判断 Harness 是否有效
- 8.10 本章小结：从最后一个“3”走向 SCOPE-V

第三部分 | 验证工程与 3-4-3 落地：让操作系统真正运行

A. SCOPE-V 运行时控制

第九章 | SCOPE-V：人机协同研发的持续控制回路

- 9.1 它不是六步瀑布，而是六个持续运行的控制面
- 9.2 SCOPE-V 与传统 SDLC 到底有什么不同
- 9.3 为什么人机协作时代需要这样的控制回路
- 9.4 六个控制面分别控制什么
- 9.5 为什么 E 必须在 V 之前
- 9.6 快内环：P $\rightleftharpoons$ E 的诊断与自愈
- 9.7 独立裁决：V 的审计与放行
- 9.8 慢外环：Telemetry $\rightarrow$ S/C/O 的组织学习
- 9.9 每个控制面都必须产生可检查工件
- 9.10 五道机械门把控制状态变成不可跳过的事实
- 9.11 一个完整控制回路示例
- 9.12 SCOPE-V 设计的四条底线
- 9.13 三大自治机制怎样通过 SCOPE-V 落地
- 9.14 不要把三大机制机械映射成三个阶段
- 9.15 三大机制如何通过 SCOPE-V 处理同一次失败
- 9.16 本章小结

第十章 | Specify：从模糊需求到可签署意图

- 10.1 需求本身就是第一类治理对象
- 10.2 批判性思维与 Grill-Me：从发现问题到帮助决策
- 10.3 从事实与意图图谱裁剪任务上下文
- 10.4 意图契约就是可执行规约
- 10.5 Example Mapping 与规格建模
- 10.6 显式签署与前置门
- 10.7 规格漂移

10.8 从自然语言到可执行 AC

10.9 Spec 不是越详细越好

10.10 Specify 常见失败

10.11 本章小结

第十一章 | **Constrain**：把组织原则变成可执行边界

11.1 约束不是“请注意”

11.2 六大核心约束域：从分类目录到工程控制面

11.3 Web 扩展与 NFR

11.4 MUST、SHOULD、MAY

11.5 Tools Manifest 与最小权限

11.6 上下文安全

11.7 AI 研发的数据安全与隐私工程

11.8 例外、降级与恢复

11.9 约束冲突

11.10 约束的经济性

11.11 Constrain 常见失败

11.12 本章小结

第十二章 | **Orchestrate**：从团队协作到可计算的人机能力网络

12.1 从任务列表到 Work Graph

12.2 每个 Agent 节点的执行契约

12.3 30 秒微检查点

12.4 Handoff 与跨 Agent 协作

12.5 资源争用、超时与死锁

12.6 自适应编排的边界

12.7 并行是否真的带来收益

12.8 Orchestrate 常见失败

12.9 编排中的三个关键决策

12.10 一个四节点编排案例：为什么这样拆、这样连

12.11 本章小结

第十三章 | **Prove**：先证明，再实现

13.1 AC 不是事后检查清单

13.2 编码门：先证明 Red 可信

13.3 最小实现：只改变足以让证据收敛的部分

13.4 AI Coding 时代的多层验证工程

13.5 LLM-as-Judge 的边界

13.6 Proof-Carrying Delivery

13.7 验证门：不是只有通过和失败

13.8 测试的独立性

13.9 什么是无效全绿

13.10 Prove 常见失败

13.11 从测试数量转向证明强度

13.12 谁来验证验证者

13.13 本章小结

第十四章 | **Evolve**：让失败进入系统能力，而不是只进入复盘

14.1 失败不是例外，而是反馈

14.2 失败分类与回流路由

14.3 可自动修复的条件

14.4 修复证据：证明问题真的被消除

14.5 反思与 Failure Mining

14.6 规则债务与约束疲劳

14.7 Champion-Challenger 与有界演进

14.8 意图图谱回写：只沉淀经过验证的变化

14.9 停止条件：自治系统也必须知道何时认输

14.10 记忆的来源与衰减

14.11 Evolve 常见失败

14.12 组织记忆不是聊天记录仓库

14.13 演进也需要变更治理

14.14 Bug 回溯门：完成之后发现错误也必须回流

14.15 Agent Chaos Engineering：主动验证系统能否发现、停止与恢复

14.16 本章小结

第十五章 | **Verify**：从全绿到基于证据的责任裁决

15.1 测试通过不等于上线批准

15.2 六维验证

15.3 Evidence Bundle 的证据映射

15.4 收尾门：没有完整证据就不能宣告完成

15.5 人类发布裁决

15.6 发布裁决示例

15.7 从“全绿”到上线裁决

15.8 Verify 常见失败

15.9 本章小结

第十六章 | 价值与效能遥测：证明 AI 转型真正兑现了提效

16.0 遥测的三层聚合边界

16.1 从度量到遥测：Agentic Agile 的反馈控制系统

16.2 一份契约一张 Dashboard：任务级治理快照

16.3 全局 Dashboard：从任务快照到项目价值与效能视图

16.4 Token、算力成本与价值归因

16.5 遥测真实性与采集闭环

16.6 遥测如何驱动动作

16.7 Telemetry 常见失败

16.8 度量治理与遥测数字的解释纪律

16.9 本章小结

第十七章 | **AI 原生交付流水线**：让每次发布携带意图、证据与回滚能力

17.1 3-4-3 不取代 DevOps，而是补上意图与证据

17.2 为什么 AI 流水线必须前移到 Commit 之前

17.3 分支管理首先解决隔离，而不是命名格式

17.4 默认建议：受保护主干、短分支或 Worktree、Merge Queue

17.5 文件所有权与语义冲突

17.6 从代码版本升级为交付谱系

17.7 建立 Release Manifest

17.8 CI 如何承载五道机械门

17.9 Build Once, Verify Once, Promote Many

17.10 版本号、Tag 和配置也要受治理

17.11 CD 中的人机权限边界

17.12 灰度、Feature Flag 与条件批准

17.13 回滚不只是重新部署旧版本

17.14 从发布版本回到契约级遥测

17.15 一个端到端的携证发布案例

17.16 不同规模团队的最小配置

17.17 常见失败

17.18 本专题的工程验收

17.19 本章小结

B. 3-4-3 工程化落地与扩展

第十八章 | 跑通第一个可信闭环：新项目与既有大型系统

18.1 两条并行落地路径

18.2 新项目：从零建立第一个可验证治理闭环

18.3 既有大型系统：从真实变更建立可验证治理岛

18.4 两条路径的共同底座与选择原则

18.5 本章小结

第十九章 | **Knowledge & Memory Engineering**：让上下文和经验成为组织资产

19.1 Agentic Knowledge Engineering：用双地图连接需求与代码

19.2 Agent Memory Engineering：从任务状态到可治理组织记忆

19.3 本章小结

第二十章 | 多人、多工具、多模块：从单任务闭环到规模化协作

20.0 组织级扩展判断

20.1 先去规模化，再升级治理复杂度

20.2 三条复杂度轴

20.3 团队内多人协同的三种拓扑

20.4 全局 OA 与模块 OA

- 20.5 Protocol: 跨主体治理协议
- 20.6 跨模块契约 XC
- 20.7 集成 DAG 与发布顺序
- 20.8 多语言 checks
- 20.9 证据聚合
- 20.10 跨项目价值与效能视图
- 20.11 一个跨模块失败案例
- 20.12 两级约束
- 20.13 多工具治理
- 20.14 多人共享一个聊天窗口: 多主控制问题
- 20.15 Handoff 摩擦
- 20.16 跨项目视图反模式
- 20.17 跨模块变更的治理链
- 20.18 所有权比工具统一更重要
- 20.19 规模化的经济边界
- 20.20 跨项目遥测: 可以汇聚, 但不能直接排名
- 20.21 本章小结

第四部分 | 四维组织转型: 把工程能力变成企业运行方式

第二十一章 | 人员与组织重构: 从岗位团队到人机责任网络

- 21.1 重构单位是任务组合, 不是岗位名称
- 21.2 IO、OA、AS 不是三个新职位
- 21.3 人类不可转移的五类责任
- 21.4 从固定团队到意图单元和动态能力网络
- 21.5 决策权必须与信息 and 风险一起前移
- 21.6 能力成长: 从亲自执行到设计人机系统
- 21.7 激励与反监控边界
- 21.8 本章小结

第二十二章 | 流程与工具重构: 从旧流程加 AI 到 AI 原生价值流

- 22.1 旧流程加 AI 为什么只会局部提速
- 22.2 哪些敏捷与工程活动保留, 哪些应被替代
- 22.3 AI 原生价值流: 从任务传递到持续控制
- 22.4 工具从 Copilot 孤岛走向最小工程系统
- 22.5 跨 AI 工具是架构要求, 不是兼容性加分项
- 22.6 流程与工具共同裁剪
- 22.7 谁运营这套流程和工具
- 22.8 费用报销规则: 一条价值流怎样被真正重构
- 22.9 常见反模式
- 22.10 如何判断流程工具重构是否成功
- 22.11 本章小结

第二十三章 | 四维协同落地: 转型画布、成熟度与 90 天首轮验证

- 23.0 三层转型画布
- 23.1 四维转型画布: 先绑定同一条价值流
- 23.2 证据驱动的成熟度
- 23.3 四级演进路径
- 23.4 90 天首轮验证: 不是等 90 天才跑通闭环
- 23.5 团队运行节奏
- 23.6 Agentic Agile 治理委员会
- 23.7 成熟度不是线性升级游戏
- 23.8 不要追求零 HITL
- 23.9 治理资产运营
- 23.10 激励设计
- 23.11 采纳阻力
- 23.12 治理委员会不是新的审批官僚
- 23.13 从一个英雄试点到第二责任单元
- 23.14 变革沟通: 从威胁叙事转向能力叙事
- 23.15 本章小结

第二十四章 | 风险与行业裁剪: 操作系统相同, 参数与责任边界不同

- 24.1 不建立一套适用于所有行业的阈值
- 24.2 六个裁剪问题

- 24.3 金融场景
- 24.4 车载与安全关键系统
- 24.5 政企与涉敏环境
- 24.6 互联网与高频业务
- 24.7 医疗与生命科学
- 24.8 工业、能源与生产控制
- 24.9 风险分层与治理重量
- 24.10 Agentic 风险的七类来源
- 24.11 威胁建模
- 24.12 供应链风险
- 24.13 事故响应
- 24.14 未来演进：作为假设，而不是承诺
- 24.15 本章小结

第五部分 | 附录与行动入口

附录 A | Agentic Agile 宣言

附录 B | 开源治理资产速查

附录 C | 图书与课程学习地图

附录 D | 术语表

附录 E | 五道门速查

附录 F | 参考资料与行动入口

附录 G | 贯穿案例：从一句话需求到可验证发布

附录 H | 常见质疑与回答

附录 I | 四大动态工件实战指南

附录 J | Agentic Agile 实践社区

结语 | 从碳基协作到硅基自治

本书不是从工具选型开始，而是从生产方式的结构性矛盾开始；不是把 3-4-3、SCOPE-V、验证工程和遥测并列成一组方法清单，而是说明它们如何共同构成人机协同研发操作系统。

全书按照“为什么要换、换成什么、怎样运行、如何成为组织能力”四个问题展开。

第一部分：为什么 AI 时代需要新的研发操作系统。 从新生产力进入旧组织后的局部加速与系统拥堵出发，提出人员、组织、流程与工具四维重构，并用三大理论基石和五条 Agentic Agile 宣言建立价值判断坐标。

第二部分：Agentic Agile 人机协同研发操作系统架构。 给出全书总装图，说明四维转型、3-4-3、SCOPE-V、Evidence 与 Telemetry 所处的不同层级；随后展开三个超级角色、四大动态工件和三大自治机制。

第三部分：验证工程与 3-4-3 落地。 先用 SCOPE-V 建立从意图到裁决的运行控制回路，再进入价值与效能遥测、AI 原生交付流水线，以及新项目、既有系统、知识记忆和多主体协作中的工程化扩展。

第四部分：四维组织转型。 把已经跑通的工程闭环转化为企业运行方式，讨论人员与组织、流程与工具如何协同重构，如何借助转型画布和阶段性证据完成规模化决策，以及不同行业如何调整参数和责任边界。

四部分不是从战略到技术的单向下钻，而是一个闭环：组织转型提出新的价值和责任要求，工程系统把要求变成可执行控制，Evidence 与 Telemetry 再把运行事实反馈给组织。读者可以顺序阅读，也可以按职责选择路径：管理者重点阅读第一、二、四部分，工程负责人重点阅读第二、三、四部分，一线实践者可从第三部分和行动附录直接进入可执行工件。

第一部分 | 为什么 AI 时代需要新的研发操作系统

第一部分首先处理一个容易被工具热潮遮蔽的事实：AI 已经改变研发执行能力，但大多数组织仍沿用以人为唯一执行主体设计的岗位、团队、流程和管理节奏。结果不是没有提效，而是个人提效首先出现，组织瓶颈随之转移。

本部分依次回答四个问题：为什么局部提效没有自动成为组织提效；人员、组织、流程与工具为什么必须协同改变；认知科学、控制论与系统论为这种重构提供了哪些设计镜头；当速度、控制、责任和价值发生冲突时，Agentic Agile 优先保护什么。

这里并不否定敏捷、DevOps 或探索性的氛围编程。它们仍然提供重要的价值观、协作机制和工程基础。需要重构的，是它们形成时默认的执行主体、反馈速度和责任载体。当 Agent 从“给建议”走向“能行动”，研发组织就需要一套新的运行方式，把机器速度转化为可持续、可验证的组织提效。

第一章 | 新生产力进入旧组织：为什么局部提效没有变成组织提效

许多企业已经不再需要证明 AI 能不能提效。工程师用它理解代码、生成实现、补充测试和修复缺陷，产品经理用它整理需求，测试人员用它构造场景，管理者用它汇总信息。过去按天完成的工作开始按小时完成，提效是真实的，也是企业投入 AI 转型必须获得的结果。

但另一个事实同样真实：个人完成任务更快，不必然让需求更早上线；代码生成更多，不必然让有效价值交付更多；一个环节节省的时间，也可能转化为下游更长的等待、审核和返工。

这不是 AI 没有产生效率，而是效率首先出现在局部，组织尚未完成对新生产力的吸收。**AI 转型最先暴露的，不是模型能力上限，而是旧研发系统的吞吐上限。**

1.1 AI 改变的不只是工具，而是执行主体

传统软件工具主要等待人发出明确操作：编辑器编辑、编译器编译、流水线执行预设步骤。它们提高人的局部能力，但任务分解、上下文拼接、异常判断和下一步选择主要由人完成。

Agent 开始改变这种分工。一个 Agent 可以读取仓库和项目文档，搜索依赖关系，修改多个文件，运行测试，根据失败继续修复，调用外部工具，甚至与其他 Agent 分工协作。人不再逐步完成每一个动作，而是开始把一个包含目标、边界和验收条件的任务交给机器执行。

这带来两项同时发生的变化：

1. **执行能力跃迁**：大量分析、生成、检查和修复可以高频并发进行，单位任务的边际成本和等待时间显著下降。
2. **行动半径扩大**：一个模糊意图、错误假设或过宽权限，也可以被快速扩展为跨文件、跨模块、跨环境的连续操作。

因此，AI 研发的关键问题不再只是“怎样让模型写得更好”，而是：

当机器开始承接任务并持续行动，组织怎样重新定义价值、责任、边界、协作和完成？

1.2 局部提效为什么没有自动成为组织提效

研发交付不是个人产出相加，而是一条包含需求澄清、决策、实现、验证、发布和反馈的价值流。AI 可以把其中一个环节压缩十倍，但整条价值流的速度仍受制于没有同步改变的瓶颈。

实现从 3 天缩短到 3 小时
≠ 需求可以更早进入实现
≠ 测试与审核能够承接更多变更
≠ 发布决策更快发生
≠ 用户更早获得有效价值

当生成速度突然提高，瓶颈通常沿价值流迁移：

局部变化	新出现或被放大的瓶颈	组织表象
代码生成更快	需求边界没有同步澄清	做得快，返工也快
并行任务增多	架构、测试与集成吞吐不足	在制品增加，交付周期不降
Agent 自动修复	失败原因和修复过程不可追溯	全绿更多，信任反而下降
工具数量增加	上下文、权限和规则彼此割裂	每个人都高效，团队难以接力
执行持续加速	人类决策仍依赖会议与层层审批	机器已经完成，结果仍在等待

所以，局部提效与组织提效不是相互否定，而是前后两个层级。前者证明 AI 具备生产力，后者要求组织重新设计整条价值流，使节省出来的时间能够穿过决策、验证和发布系统，最终成为业务结果。

1.3 从工具采用到智能体组织：四种不同变化

企业经常把“已经购买 AI 工具”“部分流程接入 AI”和“完成 AI 转型”混为一谈。为了避免这种概念膨胀，可以把变化区分为四个层级：

层级	主要表现	获得的价值	尚未解决的问题
L1 工具采用	Prompt、Copilot、AI 写文档或代码	提升个人任务速度	工作和责任结构基本不变
L2 流程增强	AI 接入需求、开发、测试等局部环节	减少部分重复劳动	旧流程仍是主干，瓶颈容易转移
L3 可验证闭环	意图、约束、执行、验证、证据和反馈连通	形成可重复、可裁决的交付能力	仍需验证能否跨团队、跨场景复用
L4 硅基自治	Agent 在人类定义的意图、约束与责任边界内自主运行	形成组织级智能生产系统	价值定义、不可逆决策和责任承担仍由人类掌握

这不是要求所有企业立即追求最高自治，也不是一条只进不退的技术成熟度阶梯。不同业务可以停留在不同层级，安全关键任务甚至应主动保留更多人类责任节点。区分层级的意义在于：**不要用 L1 的工具使用量，宣称已经获得 L3、L4 的组织能力。**

四级演进由此形成一条清晰路径：**碳基主导 → 人机编队 → 受控自治 → 硅基自治**。路径描述的是默认执行方式的变化，不是责任主体从人类转移给机器。

1.4 旧研发系统的四个默认假设

今天多数研发组织形成于“人是唯一通用执行者”的时代，其运行方式建立在四个默认假设之上。

- **人员假设：**能力绑定岗位，人负责理解、执行、检查和汇报，工具只提供辅助。
- **组织假设：**部门、团队和汇报线承载能力与协作，跨边界工作依赖交接和协调。
- **流程假设：**Sprint、会议、审批和阶段门控制节奏，因为人的执行速度和沟通带宽相近。
- **工具假设：**每个专业工具服务一个局部活动，上下文和责任链由人脑与文档连接。

这些假设并非错误。Scrum、看板、DevOps、代码评审和职责分工，都是人类组织处理复杂协作的有效机制。问题在于，当 Agent 能够跨步骤、高并发、持续运行时，旧机制的时间尺度和责任载体不再完全匹配。

如果岗位不变，人可能一边把执行交给 Agent，一边仍按亲自完成任务接受考核；如果组织不变，Agent 可以跨模块工作，授权和责任却卡在部门边界；如果流程不变，机器几分钟产生的结果要等待数天会议；如果工具不变，上下文、权限、证据和遥测就散落在多个 Copilot 与聊天窗口中。

此时继续采购更强模型，就像给拥堵的道路换上更快的汽车。车辆性能提高了，系统吞吐未必提高。

1.5 氛围编程的真正风险：探索模式进入正式交付

氛围编程适合原型探索、个人实验和低风险试验。在这些场景中，快速发散、快速看到结果、随时推倒重来，本身就是效率。

问题不在于“凭感觉写代码”，而在于组织没有明确区分探索和交付：原型接触了真实数据，试验代码直接进入生产，模型从实现反向生成验收标准，Agent 自己产生测试又用同一套假设宣布完成。

当探索模式未经转换直接进入正式交付，会积累四类债务：

- **意图债**：团队没有说清要改变的业务结果、边界条件和非目标，Agent 只能补齐假设。
- **上下文债**：Agent 获得的信息过少、过期、冲突或越权；更大的上下文窗口并不会自动产生正确上下文。
- **约束债**：“注意安全”“不要影响旧功能”仍是态度，没有成为机器可执行的权限、红线和停止条件。
- **证据债**：团队展示了代码和全绿结果，却不能证明目标、约束、测试、风险和发布裁决之间的对应关系。

这四类债务会被机器速度放大。Agent 越能自主补齐空白、连续执行和快速修复，组织越不能依赖“有经验的人最后看一眼”来兜底。**氛围编程最大的幻觉，不是模型会产生幻觉，而是组织把生成完成误认为价值交付完成。**

1.6 更快的执行，需要更快而不是更多的控制

面对风险，组织最熟悉的反应是增加审批。但如果每次 Agent 行动都等待人确认，机器速度会被重新锁回人类节奏；如果完全取消控制，错误又会以机器速度扩散。真正需要改变的不是控制数量，而是控制方式。

传统控制大量依赖会议、阶段检查和个人经验。AI 原生控制需要把可以明确表达的部分前移并自动化：目标在执行前校准，边界变成可运行约束，测试从验收标准出发，失败触发停止或修复，完成由异源证据支持，不可逆决策仍由有权的人承担。

探索可以发散，正式执行必须收敛
候选结果可以由 Agent 生成，是否完成必须由证据裁决
低风险动作可以自动化，价值判断与不可转移责任必须由人承担

这就是验证工程在新操作系统中的位置：它不是 AI 转型的全部，也不是给旧流程增加更多文档，而是让高速执行能够被信任的技术内核。至于人员如何分工、组织如何授权、流程如何闭环、工具如何承载，则需要在更大的系统中共同设计。

1.7 同一个报销需求，为什么“全绿”仍然失败

某团队让 Agent 修改报销审批规则，需求只有一句：“5000 元以上增加总监审批。”Agent 很快完成代码，自动生成测试并全部通过。上线后，恰好 5000 元的报销也被送到总监，外币报销按原币金额错误判断，大量单据阻塞。

复盘得到一条熟悉的失败链：

“以上”没有澄清边界

- 验收标准从实现反向生成
- 测试确认了同一个错误假设
- 没有明确外币场景是否属于本次范围
- 代码评审只检查实现质量
- 上线前没有独立证据支持裁决
- 事故另开 Bug，原任务仍被记录为首次成功

如果只把它看成一个代码缺陷，修复边界判断即可；如果从生产系统观察，问题分布在整条链上：业务意图没有被签署，财务规则没有转化为机器边界，测试与实现缺少独立性，发布责任没有可信证据，度量还把失败记录成成功。

这也是本书沿用这个案例的原因。后续章节会把它从一个“5000 元判断”向外展开为员工提交、经理审批、财务审核、合规控制、支付和系统接口组成的端到端价值流。同一个案例将同时接受两种检验：任务尺度上能否正确交付，组织尺度上能否真正减少等待、返工和责任模糊。

1.8 判断转型是否发生，不能只看 AI 使用量

工具激活率、Prompt 数量、Token 消耗和生成代码量，可以说明 AI 被使用，却不能单独说明组织提效。甚至“完成任务数增加”也可能来自任务切得更碎、返工被另立为新任务，或风险被转移到下游。

更有意义的问题是：

- 相同或更少投入，是否交付了更多有效价值？
- 从意图提出到用户获得结果，端到端周期是否缩短？
- 首次通过率是否提高，返工和失败成本是否下降？
- 人是否从重复执行转向价值判断、系统设计和异常裁决？
- Agent 的自治是否建立在真实约束、证据和可恢复能力之上？
- 成功经验是否成为可复用的组织资产，而不是个人技巧？

这些问题不能由一次演示回答。组织需要同时观察业务价值、治理健康、工程效能与系统进化，并允许指标相互制衡。代码量上升而业务结果不变，不是成功；周期缩短但失败成本失控，也不是可持续的成功。后续的价值与效能遥测将把这套判断变成可签署、可采集和可复查的事实。

1.9 本章小结：AI 首先是一场生产方式变革

AI 带来的提效是真实的，但组织提效不会由个人效率自动累加而来。新执行主体进入旧岗位、旧组织、旧流程和旧工具链后，瓶颈会迁移，隐性债务会被放大，管理者也更容易把工具热度误认为转型成果。

因此，企业需要的不是给旧研发系统增加一个 AI 插件，而是围绕新的执行能力重新设计生产方式。治理在其中负责让速度不越界，验证工程负责让完成可被信任，遥测负责判断提效是否真实；它们都是操作系统的组成部分，而不是转型本身的全部。

下一章将把本章暴露的结构矛盾展开为人员、组织、流程与工具四个维度，说明它们为什么必须围绕同一条价值流协同重构。

本章最小实践：选择一条已经使用 AI 的真实研发价值流，不讨论“大家用了哪些工具”，只标出四类事实：AI 加速了哪里，瓶颈转移到哪里，新增了什么返工或风险，最终业务结果是否改善。

完成证据：一张“局部提效—瓶颈迁移—业务结果”诊断图。

延伸实践：推荐学习《Agentic Agile 免费认知课：AI 转型为什么不是工具升级》，先识别局部提效为什么没有转化为组织提效；线下工作坊第一天进入“AI 转型为什么总是停在工具提效”环节，并通过四维转型画布把工具问题还原为人员、组织、流程与工具的系统重构。

第二章 | 四维重构：人员、组织、流程与工具必须同时改变

第一章说明，AI 带来的局部提效会让瓶颈沿价值流迁移。代码生成加快之后，需求澄清、架构决策、测试验证、跨团队协作和发布审批可能成为新的等待点。企业如果只优化一个环节，往往只是把压力推向下游。

因此，AI 研发转型不能被拆成四个彼此独立的项目：人力资源部门做岗位培训，管理层调整组织架构，研发效能团队改流程，平台团队采购工具。四方各自完成任务，仍然可能拼不成一套能够运行的人机协同系统。

四维重构的对象不是四张管理清单，而是同一条价值流。人员决定谁判断、谁执行、谁负责；组织决定能力怎样组合、决策权放在哪里；流程决定工作与反馈怎样流动；工具把意图、权限、执行、验证和证据变成可运行机制。任何一维缺位，其他维度的收益都会被削弱。

2.1 一张图看懂四维重构

转型维度	传统组织的默认假设	Agent 进入后的新矛盾	重构方向
人员 People	人是主要执行者，能力绑定岗位	Agent 承担大量执行后，人的价值、能力和责任容易混淆	从任务执行者转向意图定义、系统编排、异常裁决与责任签署者
组织 Organization	固定团队、部门和汇报线承载协作	Agent 可以跨边界并发，授权、信任和决策仍停留在部门结构中	从岗位接力转向“人类责任节点 + Agent 能力网络”
流程 Process	Sprint、会议、审批和交接控制工作流	机器内环远快于人类节奏，等待与人工审核成为瓶颈	从活动和阶段驱动转向意图—约束—执行—验证—反馈的持续闭环
工具 Technology	工具辅助人完成局部任务	Copilot 孤岛、上下文断裂、权限分散、结果无法独立证明	从单点助手转向 Context + Orchestration + Harness + Evidence + Telemetry

这张表描述的是方向，不是未来组织的固定模板。不同业务、风险和工程基础会产生不同设计。四维模型真正要求的是：每一次改变都能回答它如何改善价值流、如何与另外三维连接、由什么事实证明有效。

2.2 必须协同改变，但不能“大爆炸式转型”

“四维必须同时改变”容易被误解为先设计目标组织、重画全部流程、建设统一平台，再让所有员工切换工作方式。这样的整体改造周期长、假设多，而且很容易在真实反馈出现之前就投入过重。

本书所说的“同时”，是指围绕一条真实价值流形成**最小完整重构**：

选择一个必须改善的业务结果

→ 选择一段真实价值流和可交付切片

→ 明确人和 Agent 的任务、判断与责任

→ 调整完成该切片所需的授权和协作边界

→ 删除、自动化或重构一个关键等待节点

→ 接入最小上下文、约束、验证、证据与遥测能力

→ 用真实结果决定扩大、调整或停止

这意味着每一维都要有动作，但每一维只改变足以跑通本轮闭环的部分。组织架构不必先整体重画，平台不必先一次建全，流程也不必为了“AI 原生”全部废弃。**转型单元应当足够小，可以快速获得证据；又必须足够完整，不把成本和风险悄悄转移给闭环之外的人。**

2.3 人员重构：迁移任务，不是预测岗位消亡

“AI 会消灭哪些岗位”是一个容易吸引注意力、却不适合作为组织设计起点的问题。岗位由多种任务、判断、关系和责任组成。Agent 通常先改变其中的任务组合，而不是在某一天完整接管一个岗位。

更可执行的方法，是把一条价值流中的工作分成四类：

任务类型	分配原则	典型示例
交给 Agent 执行	高频、可表达、可验证、可恢复	检索、代码生成、测试执行、重复修复、证据收集
人机协同增强	Agent 提供候选与分析，人作判断或组合	需求澄清、方案比较、风险识别、测试设计、故障诊断
保留人类责任	涉及价值取舍、模糊冲突、组织承诺或不可逆后果	优先级、风险接受、例外批准、发布裁决、对外承诺
禁止 Agent 自治	法律或伦理不允许委托，或者当前无法验证和恢复	未授权生产写入、模型自批权限、替责任人签署、绕过监管控制

这里需要区分**执行权与责任**。Agent 可以完成一项工作，不代表它可以承担组织责任；人不再亲手完成每一步，也不意味着人退出。人的重心从重复执行转向意图定义、边界设计、系统编排、异常处理和结果裁决。

这种变化也会重组专业能力。产品人员要把价值和目标表达得可判定，开发人员要设计规格、测试和 Agent 工作流，测试人员要建设独立证据与评估体系，架构师要设计上下文、权限和恢复边界，研发管理者要学会运营人机系统而不是只管理人的活动。

人员重构的完成证据不是“多少工作由 AI 生成”，而是关键任务是否被分配给更合适的主体、人类责任是否更清楚、重复劳动是否下降、异常是否能够在正确位置交还给人。

2.4 组织重构：从岗位接力到人机责任网络

传统组织通过稳定团队、专业部门和汇报关系降低协作复杂度。这些结构仍然有价值，但它们不应继续成为每一次信息、任务和决策流动的唯一通道。

Agent 可以按意图临时组合代码、测试、安全、数据和领域能力。相应的组织单元需要从“哪些岗位参加”转向“完成这个意图需要哪些能力，哪些节点必须由人负责”。本书把这种结构概括为：

围绕真实意图形成最小责任单元，由人类责任节点定义价值、边界与裁决，由可组合的 Agent 能力网络承担约束执行。

人类责任节点不是新的审批层。它们只应出现在机器不应自行决定的位置，例如业务价值冲突、权限扩大、约束例外、不可逆动作和发布承诺。其余可以被明确表达和验证的决定，应通过预先授权、机械规则和证据快速流动。

组织重构至少要处理四件事：

- 1. **责任可指认**：谁定义意图，谁设计运行控制，谁执行，谁接受剩余风险？
- 2. **决策权前移**：哪些决定可以交给最接近事实的人或规则，不再层层上报？
- 3. **能力可编排**：领域专家、平台能力和 Agent 如何按任务组合，而不是每次重新拉群协调？
- 4. **异常有去处**：越界、冲突、证据不足和无法收敛时，由谁在什么时限内裁决？

组织重构不是取消稳定团队，也不是让所有人进入临时项目。稳定团队可以继续承载产品知识、长期责任和心理安全；Agent 能力网络则降低跨专业任务每次都靠人工接力的成本。二者的边界应由价值流和责任设计决定，而不是由“固定”或“动态”哪一个听起来更先进决定。

2.5 流程重构：让机器运行快内环，让人承担责任外环

传统流程常用 Sprint、站会、评审、审批和交接解决信息同步、工作协调与质量控制。这些活动形成于人的执行与反馈节奏。当 Agent 可以分钟级生成、验证和修复时，继续让每个动作等待下一场会议，会主动放弃机器速度；把全部检查取消，又会失去边界和责任。

AI 原生流程需要区分两个时间尺度：

- **机器快内环**：在已签署意图和权限范围内执行、测试、诊断、修复、重试和停止，反馈以秒、分钟或小时发

生。

- **人类责任外环**：决定价值、签署边界、处理异常、接受风险、裁决发布，并根据遥测调整下一轮规则和授权。

流程重构不是把所有会议变成自动化，而是逐项判断它们承担什么功能：

现有活动	应保留的条件	可以重构的方向
Sprint 计划	仍需要阶段性投资和组合承诺	将任务级执行转为持续拉动，不等待下一代才纠偏
站会	需要处理真实障碍、冲突和责任	状态汇报由事件和看板生成，会议只处理例外
代码评审	需要架构判断、领域知识和责任确认	机械检查前移，评审对象扩展到意图、差异、风险和证据
阶段审批	涉及不可逆风险或法定责任	低风险重复审批转为预授权规则，高风险保留 HITL
回顾	需要共同解释系统性问题	高频失败自动回流，人工复盘聚焦模式、激励和规则改变

判断一个流程是否应该保留，不看它属于传统还是 AI 原生，而看它是否承担不可自动化的价值与责任。如果只是传递机器已经知道的状态，它应被事件化；如果承担风险接受和组织承诺，它就不能因为追求“零 HITL”被删除。

2.6 工具重构：从 Copilot 孤岛到可替换的工程系统

单点 AI 工具能够提高个人效率，却通常只看见局部上下文，也不天然理解组织意图、权限、历史决策和完成标准。企业如果不断增加 Agent，却没有共同的运行底座，会形成新的工具孤岛：上下文重复投喂，规则各自维护，执行无法追踪，证据无法合并，成本和风险难以比较。

工具重构不是先采购一个“大一统 Agent 平台”，而是逐步补齐六类可以组合的能力：

1. **Context**：按任务提供最小、可信、可追溯的代码、业务和知识上下文；
2. **Orchestration**：描述依赖、并发、交接、超时、重试、停止和人工升级；
3. **Harness**：把权限、约束、验证、恢复和运行规则加载到 Agent 周围；
4. **Evidence**：把验收标准、测试、约束、风险和执行事实组织成可裁决证据；
5. **Telemetry**：连接算力投入、自治过程、工程质量和业务结果；
6. **Knowledge & Memory**：只把经过验证的经验晋升为可复用知识、规则、测试或 Skill。

这些能力必须与具体模型和 AI 宿主保持适度解耦。企业可以同时使用 Codex、Claude、WorkBuddy 或其他工具，真正需要稳定的是契约、约束、证据、遥测和交接协议。否则每次模型或供应商变化，都要重新建设一套组织工作方式。

工具维度的目标也不是让人接触更多控制台。好的工程系统应把规则嵌入日常工作，让低风险任务快速通过，让高风险或证据不足的任务准确停止，并把人带到需要判断的位置。

2.7 治理与度量为什么不是第五、第六维

四维模型之外，企业还需要治理和度量，但不应把它们设计成两个独立工作流。

治理横贯四维：它明确人类不可转移的责任，规定组织授权和例外，定义流程中的机械门与 HITL，并通过工具执行权限、约束和恢复。治理如果只由一个委员会或平台团队承担，就会变成外置审批，而不是系统能力。

度量同样横贯四维：它观察人的时间是否转向高价值判断，组织等待和交接是否减少，流程周期与首次通过率是否改善，工具、上下文和算力是否产生净收益。度量如果只统计工具激活率、代码量或 Token，就会奖励局部产出，进一步掩盖价值流瓶颈。

因此：

四维重构决定生产系统怎样运行
治理决定它在什么边界内运行
Evidence 证明一次运行是否成立
Telemetry 判断长期运行是否真正提效并持续进化

Dashboard 是管理者观察和决策的入口，不是另一个“AI 绩效考核系统”。它评估的是人、Agent、上下文、工具链和算力共同构成的生产系统，不能把系统结果粗暴归因到个人，更不能用来逼迫团队减少必要的 HITL 或隐藏失败。

2.8 用报销价值流检验四维是否连通

第一章的“5000 元以上增加总监审批”看似只是一个规则变更。把观察尺度扩大后，它属于一条更长的价值流：员工提交报销，经理审批，财务审核，合规检查，支付执行，移动端与财务系统同步状态。

如果要让 Agent 参与这条价值流，四维必须同时回答：

维度	本案例需要作出的决定
人员	谁定义“以上”、币种和例外规则？哪些测试与实现交给 Agent？谁接受上线风险？
组织	业务、财务、合规和技术决策权如何分配？规则冲突由谁裁决？
流程	哪些审批可以根据金额和风险自动路由？哪些异常必须进入人工复核？
工具	规则、汇率、权限和历史决策怎样进入上下文？如何测试、留证、发布、回滚和观测？

只改工具，Agent 会更快地产生一个可能错误的实现；只改流程，自动路由可能把错误规则传播得更快；只培训人员，责任仍可能卡在旧部门边界；只调整组织，没有可执行契约和证据，协作仍然依赖口头解释。

四维连通后的最小结果，不是“建成智能报销平台”，而是完成一个真实规则切片：责任清楚、边界可执行、流程减少等待、工具能够验证和留证，并且上线后的业务结果可以被观察。这才是可以继续扩展的转型起点。

2.9 四维转型的最小诊断法

四维讨论很容易滑向“加强协同、提升能力、建设平台”之类无法验收的口号。为了让诊断进入行动，每个维度只写五项：

必填项	要回答的问题
当前事实	当前价值流中真实发生了什么，有什么数据或样本？
目标改变	本轮具体要改变哪项行为、结构或能力？
第一步动作	七天内可以完成什么最小动作？
责任人	谁对动作与结果负责，Agent 不能替谁签署？
完成证据	看到什么事实，才算改变已经发生？

诊断顺序也很重要：先选择业务结果和目标价值流，再看四维断点；先找当前瓶颈，再决定需要什么 Agent 和平台能力。若从工具清单出发，团队很容易为已有产品寻找场景，而不是为真实问题设计系统。

2.10 本章小结：四维不是四条线，而是一套系统

AI 研发转型不是工具升级，也不是组织架构调整、流程再造或人员培训中的任何单项工程。它要求人员、组织、流程和工具围绕同一条价值流形成最小完整重构，并用真实证据验证这种新组合是否产生了更快、更好、更低成本或更可持续的业务结果。

协同重构不等于大爆炸。正确起点是一条真实价值流、一个可交付切片和一个四维闭环：任务与责任重新分配，授权与协作边界得到调整，关键等待点被删除或重构，最小工程系统能够执行、验证、留证和反馈。

后续章节将先用三大理论基石和五条宣言建立设计依据与价值取舍，再通过 3-4-3、SCOPE-V、验证工程、Evidence 和 Telemetry 把四维重构变成可运行机制。

本章最小实践：选择一条真实研发价值流，为人员、组织、流程和工具分别写出“当前事实、目标改变、第一步动作、责任人、完成证据”。

完成证据：一张 Agentic Agile 四维转型诊断图，其中四维围绕同一个业务结果，不允许各自选择不同试点。

延伸实践：推荐学习《Agentic Agile 免费认知课：AI 转型为什么不是工具升级》，从知识工作、组织惯性和价值兑现的角度理解为什么 AI 转型不是采购工具。

第三章 | 理论基石：认知科学、控制论与系统论

Agentic Agile 不是因为出现了大模型，就临时拼装出一套新名词。它试图解决的三个问题，分别对应三个长期存在的科学视角：有限认知怎样承载复杂信息，反馈系统怎样发现并纠正偏差，组织怎样具备处理复杂环境所需的多样性。

三大理论不是“证明 Agentic Agile 必然正确”的权威背书，更不是为了否定传统敏捷。它们是三副观察镜：帮助我们识别纯人类协作、单模型执行和多 Agent 系统各自的能力边界，从而设计更可靠的治理机制。

三副观察镜与上一章的四维模型不在同一个抽象层级。四维说明**要重构什么**，三大理论帮助判断**为什么这样重构、设计是否合理**。它们之间不是一一对应关系，而是交叉映射：

理论视角	首先揭示的问题	主要四维落点	对其他维度的影响
认知科学	人脑与模型上下文都有限，隐性知识难以稳定传递	人员、工具	推动组织共同语言外部化，减少流程中的寻找、解释和交接负担
控制论	高速执行会放大偏差，晚反馈使错误扩散	流程、工具	重新安排人类责任节点，并要求组织明确停止、恢复与升级权
系统论	单一主体响应不足，多主体协作又会制造协调债	组织、流程	影响人员责任分工，并要求工具承载依赖、协议、证据聚合与韧性

这张映射表的作用不是给四维披上理论外衣，而是防止局部设计自治、整体系统失衡。例如，工具维度建设了更大的上下文窗口，却没有处理知识权威和人员责任，认知负荷只是从人脑转移到了模型；流程维度增加自动反馈，却没有可靠信号和停止条件，闭环反而可能加速错误。

3.1 认知科学：复杂度不能只放在人脑里

3.1.1 工作记忆为什么重要

1956 年，认知心理学家乔治·米勒在《神奇的数字7，加或减2》中讨论了人类处理信息的容量限制。后续研究对“7±2”进行了修正，指出人在没有充分组块帮助的情况下，能够稳定保持和操作的信息单元可能更少。具体数字并不是本书的重点，真正重要的结论是：**工作记忆有限，而且容易受到任务切换、干扰和信息结构的影响。**

软件研发恰好是一项高认知负荷工作。工程师不仅要理解当前代码，还要同时处理：

- 用户真正想解决的问题；
- 业务规则和边界值；
- 模块依赖与数据流；
- 历史设计决策；
- 安全、性能和合规要求；
- 当前版本和环境状态；
- 团队尚未说出口的隐性约定。

当这些内容主要依赖个人记忆时，组织会自然采用分工、文档、评审、待办列表、迭代计划和完成定义来降低认知压力。这些机制并不是“落后”，而是人类面对复杂协作时形成的有效外部化工具。

3.1.2 认知负荷的三种来源

认知负荷理论通常区分三类负荷。将其映射到研发，可以帮助我们更准确地设计上下文。

内在负荷来自问题本身。例如财务审批包含金额、币种、组织层级、预算、税务和例外规则，问题天然复杂。

外在负荷来自信息呈现和工作方式。例如规则散落在会议记录、聊天记录、代码注释和个人记忆中；工程师需要不断切换工具和寻找上下文。

有益负荷用于建立真正的理解，例如形成领域模型、识别不变量、设计测试和抽象可复用模式。

好的治理不承诺消灭内在复杂度，而是减少无效的外在负荷，把认知资源留给价值判断和系统设计。

3.1.3 邓巴数与协作网络

工作记忆解释了个人为什么难以同时处理大量信息，但软件研发还存在另一层限制：复杂知识分散在许多人之间，团队必须通过沟通才能形成共同理解。罗宾·邓巴关于社会关系规模的研究提醒我们，人能够长期维持的稳定关系数量存在现实边界；常被引用的约 150 人及其若干内层圈，是统计意义上的经验尺度，而不是适用于所有组织的硬性上限。

对研发治理更直接的启示是：团队规模扩大后，共同上下文和高信任协作不会自动按人数增长。若任意两人都可能直接协作，潜在的两两沟通关系可用 $N(N-1)/2$ 表示。它是二次增长，而不是指数增长，但已经足以说明协调负担为什么会迅速上升：10 人团队有 45 对潜在关系，20 人团队有 190 对。实际沟通不一定覆盖所有关系，组织也会用模块边界、稳定团队和接口机制降低负担，但不能假设所有成员始终拥有同一份上下文。

当 Agent 加入团队，沟通成本不会消失，而是改变形态：

- 人与人之间的口头解释，部分转化为 Agent 对上下文的解析；
- 团队之间的接口协商，部分转化为工具协议和 Handoff；
- 依赖个人经验形成的信任，需要由契约、约束和证据补充；
- 信息不同步，会表现为上下文、代码和知识版本漂移；
- 职责不清，会变成人与 Agent 都以为对方会负责。

因此，Agentic Agile 不是认为“有了机器就不需要协作”，而是承认隐性默契无法无限扩展，并把目标、边界、交接和完成依据转成机器与人都能读取和验证的结构。

3.1.4 大上下文不等于全局理解

大模型拥有越来越长的上下文窗口，但窗口容量不是理解质量。把整个代码库、所有文档和全部历史记录塞进一次任务，会产生新的问题：

- 重要信息被噪声淹没；

- 新旧规则互相冲突；
- 不应暴露的数据进入上下文；
- 模型无法判断哪个来源更权威；
- Token 成本和推理时延上升；
- 过期信息被当成当前事实。

Agentic Agile 因此采用分层上下文和任务裁剪，而不是追求“什么都给模型看”。意图图谱保存全局方向，约束矩阵保存共同底线，意图契约保存当前任务，代码发现与检索提供必要实现上下文。

3.1.5 从认知外包到认知治理

“认知外包”容易被误解为人类不再思考。更准确的表述是：**把检索、比较、执行、重复验证等可自动化认知活动交给机器，把价值判断、责任承担、冲突取舍和未知情境保留给人。**

对应的工程设计包括：

- 意图图谱：防止局部任务失去全局方向；
- 意图契约：防止当前任务无限发散；
- 上下文裁剪（Context Cropping）：提供最小必要上下文；
- 工具约束清单（Tools Manifest）：限制 Agent 能够触达的能力；
- 证据包（Evidence Bundle）：把执行过程重新压缩成人可审阅的证据。

认知科学为 Agentic Agile 提供的不是“机器优于人”的结论，而是一个设计原则：**复杂度必须被外部化、结构化和裁剪，不能仅靠人脑记忆，也不能仅靠模型上下文硬吞。**

映射到四维，人员重构要把人的认知资源留给价值判断和未知情境，工具重构要提供经过裁剪的可信上下文；组织与流程则要减少依赖隐性默契的交接，把共同意图、规则和完成依据变成可共享结构。

认知治理回答了“系统在行动前应该知道什么”，却还没有回答：Agent 行动之后，偏差怎样被发现、修正和停止。这正是控制论要解决的问题。

3.2 控制论：没有可靠反馈，速度只会放大偏差

认知外部化能够让目标和边界更清楚，但清楚的输入并不保证执行永远正确。模型输出具有概率性，工具和环境会失败，需求与代码也可能在执行过程中发生变化。控制论提供的关键视角是：系统不能只根据指令向前运行，还必须观测结果、比较偏差并采取纠正动作。

3.2.1 从开环执行到闭环控制

开环系统根据输入执行动作，却不利用结果持续调整行为；闭环系统则把输出重新带回控制过程，与目标或参考状态比较，再决定保持、纠正、降级或停止。闭环并不保证结果必然正确，但它使系统具备发现偏差和恢复的结构。

一个基本闭环通常包含六个要素：

控制要素	回答的问题	AI Coding 中的映射
目标或参考状态	期望结果是什么？	已签署意图契约、AC、业务不变量
被控对象	哪个系统正在变化？	代码、配置、数据、运行环境与 Agent 状态
执行动作	系统可以怎样改变对象？	编辑、测试、工具调用、迁移、部署准备
观测	实际发生了什么？	测试、构建、扫描、日志、差异和生产信号
比较与判断	与目标和约束相差多少？	AC 映射、约束门禁、Evidence Bundle、Verify
控制动作	偏差出现后怎么办？	修复、重试、回退、降级、补证或升级人类

氛围编程一次性给模型 Prompt、得到代码、直接合并，接近开环执行：它可能成功，却没有建立稳定发现偏差的能力。验证工程要求形成可运行的闭环：

目标与约束

→ 被授权的执行

→ 真实观测

→ 比较目标、约束与结果

→ 纠正、回退、停止或升级

→ 再次观测

这里的重点不是把瀑布、敏捷或其他方法简单归类为开环或闭环。任何生命周期方法都可以包含反馈，也可能因为反馈太晚、太弱或不可信而在局部退化成开环。真正需要检查的是：反馈是否进入了当前执行路径，而不只是存在于流程名称中。

3.2.2 负反馈、强化回路与稳定性

负反馈通过抵消偏差帮助系统趋于稳定。例如单元测试失败、CI 构建中断、安全扫描阻断、权限门拒绝越权操作，都会迫使执行回到允许范围。负反馈不是“否定创新”，而是在当前目标和边界内抑制失控扩散。

强化回路则会放大已有趋势。成功模式被固化为模板并被更多任务复用，可能形成有益强化；未经验证的错误规则进入知识库并反复影响后续任务，也会形成有害强化。因此，经验沉淀不能只看是否被重复使用，还要经过验证、版本、适用范围和撤销机制。

稳定系统通常需要两者配合：**用负反馈控制偏差和风险，用经过验证的强化回路扩大有效能力**。任何反馈都不是越快、越强越好；信号错误、增益过大或修复范围失控，同样可能造成振荡和新的故障。

3.2.3 从敏捷反馈到 Agent 运行时反馈

每日同步、迭代评审、用户验收和回顾都在缩短反馈周期；持续集成、测试自动化和 DevOps 又把反馈推进到提交、构建、发布和生产运行阶段。Agentic Agile 并不是第一次发现反馈的重要性，也不需要否定这些已有实践。

变化在于时间尺度和执行密度。Agent 可能在一次人工评审之前完成多轮修改、测试和工具调用。如果关键反馈仍主要等待会议或阶段末评审，执行吞吐就会远高于验证吞吐，早期误解会迅速扩散。因此，可自动化的反馈需要嵌入 Agent 的执行路径，高风险价值判断和责任裁决则保留给人。

3.2.4 反馈时延、噪声、增益与覆盖

反馈系统不仅怕慢，也怕错。

时延过长：Agent 已经连续修改多个模块，团队才发现最初目标理解错误，返工范围巨大。

噪声过高：测试不稳定、日志缺失或 LLM-as-Judge 未校准，系统可能根据错误信号反复修改正确代码。

增益过大：一次局部失败触发大范围重构，纠正动作本身引入更多不确定性。

覆盖不足：只验证正常路径，没有边界、权限、恢复和业务不变量，系统会对未观测风险保持盲目乐观。

因此，每个闭环都要回答：测量什么、怎样测量、证据是否可信、允许修改多少、多久必须停止，以及失败是否回到同一个检查重新验证。SCOPE-V的 $P \Rightarrow E$ 快内环（后文会讲）之所以要求“最小修复—原检查重跑”，就是为了限制纠正增益并保持证据连续性。

3.2.5 闭环自治何时必须让人接管（HITL）

闭环自治必须有退出条件。以下情况不适合让 Agent 继续自我修复：

- 业务目标或规则发生冲突；
- 需要修改已经签署的契约；
- 需要扩大数据、工具或生产权限；
- MUST 约束失败且无法在授权范围内恢复；
- 证据互相矛盾或缺少独立来源；
- 超过时间、Token 或重试预算；
- 涉及不可逆操作；
- 法律、伦理或商业责任需要人承担。

控制论带来的核心启示不是“让机器无限循环”，而是：**自治能力取决于系统能否根据可信反馈收敛，并在不该继续时停下来。**第三部分将把这一原则实现为 SCOPE-V 六个控制面、 $P \Rightarrow E$ 快内环、Verify 独立裁决、Telemetry 慢外环与机械门禁。

映射到四维，流程重构要把可自动化反馈嵌入机器快内环，把价值与风险裁决放在人类责任外环；工具重构要提供真实观测、机械门禁和恢复能力；人员与组织重构则要明确谁有权改变目标、接受例外、扩大权限和决定停止。

闭环控制解释了系统怎样沿时间持续纠偏，但复杂研发还面临结构性问题：业务、架构、安全、数据和运维包含不同种类的变化，单一主体未必拥有足够的响应方式。怎样用受治理的多样性应对这种复杂性，是系统论视角进一步回答的问题。

3.3 系统论：以受治理的多样性应对复杂性

本节所说的系统论视角，关注的不是一次反馈怎样发生，而是多个角色、模块、工具和环境怎样共同构成一个整体。控制论更强调时间上的“观测—比较—纠正”，系统论则进一步追问：面对种类繁多的变化和扰动，治理系统是否拥有足够的响应方式，又怎样防止这些响应方式彼此冲突。

下面的论证沿着一条连续的问题链展开：复杂环境首先要求足够的响应多样性；多 Agent 可以提供这种多样性，却同时产生新的协调债；Work Graph 将协作关系变成可执行结构；意图图谱、全局约束与跨模块协议再把局部执行锚定到整体目标；最后，韧性机制负责处理治理之后仍无法消除的失败。

3.3.1 必要多样性：复杂问题需要足够的响应方式

阿什比的必要多样性法则常被概括为“只有多样性才能吸收多样性”。其工程含义不是控制者必须比整个被控系统更复杂，也不是增加更多 Agent 就能消灭复杂度，而是：**面对环境中可能出现的不同状态和扰动，治理系统必须拥有足够多、足够合适的识别与响应方式，否则某些变化将无法被区分和处理。**

组织通常有两类基本策略。一类是**削减被管理的多样性**：缩小范围、隔离环境、标准化接口、限制权限、拆分系统或拒绝高风险变化。另一类是**增加有效响应多样性**：引入领域、安全、测试、运维等不同视角，建立更多验证方法和恢复策略。成熟治理往往同时使用两类策略，而不是在“简化问题”和“增加能力”之间二选一。

可以把它类比为急诊分诊。面对病因、严重程度和资源需求不同的患者，医院既需要通过分诊和限流控制进入系统的复杂度，也需要不同专科、检查手段和处置路径提供响应多样性。医生数量本身并不保证救治质量；共享病历、分诊标准、责任边界和协作流程同样重要。

映射到 AI 研发，需求裁剪、最小权限和变更边界（Change Envelope）用于降低风险多样性；领域 Agent、测试 Agent、安全 Agent 和多种验证工具用于增加响应多样性。二者共同决定系统能否处理复杂任务。

3.3.2 多 Agent 提供多样性，但不会自动形成涌现

现代软件系统同时包含产品、架构、开发、测试、安全、数据、运维和合规视角。单个模型即使能力很强，也很难在一次推理中稳定兼顾所有目标。多 Agent 的价值首先来自职责差异和视角互补：

- Domain-Agent 识别业务规则和例外；
- Dev-Agent 关注最小实现及代码影响；
- QA-Agent 从 AC、不变量和状态模型生成反例；
- Security-Agent 检查攻击面、数据流和权限；
- Ops-Agent 关注运行、恢复和可观测性；
- Critic-Agent 挑战假设、证据与局部最优。

多个执行单元可以并行探索互不依赖的方案，也可以通过相互独立的证据减少单一视角盲区。但这只提供了产生更好结果的条件，不构成结果必然正确的证明。“涌现”应理解为系统整体表现出单一执行单元不容易稳定获得的能力，而不是 Agent 数量增加后自然出现超越个体的智慧。

判断多 Agent 是否真正有价值，应看它是否增加了有效覆盖、独立证据或恢复能力，而不是看调用了多少模型、生成了多少消息。

然而，一旦这些能力单元真正进入同一条交付链，问题就会从“有没有足够视角”转向“这些视角怎样协同而不互相干扰”。多样性解决覆盖不足，也会产生新的协调成本。

3.3.3 协调债：Agent 多样性也会制造新的复杂度

增加 Agent、工具和验证方法的同时，也会增加：

- Handoff 信息损耗；
- 对目标和规则的不同理解；
- 文件、环境和计算资源争用；
- 重复工作与相互覆盖；
- 循环依赖和无法收敛的互评；
- 多个 Agent 共享同一错误假设形成的错误共识；
- Token、时延与运行成本；
- 最终责任归属的模糊。

十个没有共同契约的 Agent，可能比一个受约束的 Agent 更危险。因此，必要多样性不等于最大多样性。有效多样性必须与任务风险相称，并由共同意图、明确边界、可执行协作关系和统一证据标准约束。

这些新增成本可以统称为**协调债**：能力单元越多，交接、依赖、状态、资源和责任越需要显式治理。仅靠组织图或“大家自行协作”无法偿还协调债，下一步必须把真实协作关系表达成运行时可以执行和检查的结构。

3.3.4 Work Graph：把协调关系变成可执行结构

Work Graph 直接回应上一节的协调债。传统组织结构说明谁属于哪个团队，却不一定说明某次任务的真实依赖；流程图可以帮助人理解顺序，却未必包含机器执行所需的状态和控制条件。**Work Graph（工作图）是一张能够被人和运行时共同读取的任务关系图**：节点表示具有输入、输出、执行主体和完成判据的工作，边表示节点之间的依赖、交接或控制条件。

Work Graph 不只是把待办事项换成方框和箭头。它需要回答：

- 哪些任务可以并行，哪些必须等待上游证据；

- 哪些节点共享文件、环境或其他资源；
- 哪个失败会阻断下游，失败后进入重试、回退还是升级；
- 每个节点需要产生什么工件和证据；
- 哪个节点需要 HITL；
- 任务何时停止、重新调度或作废。

图工程（Graph Engineering）的目的，是把多样性从“许多 Agent 同时行动”转成“多个受控能力按可验证关系协作”。第 12.1 节将进一步拆解 Work Graph 的节点、边、状态和控制条件。

不过，Work Graph 主要回答“工作怎样协同执行”，并不自动保证“执行方向符合整体价值”。一张结构正确、运行顺畅的图，仍然可能高效地完成错误目标，或者让每个节点局部通过却损害系统整体。因此，解决协调问题之后，还要解决局部与整体的对齐问题。

3.3.5 系统对齐：防止局部最优破坏整体目标

多 Agent 协作中，节点通常按照自己的输入、目标和完成判据优化。如果缺少更高层的共同目标，局部合理并不会自然汇聚为系统合理。Agent 可能在局部任务上表现出色，却损害整体系统。例如：

- 为提高接口速度，引入破坏数据一致性的缓存；
- 为让测试通过，绕过真实依赖；
- 为减少 Token，省略关键上下文；
- 为提高自动化率，把应该由人批准的操作也自动化；
- 为优化单模块指标，破坏跨模块兼容。

意图图谱和全局约束把局部任务重新锚定到系统目标；跨模块 Protocol 约束接口、依赖与集成顺序；Evidence Aggregation 防止出现“每个模块都通过，系统却无法发布”。系统级治理不是压制局部优化，而是要求局部改进能够证明没有牺牲更高层目标。

系统对齐能够减少目标冲突和局部优化造成的伤害，却仍不能消除未知依赖、环境故障、模型波动和真实世界变化。既然复杂系统不可能被一次设计成永不失败，最后一个问题就不再是“怎样避免所有失败”，而是“失败发生后，系统能否限制影响并恢复”，也就是我们通常讲的系统具备韧性和反脆弱能力。

3.3.6 韧性闭环：为无法消除的失败设计恢复能力

韧性是前面几层治理的自然终点：必要多样性提供响应能力，Work Graph 管理协作关系，系统对齐守住整体目标，而韧性机制负责承接它们仍然覆盖不到的剩余不确定性。复杂系统无法消除所有失败，Agent 和工具的加入也不会改变这一事实。更现实的治理目标是：

- 失败能够被发现和隔离；
- 影响范围能够被限制；
- 关键状态能够被观测；
- 系统能够降级、补偿或回退；
- 未知风险能够被标记而不是伪装成通过；
- 人类能够在必要时接管；
- 失败能够沉淀为新的约束、测试、知识或记忆。

系统论为 Agentic Agile 提供的核心启示是：**用足够而适当的响应多样性应对复杂性，同时用契约、约束、Work Graph、证据和责任控制多样性自身带来的复杂度。**

映射到四维，组织重构要组合足够的专业与 Agent 能力，同时控制协调债；流程和工具要把依赖、交接、证据聚合与恢复变成可执行结构；人员重构则要保证多主体协作之后，价值判断和最终责任仍然有清晰归属。

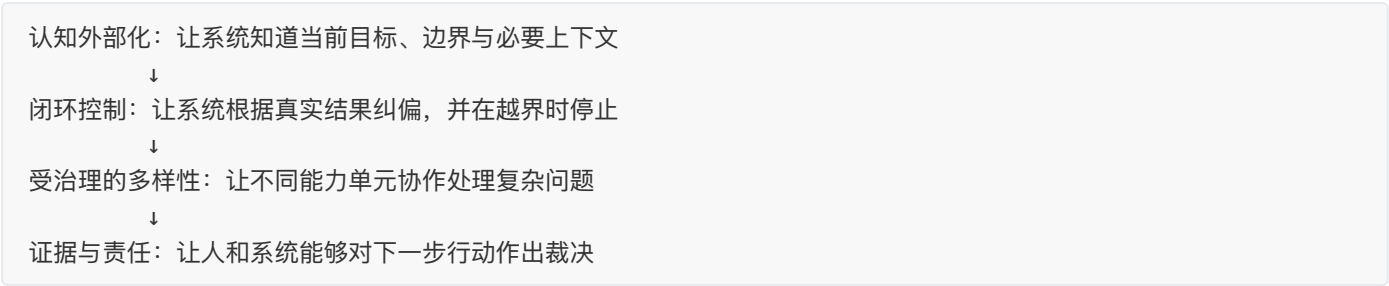
至此，三种视角形成递进：认知科学让关键信息得以外部化和裁剪；控制论让执行能够观测、纠偏和停止；系统论让多个能力单元在复杂环境中形成受治理的协作。任何一项单独存在，都不足以构成可信自治。

3.4 三大理论的融合：从理论启示到工程机制

三大理论不是用来“证明 Agentic Agile 必然正确”的权威背书，而是三副相互补充的设计镜头。它们分别暴露不同类型的治理缺口，并导出可以被工程实现和验证的机制。

理论视角	暴露的核心问题	设计原则	四维主要落点	在 3-4-3 中的主要映射	理论边界
认知科学	人脑与模型上下文都有限，信息会丢失、过载或过期	外部化、结构化、分层与任务裁剪	人员、工具	意图图谱、意图契约、Context Cropping、Knowledge Engineering	结构化信息不能替代价值判断，也不能保证执行正确
控制论	高速执行会放大偏差，晚反馈导致大范围返工	真实观测、目标比较、适度纠正、停止与恢复	流程、工具	SCOPE-V、P⇌E、Verify、五道门、Telemetry	闭环可能受噪声、延迟和错误目标影响，不能无限自治
系统论	单一主体的响应方式不足，多主体又会增加协调复杂度	增加受治理的多样性，兼顾局部与整体	组织、流程	IO/OA/AS、多 Agent、Work Graph、Protocol、Evidence Aggregation	Agent 数量不等于有效多样性，多数共识不等于事实

三者可以连成一条工程逻辑：



如果借用信息论语言，可以把契约、约束和验证理解为提高研发协作的信噪比，把遥测与反馈理解为持续减少未知和偏差。真正能够支撑 Agentic Agile 方法有效性的，仍然是可执行工件、真实证据、可重复验证和实际业务结果。

三大理论共同形成 Agentic Agile 的基本立场：

人类负责价值与责任，机器负责被授权的执行，规则负责守住边界，证据负责支持决策，遥测负责推动系统进化。

这一立场也解释了 Agentic Agile 与敏捷、DevOps 和现有工程体系的关系。已有方法继续负责产品探索、团队协作、生命周期和交付基础设施；3-4-3 与 SCOPE-V 重点补充概率性 Agent 进入执行路径后所需的意图、权限、反馈、证据和责任控制。二者可以叠加和裁剪，而不需要被描述为互相替代的两套阵营。

3.5 本章实践：用三副镜子诊断一个 AI workflow

选择一条正在使用 AI 的真实研发价值流，分别回答：

认知视角：信息是否散落？Agent 拿到的上下文是否过多、过少或过期？

控制视角：哪里产生真实反馈？失败如何修复？何时停止？

系统视角：任务怎样分解？依赖怎样表达？局部结果怎样汇聚为整体证据？

完成后，为每个缺口标记它主要发生在人员、组织、流程还是工具维度。不急着增加更多 Agent，先选择一个会阻断端到端价值、且能在本轮获得验证的缺口。

3.6 本章小结

三大理论不是为了给方法论增加学术装饰。认知科学解释为何需要把意图、边界和知识外部化；控制论解释为何必须建立反馈、停止与恢复；系统论解释为何既要利用多 Agent 的多样性，又要用契约、图谱和证据控制协作复杂度。它们共同回答了四维为什么必须协同重构，以及 3-4-3 为什么采用这样的运行设计。

本章最小实践：用认知、控制、系统三种视角诊断一条现有 AI 研发价值流，并把问题映射到人员、组织、流程和工具。

完成证据：三个主要系统缺口、对应的四维位置和一个优先改进项。

延伸实践：推荐学习《Agentic Agile 免费认知课：AI 转型为什么不是工具升级》，用认识论、控制论和系统论重新审视 AI 时代的研发协作。

第四章 | Agentic Agile 五条宣言：先有价值取舍，再有方法

方法论需要价值观，因为工具无法替组织回答“发生冲突时更重视什么”。同一套 Agent、测试和流水线，可以被用来追求短期速度，也可以被用来建立长期可信交付。五条宣言为 3-4-3 的设计和取舍提供共同方向。

宣言中的“胜过”沿用敏捷宣言的表达方式。它不意味着右项没有价值，而是在两者发生冲突时，左项应获得更高优先级。

在新的全书结构中，五条宣言不只指导一次 AI Coding 任务，也指导四维转型中的组织取舍：

宣言	四维转型中的典型冲突	优先保护什么
意图锚定	工具采用率、自动化率与真实业务结果冲突	可观察的价值意图与人类责任
多模态涌现协作	固定岗位边界与动态人机能力组合冲突	能力按意图组合，同时保留责任归属
算法规则与架构韧性	更多审批与更快、更稳定的机器控制冲突	可执行边界、恢复能力与必要的人类裁决
实时遥测与闭环对抗	漂亮汇报、个人排名与真实系统反馈冲突	可行动的系统事实与反指标
基于验证的系统工程	演示速度、生成数量与可信交付冲突	与风险相称的独立证据

五条宣言并不为所有冲突给出唯一答案。它们提供的是优先级：当短期产出与长期系统能力、机器自治与人类责任、局部速度与端到端价值发生冲突时，团队知道哪些东西不能为了看起来更快而被牺牲。

4.1 意图锚定，胜过详尽规约

为什么提出

传统需求管理常在两个极端之间摆动：一端只有一句模糊诉求，另一端试图在执行前穷尽所有细节。前者让 Agent 自由猜测，后者则容易产生大量很快过期的文档。

Agent 的危险不只是“听不懂”，还包括过度字面服从。它可能完美执行一句不完整指令，却偏离真正业务目标。

真正含义

意图锚定要求系统始终知道：

- 为什么做；
- 要改变什么业务结果；
- 什么不能被牺牲；
- 当前只做什么；
- 什么明确不做；
- 怎样判断完成。

意图不是一句愿景口号。它必须经过契约化，成为 Goal、Non-goals、Rules 和 AC。

不意味着什么

- 不意味着不写规格；
- 不意味着可以用模糊目标代替验收标准；
- 不意味着 IO 只说方向，所有细节都由 AI 决定；
- 不意味着签署后契约永远不能版本化。

意图决定方向，规格提供执行精度，两者不是敌人。

如何落地

1. 从意图图谱定位长期目标和依赖；
2. OA 对字面需求进行批判性校准；
3. 通过 Grill-Me 逐项确认关键决策；
4. 形成意图契约；
5. IO 审阅完整契约并显式签署；
6. 所有任务、测试和证据映射回契约。

费用报销案例

“超过 5000 元增加总监审批”不是完整意图。真正意图可能是降低高额差旅风险，同时不增加普通员工报销摩擦。这个意图会直接影响边界值、例外和成功指标。

如果只执行字面规则，系统可能把所有币种、所有报销类型和历史单据都纳入修改；经过意图锚定后，本次契约可以只处理境内人民币差旅，并明确不修改 5000 元及以下路径。

四维转型中的冲突

管理层可能把“AI 工具激活率达到 80%”“生成代码占比达到 50%”设为转型目标，平台团队据此推动所有岗位使用同一工具。指标很明确，却没有说明企业要改善哪条价值流、缩短什么周期、减少什么成本，也没有说明谁对业务结果负责。

意图锚定要求把工具采用降回手段：先签署要改善的业务结果和非目标，再决定人员任务怎样迁移、组织授权怎样调整、流程堵点怎样消除、工具能力怎样接入。采用率可以作为实施信号，但不能取代价值意图。

常见误区

- 把意图图谱做成巨大的知识库，却无法裁剪到任务；
- 把一句愿景当成契约；
- AC 只有“功能正常”“体验良好”；
- Grill-Me 完成后由 OA 自动代签；
- 实现失败后修改契约以获得通过。

价值观检查

如果 Agent 完全按字面完成需求，业务目标是否一定实现？如果答案是否定的，意图还没有锚定。

4.2 多模态涌现协作，胜过仅依赖跨职能人类团队

为什么提出

传统跨职能团队通过把产品、开发、测试和运维能力放进稳定团队，减少职能墙和交接。这一原则依然有效。但当 Agent 能够承担检索、分析、生成、测试和运行任务时，能力单元不再只由固定岗位提供。

文本、代码、图像、日志、指标和终端输出可以被不同 Agent 处理，多种候选方案也可以并行产生。这为协作带来新的组合空间。

真正含义

“涌现”不是让许多 Agent 自由聊天，而是让具备不同能力的执行单元，在共同意图、约束和证据标准下产生单一主体难以获得的结果。

有效涌现需要：

- 角色化能力；
- 独立或互补视角；
- 明确 Work Graph；
- 结构化 Handoff；
- 冲突和停止机制；
- 统一验证标准。

不意味着什么

- 不意味着人类团队不再重要；
- 不意味着 Agent 越多越好；
- 不意味着可以让多个 Agent 同时修改同一范围；
- 不意味着多数投票就是真相；
- 不意味着模型之间的互评可以替代真实测试。

如何落地

将一个复杂任务拆成真正独立的视角。例如：

规则分析 Agent：识别业务边界
测试 Agent：从 AC 生成反例
实现 Agent：完成最小修改
安全 Agent：检查数据和权限
证据 Agent：汇总真实结果

并行只发生在没有依赖和写冲突的节点。每个节点只获得最小必要上下文，输出必须带来源和状态。

费用报销案例

规则分析 Agent 发现“5000元是否包含边界”尚未明确；测试 Agent 生成 4999.99、5000.00、5000.01 三个反例；安全 Agent 确认测试不得访问生产报销数据。三种视角共同提高结果质量，但最终业务口径仍由 IO 决定。

四维转型中的冲突

动态 Agent 网络可以跨越开发、测试、安全和运维边界，但固定岗位仍承载长期专业能力、产品责任和组织承诺。如果组织只强调“能力随意组合”，就可能出现任务由多个 Agent 完成，却没有人对意图、例外和发布负责；如果只守住岗位边界，Agent 的跨专业并发能力又会被重新切回人工接力。

宣言优先支持按意图组合能力，但不支持责任漂移。人员维度明确不可转移的人类责任，组织维度建立责任节点，流程与工具负责让 Agent 能力按契约、依赖和证据协作。动态的是执行组合，稳定的是价值与风险责任。

常见误区

- 为展示多 Agent 而人为拆分简单任务；
- 多个 Agent 共享全部上下文，造成隐私和噪声扩散；
- 上游只输出“已完成”，下游无法验证；
- Agent 互相认同后，跳过确定性测试；
- 没有 Token、时间和重试预算。

价值观检查

新增一个 Agent 后，是否增加了独立能力或证据？如果只增加了对话轮次，它不是有效涌现。

4.3 算法规则与架构韧性，胜过仅依赖流程纪律与仪式

为什么提出

人类流程依赖记忆、培训和纪律。团队可以规定“必须先写测试”“不得提交密钥”“发布前必须有回退方案”，但在压力下，这些规则常被遗忘、误解或绕过。

Agent 不会因为读过制度就稳定遵守。规则必须进入它的上下文、权限和执行环境。

真正含义

算法规则是机器能够检查的约束，架构韧性是系统在失败时限制影响、降级、恢复和升级的能力。

它们包括：

- 约束矩阵（constraints.yaml）；
- 工具约束清单（Tools Manifest）；
- AI Coding Guide；
- 机械门禁；

- 超时、重试、熔断和补偿；
- 回滚安全；
- 例外有效期；
- HITL 触发条件。

不意味着什么

- 不意味着取消所有流程和会议；
- 不意味着所有规则都能自动化；
- 不意味着有门禁就不会发生事故；
- 不意味着可以用僵化规则替代价值判断；
- 不意味着通过检查就等于业务正确。

无法自动化的规则仍需明确责任人和人工证据。

如何落地

把“注意事项”改写为：

适用范围
检查方式
失败动作
证据位置
例外批准人
有效期
恢复方式

优先把少量高风险规则变成 MUST，而不是一次写出上百条低质量约束。

费用报销案例

“不要影响旧审批流程”应转化为 5000 元及以下路径的回归测试；“不要使用生产数据”应转化为数据环境约束；“上线前要小心”应转化为发布需人类批准和可执行回退。

四维转型中的冲突

面对 Agent 风险，一种常见做法是在原流程上增加 AI 审查表、治理委员会和更多人工签字。风险看似得到控制，结果却是每个机器动作重新等待人，组织既没有获得速度，也没有提升控制质量。

算法规则与架构韧性要求把重复、明确、可检查的控制交给工具运行，把会议和审批留给价值冲突、约束例外与不可逆风险。它不是取消流程纪律，而是让人员不再反复充当机械检查器，让组织决策集中在机器不应自行决定的地方。

常见误区

- 约束只有自然语言，没有检查命令；
- 所有规则都设为 MUST，导致团队只能绕过；
- 例外没有失效时间；
- 门禁只检查文件存在，不检查真实内容；
- 为追求自动化率，把不可逆操作也交给 Agent。

价值观检查

如果换一个从未参与项目的人或 Agent，这条关键规则还能被稳定执行吗？如果不能，它仍然只是隐性纪律。

4.4 实时遥测与闭环对抗，胜过仅依赖阶段性复盘

为什么提出

阶段性复盘能够帮助人类总结经验，但它通常发生在大量执行之后。Agent 可以在一次任务中完成多轮尝试，如果失败、重试、权限阻断和成本只在最后被汇总，团队会失去最有价值的过程信号。

真正含义

实时遥测关注任务正在怎样运行，闭环对抗关注不同验证机制怎样主动发现偏差。

需要观测：

- 目标是否命中；
- 首次是否成功；
- 哪些约束失败；
- 哪些失败被自动修复；
- 人类为何介入；
- 使用了多少时间和 Token；
- 哪些模式值得沉淀；
- 上线后价值是否出现。

不意味着什么

- 不意味着取消人类复盘；
- 不意味着采集越多指标越好；
- 不意味着用指标评价个人；
- 不意味着 LLM 生成的数字可以当遥测；
- 不意味着只关注 Token 成本。

遥测是为了改进系统，不是为了制造监控焦虑。

如何落地

每个契约 Verify 后采集单任务遥测，同时更新项目累计数据。测试数来自真实运行，Token 优先实测，无法实测时标记 estimated。团队定期观察趋势，而不是只看单次漂亮结果。

费用报销案例

如果 T-001 经历四次修复才通过，团队不应只记录“最终成功”。这可能说明边界规则不清、测试生成不足或审批模块技术债较高。下一轮应针对根因改进，而不是庆祝 Agent 终于完成。

四维转型中的冲突

当管理者用 Token 最少、自动化率最高、HITL 最低或生成代码最多给个人和团队排名时，度量会立即改变行为：必要的人类介入被隐藏，失败被拆到新任务，验证成本被转移给下游，团队开始优化仪表板而不是价值流。

实时遥测应评价人、Agent、上下文、流程和工具共同构成的生产系统，并用相互制衡的指标揭示成本转移。人员维度保护员工免受粗暴监控，组织维度根据趋势调整授权，流程与工具保证数据来自真实事件。遥测必须能够触发改进，也必须有反指标防止被单一目标劫持。

常见误区

- 手工填写 100% 通过率；
- 只展示成功任务；
- Bug 出现后另开新任务，保留原任务首次成功；
- 仪表板很多，但没有任何治理动作；
- 将人类介入一律视为失败。

价值观检查

指标变化后，团队知道应修改意图、约束、编排、验证还是模型吗？如果不知道，遥测只是装饰。

4.5 基于验证的系统工程，胜过随性提示与氛围编程

为什么提出

提示词能够改善模型输出，却无法单独建立可靠交付。模型可以生成看起来合理的代码、测试和解释，甚至可以自信地宣布任务完成。正式工程需要独立于生成者的判定机制。

真正含义

验证工程要求：

- AC 可执行；
- 测试先于实现；
- 真实运行结果被保留；
- 约束能够机械检查；
- 失败不能通过降低标准消失；
- Evidence Bundle 映射目标、约束和证据；
- 人类基于证据裁决高影响行动。

不意味着什么

- 不意味着探索阶段不能 Vibe Coding；
- 不意味着所有输出都必须形式化证明；
- 不意味着测试全绿即可无条件上线；
- 不意味着人工经验毫无价值；
- 不意味着为低风险任务引入重型治理。

验证强度应与风险、可逆性和影响范围匹配。

如何落地

从最小闭环开始：

意图契约
→ 约束矩阵
→ TDD Red
→ 最小实现
→ TDD Green
→ 全量验证
→ Evidence Bundle
→ 人类裁决
→ Telemetry

费用报销案例

AI 可以在几秒内写出 `amount >= 5000`，也可以为它生成一个恰好通过的测试。验证工程要求从契约边界出发，独立检查 5000.00 和 5000.01，并保留先红后绿的证据。只有这样，团队才能发现实现是否真正符合“严格大于”的业务规则。

四维转型中的冲突

企业突围项目可以在一天内完成真实增量，这证明机器速度能够转化为工程结果；但如果为了展示速度而让生成者自己定义标准、自己验证、自己宣布完成，演示越成功，组织越容易高估可复制能力。反过来，把所有任务都要求为重型审计，也会让验证工程成为新的流程负担。

宣言要求验证强度与风险相称：低风险探索保持轻量和可丢弃，正式交付建立契约、约束和独立证据，高风险任务增加职责隔离与有权者裁决。工具应自动完成大部分机械验证，人把时间放在证据矛盾、剩余风险和价值判断上。可信与速度不是二选一，验证系统本身也必须被设计成高吞吐能力。

常见误区

- 让生成代码的同一提示同时生成所有测试，并假设两者独立；
- 测试只覆盖正常路径；
- 用 shell grep 代替运行时行为验证；
- 把模型自评当成最终证据；
- Evidence Bundle 在发布后补写。

价值观检查

如果删除模型对自己工作的解释，只保留工件和运行结果，第三方还能判断是否完成吗？

4.6 五条宣言如何共同工作

五条宣言不是五个平行口号，而是一条递进链：

意图锚定
确保方向正确
↓
涌现协作
获得足够执行能力
↓
算法规则与韧性
限制执行边界
↓
实时遥测与闭环
持续发现偏差

缺少任何一条都会失衡：只有意图没有验证，目标仍可能被错误实现；只有验证没有意图，系统可能高质量地完成错误任务；只有多 Agent 没有约束，协作会放大风险；只有门禁没有遥测，组织无法判断治理是否改善。

从四维观察，这条链同样不能被拆开：意图为人员和组织提供共同方向，涌现协作重组能力网络，算法规则和韧性进入流程与工具，实时遥测把运行事实反馈给组织，验证工程则让人能够基于证据承担最终裁决。宣言不是操作系统的五个模块，而是贯穿四维设计的五条优先级规则。

4.7 组织级价值观冲突练习

假设公司要求一个季度内“AI 生成代码占比达到 50%，所有研发团队接入统一 Agent 平台”，并承诺研发效能提升 30%。不要先争论目标激进还是保守，选择一条真实价值流，用五条宣言依次重写这项要求：

1. **意图锚定**：30% 具体改善什么业务结果、端到端周期或总成本？哪些质量与风险不能恶化？
2. **涌现协作**：哪些任务交给 Agent，哪些判断留给人，如何组合能力而不稀释责任？
3. **规则与韧性**：哪些边界可以机械执行，哪些例外和不可逆动作必须保留人类裁决？
4. **遥测与闭环**：怎样证明提效没有转移为返工、审核、风险和员工监控？
5. **验证工程**：首个真实切片需要什么证据，才能决定扩大、调整或停止？

练习的结果不应是另一份原则声明，而应是一份可签署的转型假设：业务结果、四维改变、护栏、证据和裁决条件同时成立。价值观的作用，就是在压力和冲突中提供一致的取舍顺序。

4.8 本章小结

三大理论解释四维重构与 3-4-3 为什么这样设计，五条宣言说明发生冲突时应优先保护什么。理论不是为了制造技术优越感，价值观也不是墙上的标语。它们最终都要进入人员责任、组织授权、流程取舍、工具规则、工件、门禁、证据和日常决策。

本章最小实践：用五条宣言评审一项正在推进的 AI 转型要求，每条找出一个人员、组织、流程或工具冲突。

完成证据：一份包含业务结果、四维改变、护栏、证据和裁决条件的转型假设。

延伸实践：推荐学习《Agentic Agile 免费认知课：AI 转型为什么不是工具升级》，先建立 Agentic Agile 的价值观与判断框架，再进入具体方法。

第二部分 | Agentic Agile：人机协同研发操作系统架构

第一部分回答为什么要重构，并建立四维模型、理论基石和价值取舍。第二部分开始回答“换成什么”：人员、组织、流程与工具怎样通过一套共同架构连接起来，而不是各自推进 AI 项目。

本部分先给出人机协同研发操作系统的总装图，说明四维转型、3-4-3、SCOPE-V、Harness、Evidence 与 Telemetry 分别解决什么问题；随后展开三个超级角色、四大动态工件与三大自治机制。下一部分再把这些结构放入真实任务、流水线、既有系统和多主体协作中运行。

这里需要始终守住一个边界：Agentic Agile 不是软件平台，也不是替代敏捷、DevOps、产品管理或企业治理的万能模型。它提供的是 Agent 进入研发执行系统后，价值、责任、协作、控制、证据和反馈怎样共同工作的运行架构。

第五章 | 系统总览：四维转型、3-4-3、SCOPE-V 与遥测如何组成整体

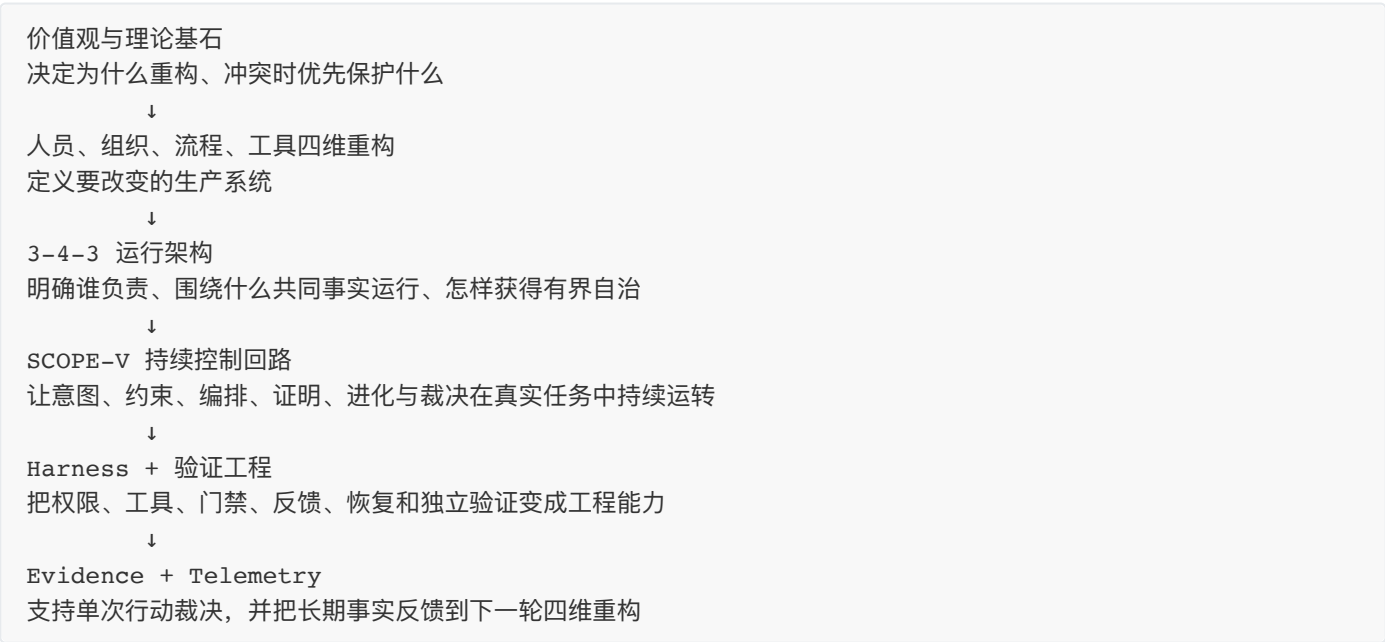
前几章已经提出多个概念：四维重构、3-4-3、SCOPE-V、Harness、验证工程、Evidence Bundle 和 Telemetry。如果不区分抽象层级，读者很容易把它们理解成多套并列框架，或者按名称拼成一条越来越长的流程。

典型误解包括：把四维当成四个转型项目，把 3-4-3 当成新的岗位与文档制度，把 SCOPE-V 当成六步瀑布，把 Harness 当成某个 Agent 平台，把 Evidence Bundle 当成发布报告，把 Dashboard 当成个人绩效系统。

这些理解都抓住了某个局部，却失去了整体。本章不展开各模块的实现细节，只回答一个问题：它们在**同一套人机协同研发操作系统**中分别处于什么位置，怎样共同把 AI 生产力转化为可信、可持续的组织提效？

5.1 总装图：从价值意图到组织进化

这套操作系统可以理解为六个彼此连接的层级：



这不是从上到下执行一次就结束的流程。业务结果和运行事实会通过 Evidence 与 Telemetry 回到组织：调整角色能力、授权范围、流程节点、工具配置、约束、知识和投资决策。操作系统因此形成闭环，而不是一套只在项目启动时使用的设计文档。

5.2 各模型回答不同问题

层级或机制	核心问题	主要产出	不应被理解为
四维重构	人机生产系统的什么必须改变？	人员责任、组织授权、价值流与工程能力的目标状态	四个独立转型项目
3-4-3	谁负责，系统围绕什么运行，自治怎样受控？	三个超级角色、四大动态工件、三大自治机制	新岗位编制或文档清单
SCOPE-V	一次真实任务怎样持续校准、执行、纠偏和裁决？	六个持续控制面及其状态转换	六步瀑布流程
Harness	规则和控制怎样进入 Agent 的实际运行环境？	上下文、工具权限、门禁、执行、恢复与观测能力	某个模型或单一供应商平台
验证工程	候选结果怎样变成可以信任的完成事实？	与风险相称的独立验证和证明链	测试数量竞赛或末端审批
Evidence Bundle	这一次任务是否有足够事实支持下一步行动？	AC、约束、测试、风险、差异、恢复与裁决证据	发布后补写的总结报告
Telemetry	这套人机系统长期是否更准确、自主、经济并持续学习？	可比较的价值、能力、效率与进化趋势	AI 使用量或个人绩效排名

这些模型可以裁剪，但不能相互冒充。SCOPE-V 不能替代责任设计，Dashboard 不能替代单次证据，工具平台不能替代业务意图，角色签署也不能替代真实运行验证。

5.3 组织级分形扩展：从任务运行到战略意图

三层分形不是企业汇报层级，而是同一套控制关系在不同范围内的递归：

层级	核心问题	主要产出	向下/向上关系
战略意图层	企业要让 AI 放大什么价值，拒绝什么风险？	价值主题、投资假设、风险红线、测量契约	约束和投资边界向下继承，组织结果向上反馈
价值网络层	多产品、多团队、多平台如何围绕价值流协同？	责任单元、Work Graph、协议、能力编排、证据聚合	继承组织边界，聚合任务证据，不替任务签署
任务运行层	每次 Agent 执行如何被授权、验证、恢复和裁决？	Intent Contract、Constraint Matrix、SCOPE-V、Evidence Bundle	在局部边界内执行，向价值网络提供可复查事实

扩展的四条规则是：

- 约束向下继承：**战略与组织规则只有转化为权限、预算、数据边界、测试门禁和升级条件，才算进入运行时。
- 证据向上聚合：**任务 Evidence 可以支持价值流判断，但必须保留来源、口径、时间窗和缺口，不能把一次 PASS 外推为组织成功。
- 能力按责任域组合：**先拆小责任单元，再为不可消除的依赖建立协议、DAG 和证据聚合，不能用中央协调掩盖架构复杂度。
- 规模化必须有净收益：**并发 Agent 带来的执行收益，必须大于上下文准备、协调、验证、重试和人工监督成

本。

因此，分形不是复制三套相同模板，也不是把 OA 变成中央审批人。它要求三个层级共享意图、约束、证据、裁决和反馈的语义，但允许字段、粒度、责任人和运行节奏不同。

本节最小实践：把一个报销任务分别放入任务运行层、价值网络层和战略意图层，写出每层已证明、未证明以及下一层的扩展条件。

5.4 3-4-3：相对稳定的运行架构

3-4-3 把人机协作中最容易混乱的内容压缩成三个问题。

第一个“3”回答谁负责。 IO 对价值意图和风险取舍负责，OA 对控制系统与编排负责，AS 对受约束执行和候选结果负责。它们是责任，不是固定职位；同一个人可以在不同任务中承担不同角色，但同一项关键责任不能无人承担。

中间的“4”回答围绕什么共同事实运行。 意图图谱连接长期方向，意图契约定义当前任务，约束矩阵建立可执行边界，Evidence Bundle 汇聚完成事实。四者把隐性认知转化为人和机器都能读取、执行与复查的组织共同语言。

最后一个“3”回答自治怎样形成。 意图注入让探索收敛为可签署任务，高频对抗与自我修正让 Agent 根据反馈纠偏，HITL 与证据验收把机器不应决定的事项交还有权的人。

3-4-3 被称为运行架构，是因为它定义相对稳定的责任、共同事实和自治逻辑；它本身并不规定每个任务必须按什么时间顺序执行。真正驱动一次任务持续运转的是 SCOPE-V。

5.5 SCOPE-V：把架构变成持续控制回路

SCOPE-V 不是在 3-4-3 之外再增加六个阶段，而是让角色、工件和自治机制进入真实执行：

控制面	控制的问题	与 3-4-3 的连接
Specify	当前意图是否足够明确、可判定并已签署？	IO、OA、意图图谱与意图契约
Constrain	哪些边界不可突破，例外和恢复怎样处理？	OA、约束矩阵、工具权限与 HITL
Orchestrate	哪些能力以什么依赖、权限和停止条件协作？	OA、AS、Work Graph 与 Handoff
Prove	什么证据能够证明实现满足意图和约束？	AS、验收标准、测试与机械门
Evolve	失败怎样诊断、最小修复并回到原检查？	高频对抗、自我修正、知识与记忆晋升
Verify	现有证据是否足以支持发布、补证、返工或停止？	Evidence Bundle、IO/OA 责任与独立裁决

六个控制面不是严格单向。Specify、Constrain 和 Orchestrate 可能因新事实重新打开，Prove 与 Evolve 会形成高速纠偏内环，Verify 发现证据矛盾时也可以要求补证或退回。把它画成一条直线只是为了阅读方便，真实运行是一组带状态、门禁和回流路径的闭环。

5.6 三种时间尺度：机器快内环、任务闭环、组织慢外环

操作系统需要同时容纳机器速度与人类责任，因此至少存在三种时间尺度：

机器快内环：Prove ⇌ Evolve
秒、分钟或小时级执行、测试、诊断、最小修复和重跑

任务控制闭环：Specify → Constrain → Orchestrate → Prove ⇌ Evolve → Verify
围绕一份已签署契约完成授权、执行、证据聚合和责任裁决

组织慢外环：Verify → Telemetry → 人员/组织/流程/工具调整
跨任务观察价值、自治、效率和学习，决定扩大、收缩、改进或停止

三个尺度不能相互替代。机器快内环中不应重新决定业务目标，任务 Verify 不能凭一次成功宣称组织转型完成，组织慢外环也不能用平均趋势掩盖某个高风险任务证据不足。

人类不需要介入机器内环的每一步，但必须出现在正确的责任节点：签署意图和不可牺牲边界，处理目标与证据冲突，批准权限扩大和不可逆动作，接受剩余风险，并根据长期事实调整授权。

5.7 Evidence 与 Telemetry：一次裁决和长期进化

Evidence Bundle 与 Telemetry 都以事实为基础，但服务于不同决策。

维度	Evidence Bundle	Telemetry
核心问题	这一次交付是否足以支持某项行动？	这套人机系统长期是否真正提效并变得更健康？
观察粒度	单契约、单任务、单次发布	多任务、项目、价值流或可比组织单元
时间范围	当前证据窗口	带分母、样本量和风险分层的时间序列
典型内容	AC、约束、测试、差异、风险、恢复、签署与裁决	目标准确率、首次通过、自愈、HITL、上下文、Token、执行效率与知识进化
主要动作	发布、灰度、补证、返工、缩小范围或停止	扩大自治、收紧权限、修改流程、补工程基础或停止试点
主要误用	由 Agent 自己写结论，缺少原始证据	用调用量和生成量排名个人，忽略成本转移与反指标

Evidence 是 Telemetry 的重要事实来源，但不是全部来源；生产结果、用户采用、故障、成本和组织等待也需要进入长期观察。反过来，Dashboard 上的平均成功率再高，也不能替某个任务补齐缺失的约束证据或发布签署。

5.8 四维重构怎样落到 3-4-3 与工程系统

四维与 3-4-3 不是一一对应关系。三个角色不只属于人员维度，四大工件也不只属于工具维度。每个运行要素都会同时改变多维关系：

四维	3-4-3 的主要承载	SCOPE-V / Harness 的主要承载	要证明的改变
人员	IO、OA、AS 的责任分配与制衡	HITL、签署、异常升级和独立裁决	人从重复执行转向价值、边界、系统设计与异常判断
组织	意图图谱、责任节点、共享约束与 Evidence 接口	Work Graph、跨模块协议、授权和证据聚合	决策权与能力能够围绕意图组合，责任不漂移
流程	三大自治机制与工件生命周期	SCOPE-V、机械门、回流、停止与恢复	等待和交接减少，机器内环与人类外环各自高效运行
工具	工件结构、约束协议和证据格式	Context、Orchestration、Harness、Evidence、Telemetry、Knowledge & Memory	不同模型和 AI 宿主能够在共同协议下执行、验证和交接

这张映射表也说明为什么不能由平台团队独自完成 Agentic Agile。平台可以提供 Context、Harness、Evidence 和 Telemetry 能力，却不能代替业务负责人定义价值，不能自行改变部门授权，也不能决定哪些风险由谁承担。

5.9 治理、验证与 Dashboard 在系统中的位置

治理是控制机制，不是转型目的。 它把人类责任、组织授权、流程边界和工具权限连接起来，使 Agent 能够在明确范围内提速，而不是靠层层审批获得表面安全。

验证工程是可信执行内核，不是全书终点。 它把候选结果转化为可复查的事实，防止生成者自我声明完成；但它不能独自决定业务是否值得做、组织怎样授权或人员怎样成长。

Dashboard 是价值与效能管理的决策入口，不是管理体系本身。 它帮助管理者判断提效是否真实、自治是否健康、成本是否转移、系统是否学习；原始证据、责任结构和改进行动仍然必须存在于看板之外。

三者共同服务于一个结果：让 AI 带来的速度能够穿过组织系统，形成可持续的业务价值。任何一项一旦成为自我目的，都会产生反效果：治理变成官僚，验证变成测试堆积，Dashboard 变成数字表演。

5.10 一个报销规则如何穿过整套操作系统

继续使用“超过 5000 元增加总监审批”的案例，可以看到各层并不是概念堆叠：

- 价值与四维：**目标是降低高额报销风险，同时不增加普通报销摩擦；业务、财务、合规和技术责任被明确，审批流程和工程工具只改本轮必要部分。
- 3-4-3：**IO 签署金额、币种、范围和成功标准，OA 设计上下文、约束、验证和异常路由，AS 执行规则分析、测试与最小实现；契约、约束和 Evidence 形成共同事实。
- SCOPE-V：**需求先澄清“超过”是否包含 5000，生产数据和权限被限制，任务按依赖编排，边界测试先红后绿，失败进入最小修复，最终由证据支持发布或补证。
- Harness 与验证：**工具只能访问允许的仓库与测试环境，机械门检查范围、测试和约束，真实运行结果进入 Evidence Bundle，发布和回退仍由有权的人控制。
- Telemetry：**上线后观察普通报销周期、高额风险命中、首次通过、返工、HITL、Token 和人工验证成本，判断这次改变是否真的提效，以及是否值得扩展到外币和其他报销类型。

如果缺少任何一层，系统都会留下断点：没有意图会高效实现错误目标，没有责任会让例外无人裁决，没有运行约束会让风险以机器速度扩散，没有证据会让“全绿”无法信任，没有遥测则无法知道一次成功能否成为组织能力。

5.11 最小可运行操作系统

企业不需要先建成完整平台才能开始，但第一次真实闭环至少要有：

- 一个必须改善的业务结果和一条目标价值流；
- 一位能够签署意图与承担结果责任的人；
- 明确的执行范围、非目标、验收标准、权限和停止条件；
- 一个能在约束内执行、测试、失败并留下记录的 Agent 路径；
- 与风险相称的独立验证、Evidence Bundle 和发布裁决；
- 少量具有决策价值的遥测，以及扩大、调整或停止条件。

“最小”不等于字段越少越好，而是删除对本轮裁决没有价值的负担，同时保留责任、边界、验证、恢复和反馈。低风险任务可以使用轻量契约与自动验证，高风险任务则必须增加职责隔离、行业控制和独立证据。

最小系统的验收标准也不是“3-4-3 文件都存在”，而是一个真实任务能够从意图进入执行，错误会被发现和阻断，结果能被有权者裁决，运行事实会进入下一轮改进。

5.12 六个架构反模式

1. **模型堆叠**：把四维、3-4-3、SCOPE-V 和 Dashboard 画在同一层，读者不知道哪个负责决策、哪个负责执行。
2. **文档治理**：契约、约束和 Evidence 都存在，但不进入权限、门禁、测试和发布路径。
3. **平台先行**：先建设统一 Agent 平台，再寻找价值流和业务责任人，最终只有使用量没有业务结果。
4. **零 HITL 崇拜**：把减少人类介入当成目标，导致必要的价值判断、风险接受和异常升级被隐藏。
5. **单次成功外推**：一次演示或一个全绿任务就宣布组织转型完成，没有样本窗口、基线和长期遥测。
6. **供应商绑定**：把组织契约、证据和知识绑定某个模型或宿主，工具变化时整套工作方式无法迁移。

识别这些反模式的共同方法是追问：当前设计改善了哪个业务结果，改变了哪一维，由谁负责，通过什么运行机制生效，用什么证据证明？无法回答时，架构很可能只是概念或工具的集合。

5.13 本章小结：从模型集合到运行系统

Agentic Agile 的各个组成部分不在同一层级。四维定义转型对象，3-4-3 建立责任、共同事实和自治架构，SCOPE-V 驱动任务控制回路，Harness 与验证工程提供可信执行能力，Evidence 支持单次裁决，Telemetry 推动跨任务学习与组织进化。

它们共同形成三种时间尺度：机器在快内环中执行与纠偏，任务在完整控制回路中被签署和裁决，组织在慢外环中根据事实调整人员、组织、流程和工具。治理、验证与 Dashboard 都服务于把 AI 速度转化为业务价值，而不是成为新的管理目的。

本章最小实践：选择一个真实 AI 研发任务，分别写出它的四维改变、3-4-3 责任与工件、SCOPE-V 控制路径、Harness 能力、Evidence 裁决和 Telemetry 反馈。

完成证据：一张没有模型混层、能够从业务意图走到组织反馈的系统总装图。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，亲手完成从意图、约束到证据裁决的第一条任务闭环。

第六章 | 三个超级角色：从岗位分工到人机责任网络

四维重构首先改变人员与组织中的一个基本问题：工作不再只能分配给岗位，也可以分配给 Agent；但价值判断、组织承诺和风险责任仍然必须由人承担。旧岗位名称无法直接说明一次人机任务中谁定义目标、谁设计控制、谁执行、谁有权裁决。

IO、OA、AS 因此不是三个新岗位，而是一组最小责任内核。它们把价值、治理和执行分开，使组织可以动态组合人员与 Agent 能力，同时保持关键责任可指认、可制衡、可追溯。

6.1 为什么不是按传统岗位映射

Agentic Agile 的三个角色描述的是责任，而不是固定职位。一个人在不同任务中可以承担不同责任，但同一项关键决策必须明确由谁负责。

IO：决定什么值得做，并对价值和风险取舍负责

OA：把意图变成可治理的执行系统

AS：在授权范围内产生并验证候选结果

三个角色不是为了制造新的组织层级，而是避免价值判断、治理设计和执行行为混在一起。

传统岗位可以承担这些责任，但不存在固定的一一映射：

现有岗位或专业	可能承担的责任	不能由岗位名称自动推定的事项
业务负责人、产品负责人	通常承担 IO，也可提供领域约束	是否真正拥有价值取舍、风险接受和发布签署权
架构师、技术负责人	通常承担 OA，也可能对技术意图承担 IO	是否具备业务目标最终解释权，是否独立于实现结果
工程经理、项目经理、敏捷教练	可以承担 OA 的协调与系统运营责任	是否具备 Context、约束、Harness、验证和恢复设计能力
开发、测试、安全、运维专家	可以设计专业 Agent、承担 OA 子责任或参与 AS	是否有权改变业务目标、批准例外或签署发布
研发效能与平台团队	提供 Harness、Evidence、Telemetry 和共享治理资产	不能因为拥有平台就成为所有任务的 IO 或中央审批者

低风险、小范围任务中，一个人可以同时承担 IO 与 OA，但两项责任仍需分别表达；高影响任务则应根据行业要求、利益冲突和证据独立性进行职责分离。灵活分配角色，不等于把责任混在一起。

6.2 IO：意图主理人

IO 拥有目标的最终解释权和关键取舍权。它可能由产品负责人、业务负责人、技术负责人或其他对结果负责的人承担。

IO 的核心职责包括：

- 明确业务目标和成功标准；
- 选择当前最有价值的范围；
- 确认明确的非目标；
- 决定风险偏好和不可牺牲底线；
- 处理相互冲突的业务规则；
- 审阅完整意图契约并显式签署；
- 基于 Evidence Bundle 作出上线、灰度、返工、缩小范围或停止决策。

IO 不需要亲自编写所有技术细节，但不能把价值判断交给模型。

成熟 IO 不只是“把需求说清楚”，还要能够用业务结果说明为什么值得做，用 Non-goals 主动控制机会成本，用证据与反指标防止局部优化，并在目标、成本、风险和时间冲突时作出可追溯取舍。IO 的价值不由提出多少需求衡量，而由意图是否准确、责任是否明确、最终结果是否值得交付衡量。

6.3 OA：编排架构师

OA 是治理系统的设计者和运行维护者。OA 的价值不在于写出更花哨的 Prompt，而在于让 Agent 获得正确上下文、工具、约束、反馈和停止条件。

OA 的核心职责包括：

- 对模糊或可疑需求进行批判性校准；
- 通过 Grill-Me 引导 IO 逐项作出关键决策；
- 起草意图契约，但不得代替 IO 签署；
- 维护约束矩阵和 AI Coding Guide；
- 设计 Work Graph、权限和工具白名单；
- 选择验证策略并维护机械门禁；
- 识别需要 HITL 的异常；
- 聚合 Evidence Bundle 和遥测；
- 把新教训反馈到意图图谱和治理资产。

OA 可以由架构师、研发效能负责人、工程经理或具备治理能力的高级工程师承担。

OA 不是换了名称的项目经理，也不是所有任务都要经过的“AI 审批员”。项目协调、会议组织和状态跟踪可以是 OA 工作的一部分，但核心责任是设计一套能够自行运行、在边界处停止并产生可信证据的控制系统。成熟 OA 应持续减少重复人工协调，把一次裁决沉淀为契约字段、约束、路由、验证器、恢复策略或共享 Skill。

OA 也不能为了让系统看起来完整而过度治理。对于低风险、可丢弃的探索任务，简单范围、沙箱和预算限制可能已经足够；对于资金、隐私、生产写入等高影响任务，OA 则必须增加职责隔离、独立验证和恢复证明。治理重量由风险与可证明性决定，不由 OA 的流程偏好决定。

6.4 AS：自治蜂群

AS 是一个或多个执行 Agent 的集合。它们可以承担代码实现、测试、安全扫描、数据处理、文档更新、部署准备等任务。

AS 的“自治”始终是有界自治：

- 只接收当前任务所需的上下文；
- 只能使用 Tools Manifest 中允许的工具；
- 只能修改授权范围；
- 必须遵守 MUST 约束；
- 失败修复不得改变已签署目标；
- 超出预算、权限或风险边界时必须停止；
- 自己宣布完成不构成完成证据。

模型输出具有概率性。确定性来自它外部的契约、约束、工具权限、测试、门禁和证据，而不是来自对模型的想象。

“自治蜂群”也不等于一群被拟人化的“数字员工”。AS 可以是一个 Agent、多个专业 Agent、确定性工具与模型的组合，也可以在不同任务中更换模型和宿主。组织真正依赖的应是稳定的输入、权限、协议、完成判据和证据接口，

而不是某个 Agent 名称或人格设定。

AS 可以提出方案、发现矛盾和请求扩权，但不能自行批准自己的请求。它的成熟度不以自主运行时间衡量，而以能否稳定遵守最小权限、保留失败事实、生成可验证 Handoff、在边界处停止并接受外部裁决衡量。

6.5 分开执行、判断与责任

“这项任务交给 人 还是 AI”过于粗糙。同一项工作可以由 Agent 执行、由 人 判断，并由另一位具备授权资格的人承担最终责任。分工时至少要拆开三件事：谁执行，谁判断结果是否成立，谁对后果和组织承诺负责。

工作性质	Agent 可以做什么	人必须做什么	责任设计
高频、可表达、可验证、可恢复	检索、生成、测试、重复修复、证据收集	设定目标、边界与抽查策略	可预授权 AS 执行，OA 对控制有效性负责
复杂但可提供候选	比较方案、识别风险、生成反例、辅助诊断	处理语义冲突、选择方案、接受剩余不确定性	AS 提供候选与来源，IO/OA 在各自权限内判断
高影响或不可逆	分析影响、准备脚本、模拟和预演	批准或执行生产写入、资金操作、外部发布和高风险例外	有权的人签署，职责隔离与恢复证据按风险增加
当前禁止自治	识别缺失信息并停止	完成法律、伦理、资质或组织规定不可委托的判断	工具层阻断，不允许通过提示词临时绕过

判断标准不是“让人介入多少”，而是每类工作是否交给最适合的执行主体，判断是否发生在正确位置，最终责任是否真实存在。减少无价值的人工接力是提效，删除必要的人类责任不是提效。

6.6 显式签署为什么重要

Grill-Me 的逐项确认只是决策形成过程，不等于签署。签署意味着 IO 已经看到完整契约，并愿意对目标、非目标、规则和验收标准承担责任。

因此：

- OA 可以起草契约；
- AS 可以建议 AC；
- 系统可以检测缺失字段；
- 只有 IO 可以完成签署；
- 签署前不得进入业务编码。

这条规则不是形式主义。它防止“大家似乎都同意”被误当成真实授权。

6.7 三个角色之间的制衡

三个角色的价值不仅在于分工，还在于形成制衡。

IO 可能过度追求业务速度，OA 需要指出需求中的歧义、权限扩张和不可验证目标；OA 可能为了治理完整而引入过多流程，IO 需要判断治理成本是否与业务风险匹配；AS 可能提出实现便利的替代方案，但不能自行改变业务目标；Evidence Bundle 可能显示技术上可行，却仍需要 IO 判断商业风险是否值得承担。

IO 不能绕过约束直接命令 AS
OA 不能代替 IO 决定价值和签署契约
AS 不能修改标准证明自己正确
任何角色都不能用口头判断替代 MUST 门禁

这种制衡使“人类在环”从随机介入变成清晰责任设计。

6.8 授权矩阵

决策	IO	OA	AS
定义目标与非目标	决定并签署	挑战、起草	提供可行性信息
制定约束	批准业务底线和例外	设计、维护	执行并报告失败
选择执行路径	关注重大取舍	编排	在授权范围内选择
修改业务规则	决定并重新签署	识别影响	不得自行修改
自动修复实现	知情或按风险授权	设定边界	在边界内执行
扩大工具或数据权限	批准高风险事项	审查并配置	不得自行扩大
发布生产	决定或指定批准人	提供证据建议	准备发布，不自行批准
更新组织知识	确认价值教训	审核并回写	提供候选事实和证据

授权矩阵应根据任务风险调整，但价值判断和不可逆操作不能因为追求自治率而被悄悄下放。

6.9 最小人机责任单元

三个角色进入组织后，不应变成新的纵向部门。更可行的结构是围绕一份真实意图形成最小人机责任单元：一组承担 IO、OA 责任的人类节点，加上一组可以按任务组合和替换的 AS 能力。



这张图表示责任与能力关系，不是汇报关系。IO 不直接逐步指挥每个 Agent，OA 也不成为所有专业判断的唯一中心。领域专家可以把知识转成上下文、约束、测试和验证器，稳定团队继续承载长期产品责任，平台团队提供共享能力；责任单元只组合完成当前意图所需的部分。

责任单元的规模应由任务复杂度和风险决定：

- 低风险单仓任务可以由一人承担 IO/OA，并调用一个通用 Agent；
- 跨模块交付可以设置全局 OA 与模块 OA，保持单一 IO 或明确的意图层级；
- 高风险任务应分离价值签署、控制设计、执行与独立验证，不能由同一主体自定义标准并批准自己；
- 无论规模大小，异常升级只能进入预先指认的人类责任节点，不能在群聊中等待“谁有空谁决定”。

人机责任网络的组织价值，是让执行能力可以动态扩展，而价值与风险责任不会随 Agent 数量增加而稀释。

6.10 一个任务中的角色协作

以费用报销 T-001 为例：

1. IO 提出高额报销增加审批的业务意图；
2. OA 发现“超过”“金额口径”“报销类型”存在歧义；
3. Grill-Me 帮助 IO 逐项确认；
4. OA 起草契约，IO 审阅后签署；
5. OA 从图谱和约束中裁剪上下文，生成 Work Graph；
6. AS 分别完成测试、影响分析和最小实现；
7. 约束失败在授权内自动修复，业务冲突升级给 IO；
8. OA 聚合 Evidence Bundle；
9. IO 根据证据批准限定范围发布；
10. OA 采集遥测并回写图谱。

整个过程中，人并没有逐行指挥 Agent，Agent 也没有替人承担业务责任。

6.11 角色反模式

IO 只提需求，不承担签署

结果是所有人都可以说“我以为不是这个意思”。解决办法是让签署成为独立、可审计动作。

OA 退化为 Prompt 工程师或流程协调者

OA 花大量时间润色提示词、组织会议和追踪状态，却没有建立契约、约束、工具权限和证据标准。提示词与协调活动可以改进局部执行，但不能替代可运行的治理系统。

AS 被描述为确定性机器

模型输出仍然具有概率性。AS 的可靠性来自外部 Harness，而不是角色名称。

人类只在出事故后介入

HITL 既包括异常升级，也包括事前价值签署和不可逆操作批准。介入点需要预先设计。

用自治率评价角色成功

如果减少 HITL 是因为 Agent 绕过了应有审批，自治率越高反而越危险。应同时观察目标、约束、质量和责任。

6.12 角色能力如何成长

IO、OA、AS 的成熟不取决于职级或工具熟练度，而取决于能否在真实任务中稳定履行责任并产生可复查结果。

角色	起步能力	成熟能力	能力证据
IO	说明要做什么，确认基本 AC	定义业务结果与 Non-goals，识别激励冲突，权衡风险并根据证据裁决	意图准确率、返工原因、业务结果、例外与发布签署质量
OA	编写任务说明，配置 Agent 和检查清单	设计 Context、Work Graph、约束、Harness、验证、恢复与遥测，并持续降低无效协调	门禁有效性、异常路由、验证成本、自愈与恢复、治理资产复用
AS	在明确任务中生成候选结果	遵守最小权限，生成可验证 Handoff，保留失败证据，在边界处停止并跨工具稳定执行	目标与约束通过、首次成功、自愈质量、证据完整度、越界与停止记录

传统专业经验不会失效，而会改变承载方式。测试人员对可判定性和独立证据的敏感度，架构师对边界和韧性的判断，产品人员对价值与非目标的取舍，安全人员对威胁与权限的理解，都可以被转化为更高杠杆的契约、约束、验证器、知识和 Skill。

这些能力不会在一天内由新岗位替代旧岗位。更现实的路径是在一个真实任务中明确责任、观察缺口，再通过连续任务积累可复用资产。角色评估也不能只看 AI 生成量、Token、自治率或 HITL 数量，否则会奖励隐藏失败和责任逃逸。

6.13 本章小结

三个超级角色不是新岗位，而是人机协作的最小责任内核。IO 保护价值与责任，OA 设计可运行的控制系统，AS 在授权范围内提供可替换、可验证的执行能力。三者把执行权与最终责任分开，又通过签署、授权、证据和制衡重新连接。

人类没有退出系统，而是从重复执行转向意图、边界、系统设计、异常和裁决；Agent 获得更大执行空间，但不能自批权限、自改标准或自证完成。最小人机责任单元允许能力动态组合，同时确保责任不会在多 Agent 网络中漂移。

本章最小实践：选择一个真实任务，分别指定 IO、OA 与 AS，拆开执行、判断和最终责任，并画出异常升级路径。

完成证据：一张人机责任网络图、一份授权矩阵和三项不可转移的人类责任。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，在一次真实任务中区分执行、编排和最终责任。

第七章 | 四大动态工件：让组织共同语言可以被机器执行

人机协作真正稀缺的不是文档，而是人和 Agent 对目标、授权、边界与完成事实拥有同一套可执行理解。四大动态工件承担的正是这种“组织共同语言”：它既要让不同岗位能够讨论和签署，也要让不同模型、工具与运行环境能够读取、检查和交接。

7.0 三层架构中的工件分辨率

四大工件并不只服务于单个任务，也不意味着每个组织层级都要复制一套同样的文件。它们在三层架构中拥有不同的分辨率：战略意图层维护方向、价值网络层维护能力与责任关系，任务运行层冻结一次可执行授权。

层级	主要承载	变化方式
战略意图层	组织级 Intent Graph、价值假设与长期红线	由战略变化、经营结果和重大风险触发
价值网络层	价值流/产品域 Intent Graph、跨团队 Constraint Matrix 与协议	由依赖、交接、资源和组织责任变化触发
任务运行层	Intent Contract、任务级 Constraint Matrix、Evidence Bundle	由一次授权、执行事件、验证和裁决触发

上层约束向下继承，下层证据向上聚合；下层不得放宽上层红线，但可以为具体任务增加更严格的限制。任务 Evidence 只能证明该任务，不能自动证明价值流或组织成功；只有经过相同口径的聚合和人工解释，才可以进入更高层的遥测与转型决策。这样既避免把治理工件复制成文档森林，也避免用一个“任务完成”冒充“组织转型完成”。

7.1 四大动态工件的闭环关系

在传统软件研发中，需求通常沿着“战略—规划—需求—任务—代码”逐层分解。产品路线图、PRD、Product Backlog、用户故事和验收标准分别承担不同层次的信息表达。这套方式并没有因为 AI 出现就失效，但它有一个默认前提：人类团队会通过会议、追问、经验和组织关系，主动补齐文档之间的空白。

AI Coding 改变了这个前提。Agent 不会天然知道某条需求与公司北极星的关系，不会因为“大家都知道”就理解隐含规则，也不会自动区分会议中的讨论意见和最终决定。更重要的是，它可以在模糊目标下立即行动，并把一种猜测快速传播到代码、测试、文档和多个模块。

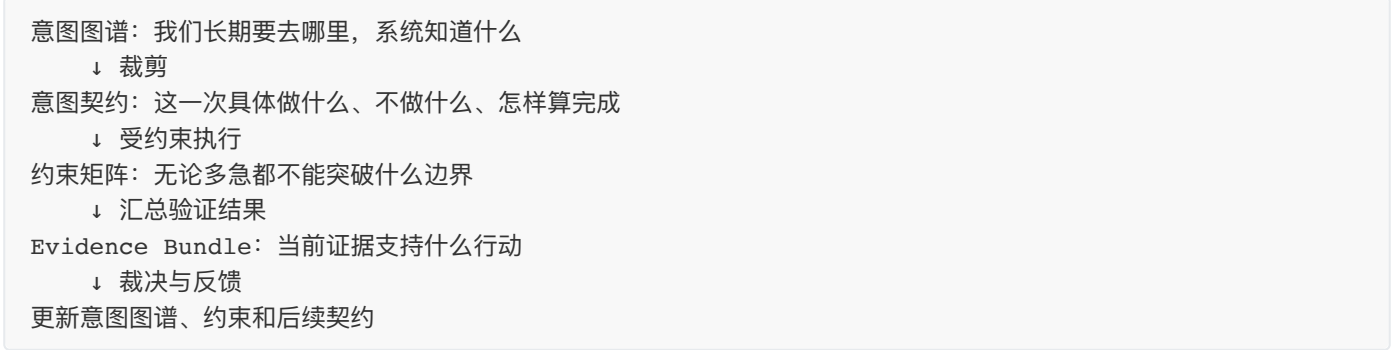
因此，四大动态工件不是为了制造四份新文档，而是把过去依赖人脑和组织默契完成的四种关键功能显式化：

- 用意图图谱保存全局方向与关系；
- 用意图契约冻结一次行动的明确授权；
- 用约束矩阵把跨任务红线变成可执行门禁；
- 用 Evidence Bundle 把执行结果变成可裁决证据。

动态工件	组织共同回答的问题	机器怎样使用	主要责任
意图图谱	我们为什么做，目标、能力、规则、风险与历史怎样关联？	导航和裁剪任务上下文，追溯来源与影响	IO 负责价值方向，OA 负责结构和新鲜度，领域专家确认专业事实
意图契约	这一次被授权做什么、不做什么、怎样算完成？	作为任务执行、测试、变更围栏和门禁的基准	OA 挑战并起草，IO 完整审阅后签署
约束矩阵	无论多急，哪些组织边界仍不能突破？	配置权限、检查、阻断、告警、升级和恢复动作	OA 维护，安全、架构、合规和领域责任人批准各自规则
Evidence Bundle	当前事实足以支持发布、补证、返工还是停止？	聚合 AC、约束、测试、差异、风险与恢复证据供 Verify 使用	AS 和工具产生原始证据，OA 聚合，独立验证者与有权者裁决

“可以被机器执行”不意味着把所有内容强行写成 YAML，也不意味着自然语言从此失效。价值意图、风险取舍和裁决理由仍需要人可理解的表达；稳定 ID、状态、版本、来源、验收标准、约束和证据引用则应尽可能结构化。真正需要统一的是语义、责任和协议，而不是绑定某一种文件格式或 AI 宿主。

四大动态工件之所以是“动态”的，是因为它们都会随着真实事件改变状态：图谱因战略、任务和事故更新，契约因关键意图变化而重新签署，约束因风险与教训版本化，Evidence 随执行、失败、修复和裁决持续积累。没有所有者、状态、版本和触发事件的文件，只是静态材料，不是运行工件。



四大动态工件不是四份孤立文档，而是一条责任链。意图契约必须能追溯到图谱，执行必须同时满足契约和约束，证据包必须映射 AC 与约束，最终结果和教训必须回写图谱。

可以把它们理解成一次航行中的地图、航次任务书、航行禁区与航行记录。地图很重要，但不能代替某一次航行的目的和批准；任务书很明确，但不能因此允许船只穿越禁区；船到了目的地，也需要航行记录证明它走了哪条路线、经历了什么风险、是否值得继续扩大航程。

7.2 意图图谱：全局导航与组织记忆

为什么传统需求列表在 AI Coding 时代还不够

传统 Backlog 擅长表达“接下来可能做什么”，通常以 Epic、Feature、Story 或任务的线性列表呈现。它对优先级和计划非常有用，但不一定显式保存需求背后的战略原因、业务规则之间的依赖、历史事故、长期约束和任务完成后形成的新知识。

人类团队可以依赖产品经理、架构师和老员工把这些关系串起来。Agent 却需要机器可检索的连接：这个需求服务哪个目标，涉及哪些用户和能力，依赖哪些模块，违反过什么规则，过去发生过什么事故，哪些知识已经失效。

意图图谱因此不是“更复杂的需求清单”，而是把需求背后的关系网络显式化。列表回答“有哪些事项”，图谱还要回答“它们为什么存在、怎样关联、依据来自哪里”。

传统需求列表常见视角	意图图谱补充的视角
一条需求的标题和优先级	它连接的战略目标与价值假设
当前待办事项	已完成任务、证据和失败教训
上下级拆分关系	用户、能力、模块、规则、风险的多向关系
由人理解背景	可被 Agent 检索和裁剪的上下文来源
完成后关闭	完成后回写知识并影响后续任务

这并不意味着要抛弃 Backlog。Backlog 仍可管理承诺与顺序；意图图谱提供 Backlog 往往没有承担的全局语义和知识导航。二者可以通过任务 ID 相互链接。

为什么 AI 越强，越需要意图图谱

模型能力增强，意味着 Agent 能处理更大范围的任务，也意味着错误意图会影响更大范围。一个只补全函数的工具，误解可能局限在十几行代码；一个能搜索仓库、调用工具、修改多个模块的 Agent，可能把错误假设扩散到整个交付链。

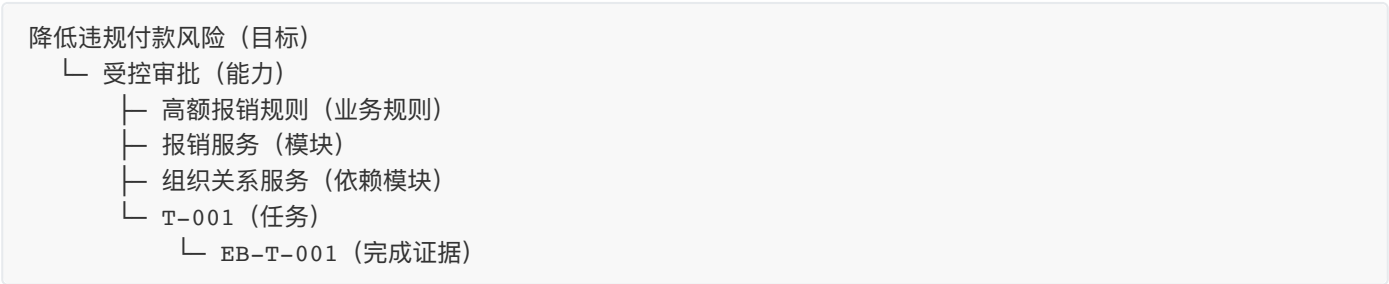
长上下文窗口也不能替代图谱。把所有 PRD、会议纪要和代码一次性塞给模型，会引入过期信息、重复规则、矛盾版本和敏感数据。图谱的作用不是存储一切，而是建立导航：先找到与当前任务相关的节点，再裁剪最小必要上下文。

意图图谱包含什么

意图图谱保存项目长期存在的方向和关系，包括：

- 北极星目标；
- 用户和利益相关者；
- 核心能力与业务域；
- 关键依赖；
- 长期约束；
- 已完成任务；
- 失败教训与模式；
- 后续演进方向。

图谱中的“节点”可以是目标、用户、能力、业务规则、模块、任务、风险或证据；“边”说明它们的关系，例如“支持”“依赖”“受约束于”“由某证据验证”“替代旧版本”。



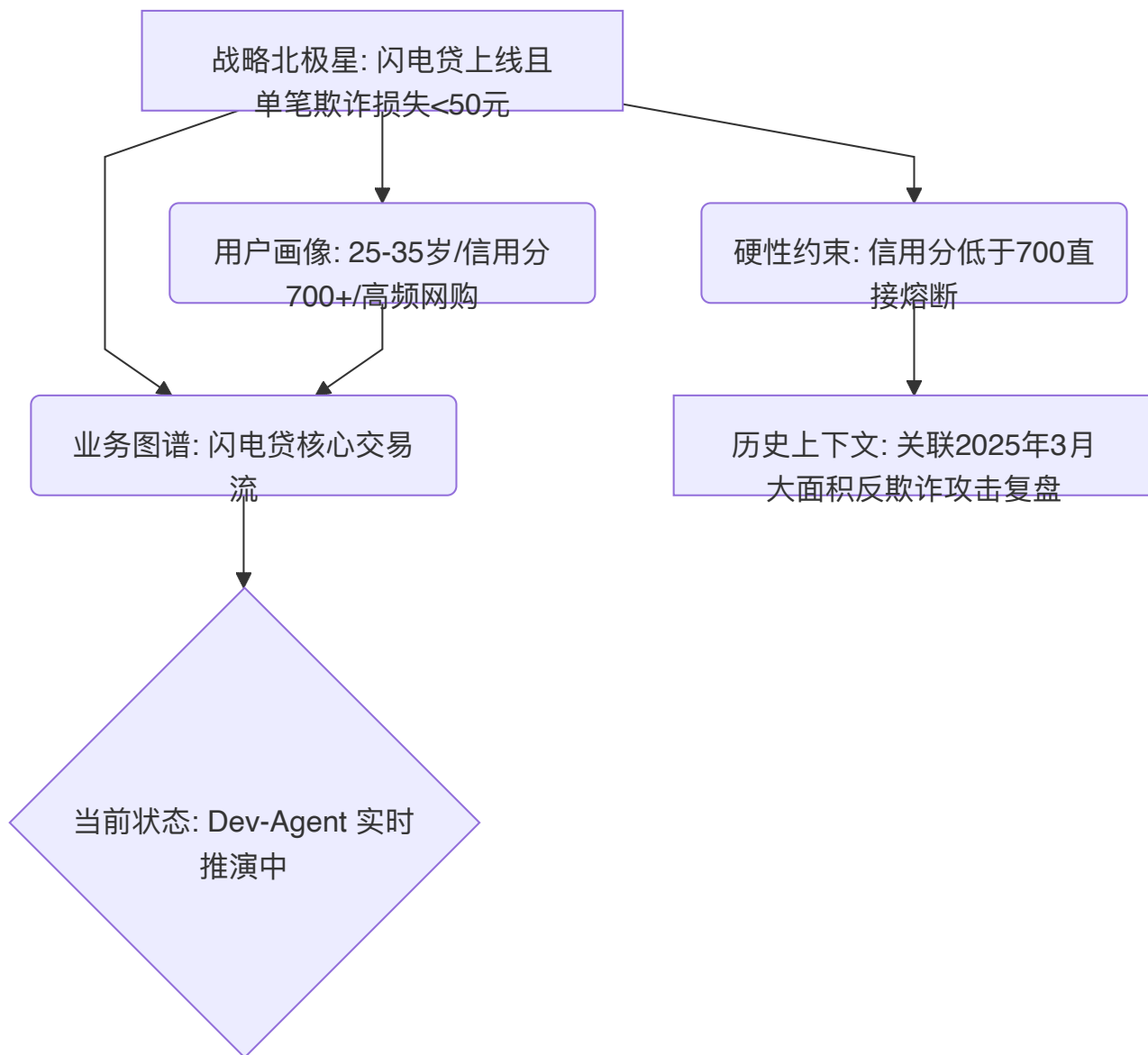
意图图谱怎样产生

它不是由 IO 一次性画完，也不应完全由模型自动生成后直接当作事实。一个可行过程是：

1. IO 提供战略目标、用户问题、现行业务和原始材料；
2. OA 从 PRD、会议纪要、代码、事故记录中提取候选节点与关系；
3. 使用批判性思维发现目标冲突、过期规则和缺失来源；
4. 通过 Grill-Me 逐条确认北极星、关键规则、用户范围和硬边界；
5. 把确认内容写入图谱，未确认内容标记为假设或待验证；
6. 每次任务完成后，从 Evidence Bundle 回写新能力、教训、风险与后续方向。

模型可以加速提取，不能自行决定哪些信息是组织事实。IO 对价值方向负责，OA 对结构、来源和新鲜度负责，领域专家对专业规则提供权威输入。

- **示例形态：**不再是 作为用户，我想要...以便于... 的孤立卡片，而是系统可解析的节点网络：



一个费用报销图谱是怎样长出来的

第一版可能只有“降低违规付款风险”这一北极星和“费用报销”业务域。T-001 启动后，OA 增加高额审批能力、成本中心总监、报销服务和组织关系服务等节点。任务中发现代理总监冲突，完成后又把“多个生效代理时转人工财务队列”作为经验证规则回写。

图谱不是先完整再使用，而是在真实任务中逐步生长。每次生长都应有来源和证据，不能因为 Agent 推测合理就自动成为长期知识。

图谱的常见失效

- 只画战略概念，不连接代码、任务和证据；
- 把所有文档全文塞进图谱，失去导航功能；
- 自动抽取后无人确认，假设与事实混杂；
- 只增加节点，从不淘汰旧规则；
- 没有责任人、版本和最后验证时间；
- 完成任务后不回写，图谱永远停留在项目启动时。

意图图谱不必一开始就是图数据库。一份结构清晰、可维护的 Markdown 就能起步。判断它是否有价值，只需问：当一个新 Agent 接手任务时，能否从任务追溯到目标、规则、依赖和历史教训，并裁剪出可信上下文？

7.3 意图契约：单次任务的执行基准

为什么有了图谱，还需要契约

这是理解四大动态工件的关键。图谱是持续演进的全局知识空间，内容广、关系多，并且可能同时包含事实、假设、候选方向和多个任务。它适合导航，却不适合直接作为一次执行的唯一依据。

如果 Agent 直接依据图谱行动，会遇到三个问题：第一，不知道当前任务到底选中了哪些目标和范围；第二，图谱在任务执行中仍可能更新，导致执行基准移动；第三，图谱没有天然表达“由谁在何时批准了这一次行动”。

意图契约就是从动态全局图谱中裁剪出来的一张“运行态切片”。它把这一次做什么、不做什么、遵循什么规则、如何验收以及由谁授权冻结下来。图谱回答“整个系统知道什么和长期要去哪里”，契约回答“本次被允许做什么”。

契约怎样从图谱产生

产生契约不是复制几个图谱节点，而是一个从模糊到确定的决策过程：

IO 发起本次意图

→ OA 从图谱检索相关目标、用户、规则、模块和历史教训

→ 批判性思维识别歧义、冲突、不可验证项和过宽权限

→ Grill-Me 逐条帮助 IO 作出关键决定

→ Example Mapping 将抽象规则展开为示例、反例和问题

→ OA 形成 Goal、Non-goals、Rules、AC、风险与边界

→ IO 审阅完整契约并显式签署

这里有两个不能混淆的动作：Grill-Me 是逐条形成决策，签署是 IO 对完整契约承担最终责任。回答了五个问题，不等于已经批准一份可能还有遗漏的完整契约。

契约至少包含什么

意图契约至少包括：

```
task_id: T-001
goal: 本次真正要改变的结果
non_goals:
  - 明确不做的事项
rules:
  - 必须成立的业务规则
acceptance_criteria:
  - id: AC-01
    description: 可判定的验收标准
    verify: assert/http/db/shell
sign_off:
  intent_owner: 由 IO 本人填写
```

除了这些字段，复杂任务还需要输入输出、边界条件、未决事项、假设、回退要求、相关模块和数据权限。字段多少可以裁剪，但“目标、非目标、可判定 AC、授权”不能被省略。

契约如何防止两个方向的失控

- 防缺漏：把关键目标、规则和成功标准说清楚；
- 防发散：通过 Non-goals 限制顺手扩展和过度设计。

费用报销契约中的 `Non-goals`：不迁移历史草稿，会阻止 Agent 因为发现旧数据结构而“顺手完成迁移”； `AC`：5000.00 元保持原路径，会阻止执行者用模糊的“高额报销测试通过”掩盖边界错误。

契约与传统 PRD、用户故事有什么不同

意图契约不是要取代 PRD 或用户故事。PRD 可以解释产品背景和完整方案，用户故事适合沟通用户价值；契约重点处理一次 Agent 执行的授权与判定。

常见需求材料	主要用途	意图契约额外强调
PRD	产品背景、方案和整体体验	本次执行范围、非目标、机器可验证 AC
用户故事	从用户视角表达价值	规则边界、权限、签署和证据映射
任务卡	安排具体工作	与全局意图和约束的可追溯关系
Prompt	引导模型生成	组织授权、版本、责任和稳定执行基准

Prompt 可以从契约生成，但 Prompt 不是契约。Prompt 面向模型交互，契约面向组织责任和后续验证。

契约什么时候需要重新签署

实现技术选型改变但目标和 AC 不变，通常只需记录技术决定；若目标、非目标、业务规则、关键 AC、风险承担或权限范围发生变化，就必须形成新版本并重新签署。签署后的契约不能为了适配错误实现而被悄悄修改。

读者可以用一句话检验契约：**如果换一个没有参加过会议的 Agent，它能否在不猜测关键业务决定的情况下执行，并让第三方客观判断完成与否？**

7.4 约束矩阵：可执行的安全边界

为什么契约明确了，还需要约束矩阵

契约定义单次任务的目标和授权，但组织中还有大量跨任务长期有效的底线：不能泄露密钥、不能未经批准写生产数据、必须先签后码、关键接口必须向后兼容、资金金额必须使用精确类型。这些规则如果复制到每份契约，不仅冗长，而且容易版本不一致。

约束矩阵因此回答另一个问题：**无论当前任务多急、契约要求什么、模型多强，哪些边界都不能被突破？契约是“本次要做什么”，约束矩阵是“做任何事情都必须怎样”。**

还有一个更重要的原因：契约可能本身有错。IO 可能为了赶进度要求直接修改生产数据，但法律、安全和组织红线不能因为某份任务契约被签署就失效。因此需要明确优先级：

法律与安全约束 > 约束矩阵 > 已签署意图契约 > 实现便利性

约束怎样产生

约束不是 OA 凭经验写一套“最佳实践大全”。来源至少包括：

- 法律、监管与行业标准；
- 企业安全、数据和发布政策；
- 系统架构中的不变量与兼容要求；
- Intent Graph 中的长期业务边界；
- 当前契约暴露出的任务特定风险；
- 历史事故、Bug 回溯和混沌实验教训；

- 工程基线，如测试、性能和可观测要求。

OA 负责把自然语言原则转成可执行候选约束，安全、架构、领域和 IO 按责任范围确认。每条约束都需要唯一 ID、适用范围、验证方式、失败动作、Owner 和证据位置。

从一句原则到一条可执行约束

原则：“注意保护财务数据。”这句话没有检查对象，也没有失败动作。

转成约束后可以是：

```
id: C-DATA-07
level: MUST
rule: Agent 不得读取生产报销明细；测试仅可使用脱敏数据集
check: audit_tools --deny production-expense-db
enforcement: block
owner: data-security-owner
recovery: revoke_session_and_escalate
evidence: tool-audit-T-001.json
```

现在系统知道检查什么、失败时做什么、由谁负责、证据在哪里。约束才从“提醒”变成 Harness。

六个核心域

约束矩阵回答：无论任务多急、模型多强，什么都不能被突破？核心六域包括：

核心六域包括：

域	关注点
STRUCT	项目结构与工件完整性
DATA	数据范围、隐私和完整性
BEHAVE	业务行为和不变量
QUAL	测试、构建、性能和质量门槛
PROC	签署、TDD、证据和流程合规
STYLE	AI Coding Guide 与代码规范

Web 服务可按风险扩展 SEC、REL、OBS 等非功能域。

约束应包含：

- 唯一 ID；
- MUST / SHOULD / MAY 等级；
- 适用范围；
- 可执行检查；
- 所属门禁；
- 失败动作：Block、Warn、Log 或 Escalate；
- 例外批准人和有效期；
- 恢复方式。

“注意安全”不能被执行；“禁止提交密钥，检测到即阻断”才能成为护栏。

Block、Warn、Escalate、Log 有什么区别

- `block`：违反后立即停止，适用于不可接受的 MUST 风险；
- `warn`：保留风险并允许继续，适用于可接受但需要关注的 SHOULD；
- `escalate`：系统无权决定，暂停并提交人类裁决；
- `log`：记录信息，用于趋势观察和未来改进。

不是所有规则都写成 Block 才安全。过多硬门禁会制造约束疲劳和绕过行为；过多 Warn 又会让红线失去意义。选择执行方式时，应考虑影响、可逆性、判断确定性和责任归属。

约束矩阵的生命周期

项目初始化时先建立少量高价值约束，真实任务和事故再驱动增加。约束需要观察命中率、误报、漏报、执行成本和例外使用情况。长期不命中且无法说明风险的规则应复核；经常被申请例外的规则可能与现实脱节，也可能揭示团队在系统性规避底线。

约束矩阵不是越厚越成熟。成熟的标志是关键红线能够被机器稳定执行，例外有负责人和有效期，失败能够阻断或恢复。

7.5 Evidence Bundle：支持裁决的证据包

为什么测试报告还不够

代码完成、测试全绿，并不自动等于可以上线。测试可能没有覆盖关键 AC，约束可能未执行，测试环境可能与当前提交不一致，Agent 可能越权访问工具，回退也可能从未验证。更重要的是，测试报告通常面向工程师，而发布决定还涉及业务价值、残余风险、运行准备和责任承担。

Evidence Bundle 不是项目总结，也不是把日志打包下载。它把“我们做了什么、依据什么相信、还有什么不知道、当前证据支持哪种行动”组织成面向裁决的产品。

Evidence Bundle 怎样产生

证据包不是开发结束后由 OA 回忆补写，而是伴随任务不断生长：

1. 契约签署时，记录任务、版本和意图证据；
2. 编码门保存可信 Red，证明测试确实能发现缺失行为；
3. 实现过程中记录代码差异、工具调用、失败和修复；
4. Prove 阶段汇总 AC、测试、构建、性能和安全证据；
5. Verify 阶段检查约束覆盖、未知项、回退和证据独立性；
6. Telemetry 写入时长、Token、首次成功率、HITL 和价值信号；
7. 人类裁决及其条件被写入；
8. 发布后发现 Bug 时，原证据包被追加修正，而不是保持虚假全绿。

AS 负责产生原始运行证据，自动化工具负责采集与校验，OA 负责映射和完整性，IO 或授权责任人负责最终裁决。任何一个角色都不能只用自己的总结证明自己正确。

五类证据

一份成熟证据包至少考虑五类：

证据类型	回答的问题	示例
意图证据	做的是不是已确认的事	契约版本、签署、规则来源
变更证据	实际改了什么、没改什么	Git diff、文件清单、排除范围
行为证据	功能是否满足 AC	单测、接口测试、数据库断言、演示记录
约束证据	红线是否真实执行	权限审计、扫描、门禁与批准记录
运行与价值证据	成本和真实效果如何	Token、时长、失败率、用户与业务指标

证据不是越多越好，而是要映射到判断

一千行日志如果不能说明它支持哪项 AC，就只是材料。证据包需要一张映射表，把目标或约束连接到证据来源、结果、独立性和缺口。例如 AC-02 的“5000.00 元不触发总监审批”对应边界接口测试；C-DATA-07 对应工具审计；回退可用性对应一次实际演练。

高风险结论还要注意证据独立性。同一个 Agent 根据自己的实现生成测试、执行测试并总结通过，可能形成同源自证。关键场景可以使用独立流水线、第二模型、人工审查、外部基准或受控真实数据交叉验证。

最低内容

Evidence Bundle 最低内容包括：

- 契约和约束版本；
- 实际变更范围；
- AC 逐条验证结果；
- MUST 约束检查结果；
- 测试、构建、安全、性能等原始证据；
- 失败与修复历史；
- 未知项和已知风险；
- 回退与恢复方案；
- 单任务遥测摘要；
- 建议裁决及适用条件。

常见裁决包括：

1. Approve；
2. Approve with conditions；
3. Return；
4. Reduce scope；
5. Stop；
6. Escalate。

实际模板还可以支持 `request_clarification` 等裁决。只要任一 MUST 为 FAIL 或 UNKNOWN，建议就不能是无条件 Approve。

一个证据包怎样改变发布决定

报销任务所有单元测试都通过，但 Evidence Bundle 显示外币字段未验证、功能开关回退已通过、当前只影响两个试点部门。IO 因此作出“仅人民币场景灰度、外币入口阻断、24 小时后复核”的条件批准。

如果只看测试报告，团队可能全量上线；如果因为存在未知项一律拒绝，又会失去合理的渐进交付。证据包的价值，是让人能够在看清已知、未知和边界后作出更精细的裁决。

证据包的常见失效

- 只有成功截图，没有原始命令、环境和退出码；
- 测试很多，但没有与 AC 和约束映射；
- 只记录最终全绿，删除失败与修复历史；
- 使用上一提交的测试结果证明当前代码；
- Agent 的自然语言总结被当成运行证据；
- 未验证事项被留空，而不是标记 UNKNOWN；
- 证据包完成后发现 Bug，却不修正首次成功率和原裁决。

证据包不替人决策。它让人的决策可解释、可追溯，也允许人基于真实证据选择“不上线”。

7.6 涌现式增量与遥测的位置

涌现式增量是执行系统产生的候选交付结果。它只有通过契约、约束和验证，才能成为可发布增量。因此，它是被治理的结果，不是治理工件本身。

遥测则贯穿四大动态工件的运行事实：

- 判断意图是否准确；
- 发现约束失败和自愈情况；
- 衡量执行效率和人类介入；
- 验证交付是否产生预期价值；
- 识别需要更新图谱、契约或约束的模式。

每份契约在 Verify 后都会形成一份独立遥测记录和 `dashboard-T-XXX.html`，与该任务的 Evidence Bundle 对应；所有契约又聚合进入项目级 `dashboard.html`。因此，Evidence Bundle 回答“本次凭什么裁决”，单契约 Dashboard 回答“本次价值、能力、效率和进化表现如何”，全局 Dashboard 回答“整个治理系统是否在持续改善”。

这也是遥测没有被列为第五个动态工件的原因：它不是只在末尾形成的一份静态材料，而是贯穿图谱、契约、约束与证据的反馈系统。任务完成后的准确率、首次成功、Token、HITL 和知识沉淀，会反过来驱动图谱更新、约束调整和下一份契约的上下文裁剪。

7.7 三方与四方一致性

治理系统至少需要检查：

意图图谱 ↔ 意图契约 ↔ 约束矩阵

执行完成后进一步检查：

意图契约 ↔ 约束矩阵 ↔ 实际实现 ↔ Evidence Bundle

任何一方漂移都可能产生“测试通过但任务错误”的假成功。

7.8 工件的生命周期

四大动态工件各自有不同生命周期：

工件	创建与所有者	主要触发事件	变更规则	完成后的状态
意图图谱	项目或价值流启动；IO 负责方向、OA 负责维护	战略调整、任务完成、事故、知识验证或失效	保留来源、状态和关系变化，不把候选假设直接晋升为事实	持续存在并为后续任务提供导航
意图契约	单任务 Specify；IO 对签署负责	目标、非目标、业务规则、关键 AC、风险或权限变化	形成新版本并重新签署，不能静默覆盖	完成后冻结并保留
约束矩阵	项目或稳定模块初始化；OA 与领域责任人共同维护	架构、法规、风险、事故、例外和验证教训变化	明确适用范围、级别、检查、失败动作、例外和有效期	持续执行，过期规则必须撤销或重签
Evidence Bundle	从任务开始持续积累；OA 负责完整性	每次执行、验证、失败、修复、提交和裁决	只引用当前版本与新鲜证据，保留失败历史和 UNKNOWN	完成后成为裁决与审计记录

工件不是在最后一次性补写。契约和约束必须在编码前存在，证据应随执行产生，图谱必须在收尾前回写。

7.9 工件版本与时效

常见失败不是工件不存在，而是版本不一致：

- Agent 使用了旧契约；
- 约束已经改变，Evidence Bundle 仍引用旧结果；
- 代码提交发生变化，测试证据来自上一提交；
- 图谱中的业务规则已经失效；
- 例外已经过期仍被继续使用。

因此工件需要任务 ID、版本、时间、代码提交或产物标识。`verify_freshness.py` 等工具的目的，是检查证据是否仍然对应当前状态。

跨 Codex、Claude、WorkBuddy 或其他宿主协作时，稳定 ID、状态枚举、签署语义、证据引用和版本关系尤其重要。宿主可以使用不同界面和模型，但不能各自解释“SIGNED / CLOSED”“PASS”“UNKNOWN”或“当前版本”。跨工具能力来自共享协议，不来自所有工具长得一样。

7.10 工件粒度

工件过粗会让一次任务携带整个项目的复杂度；过细则会制造大量维护成本。

合理粒度通常满足：

- 一个契约对应一个可独立判断完成的任务；
- 一个 Evidence Bundle 对应一个契约；
- 约束矩阵以项目或稳定模块为边界；
- 图谱维护长期稳定关系，不记录每个临时执行步骤；
- 跨模块任务由模块契约和 Release Evidence Bundle 聚合。

7.11 一份完整证据包如何被阅读

管理者不应先阅读全部日志，而应按决策顺序阅读：

1. 这次目标和非目标是什么？
2. 实际修改范围是否一致？
3. AC 和 MUST 是否全部通过？
4. 发生过哪些失败，怎样修复？
5. 哪些事项仍然未知？
6. 回退是否可执行？
7. 遥测说明成本、质量和自主性如何？
8. 当前建议是什么，适用什么条件？

如果证据包不能让决策者快速回答这些问题，它可能只是材料堆积，而不是决策产品。

7.12 工件反模式

- 图谱只增加节点，从不删除过期知识；
- 契约写成需求散文，没有 Non-goals 和可执行 AC；
- 约束矩阵复制模板后全部保留，包含大量不适用检查；
- Evidence Bundle 只有“全部通过”，没有原始命令和失败历史；
- 用一个项目级证据包覆盖几十个无法追溯的任务；
- 完成后才补签契约；
- 将模型总结当成运行证据。

7.13 本章小结

四大动态工件把全局方向、单次授权、执行边界和完成事实连接为一套人机共同语言。它们不是四份额外文档，而是带所有者、状态、版本、触发事件和运行效果的组织接口：图谱负责导航，契约负责授权，约束负责守界，Evidence 负责支持裁决。

Telemetry 不属于第五个工件，而是把工件运行事实连接成组织慢外环。格式可以按风险与工具裁剪，但目标、责任、状态和证据语义必须稳定，才能支持不同人员、Agent 和宿主持续交接。

本章最小实践：为同一个真实任务创建四大动态工件的最小版本，标明所有者、状态、版本、触发事件和下游机器用途。

完成证据：四份可相互追溯的工件、统一 ID 映射，以及至少一个能够读取契约或约束并产生真实证据的机械检查。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，把意图、契约、约束和证据变成机器可读、可检查的共同语言。

第八章 | 三大自治机制与 Harness：让 Agent 获得有边界的执行权

8.1 三大自治机制

这里的“自治”不是把责任整体交给 AI，而是让 Agent 在已签署意图、明确权限和可验证边界内获得执行权。三大机制回答三个连续问题：**任务从哪里获得确定目标，执行偏差怎样被纠正，机器不能决定时由谁接管**。如果没有这三个机制，Agent 即使拥有很强的编码能力，也只是在一个缺少方向盘、刹车和驾驶责任人的系统里高速行驶。

机制一：意图注入与对话到契约——给自治执行一个稳定起点

“意图注入”听起来像一个技术动作，很容易被理解为把一段 Prompt 发给 Agent。它真正要解决的是：探索阶段充满讨论、假设和变化，而执行阶段需要一份稳定、可判定、经过授权的任务基准。两者如果混在一起，Agent 可能把一句试探性意见当成最终决定，也可能在对话后半段忘记前面的关键限制。

意图注入通常经过四步：

1. IO 提出目标或问题，不要求第一句话已经完整；
2. OA 通过批判性思维和 Grill-Me 揭示歧义、冲突与风险；
3. OA 把已经确认的决定整理成 Intent Contract；
4. IO 审阅完整契约并显式签署，系统再把任务所需上下文、约束和 AC 注入 AS。

以“超过 5000 元的报销增加总监审批”为例，真正注入 Agent 的不只是这句话，而应包括金额口径、边界值、适用组织、非目标、异常路径、权限限制以及可以证明完成的 AC。于是 Agent 面对的不是一个需要自行猜测的愿望，而是一项获得授权的工程任务。

意图注入的核心价值是建立“探索—承诺”的分界线。契约签署前可以充分讨论；签署后，AS 不能静默改变目标。如果执行中出现新事实，需要判断它属于实现细节，还是已经改变契约。后者必须暂停并重新签署。

常见误解有三个：一是把全部会议纪要都塞进上下文，导致噪声大于信息；二是把 IO 的第一句话直接视为意图，跳过校准；三是由 OA 或 Agent 自动写上“已签署”，让授权变成形式。真正的意图注入不是“信息越多越好”，而是“进入执行的信息已经过筛选、确认和授权”。

机制二：高频对抗与自净化——让偏差在小范围内尽早暴露

“对抗”并不是让多个 Agent 互相争吵，而是用测试、规则、独立检查和反例不断挑战当前结果。传统交付常在开发末尾才集中测试；Agent 的执行速度更快，如果反馈仍然滞后，错误会在几分钟内跨越多个文件和模块。高频对抗的目的，是把反馈嵌入执行过程，使错误尚未扩散时就被发现。

一次最小对抗循环包括：

提出候选结果

- 用 AC、测试和约束挑战
- 记录失败类型与原始证据
- 判断是否在授权范围内可修复
- 最小修复并重跑原检查
- 收敛或停止升级

例如测试发现 5000.00 元也进入总监审批。系统先保存输入、期望、实际结果和规则版本，再判断这是普通实现错误、规格冲突、环境问题还是约束违规。如果只是比较符号写错，AS 可以在限定次数内修复；如果财务制度与已签署规则冲突，AS 就没有权力“选择更合理的一边”，必须升级。

“自净化”也不等于自动修到成功为止。每次循环都需要预算、最大次数、允许修改范围和停止条件。连续三次出现不同失败、修复需要降低测试强度、必须扩大权限、或错误根因无法解释时，都应停止。无限自修复只会隐藏系统没有收敛的事实，并消耗更多时间与 Token。

高频对抗的结果不只是一份全绿报告。失败分类、修复次数、被触发的约束、人工介入点和最终残余风险，首先应进入当前任务的 Evidence Bundle，支持本次裁决；其中具有跨任务分析价值的运行事实，再按 Measurement Contract 汇入 Telemetry。这样组织才能判断 Agent 是一次做对，还是经过大量试错勉强通过。

机制三：人类异常裁决与证据包验收——把机器不该决定的事情交还责任主体

HITL（Human in the Loop）常被翻译为“人在回路中”。很多团队据此在每一步增加人工审批，最后既没有自治，也没有效率。另一种极端是只在线上前让人点一下同意，使人类成为无法真正审查的橡皮图章。

Agentic Agile 中的人类介入按任务性质触发，而不是按固定流程节点机械出现。至少四类事项应交给人类：

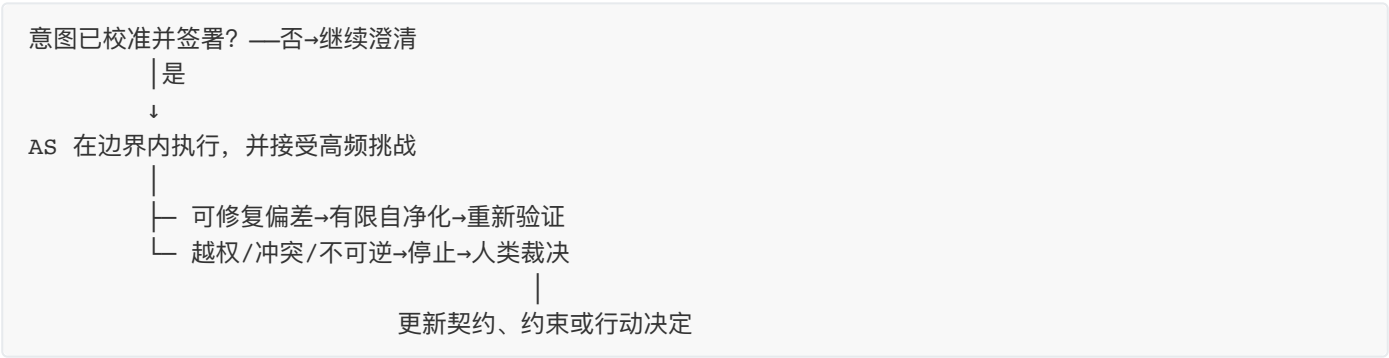
- **不可逆操作**：删除生产数据、对外发布、资金支付、扩大生产权限；
- **价值判断**：范围取舍、风险偏好、用户体验与成本冲突；
- **不确定验证**：证据互相矛盾、语义依赖业务背景、自动测试无法覆盖；
- **例外授权**：临时放宽约束、接受已知风险、改变签署目标。

异常升级时，系统不应只抛出一句“请决定”。OA 要把问题整理为可裁决包：发生了什么、违反了哪条规则、有哪些原始证据、继续或停止各有什么影响、推荐选项和理由是什么。人类作出决定后，决定本身也要进入契约、例外记录或证据包，形成可追溯责任链。

因此，好的 HITL 不是人越多越安全，而是**正确的人在正确的信息基础上，对机器不应决定的事情负责**。如果人类经常被迫处理可自动化的重复问题，说明 Harness 不成熟；如果重大价值和不可逆操作从不触发人类，说明自治已经越界。

三大机制如何连成一条运行链

意图注入提供目标与授权，高频对抗持续检查执行是否偏离，人类异常裁决处理机器边界之外的决定。三者不是三个孤立功能，而是一条从“可以开始”到“可以继续”再到“必须停下”的控制链：



读者可以用一个问题检查自己的流程：**Agent 在什么时候获得执行授权，什么失败允许它自行修复，出现什么信号时它必须停下？** 如果三个答案都不清楚，就还没有形成自治机制。

8.2 从三大自治机制到 Harness 工程承载

三大自治机制定义的是治理逻辑：任务怎样获得授权，偏差怎样被纠正，机器不能决定时怎样交还人类。但如果这些逻辑只存在于制度、培训或 Prompt 提醒中，Agent 仍然可以因为上下文错误、权限过大、状态丢失或门禁可跳过而失控。自治机制必须由能够运行的工程环境承载，这个环境就是广义 Harness。

可以用四个层级理解本章与下一章的关系：

层级	回答的问题	本书中的内容
三大自治机制	自治系统必须完成什么治理动作？	获得意图、纠正偏差、异常交还人类
Harness 六大支柱	为支撑自治，系统必须具备什么能力？	上下文、工具、编排、状态记忆、评估观测、约束恢复
四类工程方法	六种能力具体怎样建设？	Context、Graph、Loop、Harness Engineering
SCOPE-V	这些机制在一次任务中何时运行？	S、C、O、P⇌E、V 与 Telemetry

因此，三大机制不是六大支柱的上位替代品，六大支柱也不是另一套 3-4-3。前者描述运行逻辑，后者描述能力结构；四类工程提供实现路径，SCOPE-V 再把它们放入运行时控制回路。

三大机制与六大支柱也不是一一对应。一项机制往往需要多根支柱共同支撑：

自治机制	主要依赖的 Harness 能力
意图注入与对话到契约	上下文管理、状态与记忆、约束与恢复
高频对抗与自净化	评估与观测、执行编排、状态与记忆、约束与恢复
人类异常裁决与证据包验收	评估与观测、工具系统、状态与记忆、约束与恢复

下面先固定唯一的一套六大能力支柱，再说明它们怎样通过四类工程方法落地。

8.3 Harness 的六大能力支柱

Harness 原意是“挽具”或“安全带”：它不替代马匹、车辆或驾驶者，却把力量约束在可控制方向上。在 AI Coding 中，广义 Harness 不是某一个脚本，也不是一套额外平台，而是包裹 Agent 的完整工程控制环境。六大支柱分别解决信息、能力、顺序、连续性、判断和安全恢复问题。它们是六种能力域，不是六套必须分别采购的平台。

支柱一：上下文管理——让 Agent 看到“足够且正确”的信息

模型并不会因为读到更多文件就必然理解得更好。无关材料会挤占注意力，过期文档会提供错误规则，敏感信息会扩大泄露风险。上下文管理因此不是简单扩充上下文窗口，而是持续回答：当前任务必须知道什么、可以不知道什么、哪些信息不能看到、信息是否仍然有效。

3-4-3 使用分层注入：意图图谱提供全局导航，约束和编码规范提供跨任务边界，当前契约和代码切片提供任务上下文。费用报销 Agent 需要看到金额规则、审批接口和相关测试，不需要读取整套薪酬数据、所有历史会议记录和生产凭证。

上下文支柱失效时，常见表现包括 Agent 反复询问已存在信息、引用旧接口、修改无关代码、不同 Agent 对同一术语理解不一致。改进它时，重点不是继续添加文档，而是建立来源、新鲜度、裁剪规则和任务完成后的回写。

支柱二：工具系统——区分“知道什么”与“能够做什么”

Agent 知道数据库结构，不等于它应该获得生产数据库写权限；它知道发布命令，也不等于可以自行发布。Tools Manifest 像一份机器员工的岗位授权书，声明允许调用哪些工具、访问哪些目录和网络域、执行哪些操作，以及哪些动作必须取得人工批准。

权限应遵循最小必要原则，并同时限制“工具”和“作用范围”。只允许 Git 并不够，还要区分只读、创建分支、提交、推送和合并；允许数据库工具也要区分测试环境查询、生产只读与生产写入。工具调用应留下审计记录，越权请求应被阻断而不是仅告警。

工具系统的典型失败，是为了减少中断给 Agent 一个过大的万能权限。短期看执行更顺畅，长期却让一次 Prompt 注入、错误路径或模型误判获得真实破坏能力。

支柱三：执行编排——把复杂任务从聊天顺序变成工作关系

一个复杂任务往往包含需求澄清、测试、实现、接口适配、安全检查和证据汇总。仅靠 Agent 在对话中自行决定下一步，依赖关系很容易丢失。Work Graph 用 DAG（有向无环图）表达“什么必须先发生、什么可以并行、什么结果是下一步的输入”。

DAG 可以先理解成一张带箭头的任务图：箭头表示依赖，节点表示可验证工作。测试节点完成可信 Red 后，实现节点才能开始；实现和回退准备可以在边界清晰时并行；所有检查结束后，证据汇总节点才有资格运行。

成熟编排不仅描述成功路径，还描述超时、失败、重试、回滚和升级。它追求的不是让最多 Agent 同时工作，而是减少等待、冲突和无效交接，让系统始终知道“现在在哪、为什么停、接下来由谁做”。

支柱四：状态与记忆——让跨步骤、跨会话的执行保持连续

模型会话结束后，临时推理不会自然变成组织知识。状态用于记录当前任务进行到哪里，记忆用于保存经过验证、未来可能复用的事实与教训。两者混在一起，会把临时猜测永久化；两者都不保存，则每次执行都从头开始。

Loop Memory 可以记录节点状态、关键决定、失败与恢复；Intent Graph 保存稳定的业务关系和经过验证的知识；反思日志记录某次任务值得复用的模式。每条记忆都应有来源、适用范围、版本、责任人和失效条件。

例如“外币报销必须先折算为本位币”来自一次真实缺陷，可以进入业务规则；“某接口可能很慢”只是一次观察，需要更多证据，不应直接升级为全局事实。记忆支柱的质量，不看保存了多少，而看下次是否能在正确场景被正确使用。

支柱五：评估与观测——既判断结果，也看见过程

评估回答“做得对不对”，观测回答“系统正在怎样做”。只有最终测试结果，团队看不到 Agent 是否越权、重试多少次、花费多少 Token、在哪个环节偏离；只有过程日志，又无法证明业务目标完成。

因此这一支柱同时包含 AC 验证、测试、证据审计、图谱—契约—约束一致性、NFR 检查和遥测。它要把目标命中、质量、成本、异常、人类介入与学习联系起来。

“测试全绿”只是一个信号，不是完整结论。若测试没有覆盖关键 AC，或者测试与错误实现来自同一误解，全绿仍然可能无效。评估必须能追溯到签署意图，观测必须保留真实来源，二者共同支撑发布裁决。

支柱六：约束与恢复——给高速执行装上护栏、刹车和逃生通道

约束说明 Agent 不能做什么、做到什么程度必须停止；恢复说明失败以后怎样回到安全状态。只有约束没有恢复，系统一出错就只能人工救火；只有恢复没有约束，错误会反复发生并不断扩大影响。

可执行约束可以限制文件范围、依赖、权限、性能、安全、Token 预算和发布条件。恢复策略则定义自动修复次数、功能降级、事务回滚、数据补偿、人工接管和事故记录。

以生产发布为例，约束可以规定 Agent 无权直接持有生产凭证；恢复要求功能开关已经验证、回退命令可用、监控阈值和负责人明确。安全不是“希望不出问题”，而是即使出问题，影响也能被限制并且系统能恢复。

六大支柱的木桶效应

六大支柱并非各自打分后求平均。某一根明显缺失，就可能成为整个自治系统的失效点：上下文正确但工具权限过大，Agent 会准确地做出未经授权的操作；约束严格但不可观测，团队不知道门禁是否真正运行；证据充分但没有恢复，发布后仍无法控制事故。

因此建设 Harness 时应先寻找“最危险的短板”，而不是平均铺开六个平台。读者可以逐项回答：信息是否可信、权限是否最小、依赖是否显式、状态是否连续、结果是否可证、失败是否可恢复。回答不出的地方，就是下一项建设重点。

8.4 六大支柱如何通过四类工程方法落地

六大支柱描述广义 Harness 需要具备哪些能力，四类工程方法则描述这些能力如何被建设、装配并在一次任务中协同运行。Context Engineering、Graph Engineering、Loop Engineering、Harness Engineering 不是四个流行术语的拼盘，也不是四套互相独立的平台。它们分别治理 Agent 的认知输入、协作拓扑、反馈收敛和行动边界。

Context Engineering: 让正确的 Agent，在正确时刻，只获得完成当前任务所需的可信信息

Graph Engineering: 把目标、依赖、角色、并行与汇合点变成可执行的工作拓扑

Loop Engineering: 把失败、评价和遥测变成有预算、有终点的纠偏回路

Harness Engineering: 把组织规则变成执行前、执行中和执行后的可运行护栏

如果把 SCOPE-V 比作整套治理系统的控制回路，那么四类工程就是支撑这条回路运行的工程方法；如果把六大 Harness 支柱比作系统所需的能力部件，那么四类工程就是把这些部件装配、运行和校验起来的方法。三者不是竞争关系：SCOPE-V 回答“控制如何流转”，六大支柱回答“系统要具备什么”，四类工程回答“这些能力怎样落地”。这里

的四类工程也不同于人员、组织、流程、工具四维转型：前者是工具与运行环境的工程方法，后者是企业转型的观察与设计维度。

8.4.1 Context Engineering：从写好 Prompt 到治理 Agent 的认知环境

Prompt Engineering 关注“这一句话怎样写得更清楚”，Context Engineering 关注的范围更大：模型在当前时刻能看到哪些目标、规则、代码、历史、工具说明和证据；这些信息从哪里来、是否最新、彼此是否冲突；哪些内容必须隔离；任务阶段变化以后，又应当替换哪些内容。

这一区别在 AI Coding 中很关键。人类开发者拥有持续的项目记忆，会主动翻代码、问同事并识别过期资料；Agent 的一次会话却可能同时面对几万行代码、多个版本的文档和相互矛盾的聊天记录。把更多内容塞进上下文并不等于理解更准确。无关信息会争夺注意力，陈旧信息会制造规格漂移，敏感信息的过度注入还会扩大数据泄露和越权风险。

因此，Context Engineering 的目标不是“上下文越长越好”，而是让上下文具备五种质量：

- **相关**：只包含当前节点真正需要的信息；
- **可信**：能追溯到已签署契约、权威代码或有效证据；
- **新鲜**：带有版本和时间，过期内容能够失效；
- **隔离**：不能让一个节点看到无权访问的数据和凭证；
- **可压缩**：在不丢失关键语义的前提下减少噪声和 Token 成本。

四层上下文，而不是一个巨型提示词

3-4-3 可以把上下文组织成四个层次：

1. **L1：任务意图层**。来自 Intent Graph 和当前 Intent Contract，说明为什么做、为谁做、成功如何判定、明确不做什么。
2. **L2：组织约束层**。来自 Constraint Matrix、Tools Manifest 和合规规则，说明哪些边界不可突破。
3. **L2+：工程惯例层**。包括代码结构、接口约定、测试方式、命名规范和仓库级 AI Coding Guide，说明在这个系统里应怎样工作。
4. **L3：节点切片层**。只为当前 Work Graph 节点裁剪所需文件、相邻接口、AC、测试和上游产物。

L1 和 L2 保证方向与底线稳定，L2+ 提供团队的工程常识，L3 随节点快速变化。测试 Agent 与实现 Agent 即使服务于同一个契约，L1 和 L2 可以相同，L3 也不应相同：前者需要边界样例和验收口径，后者需要失败测试、受影响代码和允许修改的文件范围。

一条可运行的上下文流水线

Context Engineering 至少包含六个连续动作：

发现候选信息

→ 判断来源、版本和权限

→ 按当前节点裁剪

→ 检查冲突、过期与敏感信息

→ 注入 Agent 会话

→ 在阶段切换或事实变化时刷新

“发现”可以通过代码索引、Intent Graph 和依赖关系定位材料；“裁剪”不是随意摘要，而是保留当前 AC、约束、输入输出和相邻接口；“隔离检查”要证明未注入禁止目录、生产数据或无关客户信息；“刷新”则防止 Agent 在契约已更新后继续使用旧目标。

上下文也必须接受评估。除了压缩率和 Token 数量，团队还应观察关键事实召回率、过期内容命中率、冲突数量、敏感信息泄露次数，以及由于上下文缺失造成的返工。压缩到原来的 30% 却删掉一个 MUST 约束，不是优化；上下文翻倍却没有提高任务命中率，也不是工程进步。

报销案例：同一任务为什么需要不同上下文

“单笔报销不得超过 5000 元”的测试节点，需要看到金额边界、币种处理、AC-001、禁止修改生产数据等内容，却不需要整个财务系统源码。实现节点在接到可信 Red 后，需要看到失败测试、金额校验函数、异常返回规范和允许修改的文件列表，但不需要员工薪资数据。证据节点则主要需要契约版本、检查命令、退出码、测试报告和代码差异。

Context Engineering 在 SCOPE-V 中主要支撑 S、C 和 O，并在 P、E、V 每次交接时重新裁剪输入。它与“上下文管理”支柱关系最直接，同时依赖状态与记忆、约束、评估和工具系统。它不是一次准备动作，而是贯穿控制回路的认知供给系统。

常见误区包括：把整个仓库一次性塞给模型；用旧聊天记录代替签署契约；只有摘要没有来源；所有 Agent 共用同一份上下文；只计算 Token 节省，不验证关键信息是否保留。

8.4.2 Graph Engineering：从任务清单到可计算的协作拓扑

Graph Engineering 不是把待办事项画成一张漂亮流程图，也不只是传统项目计划的可视化。它要把目标、任务、角色、依赖、输入输出、并行关系、汇合点、失败分支和停止条件表示为机器可以查询和调度的图结构。

AI Coding 放大了 Graph 的必要性。一个人类团队按日协调时，很多依赖可以靠口头默契补足；多个 Agent 在分钟级并行执行时，一个模糊依赖会迅速变成冲突写入、基于旧结果继续工作、重复修复和错误汇合。Agent 越多，并不天然越快；只有工作能够被正确分解、隔离和重新汇合，并行才会转化为吞吐量。

两张图：组织能力图与任务工作图

Graph Engineering 通常包含两类图：

- **Org Graph** 描述有哪些人、Agent、Skill、工具和权限，谁能够承担哪类任务，谁拥有批准权和责任；
- **Work Graph** 描述当前任务怎样拆分，节点之间如何依赖，哪些可以并行，何时汇合，失败后回到哪里。

Org Graph 回答“谁有能力且有权做”，Work Graph 回答“这一次谁在何时做什么”。如果只有 Work Graph 而没有 Org Graph，调度器可能把安全审计交给没有权限或能力的 Agent；如果只有 Org Graph 而没有 Work Graph，团队知道有哪些角色，却无法治理一次具体协作。

一个节点需要的不只是任务名称

可执行节点至少应包含：

- 节点目标和对应的 Intent Contract、AC；
- 前置依赖、输入产物与可信来源；
- 负责人、执行 Agent 和必要的 HITL 角色；
- 允许使用的工具与可修改范围；
- 预期输出、完成条件和证据要求；
- 时间、Token、并发和重试预算；
- 失败分支、回滚方式与升级条件。

边也不只是“先做 A，再做 B”。它可以表示数据依赖、审批依赖、互斥写入、证据汇合和失败路由。Graph Engine 因此应能回答：当前瓶颈在哪里，哪些节点真正可并行，关键路径是什么，一个上游产物失效会影响哪些下游，以及哪个节点需要重新调度。

报销案例：并行不是同时开四个 Agent

费用规则任务可以先并行生成金额边界测试、分析审批规则影响、检查权限约束和准备证据结构；这些节点只读共享契约，在各自隔离范围内产生输出。实现节点必须等待边界测试形成可信 Red，并在规则影响分析完成后启动。集成验证等待实现和安全检查汇合，Evidence Bundle 又必须等待全部规定检查完成。

如果金额测试与实现节点同时修改同一个测试文件，看起来是并行，实际是在制造争用；如果证据节点在集成验证前就宣告完成，图上的顺序正确但语义依赖错误。Graph Engineering 的价值，是把这些隐含关系变成可检查的结构。

Graph Engineering 在 SCOPE-V 中最直接地实现 O，但图中的节点可以覆盖 S、C、P、E 和 V。Intent Graph 负责保持“为什么”的结构，Work Graph 负责组织“怎样执行”的结构，Evidence Bundle 则记录图上关键节点是否真实完成。它主要调动编排、工具、状态记忆和评估支柱。

常见误区包括：把任务列表误称为 DAG；只画成功路径，不画失败和恢复；按角色并行，却忽略文件写冲突；节点只有标题，没有输入输出契约；认为多 Agent 数量越多，系统能力就越强。

8.4.3 Loop Engineering：从机械重试到有证据的自我纠偏

Loop Engineering 解决的不是“让 Agent 多试几次”，而是“系统收到偏差信号后，如何定位原因、采取获准的最小纠正、用同一个判据复验，并在正确时刻停止”。无限重试会消耗 Token、掩盖根因，甚至让 Agent 为追求全绿而削弱测试；可治理的 Loop 必须有输入信号、纠偏策略、预算、退出条件和升级路径。

一个完整反馈环至少包含六步：

接收真实信号

→ 对失败分类

→ 定位契约、约束、实现或环境原因

→ 执行最小且获授权的修复

→ 重跑原始检查

→ 收敛、停止或升级人工

“重跑原始检查”非常重要。若第一次因性能阈值失败，修复后却换成一个更宽松的阈值，系统并没有收敛，只是移动了球门。Loop 还要保存失败、尝试、退出码和最终结果，使“自愈”能够被复盘，而不是只留下一个全绿截图。

三层 Loop：任务内自检、失败后反思、组织级学习

3-4-3 可将 Loop 分成三层：

1. **LOOP-1 Self-Consistency**：发生在 O 之后、正式 P 之前。Agent 复述目标、范围、约束和计划，检查自身理解是否一致。它用低成本阻止方向性错误进入高成本执行。
2. **LOOP-2 Reflection**：发生在 P 暴露测试失败、错误或约束拦截之后。Agent 在当前上下文内诊断原因，完成修复或重构 E，再回到 P 取证。这就是 SCOPE-V 的 $P \rightleftharpoons E$ 快内环。
3. **LOOP-3 Feedback-to-Graph**：发生在 V 的人工裁决或生产遥测之后。经过确认的教训被反馈到 Intent Graph、约束、上下文规则、Skill 或路由策略，影响下一轮 S/C/O。这形成跨任务的慢外环。

三层 Loop 的时间尺度不同。LOOP-1 可能只需几十秒，LOOP-2 在一个任务内循环数次，LOOP-3 则跨发布和跨任务积累。把三者混成一个“让 Agent 自己反思”的按钮，会失去不同层级的责任边界。

为什么 Evolution 位于 Verification 之前

P 产生机器事实，E 依据这些事实快速诊断和修复，二者形成高频内环；当任务达到退出条件后，V 才对契约、约束、证据和新鲜度进行独立裁决。V 不负责替 Agent 生产证据，而负责审查证据是否足以放行。上线后的遥测再把成本、质量摩擦和价值偏差反馈到下一轮规划，形成慢外环。

这与传统“开发—测试—修复—再测试”的表面顺序相似，但控制目标不同：Loop 不是固定部门间的阶段回退，而是围绕每个微课题的高频自治收敛；同时它又明确限制自治预算，并保留独立 V 和人类价值裁决。

报销案例：一次失败怎样成为受控回路

5000.00 元边界测试失败后，Loop 先判断是实现使用了 `<` 而非 `<=`，而不是契约含糊或测试环境损坏；E 只修改金额比较逻辑，不得删除测试、修改 AC 或扩大文件范围；P 重跑同一边界测试和相关回归。若连续两次仍失败、触发安全约束，或发现人民币与外币规则冲突，系统停止自动修复并升级 OA/HITL，而不是继续猜测。

Loop 的健康度可以通过首轮成功率、平均收敛轮次、重复失败率、无效重试 Token、人工接管率，以及进入组织记忆后同类问题的复发率来观察。它主要连接评估观测、状态记忆、编排和恢复支柱。

常见误区包括：把“再生成一次”称为反思；没有最大重试次数；允许 Agent 修改判据以获得成功；由产生实现的同一主体单独宣布验证通过；失败得到修复，却没有沉淀可复用的规则。

8.4.4 Harness Engineering：把原则变成 Agent 无法绕过的运行时护栏

这里的 Harness Engineering 使用狭义含义，是四类工程方法之一，重点把约束、工具权限、门禁和恢复动作变成不可绕过的运行时控制。它服务于广义 Harness，但不等于全部六大支柱。Harness 在传统软件语境中有时指测试支架或工具框架；Agentic Agile 更关注它怎样包围概率性执行者并形成工程控制环境。它把意图、约束、工具权限、门禁、观测和恢复编织成可执行机制，使 Agent 知道可以做什么、必须证明什么、何时应当停止，以及失败后怎样安全返回。

它与写一份治理制度的根本差别在于“能否运行”。文档写着“不得读取生产数据”，但 Agent 仍持有生产凭证，这不是 Harness；规则写着“必须通过安全检查”，但流水线可以跳过检查直接发布，也不是 Harness。真正的护栏应尽可能进入 Policy as Code、工具白名单、沙箱、CI/CD 门禁和可验证的恢复动作。

Harness 同时作用于行动前、行动中和行动后

- **行动前**：校验契约状态、权限、上下文隔离、允许工具和资源预算；
- **行动中**：监控文件范围、命令、依赖变化、超时、Token 和约束命中，必要时阻断或降级；
- **行动后**：执行测试与门禁，聚合 Evidence Bundle，验证回退能力，并把真实遥测送入 Dashboard。

一条可执行约束至少应说明检查什么、何时检查、失败怎样处理、谁负责，以及证据存在哪里。例如：

```
- id: C-SEC-003
  rule: "Agent 不得读取或输出生产凭证"
  level: MUST
  gate: pre_action
  check: "python scripts/check_secret_access.py"
  watch_paths: ["config/**", ".env*"]
  on_failure: block_and_escalate
  owner: Security Owner
  evidence: "evidence/security/secret-access.json"
```

`on_failure` 不能统一写成“报错”。不同风险可以选择自动修复、阻断、回滚、降级、隔离或升级人工；例外必须有批准人、原因和到期时间。Harness 还需要恢复演练，否则“可以回滚”只是一项未经验证的愿望。

从约束到五道门

在 SCOPE-V 中，Harness 应把约束分布到控制回路，而不是只在最后检查：前置门确认契约、签署、约束和 AC 已准备；编码门确认测试先写并产生可信 Red；验证门检查测试、AC、构建、NFR 与证据；收尾门审计 Evidence Bundle、单任务遥测、Dashboard 和图谱回写；Bug 回溯门在完成事实被生产缺陷推翻时修正证据、度量与组织记忆。V 是独立裁决点，但 V 所审查的证据由此前运行与门禁真实产生。五道门裁决控制状态转换，不是五个额外阶段。

报销案例中，Harness 可以强制金额 AC 在前置门存在，限制实现 Agent 只能修改规则模块，禁止删除边界测试，监视新依赖和敏感数据访问，规定最多两轮自动修复，并要求发布前同时具备回归报告、性能结果、代码差异和回退证明。即使 Agent 判断“代码看起来没问题”，也无法绕过这些条件自行上线。

Harness Engineering 贯穿整个 SCOPE-V，并综合六大支柱：它不仅是“约束与恢复”这一根支柱，还要借助上下文、工具、编排、状态记忆和评估观测，才能形成真正的执行环境。

常见误区包括：只有规则文档没有自动检查；只在 CI 最后设门，执行过程完全不可见；工具白名单允许任意 shell 和任意路径；每种失败都无限自动重试；例外永久有效；检查通过却没有保存真实输出和来源。

8.4.5 四类工程与六大支柱的对应关系

四类工程不是另外四根支柱，也不是四套独立平台。每类工程都有主要建设对象，同时会调用其他支柱形成闭环：

工程方法	主要建设的支柱	需要协同的支柱	典型产物
Context Engineering	上下文管理	状态与记忆、约束与恢复	上下文切片、来源与版本、新鲜度和隔离检查
Graph Engineering	执行编排	工具系统、状态与记忆、评估与观测	Work Graph、节点契约、资源关系和失败路由
Loop Engineering	评估与观测、状态与记忆	执行编排、约束与恢复	失败分类、重证记录、预算、停止与反馈入图
Harness Engineering（狭义）	约束与恢复、工具系统	上下文管理、执行编排、评估与观测	Policy as Code、Tools Manifest、Gate、Recovery 与审计证据

广义 Harness 是六大支柱共同形成的完整控制环境；狭义 Harness Engineering 是其中一种建设方法。Context、Graph 和 Loop 负责让信息、协作与反馈可治理，狭义 Harness Engineering 负责把关键边界落实现成机器不能静默绕过的控制。

8.5 一个任务中的四类工程接力

仍以报销规则为例，四类工程的协同可以展开为一条完整链路：

- Context Engineering 从签署契约、约束和代码库中，为测试节点裁剪金额边界、币种规则与允许范围。
- Graph Engineering 确认测试节点先行，安全分析可以并行，实现节点必须等待可信 Red，集成与证据节点在后方汇合。
- Harness Engineering 在节点启动前检查上下文隔离、工具权限和预算；在执行中阻止越权文件修改与删除测试。
- Loop Engineering 根据 Red 和约束信号进入 $P \rightleftharpoons E$ ，完成最小修复，重跑同一检查，并在超预算或目标冲突时升级人工。
- Graph 根据节点状态释放集成验证；Context 随之切换为代码差异、测试结果、NFR 与发布约束。
- Harness 聚合各门证据，V 独立审查契约命中、约束、新鲜度和恢复条件；人类作出上线裁决。
- 生产遥测进入契约级和全局 Dashboard，经 LOOP-3 反馈到下一轮 Intent Graph、约束、路由和上下文策略。

四类工程的职责边界可以概括如下：

工程方法	核心问题	SCOPE-V 主要落点	关键工件或机制	缺失后的典型风险
Context Engineering	Agent 此刻应知道什么	S/C/O，并在 P/E/V 交接时刷新	上下文切片、来源与版本、隔离检查	用错目标、过期资料、信息泄露
Graph Engineering	谁在何时依赖谁完成什么	以 O 为中心，覆盖全环	Org Graph、Work Graph、节点契约、关键路径	冲突并行、错误汇合、责任不清
Loop Engineering	偏差如何收敛并停止	LOOP-1、 $P \rightleftharpoons E$ 、Telemetry 外环	失败分类、反思记录、重试预算、反馈入图	无限重试、移动判据、问题复发
Harness Engineering	哪些动作获准，怎样证明与恢复	贯穿 SCOPE-V，各门实施	约束即代码、Tools Manifest、Gate、Evidence、Recovery	文档护栏被绕过、越权、无法回退

可以用一句话记忆：**Context** 决定 **Agent** 能看见什么，**Graph** 决定工作怎样流动，**Loop** 决定偏差怎样收敛，**Harness** 决定系统允许发生什么。

8.6 轻量模式与完整模式：按风险裁剪

不是所有项目都需要相同治理重量。

单人单模块项目也应保留四大动态工件的语义，但不必部署复杂平台：用一份最小 Intent Graph 或项目上下文索引提供方向，以意图契约、约束矩阵和 Evidence Bundle 管理当前任务。多人、多工具、多模块项目再增加 Protocol、模块治理、跨模块契约、集成 DAG 和证据聚合。

治理复杂度应与以下因素匹配：

- 变更影响范围；
- 数据敏感度；
- 操作可逆性；
- 业务关键性；
- 参与者和模块数量；
- 自动化验证能力；
- 当前团队成熟度。

8.7 风险驱动的最小建设路径：先闭环，再扩大自治

团队不应同时建设六大支柱的所有高级能力。推荐顺序：

契约与核心约束

→ 可执行 AC 和测试

→ Evidence Bundle

→ 上下文与工具权限

→ Work Graph 和多 Agent

→ 遥测与组织级价值效能视图

先建立完成闭环，再扩大自治范围。没有证据基础时引入更多 Agent，只会增加不可见复杂度。

上面的顺序是通用起点，不是所有团队必须照搬的成熟度阶梯。实际优先级应由能力缺口和风险决定：

不同团队的第一短板并不相同。可以用四个问题判断优先级：

1. 团队最常在哪个阶段发现错误？越晚发现，越应优先前移相应门禁。
2. 哪类错误代价最高？高代价风险优先于高频但低影响的摩擦。
3. 哪项证据最难获得？缺失证据往往揭示系统不可观测区。
4. 哪个环节最依赖某个“知道所有事情的人”？这通常是尚未资产化的隐性治理能力。

例如，一个需求经常反复变更的团队，应先强化 Specify 和签署，而不是急于增加 Agent 并发；一个发布后事故多、测试却常常全绿的团队，应优先检查 AC、测试独立性与 Evidence Bundle；一个多人协作频繁冲突的团队，应先建设 Work Graph、所有权和 Handoff 协议。

8.8 常见概念混淆与机制反模式

本章最容易出现的是不是工具不足，而是概念层级混用：

- **六大支柱不是六套平台。** 它们是六种能力域，可以由同一仓库、现有 CI/CD、YAML 与少量脚本共同实现。
- **四类工程不是四根新支柱。** 它们是建设和组合六大能力的方法。
- **狭义 Harness Engineering 不等于广义 Harness。** 前者重点建设约束、工具、门禁与恢复，后者是六大支柱形成的完整环境。
- **证据和遥测不是第二套支柱。** 它们主要属于评估与观测能力，并贯穿其他能力域。
- **Graph 不是额外支柱。** Work Graph 是执行编排的核心工程载体，Intent Graph 则服务于意图与系统对齐。
- **SCOPE-V 不是第五套架构。** 它是三大机制、六大能力和四类工程在任务运行时的控制回路。

常见实施反模式包括：

- 把意图注入理解为复制一段超长 Prompt；
- 允许 Agent 无限自我修复；
- 失败只记录最终结果，不保留原始信号与修复过程；
- HITL 没有预设触发条件，完全依赖人临时观察；
- 六大支柱各自建设，彼此没有任务 ID 和证据映射；
- 只有治理文档，没有机器门禁、权限隔离和恢复演练；
- 为展示多 Agent 而增加不必要节点，导致成本和协调债上升；
- 单人小项目照搬多模块企业治理，使治理成本高于任务风险。

识别这些反模式的共同问题：系统讲得出流程，却无法用真实工件和机器输出证明流程确实运行。

8.9 管理者如何判断 Harness 是否有效

管理者不必审阅每条 Prompt，却应要求系统回答五个问题：

- 本次任务的价值目标和非目标是什么？
- 哪些风险被机器门禁阻断，哪些仍由人承担？
- 当前绿色状态由哪些独立证据支持？
- 发生异常时是否可停止、可恢复、可追责？
- 这次执行给下一次留下了什么可复用资产？

如果系统只能展示生成代码量、调用次数和节省工时，就还没有证明治理有效。Harness 的价值不是让 Agent 看起来更忙，而是让团队能够在更大自治范围内保持判断力。

8.10 本章小结：从最后一个“3”走向 SCOPE-V

3-4-3 的最后一个“3”定义了自治系统必须完成的三项治理动作：从探索中获得经过授权的意图，在执行中通过高频对抗纠正偏差，并在机器不应决定时把问题与证据交还责任主体。

三大机制要真正运行，必须由广义 Harness 承载。唯一正式的六大支柱定义完整 Harness 所需的能力结构；Context、Graph、Loop 与狭义 Harness Engineering 提供建设这些能力的四类工程方法；下一章的 SCOPE-V 则进一步说明它们怎样在一次任务中流转、回环、停止并形成遥测反馈。

本章最小实践：选择一个真实任务，分别写出三大自治机制的当前实现、六大支柱的最危险短板，以及优先补强的一项工程能力。

完成证据：一张“机制—支柱—工程方法”映射图和一个最小改进动作。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，在受控任务中练习 Context、Harness、Loop 与异常升级。

第三部分 | 验证工程与 3-4-3 落地：让操作系统真正运行

如果说 3-4-3 定义了治理系统的角色、工件和运行机制，那么 SCOPE-V 解决的是这些要素怎样在一次真实任务中持续运转。它把“**目标要明确、风险要受控、结果要验证**”从原则变成控制回路，使 Agent 的每一次行动都能找到方向、边界、执行关系、证明方式、修复路径和最终裁决依据。

SCOPE-V 不是另一套六步瀑布流程，而是六个持续运行的控制面：Specify 控制方向，Constrain 控制可行动空间，Orchestrate 控制协作关系，Prove 产生可检查证据，Evolve 根据失败诊断、修复和重证，Verify 保持独立视角并决定证据支持什么行动。Prove 与 Evolve 构成高频快内环，Verify 后的 Telemetry 构成慢外环，把真实质量、风险、成本和价值反馈到下一轮 S、C、O。

本部分分为两组。A 组“SCOPE-V 运行时控制”依次展开规格校准与显式签署、约束工程、Work Graph 与多 Agent 编排、TDD 与多层测试、失败驱动的演进、独立验证与上线裁决；随后把度量、遥测、契约级与全局 Dashboard 纳入反馈系统，并连接现有 CI/CD、分支、版本、构建、发布和回滚机制。B 组“3-4-3 工程化落地与扩展”再进入新项目与既有系统、Knowledge & Memory Engineering，以及多人、多工具、多模块协作。

读完本部分，读者不只应当“知道 SCOPE-V 的六个字母”，还应能够为一个任务建立可签署契约、可执行约束、可失败的验证、可收敛的修复回路和可审计的 Evidence Bundle，并理解为什么测试全绿、扫描通过或模型自评良好，都不能单独等同于可信发布。

A. SCOPE-V 运行时控制

第九章 | SCOPE-V：人机协同研发的持续控制回路

前面的三大理论解释了 3-4-3 为什么需要外部化认知、建立反馈并治理复杂协作；三大自治机制则说明系统怎样获得意图、纠正偏差并把机器不该决定的事情交还人类。但这些原则还需要一套能够真正运行的控制架构。SCOPE-V 正是连接方法论与工程执行的中枢。

本章先给出 SCOPE-V 的整体结构，再讨论它与传统 SDLC 的关系、六个控制面、快内环与慢外环、机械门禁，以及三大自治机制怎样通过这条控制回路落地。后续各章再分别展开 Specify、Constrain、Orchestrate、Prove、Evolve，以及 Verify 与 Telemetry。

9.1 它不是六步瀑布，而是六个持续运行的控制面

第一次看到 SCOPE-V，人们很容易把它读成另一套六阶段流程：先 Specify，再 Constrain，然后 Orchestrate、Prove、Evolve、Verify，仿佛把需求、设计、开发、测试换了一组英文名称。

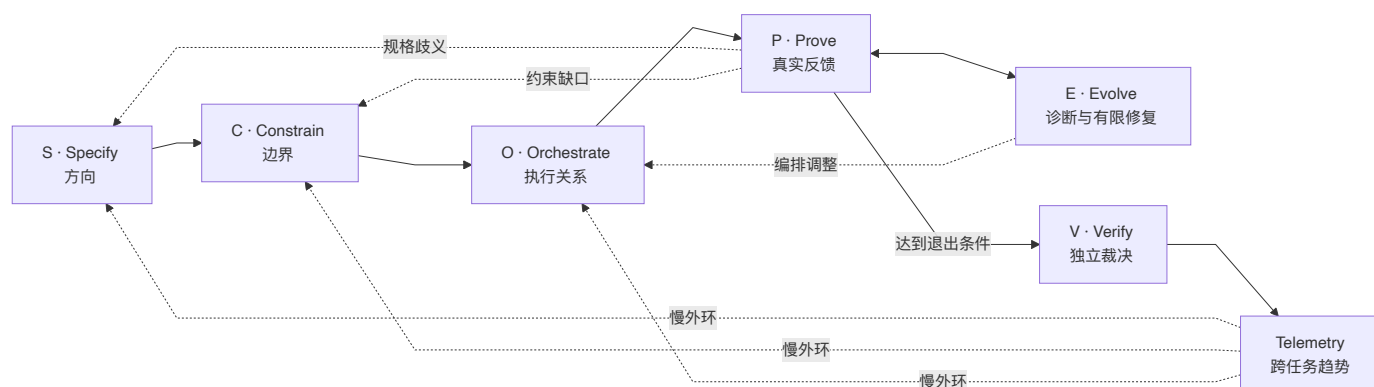
这种理解会错过它最重要的设计。SCOPE-V 不是六个只能依次走完一次的箱子，而是六个在任务生命周期中持续发挥作用的控制面：

- **Specify** 持续回答目标是否仍然明确、规格是否发生漂移；
- **Constrain** 持续检查权限、数据、质量和过程边界；
- **Orchestrate** 根据依赖、失败和资源状态动态调整执行关系；
- **Prove** 不断用测试与可执行证据挑战候选结果；
- **Evolve** 对失败进行诊断、修复、重构与知识回写；
- **Verify** 保持独立视角，决定现有证据支持哪一种行动。

Telemetry 汇聚任务执行中产生并经 Verify 确认的信号，在跨任务尺度形成慢速外环，把趋势反馈到下一轮 S、C、O。于是 SCOPE-V 的真实结构不是：

第一步 → 第二步 → 第三步 → 第四步 → 第五步 → 第六步 → 结束

而更接近：



在整个过程中，S、C、O 的结果仍然约束 P 与 E；P 暴露的失败可能要求回到 S 重新澄清、回到 C 增加边界，或回到 O 调整 Work Graph。它是一个可回流、可停止、可重新签署的控制系统，而不是把所有变化都塞进当前步骤继续向前。

9.2 SCOPE-V 与传统 SDLC 到底有什么不同

SDLC（Software Development Life Cycle，软件开发生命周期）回答的是软件从构想到运行通常经历哪些生命周期活动。不同组织可以采用瀑布、迭代、敏捷、DevOps 或持续交付，但仍然需要需求、设计、实现、测试、部署和运维。

SCOPE-V 不否定这些活动，也不试图再发明一套项目管理生命周期。它解决的是另一个问题：当执行主体中出现能够高速生成、调用工具、并行协作和自主修复的 Agent 时，怎样在运行时持续控制它的方向、权限、反馈和责任。

比较视角	传统 SDLC 主要关注	SCOPE-V 额外关注
核心对象	软件产品的生命周期活动	Agent 行为及其治理状态
执行主体	以人类团队和确定性工具为主	人类责任主体 + 概率性 Agent
需求入口	文档、Backlog、评审和沟通	可签署意图契约与规格漂移控制
流程推进	阶段、迭代或流水线状态	证据、门禁、停止和回流条件
质量反馈	测试、评审、CI/CD、运行监控	$P \rightleftharpoons E$ 快内环、独立 V、证据映射
权限治理	角色权限和流程审批	任务级工具能力、最小权限、越权阻断
完成判断	阶段完成、故事完成、发布成功	契约、约束与 Evidence 支持单任务裁决；Telemetry 支持跨任务调整
学习方式	复盘、事故分析、流程改进	Telemetry 外环回写图谱、约束与编排

二者可以叠加。一个采用 Scrum 的团队，可以在每条高风险用户故事内部运行 SCOPE-V；一个 DevOps 团队，可以把五道门嵌入现有 CI/CD；一个传统阶段式项目，也可以用意图契约、约束矩阵和 Evidence Bundle 治理 Agent 执行。

因此更准确的说法是：**SDLC 描述软件怎样走过生命周期，SCOPE-V 描述智能体在每次行动中怎样被持续控制**。前者更像路线，后者更像车辆上的方向、刹车、仪表、驾驶辅助和事故恢复系统。

9.3 为什么人机协作时代需要这样的控制回路

人类开发者与 Agent 的差异，不只是速度。Agent 还具有四个会改变治理设计的特征。

第一，**执行速度快于人类审查速度**。一个人在准备评审时，Agent 可能已经修改几十个文件。如果反馈仍然只发生在阶段末尾，方向性错误会快速扩散。

第二，**输出具有概率性**。同一个 Prompt 在不同上下文、模型或时间可能产生不同结果。仅靠“流程执行过”不能证明行为正确，必须要求真实测试、约束结果和可复查证据。

第三，**Agent 能够行动，而不只是建议**。当它能读写仓库、调用数据库、访问网络和执行发布命令时，错误不再停留在文本层面。Constrain 与 Tools Manifest 必须成为运行时边界。

第四，**多 Agent 会放大协调复杂度**。每个 Agent 都可能局部合理，但 Handoff、共享文件、上下文版本和目标理解可能互相冲突。Orchestrate 必须把协作关系变成可执行 Graph。

于是传统“人先理解—人再编码—人最后评审”的隐性控制，必须转变为显式的机器可执行控制：目标写进契约，风险写进约束，协作写进图，成功写进测试，判断依据写进证据，系统表现写进遥测。

9.4 六个控制面分别控制什么

9.4.1 S：Specification——控制方向

S 不只是“写需求”。它把模糊意图转成 Goal、Non-goals、Rules 和可执行 AC，并由 IO 显式签署。它控制的是系统朝哪个目标前进，以及什么变化已经构成规格漂移。

S 的典型工件是 Intent Contract、Example Mapping 和签署记录。没有这些工件，Agent 很容易把讨论、建议或历史规则误当成当前授权。

9.4.2 C：Constraints——控制可行动空间

C 把法律、安全、数据、架构、质量、工具和过程边界转成可执行规则。它不告诉 Agent 如何实现，而是限制哪些实现路径不可接受。

风险只有转换为规则、测试、权限或恢复动作，才真正进入控制系统。“注意隐私”不是控制；“Agent 不得读取生产报销明细，触发即阻断并升级”才是控制。

9.4.3 O：Orchestration——控制执行关系

O 用 Work Graph 定义节点、依赖、并行、超时、重试、Handoff 和停止条件。它控制的不只是先后顺序，还包括失败后返回哪里、哪个结果可以被下游信任、什么时候需要人类。

传统任务列表告诉团队“要做什么”；Orchestration 还要让运行时知道“当前谁能做、拿什么输入做、做到什么程度算完成”。

9.4.4 P：Proof——控制反馈真实性

P 的目标不是生成“看起来正确”的实现，而是让候选结果接受挑战。AC、TDD Red、运行时断言、边界测试、安全检查和性能基准都属于证明机制。

证据先于结论。单一 LLM-as-Judge 或 Agent 自述不能作为成功证明，因为同一个误解可能同时影响实现和评价。关键结论需要可复现运行证据，必要时还需要独立来源。

9.4.5 E：Evolution——控制修复与学习

E 接收 P 暴露的失败，判断它属于实现错误、规格冲突、约束缺失、编排问题还是证据不足。只有在授权边界内，系统才允许进行有限修复；超出范围则回流到 S、C、O 或触发 HITL。

E 不只是把测试改绿，还要保留失败、修复和重验记录。尚未裁决的发现先进入 Evidence Bundle 与 Loop Memory，经过 V 确认且具有复用价值后，才能晋升为图谱、约束或测试资产，避免把一次猜测写成组织事实。

9.4.6 V：Verification——控制最终裁决

V 站在执行与自愈内环之外，独立检查契约、约束、实现、证据和时效是否一致。它不生产用于证明自己的证据，而是审查证据并决定：批准、条件批准、补证、缩小范围、拒绝或升级。

V 的独立性非常重要。如果负责修改结果的同一 Agent 同时决定证据是否充分，就可能出现自我证明。Verify 可以使用自动化工具，但价值取舍、不可逆操作和残余风险仍由人类责任主体裁决。

9.5 为什么 E 必须在 V 之前

“为什么不先 Verify，再 Evolve？”这是 SCOPE-V 架构中很有价值的问题。

因为 P 已经承担了执行内环中的失败发现。P 暴露测试失败、约束拦截或证据缺口后，E 负责在当前授权范围内诊断和修复，再回到 P 重新取证。这个 $P \rightleftharpoons E$ 回路可以快速运行多次：

P: 边界测试失败
→ E: 判断为实现错误, 完成最小修复
→ P: 重跑同一测试并取得新证据
→ E: 检查是否需要反思与知识回写
→ 满足退出条件后提交 V

V 的职责不是参与每一次试错, 而是在内环达到退出条件后进行独立审查。如果把 V 放在 E 前面并让其参与修复, 裁判就进入了比赛: 验证者既发现问题、又修改结果、再宣布通过, 独立性会被削弱。

E 在 V 之前还带来三个好处:

1. **减少人类审查噪声**: 可在边界内自动修复的普通错误先被收敛;
2. **保留失败历史**: V 看到的不只是最终绿灯, 还包括系统怎样从红到绿;
3. **明确退出条件**: 只有测试、约束、证据和修复记录达到要求, 才进入裁决。

但 E 不能无限运行。超过重试、Token 或时间预算, 必须修改契约、降低约束, 或出现证据矛盾和不可逆风险时, 内环立即停止并交给人类。

9.6 快内环: $P \rightleftharpoons E$ 的诊断与自愈

快内环针对当前契约、当前上下文和当前执行结果运行。它的时间尺度可以是秒到分钟, 目标是让局部偏差尽早收敛。

快内环必须具备:

- 明确失败信号, 而不是模型觉得“不太好”;
- 失败分类与路由;
- 最小修复范围;
- 原检查重跑;
- 最大轮次与资源预算;
- 失败和修复证据;
- 超界后的 HITL 条件。

费用报销任务中, 5000.00 元边界测试失败。E 判断为比较符号错误, 修改一行实现, 再由 P 重跑相同边界测试。如果 E 发现财务制度与签署契约冲突, 它不能选择一边修复, 而应停止内环, 返回 S 请求 IO 重新决策。

9.7 独立裁决: V 的审计与放行

当快内环达到退出条件, V 执行三角验证:

Intent Contract: 做的是不是被批准的事?
Constraint Matrix: 执行是否守住边界?
Evidence Bundle: 结论是否由真实、当前、足够独立的证据支持?

V 还检查工件版本、代码提交、UNKNOWN 项、回退准备和人工批准。它可以得出“测试通过但暂不发布”, 也可以给出带灰度、时间和监控条件的批准。

9.8 慢外环: Telemetry → S/C/O 的组织学习

V 之后的 Telemetry 形成慢速外环。它不是修复当前一行代码, 而是观察多个契约后的累积趋势: Token 是否持续上升、首次成功率是否下降、HITL 是否集中在某类问题、约束自愈是否改善、知识沉淀是否减少重复失败。

这些趋势反过来改变下一轮控制面:

- 目标准确率下降，回到 S 检查意图和契约质量；
- MUST 失败增加，回到 C 检查约束表达与工具权限；
- Token、轮次和 Handoff 摩擦上升，回到 O 调整上下文和编排；
- 重复出现同类失败，回到 E 改进诊断与修复策略，并把经验证的模式更新到图谱和组织知识。

快内环保证当前任务收敛，慢外环保证整个系统不会在长期中退化。前者像即时纠偏，后者像根据连续航行数据改进航线、规则与训练。

9.9 每个控制面都必须产生可检查工件

控制面不能只停留在概念层。每一面至少要有可读取、可版本化、可审计的产物：

控制面	关键工件或证据
S	Intent Contract、Example Mapping、IO 签署
C	Constraint Matrix、constraints.yaml、Tools Manifest
O	Work Graph、节点契约、Handoff 记录
P	TDD Red/Green、AC 运行结果、NFR 检查、Evidence Bundle 原始证据
E	失败分类、修复与重验记录、Loop Memory、Evidence Bundle 更新
V	三方一致性审计、Evidence Bundle 裁决状态、发布或退回决定
Telemetry	单任务事件与度量、聚合 Dashboard、跨任务趋势

这就是“每节必产工件”的含义：没有工件，控制状态无法被机器检查；没有原始证据，人类只能相信 Agent 的叙述。

9.10 五道机械门把控制状态变成不可跳过的事实

五道机械门进一步防止系统跳过关键状态。它们是状态转换的机械裁决点，不是把 SCOPE-V 重新切成五个线性阶段：

前置门：契约、IO 签署、约束和 AC 是否存在？

编码门：测试是否先写并产生可信 Red？

验证门：测试、AC、构建和证据是否通过？

收尾门：Evidence Bundle 是否可裁决，本次真实信号是否已进入单任务遥测与 Dashboard，合格知识是否回写？

Bug 回溯门：完成后发现问题，是否如实修正证据和组织记忆？

Agent 不能仅凭自然语言声明“我已经完成前置检查”。门禁需要读取真实工件、时间、版本和运行结果。

收尾门检查遥测入口是否完整，不负责用一次任务解释长期趋势。组织层结论必须等待多个可比任务按 Measurement Contract 聚合后，才能进入 Telemetry 慢外环。

9.11 一个完整控制回路示例

仍以“超过 5000 元的报销增加总监审批”为例：

1. S 把“超过”、金额口径、适用范围和异常路径写成签署契约；
2. C 禁止访问生产明细、禁止修改历史已审批单，并规定回退开关；
3. O 安排边界测试先于实现，组织关系检查与回退准备可并行；

4. P 先证明 5000.00 与 5000.01 的测试在旧实现上失败；
5. E 完成最小实现，处理代理总监冲突，并把候选规则及来源写入 Evidence Bundle；
6. P 重跑原测试和约束，生成当前提交的证据；
7. V 独立检查契约、约束、证据、UNKNOWN 与回退，批准两个部门灰度，并将已确认规则晋升为组织知识；
8. Telemetry 记录首次成功、修复轮次、HITL、Token 和知识沉淀；
9. 多个契约后发现代理关系类 HITL 持续偏高，外环驱动 S 增加前置问题、C 增加组织关系约束、O 提前安排领域检查。

这个例子说明 SCOPE-V 的完成点不是“代码写完”，而是当前任务已经由证据支持裁决，并且执行经验已经进入下一轮控制。

9.12 SCOPE-V 设计的四条底线

最后可以用四条原则概括其控制论立场：

1. **证据先于结论**：不能以“看起来正确”或单一 LLM Judge 作为成功证明；
2. **风险转化为控制**：热点名词必须落为规则、测试、权限、门禁或恢复动作；
3. **价值判断不可让渡**：AI 可以有限自治，但目标、公平、伤害取舍和不可逆操作由人负责；
4. **每个控制面必须可检查**：不堆砌概念，每一面都要产生工件、矩阵、模型、策略或证据。

因此，SCOPE-V 的价值不在于记住六个英文单词，而在于为概率性、高速度、可行行动的 Agent 建立一套能够持续校准、受控修复、独立裁决并长期学习的工业级自循环模型。

9.13 三大自治机制怎样通过 SCOPE-V 落地

三大自治机制与 SCOPE-V 处在不同的抽象层，不能简单一一对应，也不应被理解成两套并列框架。

- **三大自治机制**描述系统怎样运行：如何获得意图、如何纠偏、何时把决定交还人类；
- **SCOPE-V**描述运行时的控制面：目标、边界、编排、证明、演进与裁决分别由什么环节承担；
- **六大 Harness 支柱**提供工程条件：上下文、工具、图、状态、评估、约束与恢复怎样真正执行。

可以把它们理解为“运行原则—控制回路—基础设施”的关系：

三大自治机制：系统以什么方式自治
↓ 借助
SCOPE-V：自治在哪些控制面中发生
↓ 依托
六大 Harness 支柱：控制怎样被机器执行和观测

因此，SCOPE-V 不是三大机制之后才启动的一段流程，也不是额外的第四个自治机制。每一种机制都横跨 SCOPE-V 的多个控制面。

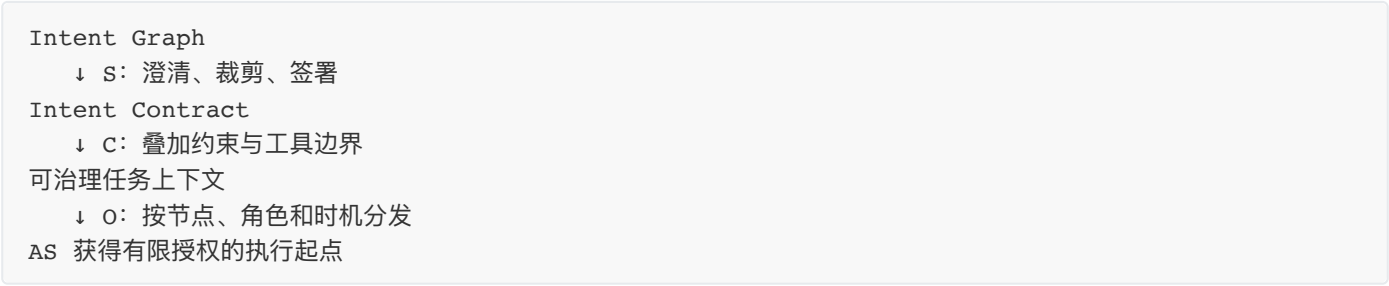
9.13.1 机制一：从意图注入到可执行起点

意图注入主要发生在 S、C、O 三个控制面，但它不是简单地“依次填三份表”。

Specify 控制注入什么目标。 OA 从 Intent Graph 中裁剪相关背景，通过批判性思维、Grill-Me 和 Example Mapping，把 IO 的模糊意图转成 Goal、Non-goals、Rules 和 AC。IO 对完整 Intent Contract 显式签署，建立执行授权。

Constrain 控制注入哪些边界。 已签署契约只说明本次要做什么，Constraint Matrix、AI Coding Guide 和 Tools Manifest 还要说明不能做什么、可以使用什么工具、遇到风险怎样停止。目标与边界共同组成 Agent 的任务上下文。

Orchestrate 控制把意图注入给谁、在什么时候注入。 OA 不会把整个项目知识库复制给所有 Agent，而是根据 Work Graph 为不同节点裁剪不同 Context。测试 Agent 获得 AC、接口和约束；实现 Agent 获得签署规则、可信 Red 和允许修改范围；验证节点获得原始证据，但不必继承实现 Agent 的全部推理。



前置门是这套机制的机械保证。契约不存在、IO 未真实签署、约束缺失或 AC 不可执行时，意图注入就没有完成，Orchestrate 不得进入业务编码。

9.13.2 机制二：P⇌E 快内环中的高频对抗与自净化

高频对抗与自净化最集中地体现在 P⇌E 快内环中，但 S、C、O 始终在外围提供目标、边界和执行结构。

Prove 负责制造可信挑战。 它从签署 AC 生成测试，通过 TDD Red 证明测试能够发现缺失行为，再用边界、异常、安全、性能和约束检查挑战候选实现。P 不是为了尽快得到绿灯，而是为了尽早暴露偏差。

这里的“挑战”不仅包括功能测试。SAST、SCA、Secret Scan、IaC Scan、DAST、Fuzzing，以及经过授权的红队攻击，都可以成为 P 的误差信号；E 根据发现完成代码、配置、权限、检测与恢复能力的最小修复，再由 P 使用原规则、原攻击路径或等价强度的测试重新取证。区别在于反馈速度与独立性：SAST 等确定性扫描可以高频进入每次提交，DAST 需要可运行环境，红队与渗透测试还需要独立授权、隔离范围和证据保全，不能全部压成同一种自动循环。

Evolve 负责分类与受控修复。 同样一个测试失败，可能来自实现错误、规格歧义、约束冲突、依赖异常或证据缺口。E 先判断失败应该回到哪里，再决定是否允许 AS 在当前授权范围内最小修复。

修复后必须回到 P 重新取证。 E 不能自己声称已经修好，也不能换一个更宽松的检查。原测试、原约束和相关回归必须再次运行，形成从 Red 到 Green 的证据链。



如果失败需要改变契约，就回到 S；需要放宽或新增红线，就回到 C；需要调整依赖、超时或责任节点，就回到 O；涉及不可逆操作或超出自治预算，就转入机制三的人类裁决。

因此，高频自净化不是只属于 E。P 提供真实误差信号，E 负责纠偏，S/C/O 决定纠偏还能否在原授权内继续。三者共同保证内环快速但不失控。

9.13.3 机制三：人类裁决是横跨全程的异常出口

人类异常裁决不只发生在最后的 Verify。它是 SCOPE-V 每个控制面都可以触发的异常出口：

控制面	典型 HITL 触发	人类需要决定什么
S	目标冲突、价值取舍、规则歧义	做什么、不做什么、怎样算成功
C	法律安全冲突、权限扩大、约束例外	哪条边界优先、是否授权例外
O	多模块责任冲突、不可安全回滚	怎样重排、暂停或拆分任务
P	证据互相矛盾、自动判断不确定	还需要什么独立验证
E	超过重试预算、修复需要改契约	停止、重签、降级还是放弃
V	残余风险、灰度范围、生产发布	批准、条件批准、退回或拒绝
Telemetry	长期退化、成本异常、HITL 激增	收缩还是扩大自治范围

Verify 是人类裁决最集中的位置，因为它负责独立审查与发布建议；但如果把所有问题都拖到 V 才找人，Agent 可能已经在错误方向上消耗大量算力并扩大修改范围。

HITL 的设计原则是“事件触发”，不是“阶段打卡”。系统一旦检测到需要人承担的价值、法律、伦理、不可逆或例外责任，就应冻结相关节点、保存 Evidence、说明可选方案，并把决定交给正确责任人。

9.13.4 Verify 与 Telemetry 如何把三个机制闭成完整回路

$P \rightleftharpoons E$ 收敛后，Verify 不属于高频自净化的一部分，而是站在内环之外检查三个问题：

- 1. 注入的意图是否仍与签署契约一致；
- 2. 自净化过程是否始终遵守约束，没有通过弱化标准获得全绿；
- 3. 所有需要人类裁决的事件是否由正确责任主体处理并留下记录。

Verify 通过后，Telemetry 将三大机制的运行质量变成数据：

- 目标准确率反映意图注入是否有效；
- 首次成功率、修复轮次和自愈率反映高频对抗是否收敛；
- HITL 升级率与类型反映人类裁决是否发生在正确位置；
- Token 与执行效率反映自治成本；
- 知识沉淀率反映失败是否真正反哺下一轮。

这些数据通过慢外环返回下一轮 S、C、O。若阈值歧义反复触发 HITL，S 应在 Grill-Me 中提前询问；若越权失败增加，C 应收紧 Tools Manifest；若 Token 和交接摩擦上升，O 应调整上下文裁剪与 Work Graph。



9.13.5 一张映射表看懂三者关系

自治机制	主要 SCOPE-V 控制面	核心工件与证据	关键机械门
意图注入	S、C、O	Intent Graph、Intent Contract、Constraint Matrix、任务 Context、Work Graph	前置门
高频对抗与自净化	O、P、E	TDD Red/Green、AC 结果、约束失败、修复记录、Loop Memory	编码门、验证门
人类异常裁决	贯穿 S/C/O/P/E, 集中于 V	决策记录、例外批准、Evidence Bundle、发布裁决	前置门、验证门、收尾门、Bug 回溯门
长期反馈（贯穿三机制）	Telemetry → S/C/O	单契约遥测、Dashboard、趋势与知识回写	收尾门、Bug 回溯门

这张表也解释了为什么三大机制不能离开 SCOPE-V 单独运行。没有 S/C/O，意图注入只是一次 Prompt；没有 P/E，高频对抗只是反复生成；没有独立 V 和 HITL，人类裁决只是最后盖章；没有 Telemetry，组织无法知道三种机制是在改善还是退化。

9.14 不要把三大机制机械映射成三个阶段

理解三大机制与 SCOPE-V 的关系后，还要避免三个看似直观但错误的映射。

9.14.1 误解一：意图注入只等于 Specify

Specify 负责形成签署目标，但如果没有 Constrain 提供边界、没有 Orchestrate 按节点裁剪并分发 Context，Agent 仍然只是拿到一份较长的需求文档。完整意图注入至少覆盖 S、C、O，并由前置门确认可以进入执行。

9.14.2 误解二：高频对抗只等于 Prove

Prove 负责产生反馈，Evolve 负责诊断与修复，随后必须回到 Prove 重验。与此同时，S/C/O 仍然规定修复目标、允许范围和预算。只有 P 没有 E，系统只会不断报错；只有 E 没有可信 P，系统可能在错误信号上自我优化。

9.14.3 误解三：人类裁决只等于 Verify

Verify 是发布裁决的集中位置，但人类责任贯穿全部控制面。目标冲突应在 S 解决，约束例外应在 C 解决，不可安全回滚的编排应在 O 解决，超过修复预算应在 E 停止。把全部问题拖到 V，只会让早期错误以更高成本到达终点。

正确理解不是寻找一张简单的一对一对应表，而是看每种机制怎样借助多个控制面完成闭环。三大机制定义运行原则，SCOPE-V 将原则分解为可执行控制，Harness 再把控制落实为工具、权限、门禁与证据。

9.15 三大机制如何通过 SCOPE-V 处理同一次失败

费用报销任务中，P 的边界测试发现 5000.00 元错误进入总监审批。

- 意图注入机制提供判定基准：**S 中的签署契约明确“严格大于 5000 元”，C 禁止通过修改契约或删除测试获得通过；
- 高频对抗机制产生反馈：**P 保存失败输入、期望、实际值和测试日志；
- E 执行自净化：**判断为实现比较符错误，在原文件范围和轮次预算内完成最小修复；
- P 重新取证：**重跑 4999.99、5000.00、5000.01 三个边界测试及相关回归；
- 若发现制度冲突则触发机制三：**系统停止，OA 整理旧制度、当前契约和失败证据，由 IO 与财务责任人决定是否重签；

6. **V 独立裁决**：确认最终实现没有改变签署目标，自净化未突破约束，证据属于当前提交；

7. **Telemetry 记录机制表现**：首次成功率、修复轮次、自愈次数、HITL、Token 和知识回写进入单契约 Dashboard。

同一个失败在这里不是“测试失败—改代码—测试通过”三步，而是三大自治机制通过 SCOPE-V 协同运行的一次完整治理事件。

9.16 本章小结

SCOPE-V 不是一套替代 Scrum、DevOps 或 SDLC 的六阶段流程，而是面向概率性 Agent 的运行时控制架构。S、C、O 建立方向、边界和执行关系， $P \rightleftharpoons E$ 用真实反馈形成快速纠偏内环，V 保持独立裁决，Telemetry 再把多次任务的运行信号反馈给下一轮治理。三大自治机制由此不再停留在原则层，而是进入可执行、可停止、可审计和可进化的控制系统。

本章最小实践：选择一项真实 AI Coding 任务，画出它的 SCOPE-V 控制回路，并标出三大自治机制、 $P \rightleftharpoons E$ 快内环、HITL 异常出口、五道门和 Telemetry 慢外环。

完成证据：一张带工件、门禁、回流与责任人的 SCOPE-V 运行图。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，完整跑一遍 SCOPE-V，并观察每个状态如何产生下一步证据。

第十章 | Specify：从模糊需求到可签署意图

10.1 需求本身就是第一类治理对象

很多 AI 项目从收到需求后才开始治理：规定编码规范、限制工具权限、要求测试覆盖。但如果需求本身模糊、冲突或不可验证，后续执行得越严格，越可能精确地产生错误结果。

Specify 的任务不是把用户原话改写得正式，而是发现字面需求背后的真实意图、假设和风险，并把探索性对话转化为组织愿意承担责任的执行授权。它优化的不是 Prompt，而是“这一次究竟允许人机系统改变什么”的共同承诺。

常见可疑信号包括：

- 目标只有动作，没有业务结果；
- 成功标准使用“优化一下”“更智能”等不可判定词语；
- 需求同时包含过多激励，模型可能通过钻空子刷指标；
- 范围看似很小，却要求修改高风险核心模块；
- 权限要求远大于任务所需；
- 多个业务规则互相冲突；
- 要求跳过验证、审计或人工批准；
- 所谓“完成”无法被独立验证。

OA 的职责是先校准，再执行。盲从 IO 的第一句话，不是尊重需求，而是放弃治理。

10.2 批判性思维与 Grill-Me：从发现问题到帮助决策

Grill-Me 不是一套孤立的提问技巧。它与批判性思维组成 Specify 阶段的前后两步：**批判性思维**负责发现什么值得质疑，**Grill-Me** 负责把质疑转化为 IO 能够逐项作出的决定。

如果没有批判性思维，Grill-Me 很容易退化成模板问卷，机械地询问技术栈、时间和颜色，却没有触碰真正会改变价值与风险的问题；如果只有批判性思维而没有 Grill-Me，OA 又可能只会指出“需求不合理”，把问题原样退回给 IO。

批判性思维不是反对用户

这里的“批判”不是挑错、抬杠或证明 OA 更专业，而是拒绝把用户第一句话直接等同于实现规格。IO 往往最了解业务目标，却不一定在第一句话里说清所有边界；OA 最了解系统影响，也不应替 IO 作价值判断。两者需要通过结构化挑战共同形成可执行意图。

OA 至少要检查七类可疑信号：

- 1. 自然语言歧义：一句话存在两种以上合理解释；
- 2. 不可验证：使用“更智能”“体验更好”等无法判定的表达；
- 3. 与北极星或既有契约冲突：局部需求损害全局目标；
- 4. 激励或凭证过密：系统可能为了刷指标制造大量低价值动作；
- 5. 权限过宽或操作不可逆：任务要求的能力超过目标所需；
- 6. 局部最优、全局劣化：一个功能方便，却显著增加整体复杂度；
- 7. 与刚确认的决定矛盾：规则在短时间内翻转但没有明确取舍。

发现这些信号后，不得直接进入编码。OA 先说明自己理解的意图，再解释按字面执行的风险，并给出推荐默认与备选方案。

Grill-Me 是什么

Grill-Me 可以理解为“逐项追问并辅助决策”。它不是一次抛出几十个问题，也不是让 IO 自己通读一份复杂模板。每一轮只聚焦一个会改变执行结果的关键问题，同时给出推荐默认值、简短理由和可选回答。

推荐答案很重要。只问“你希望怎么处理外币？”会把全部分析负担推回 IO；给出“推荐本期排除外币，因为汇率口径尚未确认，并将外币请求转入人工队列”，则把 OA 的专业判断变成可评价选项。IO 可以接受、否决或自定义。

一次完整的组合动作

IO 提出：“员工报销超过 5000 元时，自动交给总监审批，明天上线。”

OA 先用批判性思维扫描：

- “超过”的边界不清；
- “5000 元”的金额口径不清；
- “总监”的组织关系不清；
- “自动”可能隐藏异常处理；
- “明天上线”可能压缩验证和回退准备。

然后不是把五个问题一起扔给 IO，而是逐项 Grill-Me：

[Specify] 决策点 1/5：金额口径

【我理解的意图】
对真正意义上的高额报销增加更高级别审批。

【字面实现的风险】
“5000 元”可能指单张发票、报销单总额、含税或未税金额，不同解释会产生不同审批结果。

【推荐】
按整张报销单的含税本位币总额判断。
理由：口径稳定，容易审计，也能覆盖多张发票场景。

【备选】
按单张发票金额判断。

请选择：推荐 / 备选 / 自定义。

IO 确认后进入第二个决策点：

[Specify] 决策点 2/5：边界值

【推荐】5000.00 元不触发，5000.01 元触发。
理由：“超过”通常表示严格大于，也最符合字面语义。

请选择：推荐 / 改为 5000 元及以上 / 自定义。

后续再处理外币、总监缺席和灰度范围。每次确认都会形成一条决策记录，并最终进入规则、Non-goals、AC 或约束。

什么问题值得问，什么问题不值得打断 IO

不是所有未知信息都需要 Grill-Me。判断标准是：答案不同，是否会显著改变价值、范围、风险、验收、权限或不可逆行动？

值得问：

- 等于阈值时如何处理；
- 是否影响历史数据；
- 无法找到审批人时是拒绝、人工处理还是自动通过；
- 是否允许使用生产数据；
- 风险与上线时间冲突时优先什么。

通常不值得立即打断：

- 在既有技术规范内选择变量名；
- 不影响接口和风险的局部实现细节；
- 已经由约束矩阵明确规定的通用规则；
- 能通过低成本实验验证、且可逆的技术假设。

Grill-Me 的目标是集中人类注意力，而不是增加互动次数。

从决策到契约字段

每轮回答都必须落入治理工件，不能停留在聊天记录：

Grill-Me 决策	进入哪里
本次业务结果	Goal
明确暂不处理外币	Non-goals
严格大于 5000 元	Rules + 边界 AC
审批人缺失转人工	异常规则 + AC
禁止读取生产明细	Constraint Matrix
首日只对两部门启用	发布约束 + Evidence Bundle 裁决条件

这一步非常关键。没有落入契约和约束的决定，后续 Agent 很可能无法获得；没有转成 AC 的决定，也难以被证明。

Grill-Me 决策确认不等于契约签署

逐条回答“推荐”只说明若干关键问题已经确认，并不代表 IO 已经审阅完整契约。OA 还要把所有决定汇总为 Goal、Non-goals、Rules、AC、假设、风险和约束，让 IO 检查它们放在一起是否仍然一致。

随后由 IO 显式回复“签署”或填写署名与日期。OA 不得因为 IO 在对话中连续回答了五个问题，就自动把契约标记为 SIGNED。决策确认解决局部选择，签署建立整体授权。

常见反模式

- 把批判性思维理解为一味反对，只有风险没有推荐；
- 一次提出十几个问题，让 IO 承担全部认知负担；
- 推荐答案没有说明理由，变成诱导确认；
- 每个实现细节都请求批准，制造人工瓶颈；
- IO 回答后只保留聊天记录，没有进入契约和 AC；
- 把逐条确认当成整体签署；
- 发现需求与安全红线冲突时仍提供“按原要求实现”选项。

读者可以用三个问题检验一次需求澄清：**OA 是否指出了字面需求的关键风险？是否给出了可验证的推荐默认？确认结果是否真正进入契约、约束和 AC？** 三者缺一，批判性思维与 Grill-Me 就还没有形成闭环。

10.3 从事实与意图图谱裁剪任务上下文

新项目可以从战略材料、业务规则和初始架构建立最小 Intent Graph；既有系统则必须先做只读 Recon，识别 Baseline、Preserve、Unknown、依赖和候选 Change Envelope。代码现状不等于业务意图，但也不能被新需求的语言覆盖。只有把“组织想去哪里”和“系统现在真实是什么”同时放入上下文，Agent 才不会在错误基线上精确执行。

OA 随后从可信事实与意图图谱中选择与当前任务有关的：

- 业务目标；
- 用户角色；
- 现行规则；
- 相关模块；
- 历史事故；
- 长期约束；

- 已知依赖。

裁剪的目标是让当前任务拥有足够信息，同时避免无关信息、敏感信息和旧规则污染上下文。

与第八章的 Harness 定义一致，上下文按四层注入：

```
L1: 任务意图层—目标、用户、价值、签署契约
L2: 组织约束层—Constraint Matrix、Tools Manifest、合规边界
L2+: 工程惯例层—代码结构、接口约定、测试方式、仓库级 AI Coding Guide
L3: 节点切片层—当前 Work Graph 节点所需文件、AC、测试与上游产物
```

L1 和 L2 保持方向与底线稳定，L2+ 保存团队工程常识，L3 随执行节点变化。不同 AI 工具可以采用不同注入方式，但必须消费同一契约版本、约束语义和来源标识；宿主差异不能改变授权边界。

10.4 意图契约就是可执行规约

意图契约不是开工前的汇报文档，而是后续测试、验证和证据的来源。

费用报销案例的最小契约可以是：

```
task_id: T-001
contract_version: 1
status: DRAFT
goal: 将超过 5000 元的境内差旅报销转交部门总监审批
non_goals:
  - 不改变 5000 元及以下审批路径
  - 不处理外币报销
rules:
  - 以报销单含税总额判断
  - 金额严格大于 5000 元时触发
acceptance_criteria:
  - id: AC-01
    description: 5000.01 元进入总监审批
    verify:
      type: command
      command:
        argv: [python, -m, pytest, tests/test_expense_approval.py, -k, amount_5000_01]
        timeout_seconds: 60
  - id: AC-02
    description: 5000.00 元保持原审批路径
    verify:
      type: command
      command:
        argv: [python, -m, pytest, tests/test_expense_approval.py, -k, amount_5000_00]
        timeout_seconds: 60
sign_off:
  intent_owner: "<由 IO 本人填写>"
  signed_at: null
```

这仍是一份待签署草案。只有 IO 审阅完整内容、亲自填写签署区并明确确认后，状态才能从 `DRAFT` 进入 `SIGNED`。OA 或宿主工具不得代签，也不得仅凭 Grill-Me 的逐条回答自动改变状态。

契约格式可以是 Markdown、YAML 或由平台管理的结构化记录，但跨工具语义必须稳定：任务 ID、版本、状态、Goal、Non-goals、Rules、AC 和签署主体不能因为换了 AI 工具而丢失或被重新解释。Prompt 可以从契约生成，不能反过来取代契约。

好的 AC 必须可观察、可判定，并尽量使用运行时验证。仅检查某个字符串是否出现在文件中，通常无法证明业务行为正确。

10.5 Example Mapping 与规格建模

Example Mapping 可以直译为“示例映射”。它是一种用具体例子澄清抽象规则的方法。读者不需要先掌握复杂的需求工程理论，只需要记住一个直觉：一句规则往往允许多种解释，而一个恰当的例子能够迫使大家暴露各自不同的理解。

例如业务说：“高额报销需要总监审批。”团队如果只讨论这句话，所有人都可能觉得自己听懂了。直到分别写出 5000.00 元、5000.01 元、外币报销和总监缺席的例子，隐藏分歧才会出现。

Example Mapping 的四种基本元素

通常使用四类卡片或条目：

- **Story（目标）**：这次希望实现的业务结果；
- **Rule（规则）**：系统必须遵守的一般判断；
- **Example（示例）**：规则在某个具体输入下应产生什么结果；
- **Question（问题）**：团队尚不能决定、必须向 IO 或领域专家确认的事项。

在 Agentic Agile 中还建议显式增加 **Counterexample（反例）**，用于说明哪些相似情况绝对不应触发该规则。

类型	内容
Story	对高额境内差旅报销增加总监审批，同时不改变普通报销路径
Rule	金额严格大于 5000 元进入总监审批
Example	5000.01 元进入总监审批
Counterexample	5000.00 元不能进入总监审批
Question	外币是否按申请日汇率折算？

怎样开展一次最小 Example Mapping

第一步，写出一个业务目标，不急着写解决方案。第二步，把 IO 已经明确的规则列出来，每条规则尽量只有一个判断。第三步，为每条规则至少写一个典型示例和一个边界或反例。第四步，把无法从现有信息推出答案的事项转为问题，而不是让 Agent 猜测。第五步，将已经确认的示例转成 AC 和测试候选。

费用报销案例可能经历这样的对话：

规则：超过 5000 元需要总监审批。
示例：5000.01 元人民币差旅报销 → 总监审批。
反例：5000.00 元人民币差旅报销 → 保持原路径。
问题：5000 美元是否直接按数字 5000 判断？
决定：否，先按提交日财务汇率折算为人民币含税总额。
新示例：1000 美元，汇率 7.2 → 7200 元 → 总监审批。
异常问题：提交时缺少汇率怎么办？
决定：进入人工财务队列，不得自动通过。

可以看到，示例不是为了“举例说明已经确定的规则”，而是用来发现规则还没有确定的部分。每出现一个问题，团队要么找到现有制度依据，要么请 IO 决策；不能把问号直接留给执行 Agent。

从示例走向可执行规格

经过确认的示例可以自然转为 Given—When—Then：

```
Given 一张中国区差旅报销单
And 含税本位币总额为 5000.01 元
When 员工提交报销
Then 审批链应增加成本中心总监节点
And 应记录规则版本 expense-v2
```

这一步把业务语言连接到测试，但不能让测试语法反过来绑架业务讨论。IO 不必会写 Gherkin，OA 负责把业务决定转成可执行形式，再交由 IO 检查语义。

什么时候需要更强的规格建模

Example Mapping 擅长澄清规则和边界。如果系统涉及大量状态变化，例如报销单从草稿、提交、审批、退回、撤销到支付，不同状态允许的操作不同，仅靠例子可能遗漏组合。此时可以引入状态机、决策表、不变量或属性测试。

- **决策表**适合多个条件组合决定一个结果；
- **状态机**适合事件驱动的状态转换；
- **不变量**描述任何时候都不能被破坏的性质，如“同一报销单不得重复付款”；
- **属性测试**自动生成大量输入，检查某个性质的始终成立。

规格不是越复杂越专业。只有当某种模型确实减少歧义、覆盖重要风险并能驱动验证时，才值得使用。对于简单文案修改，写清 Goal、Non-goals 和两个 AC 可能已经足够。

Example Mapping 的常见误区

- 只有正常示例，没有边界和反例；
- 由 OA 或 Agent 替业务决定所有问号；
- 写了大量例子，却没有归纳它们属于哪条规则；
- 示例确认后没有转入契约、AC 和测试；
- 需求变化后只改测试，不同步更新规则与签署；
- 把工具形式看得比业务讨论更重要。

读者可以拿一条当前需求做练习：写出一条规则、两个示例、一个反例和一个问题。如果完全写不出问题，往往不是需求已经完美，而是团队还没有触碰边界。

10.6 显式签署与前置门

完成批判性检查、Grill-Me 和必要的 Example Mapping 后，OA 把所有确认结果组织为完整契约，交给 IO 审阅。只有 IO 本人明确填写签署区或回复“签署”，契约才生效；逐条决策确认、OA 总结或工具生成 `SIGNED` 都不能替代这个动作。

前置门至少机械检查：

- 契约文件存在；
- IO 已真实签署；
- 没有 OA 代签或自动签署；
- 约束矩阵存在；
- AC 可执行性达到要求；
- 文本 grep 类验收没有占据主要比例。

```
python scripts/gate_check.py --gate pre --task T-001 --project-dir .
```

前置门没有通过，不得进入业务编码。

10.7 规格漂移

规格漂移是指执行过程中，团队真正实现和验证的目标逐渐偏离已签署规格。它可能来自合理的新发现，也可能来自为了迁就实现而偷偷改变标准。危险之处在于：最终代码、测试和证据可能彼此一致，却已经不再对应 IO 原本批准的目标。

漂移是怎样发生的

规格漂移很少以“我们决定改变需求”的形式明确出现，更多发生在看似微小的动作里：

- Agent 发现原规则难以实现，于是选择更容易的解释；
- 测试失败后，执行者修改期望值而不是修复实现；
- 新接口不支持某个边界，团队将其悄悄列为“不处理”；
- 为兼容历史行为，把“严格大于”改成“大于等于”；
- IO 在聊天中提出新想法，但没有更新完整契约；
- 多个 Agent 使用了不同版本的契约和 AC；
- 截止时间临近，团队先上线“接近的版本”，却仍沿用原批准结论。

这些动作单独看都可能很合理，累积后却造成“验证通过了另一个需求”。

先区分发现与漂移

执行中出现新事实并不等于发生错误。软件开发本来就包含发现。关键是判断新事实影响哪一层：

新事实类型	示例	处理方式
实现细节	当前模块使用既有规则引擎而不是手写判断	在契约边界内调整，记录技术决定
估算变化	接口适配比预计多半天	更新计划和遥测，不改变业务规格
新风险	旧草稿可能受到新规则影响	暂停相关路径，补充风险与验证
业务规则变化	“超过 5000”改为“5000 及以上”	新契约版本，IO 重新签署
范围变化	增加外币报销和历史数据迁移	重新评估目标、约束、DAG 和证据
权限变化	实现需要生产数据写权限	停止，走权限与人工裁决流程

判断标准不是“代码改动大不大”，而是它是否改变目标、非目标、业务规则、关键 AC、风险承担或授权边界。一行比较符号变化，也可能是重大的规格变化。

规格漂移的治理流程

当 AS 或 OA 怀疑发生漂移时，应执行以下步骤：

1. **停止受影响节点**，避免其他 Agent 继续基于旧假设扩散修改；
2. **保留现场**，记录触发输入、现规格、当前实现、失败证据和新事实来源；
3. **评估影响**，识别哪些 AC、约束、模块、测试和发布计划会变化；
4. **提出选项**，例如保持原规格并增加实现成本，或改变范围并接受相应影响；
5. **由 IO 裁决**，涉及安全、合规或架构责任时加入相应责任人；
6. **生成新版本并重新签署**，旧版本保持可追溯，不直接覆盖；

7. 重新裁剪 Context、更新 Graph 与测试，确保所有节点使用同一版本；

8. 在 Evidence Bundle 中记录漂移及其处理。

例如实现过程中发现历史接口一直把 5000.00 元视为高额报销。团队有两个选择：保持新契约的“严格大于”，处理兼容差异；或者让 IO 决定延续历史语义，把规则改为“大于等于”。无论选择哪一种，都不能由 Agent 静默决定，更不能只修改测试使其变绿。

如何发现漂移

人工审查固然有用，但高速 Agent 工作流更需要机械信号：

- 契约内容摘要与签署摘要不一致；
- AC 或测试在签署后被修改；
- 不同节点引用不同契约版本；
- Evidence Bundle 中的规则文本无法回链到契约；
- 新增文件或模块超出原影响范围；
- Non-goals 中的事项出现在代码差异；
- 测试从 Red 变 Green 的过程中断言被削弱或删除；
- 约束例外没有批准记录。

这些信号可以进入前置门、验证门和证据审计。机械检查不能理解全部业务语义，却能发现“签署后发生了什么变化”，把最值得人审查的地方暴露出来。

规格版本不是文档管理小事

每个有效版本至少需要版本号或内容摘要、签署人、签署时间、变更原因和替代关系。新的签署不是抹掉旧决定，而是说明组织在什么证据基础上改变了什么。这样发生事故时，团队才能判断问题来自错误执行、错误规格，还是正确规格后来发生变化。

规格漂移管理的目的也不是冻结需求。Agentic Agile 接受变化，但要求变化从暗处走到明处：可以改变目标，不能伪装成目标从未改变；可以接受新风险，不能让 Agent 替责任人接受。

10.8 从自然语言到可执行 AC

AC（Acceptance Criteria，验收标准）是意图契约中最接近执行的一层，也是从“业务想要什么”跨越到“机器怎样证明已经做到”的关键接口。Goal 给出方向，Non-goals 划定范围，Rules 表达业务判断，而 AC 把这些内容转成可以观察、执行和裁决的条件。没有 AC，Agent 只能根据语言相似度判断“似乎完成”；有了可执行 AC，系统才能根据真实行为回答“是否完成”。

这也是为什么 AC 不能等到编码结束后再由实现反推。实现者如果同时决定目标、编写实现并定义验收方式，很容易把当前实现包装成正确答案。Agent 的生成速度越快，这种“先有答案、再修改题目”的风险越高。关键 AC 应在 Intent Contract 签署前形成基线，由 IO 确认业务含义；签署后若要改变，必须按规格漂移处理，而不能为了让测试变绿而静默削弱。

AC 不是需求复述，也不等于测试用例

自然语言需求说明期望和背景，AC 规定可接受结果的边界，测试用例则用具体数据和操作验证某一条 AC。三者相互连接，但不能混为一谈。

例如，需求是“高额报销需要总监审批”。如果 AC 仍然写成“高额报销正确进入审批”，只是把原句重复了一遍，并没有消除“高额是多少”“边界是否包含 5000 元”“谁是总监”“怎样证明进入审批”等歧义。更可判定的 AC 是：

当境内人民币差旅报销单含税总额为 5000.01 元时，提交成功后当前审批角色为 `director`，审批轨迹记录规则版本 `expense-v2`。

这条 AC 随后可以派生接口测试、数据库断言或事件检查。测试可以有多个，测试工具也可以更换，但它们共同证明的业务判据不应随实现变化。

内容层次	回答的问题	报销示例
Goal	为什么做、要产生什么价值	高额报销进入更高等级审批，降低财务风险
Rule	业务怎样判断	含税总额严格大于 5000 元时需要总监审批
AC	什么可观察结果算满足规则	5000.01 元进入 <code>director</code> ，5000.00 元保持原审批路径
Test	用什么数据、步骤和断言证明	调用提交接口，检查响应、审批轨迹和规则版本

一条高质量 AC 的六个组成部分

一条 AC 不一定要写成很长的模板，但至少应让人和机器找得到以下要素：

- 1. **前置状态**：在什么身份、数据、环境或系统状态下执行；
- 2. **触发动作**：用户、Agent 或外部系统做了什么；
- 3. **输入与边界**：使用什么关键数据，边界值和单位如何解释；
- 4. **预期行为**：系统必须产生什么业务结果；
- 5. **可观察证据**：从响应、状态、事件、数据库、日志或界面哪里判断结果；
- 6. **判定口径**：怎样算通过或失败，是否存在时限、误差或容忍范围。

Given-When-Then 是常用表达，但不是唯一格式：

```
Given 报销单为境内人民币差旅，当前状态为 draft
And 含税总额为 5000.01 元
When 具备提交权限的员工提交报销单
Then 提交成功
And 当前审批角色等于 director
And 审批轨迹包含规则版本 expense-v2
And 以上状态在 2 秒内可查询
```

格式的价值不在于模仿 Gherkin 语法，而在于迫使团队说清前置条件、动作和结果。若业务规则复杂，可以先通过 Example Mapping 找出规则、典型例子、反例和未决问题，再把已经确认的例子转成 AC；尚未确认的问题应进入 Grill-Me，而不是由 Agent 自行补全。

从“可读”到“可执行”的三个层级

“可执行 AC”不意味着每条文字都必须直接变成自动化代码，而是每条 AC 都有明确、可重复的验证协议。可以按以下层级推进：

层级	表现	适用情况
L1 人工可判定	写明观察对象、步骤、预期和责任人，可重复人工验收	审美、价值取舍、探索性体验
L2 半自动验证	脚本采集结果，由人对照判据裁决	复杂报表、跨系统 UAT、语义判断
L3 自动可执行	通过安全 <code>predicate:</code> 、 <code>http:</code> 、 <code>db:</code> 、测试命令或策略检查自动给出结果	接口、规则、状态、不变量、性能与安全门槛

对 AI Coding 任务，应尽可能让关键业务行为进入 L3，并保留确实需要人的价值判断。仅用 `grep` 检查某个字符串、文件或函数是否存在，通常只能证明代码长得像预期，不能证明运行行为满足业务意图。按照 3-4-3 的机械门禁，使用 `shell:grep` 的 AC 不应超过总数的一半，至少一半应通过安全 `predicate:`、`http:`、`db:` 或真实测试命令等运行时方式验证。任意 Python `assert:` 与 `setup` 形式已经因执行风险停用；自定义业务验证应进入受控测试脚本，而不是把可执行代码藏进契约。

例如，可以在 Intent Contract 中直接声明验证方式：

```
acceptance_criteria:
- id: AC-EXP-001
  rule: 金额严格大于 5000 元时进入总监审批
  verify:
    type: command
    command:
      argv: [python, -m, pytest, tests/test_expense_approval.py, -k, amount_5000_01]
      timeout_seconds: 60
    evidence: evidence/ac/AC-EXP-001.json
- id: AC-EXP-002
  rule: 金额等于 5000 元时保持原审批路径
  verify:
    type: command
    command:
      argv: [python, -m, pytest, tests/test_expense_approval.py, -k, amount_5000_00]
      timeout_seconds: 60
    evidence: evidence/ac/AC-EXP-002.json
```

具体语法可以因工具链而异，关键是验证命令、输入、断言和证据位置都可追溯，任何人或 Agent 在同一版本上重跑，都应得到可解释结果。

AC 要形成覆盖组合，而不是只写“快乐路径”

单条成功示例不足以定义规则边界。AC 组合至少应按风险考虑：

- **典型成功路径**：最常见输入产生正确结果；
- **边界值**：阈值前、阈值本身和阈值后，例如 4999.99、5000.00、5000.01；
- **明确反例**：看起来相似但不应触发规则的情况；
- **权限拒绝**：无权限主体不能通过另一入口完成同一动作；
- **异常与恢复**：依赖失败、超时或重复提交后，状态是否一致并可恢复；
- **关键不变量**：无论路径怎样变化都不能被破坏的规则；
- **必要 NFR**：对该意图真正关键的性能、安全、隐私和可用性门槛；
- **可观测性**：不仅结果正确，还能识别使用了哪个规则版本、发生了什么裁决。

并非每个任务都要机械凑齐八类 AC。选择标准是风险：哪一种误解最可能发生，哪一种失败代价最高，哪一项若没有证据便无法上线裁决。简单文案可以只有两三条 AC；资金、权限、隐私和跨系统状态变化则需要更完整的组合。

AC 如何贯穿 SCOPE-V 与证据链

AC 一旦签署，就不再只是 Specify 阶段的一段文字：

自然语言意图

→ Example Mapping / Grill-Me 消除歧义

→ Intent Contract 中形成带 ID 的 AC

→ Work Graph 把 AC 分配到测试、实现和验证节点

→ Prove 先产生可信 Red，再挑战候选实现

→ Evidence Bundle 按 AC ID 收集命令、结果、时间与来源

→ Verify 逐条裁决通过、失败、豁免或证据不足

→ 生产遥测继续观察 AC 所代表的业务效果

每条关键 AC 都应拥有稳定 ID，并能回链到规则和目标，向前映射到测试与证据。Evidence Bundle 不能只写“所有测试通过”，而要说明 `AC-EXP-001` 由哪项检查证明、检查针对哪个契约和提交、何时运行、结果是否新鲜、是否存在例外。若某条 AC 没有证据，正确状态不是“默认通过”，而是“未证明”。

同样，测试全绿也不代表所有 AC 都满足：测试可能漏掉某条 AC，可能运行在旧提交上，也可能与实现共享同一个错误假设。因此 Verify 必须检查“AC—测试—证据”的映射完整性，而不能只看测试总数和绿色比例。

防止 AC 被 Agent 反向操纵

在 $P \Leftrightarrow E$ 快内环中，AS 可以修改实现并重跑测试，但不能自行降低 AC、删除失败断言或把边界值移出范围。建议把以下内容纳入 Harness 门禁：

- 签署后的 AC 使用版本号或摘要锁定；
- AC、关键测试和断言发生变化时触发漂移检查；
- 实现 Agent 不拥有批准关键 AC 变更的权限；
- Red 到 Green 过程中保存测试差异和原始失败证据；
- AC 豁免必须记录原因、风险承担人和有效期限；
- Verify 由独立角色或独立上下文审查证据，而不是由实现者自行宣布完成。

判断一条 AC 是否合格，可以做最后的“五问检查”：它是否回到某个已确认的目标或规则？不同的人是否会作出相同判定？机器或责任人是否知道到哪里取证？它是否覆盖最危险的边界或反例？实现失败时，Agent 能否通过修改 AC 逃避失败？如果其中任何一问答不上来，AC 仍然只是验收愿望，还没有成为可执行规约。

10.9 Spec 不是越详细越好

规格过少会导致猜测，规格过多会带来维护和漂移成本。可以按风险选择层级：

场景	推荐规格
低风险界面调整	Goal、Non-goals、少量 AC
规则密集业务	Rules、Example Mapping、边界 AC
状态机或协议	Model-Based Specification、状态不变量
高风险跨模块	契约、XC、回滚和审计要求

10.10 Specify 常见失败

- 把 IO 第一句话直接写进 Goal；
- 一次提出十几个问题，让 IO 无法决策；
- 推荐默认值没有理由；
- Non-goals 为空；

- 业务术语没有统一口径；
- AC 只检查文件或字符串；
- 签署区由 OA 填写；
- 签署后仍允许 Agent 静默修改。

10.11 本章小结

Specify 把需求从一句自然语言变成可判定、可签署、可追溯的组织承诺。批判性思维负责发现问题，Grill-Me 负责帮助决策，Example Mapping 与 AC 把决定转成验证协议，显式签署负责建立授权边界。它不是 Prompt 优化阶段，而是人机系统取得执行权之前的责任形成过程。

本章最小实践：对一个模糊需求完成四轮 Grill-Me，生成并由 IO 签署 T-001。

完成证据：带 Goal、Non-goals、Rules、AC 和签署区的契约。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，把模糊需求改写为可以被人签署、被 Agent 执行的意图契约。

第十一章 | Constrain：把组织原则变成可执行边界

11.1 约束不是“请注意”

“不要破坏旧功能”“确保数据安全”“尽量少花 Token”无法直接驱动系统行为。Constrain 要把这些原则变成机器能够检查、阻断和升级的条件。

```
- id: C-DATA-01
  domain: DATA
  level: MUST
  description: 测试不得读取生产报销数据
  check_type: predicate
  check: "not is_file('.env.production')"
  gate: G2
  on_failure: block
  owner: Data Owner
  evidence: evidence/constraints/C-DATA-01.json
```

一条有效约束至少回答：适用于哪里、如何检查、失败后怎么办、谁能批准例外、例外何时失效。

11.2 六大核心约束域：从分类目录到工程控制面

第五章介绍约束矩阵时，已经用一张表给出了 STRUCT、DATA、BEHAVE、QUAL、PROC、STYLE 六个核心域。那张表回答的是“约束可以分成哪些类别”；本节不再重复名词，而要进一步回答三个工程问题：**每个域保护什么风险，怎样变成机器检查，违反后留下什么证据并触发什么动作。**

六域也不是为了把规则整理得更好看。它们共同覆盖 Agent 执行中的六种典型失控：工件缺失、数据越界、行为走样、质量退化、过程绕过和代码失序。缺少任何一域，系统都可能出现“局部全绿、整体失控”。

STRUCT：确保治理骨架没有被拆掉

STRUCT 保护的是项目和治理制品的**结构完整性与可寻址性**。Agent 不仅可能写错代码，也可能把契约放错目录、删除关键测试、改变证据命名、制造两个任务共用同一 ID，最终让自动化找不到应检查的对象。

典型 STRUCT 约束包括：必需目录和文件存在；契约、约束矩阵、Evidence Bundle 使用同一任务 ID；关键模块入口不能被绕过；生成文件不得覆盖源文件；测试和回退脚本位于约定位置；图谱节点引用的产物真实存在。

它适合用目录扫描、Schema 校验、引用完整性检查和内容摘要验证来执行。例如：

```
- id: C-STRUCT-003
  level: MUST
  rule: 每个已签署任务必须拥有同 ID 的约束矩阵与证据目录
  check: "python scripts/check_artifact_links.py --task ${TASK_ID}"
  gate: preflight
  on_failure: block
  evidence: "evidence/structure/artifact-links.json"
```

STRUCT 通过只能证明“治理骨架存在且能连接起来”，不能证明流程真实执行过。这正是它与 PROC 的区别：前者检查有没有正确的轨道，后者检查列车是否真的按规则运行。

DATA：限制 Agent 可以接触和改变的数据世界

DATA 保护数据的**范围、机密性、完整性、用途和生命周期**。它不仅回答“能不能读生产数据库”，还要回答哪些字段可以进入上下文、是否需要脱敏、能否写回、保存多久、是否允许跨客户或跨地域流动，以及失败后怎样恢复。

典型 DATA 约束包括：生产数据不得进入开发上下文；PII 必须脱敏；测试数据必须可追溯到合成或批准来源；Agent 只能访问当前租户；数据库写操作必须限定环境与表；数据导出必须经过出域扫描；临时上下文和日志按期限删除。

DATA 不能只靠扫描最终代码，因为越界可能已经发生在提示词、工具调用或日志输出中。有效控制通常需要数据分级标签、上下文注入过滤、工具网关、读写权限、DLP 扫描和审计日志共同完成。违反生产数据、密钥或客户隔离等 MUST 时，应立即阻断、撤销会话能力并升级，而不是让 Agent“注意下次不要再犯”。

DATA 与安全高度相关，但并不等于 SEC。DATA 关注被保护对象及其允许流动方式；SEC 扩展域还会覆盖身份认证、依赖漏洞、攻击面和威胁响应。9.7 节将进一步展开完整的数据安全与隐私工程。

BEHAVE：把业务语义和系统不变量锁进运行行为

BEHAVE 保护的是**系统做出的业务决定和状态变化**，防止 Agent 生成了结构合理、测试不少，却在关键语义上偏离意图的实现。它通常直接来自 Intent Contract 的 Rules、边界 AC、状态模型和历史兼容承诺。

以报销规则为例，“金额严格大于 5000 元才进入总监审批”属于 BEHAVE；“同一报销单重复提交不得重复生成审批实例”也是 BEHAVE；“已支付单据不能退回草稿状态”则是状态不变量。它们应通过接口测试、数据库断言、属性测试、状态机测试或契约测试证明，而不是通过 `grep` 判断代码里是否出现了某个比较符号。

BEHAVE 最危险的失败不是程序崩溃，而是程序稳定地做出错误业务决定。因此关键行为约束通常是 MUST，必须覆盖典型例、边界和反例，并在 Evidence Bundle 中按 AC 或 Rule ID 建立映射。如果实现需要改变行为约束，应触发规格漂移和 IO 重新签署，AS 无权为了兼容现有代码自行改写。

QUAL：定义“做到了”之外的工程可接受程度

QUAL 保护的是交付结果的**工程质量底线**。BEHAVE 回答业务行为是否正确，QUAL 回答结果是否足够可靠、可维护、可构建和可持续运行。一个接口可能正确地把 5000.01 元送入总监审批，却因为响应需要十秒、存在高危漏洞或一并破坏了其他模块，仍然不能交付。

典型 QUAL 约束包括：编译和构建必须通过；增量与回归测试满足要求；静态分析不得出现新增阻断问题；关键接口达到明确的性能基准；依赖风险在允许等级内；复杂度、重复度或覆盖率不得发生不可接受退化；关键可观测信号能够产生。

QUAL 要防止“代理指标替代真实目标”。覆盖率高不等于关键行为得到验证，测试数量多不等于测试有效，平均响应时间也可能掩盖长尾。质量约束应明确环境、样本、阈值和比较基线，并尽量组合结果指标与防护指标。9.3 节将说明 SEC、REL、OBS 等 NFR 如何按风险扩展，9.10 节会进一步讨论约束成本和古德哈特风险。

PROC：证明关键治理动作真实发生，而非只留下结果

PROC 保护的是治理过程的真实性和责任链。当 Agent 能在几分钟内完成多步工作时，最终产物存在并不代表签署、TDD、独立验证、人工裁决和遥测回写真的发生过。PROC 约束负责阻止“跳过过程但伪造完成状态”。

典型 PROC 约束包括：编码前必须存在 IO 真实签署；测试必须先产生可信 Red；实现者不得独自批准关键 AC 变更；Verify 前必须生成 Evidence Bundle；高风险例外必须由授权角色批准且带失效时间；任务结束必须写入契约级与全局遥测；重大决策必须能够追溯到责任人。

PROC 需要时间戳、签署摘要、Git 历史、流水线事件、角色身份、命令退出码和审计记录等过程证据。只检查最终文件里写着“已完成 TDD”没有意义。比如 TDD 门禁要证明测试文件和失败记录先于实现变化出现，而不仅是仓库最后同时存在测试和代码。

这也解释了 PROC 与 STRUCT 的边界：Evidence Bundle 文件存在属于 STRUCT；其中包含与当前提交匹配的真实运行记录、并经过独立 Verify，属于 PROC。两域常同时命中，但证明的问题不同。

STYLE：限制高概率生成下的代码熵增

STYLE 保护的是代码库内部可持续的表达方式和架构惯例。AI Coding 可以快速产生大量“能运行但不像这个系统”的代码：命名体系不一致、错误处理各自发明、绕过现有抽象、重复创建工具函数，或把本应进入领域层的逻辑塞进控制器。单次看似无害，长期会迅速放大维护成本和上下文噪声。

STYLE 不应变成一本几百页的审美手册。真正适合约束矩阵的是少量高价值规则：必须复用的框架和公共组件；禁止绕过的分层边界；错误、日志、配置和依赖注入方式；禁止引入的模式；语言格式化与 Lint 基线。其余偏好可以进入 AI Coding Guide，按语言、模块和任务裁剪注入。

STYLE 约束可以通过 formatter、linter、架构依赖测试、AST 规则和代码评审执行。格式错误通常可自动修复；破坏架构边界可能需要阻断；带有合理原因的新模式则应升级人工评审并更新 Coding Guide。STYLE 的目的不是让所有代码看起来完全相同，而是防止 Agent 每次执行都发明一套新的局部世界。

六域如何协同：一项变更往往同时穿过多个控制面

约束的分类不是互斥标签。同一风险可能涉及多个域，但应指定一个主域作为 Owner 和主要门禁，再用标签关联其他域，避免重复维护两条含义相同的规则。

例如“禁止 Agent 删除金额边界测试”主要保护业务不变量，可以归入 BEHAVE；测试文件必须存在由 STRUCT 检查；测试必须先 Red 再 Green 由 PROC 证明；完整套件必须通过属于 QUAL；若测试数据来自脱敏数据集，则又受到 DATA 控制。六域共同作用，才构成一条完整约束链。

可以用下面的诊断顺序为新约束确定主域：

工件或引用是否完整？

→ STRUCT

数据是否被不当读取、流动或修改？

→ DATA

业务决定或状态是否错误？

→ BEHAVE

工程品质是否低于交付底线？

→ QUAL

关键治理动作是否被绕过？

→ PROC

代码表达或架构惯例是否失序？

→ STYLE

六域不是要求每项任务都平均配置六类规则。低风险文案修改可能只需要少量 STRUCT、PROC 和 STYLE；涉及资金、权限与生产数据的任务则必须强化 DATA、BEHAVE 和 QUAL。成熟的约束矩阵追求的不是规则数量，而是高风险路径没有无主、不可检查或无法恢复的空白。

11.3 Web 扩展与 NFR

NFR 是 Non-Functional Requirements 的缩写，通常译为“非功能需求”。功能需求描述系统做什么，例如“超过 5000 元进入总监审批”；NFR 描述系统以什么质量和运行条件完成，例如响应时间、安全、可靠性、可观测性和恢复能力。

“非功能”并不表示不重要。一个审批接口即使业务判断完全正确，如果泄露财务数据、经常超时、失败后重复提交或无法监控，仍然不能称为可用交付。NFR 更准确的理解是“跨越单个功能的系统品质要求”。

对 Web 服务或高风险系统，可以增加安全（SEC）、可靠性（REL）和可观测性（OBS）约束，例如：

- 密钥扫描；
- 依赖风险；
- 健康检查；
- 重试和超时；
- 结构化日志；
- 监控埋点；
- 性能基准。

NFR 也要可判定。“系统性能要好”无法进入门禁；“在 200 并发、指定数据规模下，审批接口 p95 响应时间不超过 300ms”才有环境、负载、指标和阈值。性能测试还必须使用专用基准，不能把普通测试套件的总耗时冒充性能证据。

NFR 的强度应匹配风险。内部一次性脚本可能只需要基本错误处理和日志；面向生产的资金系统则需要更严格的身份、审计、幂等、恢复与监控。CLI 工具或纯库项目不应机械保留不适用的 Web 约束。裁剪约束不是降低标准，而是让每条规则都有真实风险依据。

11.4 MUST、SHOULD、MAY

MUST、SHOULD、MAY 用来表达规则强度。它们不是“大、中、小”三个优先级，而是在规则未满足时决定系统还能不能继续。

- **MUST**：不可妥协的要求，失败即阻断。适合安全红线、核心业务不变量和完成定义；
- **SHOULD**：强烈建议满足，但在说明理由、责任人和有效期后可以例外；
- **MAY**：可选能力，本次不满足不会影响当前裁决。

例如“不得把生产密钥写入仓库”是 MUST；“新增核心模块应保持不低于现有覆盖率”可能是 SHOULD，特殊重构期间经批准暂时例外；“本次顺便优化仪表盘配色”则是 MAY。

选择等级时可以问三个问题：违反后影响是否严重，结果是否不可逆，是否存在明确责任人能够接受风险。不要为了表示重视把所有规则都写成 MUST。硬门禁过多会造成频繁阻断和绕过；规则全部是 SHOULD，又会让真正的底线消失。

MUST 也不是绝对永恒。法律、安全等最高层约束通常不可由项目角色豁免；组织工程约束若确有例外，则必须由授权人书面批准，说明范围和失效时间，不能由 AS 自行降级。

优先级遵循：

法律与安全 > 约束矩阵 > 已签署意图契约 > 实现便利性

11.5 Tools Manifest 与最小权限

Agent 的能力不仅来自模型，也来自工具。Tools Manifest 应定义：

- 允许调用的工具；
- 允许读写的路径；
- 允许访问的域名和 API；
- 数据库操作边界；
- 命令预算和超时；

- 哪些操作需要人类批准。

不可逆操作不应因为“自动化更方便”而默认开放。

11.6 上下文安全

外部网页、Issue、日志、文档和用户输入都可能包含恶意或过期指令。Agent 必须区分：

- 系统和项目约束；
- 已签署契约；
- 任务上下文；
- 外部不可信内容。

Prompt Injection 和 Context Poisoning 的防御重点不是让模型“更聪明”，而是：降低不可信内容的指令权限、隔离敏感上下文、限制工具能力、对高风险动作设置机械批准。

11.7 AI 研发的数据安全与隐私工程

AI Coding 的数据风险不只是“员工把一段代码复制进聊天框”。IDE 插件可能自动读取当前文件和相邻代码，Agent 可能搜索整个仓库、读取 Issue 和日志，网关可能保存请求，模型供应商可能保留输入输出，Prompt Cache、向量库、Memory、MCP Server、Evidence Bundle 和 Telemetry 又会形成新的副本。如果只在培训中要求“不要泄密”，却没有控制这些自动数据流，组织拥有的是行为倡议，不是安全边界。

11.7.1 先拆开四个经常混淆的问题

选择 AI 服务时，至少要分别回答：

1. **训练边界**：供应商是否使用客户输入、输出或反馈训练、改进模型？默认条款和主动反馈是否采用不同规则？
2. **留存边界**：Prompt、代码、响应、安全分类结果、缓存、文件和日志分别保存多久？是否可以删除？
3. **处理边界**：数据由哪些供应商、子处理者、地区和模型托管方处理？哪些插件、搜索或工具会把数据继续传给第三方？
4. **访问边界**：组织内谁能看到会话、日志和管理数据？是否具备 SSO、RBAC、审计、租户隔离和密钥管理？

“不用于训练”不能推导出“零留存”；“零留存”也不能推导出“不出域”。数据可能不参与训练，却仍为产品功能、安全监测或法律义务被保存；也可能由云端即时处理但不长期存储。采购、法务、安全和 OA 必须逐项核验，不能用一个营销标签替代完整判断。

产品条款还会随套餐、模型和功能变化。GitHub 当前说明 Copilot Business 与 Enterprise 客户数据不用于训练，但不同模型托管方、预览功能和缓存可能采用不同处理方式；Anthropic 当前说明标准 API 输入输出通常在后端 30 天内删除，而 Zero Data Retention 是需另行批准的 API 安排，并非 Claude for Work 或 Claude Enterprise 的默认属性，Files API、Prompt Caching、Batch 与第三方搜索还可能具有不同留存规则。书中的原则不能替代签约时对最新 DPA、服务专用条款和实际配置的核验。[GitHub Copilot 模型托管与数据处理说明](#)；[Anthropic 商业产品留存说明](#)；[Anthropic ZDR 适用范围](#)

11.7.2 “代码和数据不出域”必须变成分级路由

“核心代码、设计文档和客户数据不能直接喂给公网 LLM”可以作为安全默认值，但“什么都不出域”并不适用于所有团队，也不必然最安全。公开仓库、一般内部代码、未公开商业逻辑、客户数据、生产密钥和受监管数据的风险不同，应按组织数据分类建立模型与工具路由：

数据级别	典型内容	默认 AI 处理策略
L0 公开	开源代码、公开文档、公开 API	可使用获准公共或企业服务，仍需检查许可证与供应链
L1 内部	一般内部代码、流程文档、测试样例	仅使用组织批准的企业租户，经身份、审计和保留策略控制
L2 机密	核心算法、未发布设计、商业规则、真实客户数据	默认不进入通用 SaaS；优先脱敏、最小切片，经企业网关路由到合同与配置均获批的环境
L3 严格受限	密钥、口令、生产凭证、受监管敏感数据、涉密材料	禁止进入模型上下文；使用确定性工具、隔离环境或经正式安全论证的私有处理路径

这里的“域”必须具体化：是企业身份租户、指定云区域、VPC、专有网关、物理隔离网络，还是完全离线环境？“企业版”“私有化”和“本地模型”都不能自动证明安全。本地模型仍可能通过遥测、依赖下载、向量数据库、日志平台或 MCP 工具把数据带出；私有化部署还需要补齐模型制品来源、GPU 运维、访问控制、补丁、备份和销毁责任。

11.7.3 敏感信息控制不能只依赖员工自觉

密钥、Token、口令、私钥和生产连接串不得进入 Prompt、模型上下文、响应、日志或 Evidence Bundle。PII、客户数据和内部网络信息则遵循目的限定与最小必要原则：默认使用合成数据、掩码、Tokenization 或假名化；确有合法业务目的时，只允许进入经数据责任人批准、满足合同与技术控制的处理环境，并保留访问、用途和删除证据。

控制应部署在多层，而不是只选择 IDE 插件或网关二选一：

- **端点与 IDE 层**：阻止向未批准 AI 网站粘贴敏感内容，扫描当前文件、选区、附件和自动收集的相邻上下文；
- **仓库与上下文层**：在 Context Cropping 前执行数据分类、Secret Scan、敏感路径排除和字段级脱敏；
- **AI 网关层**：强制模型白名单、地区与租户路由、DLP、请求大小、审计和阻断策略；
- **工具与 Agent 层**：Tools Manifest 限制数据库、对象存储、浏览器、MCP、Shell 和外部域名，防止模型通过工具绕过 Prompt DLP；
- **输出与落盘层**：扫描响应中的密钥、PII 和受限代码，控制聊天记录、缓存、向量、Memory、日志、证据与遥测的保留和权限；
- **组织层**：配置数据责任人、例外审批、供应商评审、访问复核、删除演练和事件响应。

微软的 AI 安全指南同样把敏感数据保护、DLP、AI 应用发现和 Prompt Injection 防御视为需要组合使用的纵深控制，而不是依赖一次输入过滤。[Microsoft AI 应用与数据保护指南](#)

11.7.4 数据安全必须覆盖完整生命周期

团队应为 AI Coding 画出真实数据流，而不是只登记模型名称：



对每一跳都要回答：传输了什么、依据什么目的、由谁处理、是否加密、保存多久、谁能访问、怎样删除、发生泄
漏怎样发现与响应。尤其不要把完整 Prompt 和源代码写入遥测。Telemetry 默认采集 Contract ID、模型、
Token、延迟、门禁状态和错误分类等元数据；确需保留内容用于审计时，应单独分级、最小化、加密并设置更短
保留期。

11.7.5 把隐私政策转成 3-4-3 的可执行控制

数据安全与隐私必须贯穿 SCOPE-V：

控制面	应落实的数据治理动作
S	识别任务涉及的数据类别、处理目的、合法性、非目标和数据责任人
C	写入允许数据级别、模型路由、出域规则、脱敏、留存、删除和工具权限
O	把敏感任务调度到获批环境；把脱敏、DLP 和人工审批设为前置节点
P	用合成探针验证密钥、PII、禁止路径和外部域名确实会被阻断
E	泄漏或误传后立即停止、撤销凭证、隔离记录、修正规则并完成事故回流
V	审查数据流、DLP 结果、供应商与配置版本、例外、删除和残余风险证据
Telemetry	采集最小必要元数据，监测阻断、例外、出域与删除，不复制敏感正文

一个最小可执行约束可以写成：

```
- id: C-DATA-EGRESS-01
  domain: DATA
  level: MUST
  description: "L2/L3 数据不得发送到未批准模型、域名或工具"
  gate: pre_action
  check: "python scripts/check_ai_egress.py --classification governance/data-
classification.yaml"
  on_failure: block_revoke_and_escalate
  owner: Data Owner
  evidence: "evidence/security/ai-egress-check.json"
```

但配置文件存在不等于控制有效。团队还应定期用合成密钥、测试 PII、禁止域名和恶意 MCP 调用做负向验证，证
明端点、网关、Agent 工具和落盘环节真的会阻断；再通过删除演练证明会话、缓存、向量、Memory 和备份能够
按约定退出生命周期。

11.7.6 采购和上线前需要什么证据

Evidence Bundle 至少应引用：

- 当前有效的 DPA、服务专用条款、子处理者与数据区域说明；
- “是否训练、留存多久、哪些例外”的书面结论，而不是销售口头承诺；
- 企业租户、模型、缓存、日志、反馈、插件和 Web Search 的实际配置快照；
- 数据分类与模型路由矩阵；
- Secret、PII、出域、未批准工具和 Prompt Injection 的负向测试结果；
- SSO、RBAC、密钥轮换和访问复核记录；
- 删除、撤权、事故响应和供应商退出演练；
- 尚未验证的 UNKNOWN、临时例外、责任人和到期时间。

这不是要求每个低风险任务都重新审计一次供应商。组织可以维护经过批准的 AI Service Catalog 和可复用供应商证据包；单任务只需证明本次数据级别、模型、工具和配置落在已批准边界内。这样既守住底线，也避免把数据治理变成每次开发都重复签合同。

11.8 例外、降级与恢复

约束失败后不只有“阻断”或“放行”两个选择。团队需要区分三种动作：

- **例外 (Exception)**：某条约束暂时不满足，但责任人明确接受残余风险；
- **降级 (Degradation)**：不突破底线，通过缩小范围、降低权限或关闭能力继续运行；
- **恢复 (Recovery)**：执行失败后，把代码、数据、权限和任务状态带回已知安全状态。

优先顺序应是：能恢复先恢复，能安全降级就不申请例外；只有确实需要承担残余风险时才创建例外。法律、安全、生产数据隔离等不可豁免约束，不应通过项目级例外放行。

情况	推荐动作	示例
结果错误且可撤销	恢复后重新执行	回滚提交、恢复测试数据、撤销临时权限
完整能力不可用但核心路径仍安全	降级运行	关闭自动发布，改为人工发布；只开放只读工具
约束暂时无法满足且风险可承担	限时例外	MVP 暂无性能环境，由责任人批准仅限内测
风险不可逆、责任人不明或证据不足	阻断并升级	生产数据可能出域、回退脚本从未验证

一份有效例外至少要记录：约束 ID、业务原因、影响范围、残余风险、补偿控制、批准人、到期时间和退出条件。只写“项目紧急”不构成批准依据。

```
exceptions:
- id: EX-001
  constraint_id: C-QUAL-03
  reason: MVP 阶段暂时没有正式性能环境
  scope: internal-pilot
  residual_risk: 高并发容量尚未被证明
  compensating_controls:
    - 限制为 30 名内部用户
    - 关闭自动扩量
    - p95 超过 500ms 自动告警并停止试点
  approved_by: Product-Owner
  valid_until: 2026-09-01
  exit_condition: 完成正式性能基准并更新 Evidence Bundle
  on_expiry: block
```

降级策略应预先写进约束，而不是故障发生后临时发挥，例如 `full-auto → human-approval → read-only → stop`。每一级都要明确触发条件、允许能力和恢复入口，避免系统虽然“降级”，却仍持有原来的高风险权限。

恢复也不能只写“支持回滚”。至少要回答：恢复到哪个已知版本，代码、数据和权限分别怎样处理，由什么信号触发，恢复后运行哪些验证，以及谁确认业务重新可用。关键恢复路径需要定期演练，演练记录进入 Evidence Bundle。

Harness 应在例外到期时自动撤销授权并恢复原约束；无法自动撤销时必须阻断后续门禁。Dashboard 还应持续显示未关闭例外、剩余期限、重复申请次数和恢复演练状态。例外反复续期通常意味着约束不合理或技术债长期未还，不能继续伪装成“临时情况”。

11.9 约束冲突

约束可能互相冲突。例如性能要求鼓励缓存，而数据一致性要求禁止使用过期结果；成本约束要求减少模型调用，而安全约束要求双模型复核。

冲突不能由 AS 根据实现便利自行解决。OA 应识别冲突，按法律与安全、数据、业务、质量、效率的优先顺序提出方案，由拥有责任的人批准。

冲突处理需要记录：

- 涉及哪些约束；
- 为什么冲突；
- 选择了什么优先级；
- 是否产生临时例外；
- 何时重新评估。

11.10 约束的经济性

每条约束都有执行成本。团队应观察：

- 命中次数；
- 真正阻止的风险；
- 误报率；
- 执行耗时；
- 例外数量；
- 是否已经被其他门禁覆盖。

没有命中、没有风险依据、持续产生误报的约束应被审查，而不是因为“安全第一”永久保留。

11.11 Constrain 常见失败

- 从模板复制几十条约束却不裁剪；
- 将 SHOULD 全部升级为 MUST；
- 检查命令只验证文件存在；
- 工具白名单允许任意 shell 和任意路径；
- 生产权限与开发权限混在一起；
- 只凭“企业版”“零留存”或“不训练”标签批准敏感数据进入模型；
- Prompt 做了脱敏，但日志、缓存、Memory、MCP 或 Evidence 继续保存原文；
- 私有化部署没有检查遥测、依赖更新、向量库、备份和模型制品来源；
- 例外没有批准人或期限；

- 约束更新后没有重新验证相关 Evidence Bundle。

11.12 本章小结

Constrain 把组织的隐性经验变成机器可执行的护栏。Agent 获得的不是无限自治，而是在明确数据、权限、质量、预算和恢复边界内的行动能力。

本章最小实践：为 T-001 建立六域约束矩阵、Tools Manifest 和四级数据路由矩阵，并用合成密钥或测试 PII 验证一次阻断。

完成证据：至少五条 MUST 约束、模型与工具路由、DLP 负向测试和数据责任人记录。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，为真实任务配置约束矩阵、失败动作和人工升级边界。

第十二章 | Orchestrate：从团队协作到可计算的人机能力网络

12.1 从任务列表到 Work Graph

Work Graph 可以直译为“工作图”。先不要把它想成一个高深的 AI 术语：它本质上是在回答——**为了完成这次交付，哪些工作必须发生，它们彼此依赖什么，满足什么条件才可以继续？**

假设周末要在家做一顿饭。“买菜、洗菜、炒菜、摆桌、开饭”是一张任务清单；如果进一步说明“买到菜后才能洗菜，洗菜和摆桌可以并行，菜品与餐具都准备好后才能开饭”，它才开始成为工作图。箭头不是为了好看，而是在表达一个真实条件：上游没有产生合格结果，下游就不应启动。

AI Coding 中也一样。复杂任务不是一串可以任意领取的待办事项。多个 Agent 执行得很快，如果它们不知道依赖和边界，就可能同时修改同一文件、在测试尚未可信时开始实现、拿过期产物继续工作，或者在集成验证之前宣布完成。Work Graph 把这些原本藏在人脑、会议和聊天记录里的协作关系，变成可以检查、调度和留证的结构。

12.1.1 一张工作图由什么组成

最小 Work Graph 包含四类元素：

1. **节点 (Node)**：一项可独立执行和验证的工作，例如“生成边界测试”或“完成接口契约验证”。节点不是一句模糊任务名称，还要有输入、执行者、允许范围、输出和完成判据。
2. **边 (Edge)**：两个节点之间的关系。它可能表示先后依赖、数据交接、审批放行、资源互斥或失败路由，而不只是“A 做完再做 B”。
3. **状态 (State)**：节点当前是等待、可运行、执行中、通过、失败、阻塞还是需要人工裁决。状态必须来自真实执行结果，不能只靠 Agent 自报。
4. **控制条件 (Control Condition)**：什么时候启动、停止、重试、回退、汇合或升级 HITL。例如“只有测试产生可信 Red，且影响分析没有发现范围外依赖，实现节点才可启动”。

因此，一张真正可执行的工作图应能回答：

- 现在有哪些节点已经具备启动条件？
- 哪些工作可以并行，哪些只是表面上可以同时开始？
- 下游正在等待谁的什么产物或证据？
- 某个节点失败后，应该重试、返回上游、回退还是交给人类？
- 哪些节点全部通过后，才允许汇合、发布或生成 Evidence Bundle？

12.1.2 它与任务清单、流程图和项目计划有什么不同

形式	主要回答	通常缺少什么
任务清单	有哪些事情要做	依赖、输入输出、失败路线和汇合条件
流程图	大致经过哪些步骤	节点执行契约、真实状态、证据和调度能力
项目计划	谁在什么时间完成什么	面向 Agent 的上下文、工具权限、停止与重试条件
CI/CD Pipeline	代码如何构建、测试和发布	从意图澄清到实现、裁决的完整工作关系
Work Graph	当前交付如何被分解、依赖、执行、验证和恢复	它不是某种固定模板，需要按契约与风险生成

Work Graph 可以吸收项目计划和流水线中的节点，但不等于二者。项目计划可能写“周三完成测试”，工作图则要说明测试使用哪个契约版本、消费什么输入、通过判据是什么，以及测试失败后哪个节点失效。CI/CD Pipeline 通常从代码提交开始，Work Graph 可以更早地从意图澄清、约束确认和测试设计开始，并把人工裁决也纳入图中。

12.1.3 为什么经常使用 DAG，但又不能只理解成 DAG

DAG 是 Directed Acyclic Graph，即“有向无环图”。“有向”表示边有方向，例如测试结果流向实现节点；“无环”表示在同一次调度里不能出现 A 等 B、B 又等 A 的死锁依赖。DAG 很适合表达一次交付的主干依赖、并行组和汇合点。但现实中的研发会返工：验证失败后可能回到测试、约束或规格重新处理。为避免概念混乱，可以把一次运行中的可调度依赖保持为 DAG，把“失败后生成修复节点、重新签署并启动下一轮图”看作控制回路。这样既允许系统学习和返工，又不会把图变成无法判断何时结束的无限循环。

12.1.4 它与 Intent Graph 的区别

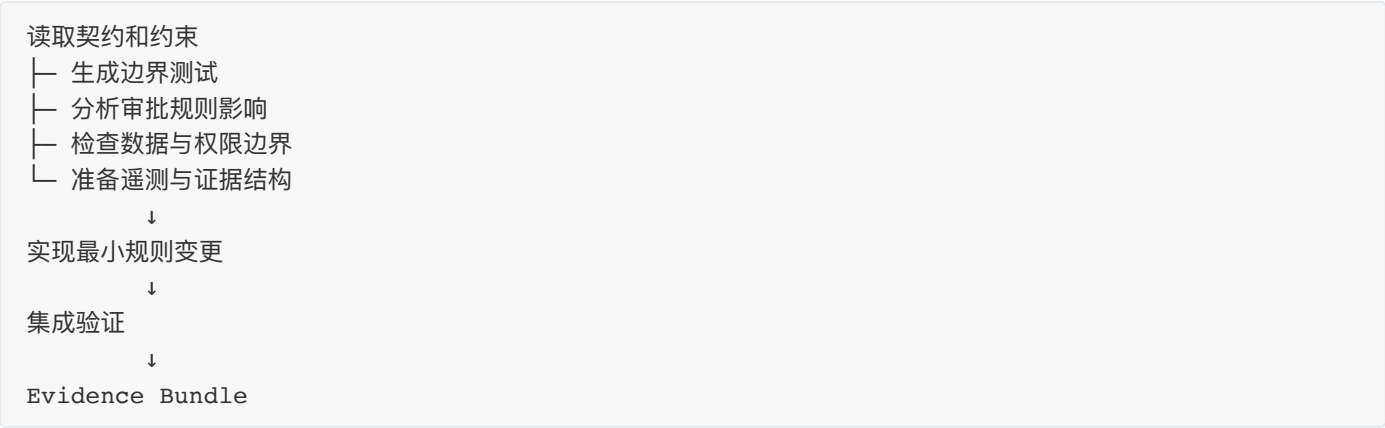
这两张图最容易混淆：

- **Intent Graph（意图图谱）** 组织“为什么做、价值如何分解、哪些意图彼此关联”；
- **Work Graph（工作图）** 组织“这一次怎样做、谁先谁后、产物如何交接、失败如何处理”。

例如，“降低虚假报销并保持员工体验”属于 Intent Graph；为了修改单笔金额规则而安排“澄清边界—生成测试—影响分析—实现—SIT—证据裁决”，属于 Work Graph。前者相对稳定，可以服务多个契约；后者面向一次具体交付，会随着契约、风险和执行状态变化。

12.1.5 费用报销案例：从清单变成可执行关系

费用报销案例可以拆成：



这张简图还只是骨架。要成为可执行 Work Graph，每个节点还要绑定契约与 AC、输入产物、负责人或 Agent、工具权限、允许修改范围、完成证据、预算和失败路线。例如：

- “生成边界测试”读取已签署的金额规则和 Example Mapping，输出处于可信 Red 状态的测试及运行记录；
- “实现最小规则变更”必须等待可信 Red 和影响分析完成，只允许修改金额策略与指定适配层；
- “集成验证”必须消费当前提交的制品，不能拿旧分支的 Green 结果冒充；
- “Evidence Bundle”只负责汇聚与核验既有证据，不能自己补造缺失证据；
- 若影响分析发现未知调用方，图进入阻塞状态并升级 OA/IO，而不是静默扩大修改范围。

只有无前置依赖、无写冲突、上下文边界明确、失败互不污染的任务才适合并行。**Work Graph 的价值不是让更多 Agent 同时开工，而是让每个 Agent 只在条件成立时开工，并让任何“已经完成”的声明都能沿图追溯到输入、规则和证据。**

12.2 每个 Agent 节点的执行契约

每个 Agent 节点至少需要：

- 当前目标；
- 非目标；
- 输入和预期输出；
- 相关 AC 和约束；
- 允许修改的文件；
- 禁止触碰的模块；
- 可用工具；
- 时间、Token 和重试预算；
- 完成条件；
- 停止和升级条件。

12.3 30 秒微检查点

30 秒微检查点是一种极轻量的方向校验：在 Agent 准备进入高成本实现或验证前，用半分钟复述自己的理解和行动边界。它借鉴了航空与医疗中的“操作前确认”思想——不是重新开一次评审会，而是在不可忽略的转折点检查大家是否正在做同一件事。

Prove 开始前，AS 向 OA 报告：

1. 我理解的目标是什么；
2. 我会修改哪些文件；
3. 我不会触碰哪些范围；
4. 我识别到哪些风险。

OA 检查的也不是代码方案是否完美，而是目标、范围和风险是否明显偏离契约。例如 AS 说“我将修改历史报销草稿以保持数据一致”，而契约 Non-goals 明确排除历史迁移，OA 可以在任何代码产生前停止这一方向。

微检查点适合放在上下文交接后、并行节点汇合前、高风险工具调用前，以及多轮修复后准备继续时。低风险、边界稳定的重复任务可以自动化比对；权限扩大、目标冲突或不可逆操作仍要进入正式 HITL。

这个检查点不是人工逐行审批。它防止的是方向性错误，而不是替代测试和 Evidence Bundle。一个好的微检查点应短、结构固定、结果可记录；如果每次都演变成成长会议，说明任务契约或节点边界还不够清晰。

12.4 Handoff 与跨 Agent 协作

Handoff 即“工作交接”。在人类团队中，接收者会追问不清楚的地方，并凭共同背景判断上游是否遗漏；Agent 之间缺少这种默契，一句“测试已经完成，可以继续”很容易把错误假设传到下游。

Handoff 因此不是聊天总结，而是一份小型、可验证的交付契约。它应传递结构化事实：

- 已完成什么；
- 产生了哪些文件；
- 哪些检查已运行；
- 哪些失败仍未解决；
- 哪些假设尚未验证；
- 下一个 Agent 需要什么输入。

还应带上任务与契约版本、产物路径、运行命令和退出码。下游收到后先做低成本验真：文件是否存在，测试是否确实处于预期 Red 或 Green，产物是否来自当前提交，上游声明的未知项是否会阻塞自己。

例如测试 Agent 的 Handoff 不能只写“边界测试已完成”，而应说明测试文件、对应 AC、运行命令、退出码为 1 且失败原因是功能尚未实现。这样实现 Agent 才能确认这是可信 Red，而不是测试环境损坏。

Handoff 的价值是隔离错误。如果下游验证失败，系统可以把问题路由回具体上游节点，而不是让整个任务在错误基础上继续。接收方应验证输入，而不是盲信上游 Agent 的“已完成”；上游也不应传递自己的全部上下文，只传递下游完成工作所需的事实、产物和未知项。

一份可验证的 Handoff 示例

```
task: WG-TEST-01
contract_version: T-001@sha256:8d31...
source_commit: a81f5c2
status: completed
outputs:
  - path: tests/expense-threshold.test.ts
    sha256: 6a72...
checks:
  - command: npm test -- expense-threshold
    exit_code: 1
    meaning: expected_red
    evidence: evidence/red/WG-TEST-01.txt
assumptions:
  - currency is CNY
unknowns:
  - historical drafts migration not evaluated
next:
  allowed_node: WG-IMPLEMENT-01
  precondition: coding_gate_passed
```

接收方至少校验契约版本、来源提交、产物摘要和证据文件，再判断 `exit_code: 1` 是否真是预期 Red，而非依赖缺失或环境故障。验证失败时，图应把任务退回 `WG-TEST-01`，不能让实现节点带着不可信输入继续执行。

12.5 资源争用、超时与死锁

多 Agent 编排首先要管理的不是“谁更聪明”，而是谁占用什么资源、占用多久，以及失败后资源怎样释放。常见故障包括同时修改同一文件、争抢测试环境、循环依赖、重试风暴、节点失败后下游继续消费陈旧结果，以及多个 Agent 互相等待人工确认。

每个节点启动前应声明三类资源：**写资源**（文件、分支、数据库表）、**独占资源**（测试环境、设备、发布槽位）和**稀缺预算**（Token、时间、并发额度）。Graph Engine 据此检查写集合冲突；无法隔离的共享写入必须串行，不能依靠 Agent 自觉避让。

一套最小控制规则包括：

- 写同一文件或状态对象的节点不得并行，除非使用独立分支并设置明确汇合节点；
- 锁必须包含 Owner、获取时间和租约，到期后进入确认或回收，不能永久占用；
- 超时要区分“仍在正常运行”“外部依赖无响应”和“节点已经失联”，不能一到时间就盲目重跑；
- 只对瞬时错误进行有限重试，规格冲突、权限拒绝和确定性测试失败不得靠重试解决；
- 节点失败后立即使其输出失效，下游只能等待、回退或改走显式降级分支；
- 图中出现循环等待，或连续两轮重调度仍无法取得资源时，停止自动编排并升级 OA。

例如测试 Agent 占用唯一集成环境超过租约时，系统应先检查心跳与当前日志；确认失联后释放环境、把未完成输出标记为无效，再决定是否重调度。直接启动第二个 Agent 可能让两个执行者同时写入环境，制造更难诊断的污染。

关键事件都应留下 Graph Event：锁的获取与释放、超时原因、重试次数、输出失效和人工解锁。否则 Dashboard 只能看到“任务变慢”，却无法判断是模型问题、资源瓶颈还是图设计错误。

12.6 自适应编排的边界

自适应编排允许系统根据任务类型、历史成功率、工具可用性和当前负载，调整执行者、并发度、节点顺序与重试策略。但它优化的是**已授权边界内的执行路径**，不是重新定义目标或替责任人接受风险。

可以自动调整的内容通常包括：在批准列表中选择 Agent 或模型；把无冲突节点串行或并行；根据资源状态重调度；在预算内切换工具；对已分类的瞬时故障有限重试。以下内容则必须锁定：

- 已签署的 Goal、Non-goals、Rules 和关键 AC；
- MUST 约束及其失败动作；
- 工具和数据权限上限；
- 人类批准与职责分离边界；
- Evidence Bundle 和 Verify 的独立性要求；
- 总预算、停止条件和不可逆操作门禁。

Learning Router 的新策略不应直接接管生产任务。至少要经过历史任务回放或影子评估，证明完成率、证据完整性和成本没有不可接受退化；随后在低风险范围灰度，并保留旧路由作为回退。路由依据、选择结果和回退原因都要进入遥测，避免算法逐渐形成无法解释的“黑箱管理者”。

可以用一句规则判断：**路径可以学习，边界不能自学；效率可以优化，授权不能推断**。一旦自适应需要扩大权限、改变 AC、跳过门禁或突破预算，Graph Engine 必须暂停并请求人类裁决。

12.7 并行是否真的带来收益

并行收益取决于可独立任务比例、协调成本、共享资源和最终集成成本。团队可以记录：

- 串行关键路径长度；
- 可并行节点数量；
- Handoff 失败次数；
- 写冲突和重做；
- 并行带来的真实时间节省；
- 增加的 Token 和验证成本。

如果两个 Agent 需要反复同步同一文件，并行可能比串行更慢。

12.8 Orchestrate 常见失败

- 把任务切得太细，Handoff 成本超过收益；
- 多个 Agent 同时修改共享核心文件；
- 下游不检查上游产物；
- DAG 只描述正常路径，没有失败分支；
- 自动重试没有上限；
- 微检查点只重复用户原话；
- Learning Router 用历史成功率掩盖任务类型差异。

12.9 编排中的三个关键决策

决策一：串行还是并行

只有当任务边界稳定、共享写入较少、验收接口明确时，并行才有意义。若两个 Agent 不断修改同一核心对象，表面的并发会转化为合并冲突、上下文重载和重复验证。OA 应比较关键路径缩短量与协调成本，而不是把 Agent 数量当作生产力指标。

决策二：自动重试还是升级人类

网络抖动、临时资源不可用、已知幂等操作可以有限重试；目标冲突、权限不足、数据不可逆、证据自相矛盾则必须停止。重试次数、退避策略和升级条件都应预先写入 Work Graph，不能让执行 Agent 在失败后临场决定自己还可以尝试多少次。

决策三：共享上下文还是隔离上下文

共享上下文有利于一致性，却增加信息泄漏、锚定偏差和上下文污染；隔离上下文减少干扰，却可能产生局部最优。合理做法是共享稳定的契约、约束和接口，隔离与本节点无关的实现细节，并通过 Handoff 传递已经验证的产物。

12.10 一个四节点编排案例：为什么这样拆、这样连

继续使用“境内人民币差旅报销含税总额严格大于 5000 元时，增加总监审批”的任务。表面看只是把比较条件改一下，但影响可能穿过领域规则、审批状态、历史接口、回归测试和发布证据。如果直接让一个 Agent 从需求一路写到上线，它很容易同时定义判据、修改实现、解释兼容问题并宣布自己通过，责任和错误来源都会混在一起。

编排前，OA 先从契约和约束中提取四个决定：

1. 5000.00 不触发、5000.01 触发，这是已经签署的业务边界，Agent 无权重新解释；
2. 历史草稿迁移属于 Non-goals，本次不能顺手修改旧数据；
3. 测试必须先产生可信 Red，实现 Agent 不得修改关键测试；
4. API 兼容性尚未被证明，发现旧语义冲突时必须升级，不能静默兼容。

这些决定说明任务至少包含四种性质不同的工作：定义挑战、改变领域行为、适配系统边界、独立汇总证据。于是得到四个节点，而不是简单地“找四个 Agent 同时干活”。

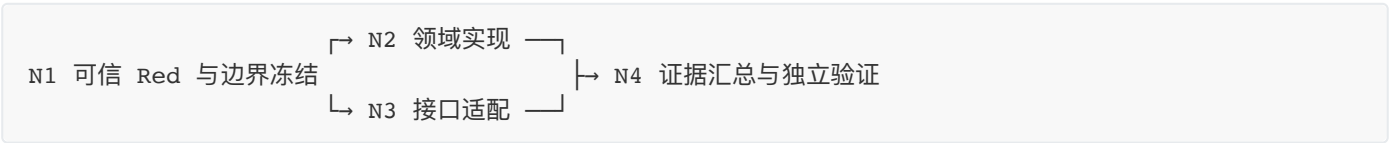
四个节点分别承担什么责任

节点	为什么独立	主要输入	允许修改	完成条件
N1 规则测试	先固定判据，避免实现反向定义正确	签署契约、AC、Example Mapping、金额口径	边界测试与测试夹具	4999.99、5000.00、5000.01 形成可信 Red；失败原因确为能力尚未实现
N2 领域实现	只负责最小业务变化，不承担接口兼容裁决	N1 Red、领域代码切片、BEHAVE/STYLE 约束	金额规则与审批领域模块	N1 由 Red 变 Green，领域回归通过，未修改测试和历史数据
N3 接口适配	隔离 API、序列化和历史调用方风险	已签署接口语义、N1 边界判据、现有 API 契约	Adapter、接口契约测试	新旧请求均可被明确解释；未发现静默语义变化；接口测试通过
N4 证据汇总与独立验证	防止实现者用自己的解释宣布完成	N1-N3 原始输出、当前提交、约束结果	Evidence Bundle，不得修改实现与判据	AC 与约束逐条有新鲜证据；来源一致；残余风险和回退条件清楚

这里有两个刻意的职责分离。第一，N1 和 N2 分开，是为了让“什么算正确”先于“怎样实现”稳定下来；第二，N4 不修代码，只核验证据，否则它发现缺口后可能顺手补一个结果，再把自己补出的东西判为通过。

边不是箭头，而是启动条件

这张图可以简化为：



但每条边都有具体语义：

- **N1 → N2**：不是“测试文件写完”，而是测试已运行、确实为 Red、失败原因与缺失行为一致，并且关键断言摘要已锁定；
- **N1 → N3**：金额边界和目标接口语义已经确定，N3 才能检查历史 API 是否兼容；
- **N2 + N3 → N4**：两个节点都完成还不够，来源提交必须能够汇合、没有写集合冲突，集成检查必须使用汇合后的当前制品。

如果边只写成 `depends_on`，却不定义这些条件，Graph Engine 最多知道前一个节点“自报完成”，不知道下游获得的输入是否可信。

为什么 N2 和 N3 可以并行，又为什么有时不能

本案例允许 N2 与 N3 在 N1 之后并行，前提是接口语义已经由契约冻结，并且写集合隔离：N2 只改领域规则，N3 只改 Adapter 和接口契约测试。这样两者共享稳定的边界，不共享可变实现。

若代码库把金额判断直接写在 Controller 中，或者 N2 仍可能改变接口字段与错误码，那么两节点实际上会争用同一设计决策和文件。此时正确选择不是强行增加并发，而是先串行完成领域接口抽取，再启动适配；必要时把任务改成五个节点。**节点数量不是预设答案，依赖和风险才是。**

并行前至少检查四件事：输入是否稳定、写集合是否隔离、单方失败是否污染另一方、汇合时是否有独立验证。任何一项答不上来，串行通常更便宜。

三类失败为什么要回到不同位置

Work Graph 的价值在失败时更明显。这个案例至少需要三条失败路由：

- 1. **实现错误**：N2 中 5000.00 仍触发总监审批。规则与测试没有冲突，返回 N2 进入有限 P⇒E，不重写 N1；
- 2. **接口兼容冲突**：N3 发现历史 API 一直把 5000.00 解释为高额报销。节点标记为 blocked_by_rule_conflict，保存旧接口、现契约和失败测试，返回 OA/IO 裁决。若意图改变，则生成新契约版本，并使 N1-N3 旧输出失效；
- 3. **证据不一致**：N4 发现领域测试来自提交 A，接口测试却来自提交 B。它不允许把两份绿色报告拼成证据包，而是退回汇合检查，在同一提交上重新运行相关验证。

还要有停止条件：N2 最多自动修复两轮；任何节点请求生产数据或扩大文件权限立即停止；规则冲突不得自动重试；N4 缺少关键 AC 证据时状态只能是 unproven，不能降级成“基本通过”。

一份最小 Work Graph 定义

```
nodes:
  N1_TEST:
    outputs: [trusted_red, locked_assertions]
    write_scope: [tests/expense-threshold.test.ts, tests/fixtures/expense-*]
    exit: red_reason == feature_missing

  N2_DOMAIN:
    depends_on: [N1_TEST.trusted_red]
    write_scope: [src/domain/expense/**]
    retry: {max: 2, only: [implementation_defect]}
    on_rule_conflict: escalate

  N3_ADAPTER:
    depends_on: [N1_TEST.locked_assertions]
    write_scope: [src/api/adapters/**, tests/contracts/**]
    on_contract_conflict: block_and_escalate

  N4_VERIFY:
    depends_on: [N2_DOMAIN.green, N3_ADAPTER.green]
    require_same_commit: true
    write_scope: [governance/evidence/T-001/**]
    forbidden: [modify_source, modify_ac, modify_tests]
    on_missing_evidence: unproven
```

真实项目还会加入超时、资源锁、Tools Manifest、成本预算和恢复动作。这个最小定义已经表达了编排的核心：每个节点为什么存在、在什么条件下开始、能改变什么、必须交出什么，以及失败后返回哪里。

回看这个案例，四节点并不是一种标准模板。它展示的是一套推导方法：先从契约和风险识别需要分离的责任，再根据可信输入建立节点，用启动条件而不是时间顺序连接节点，只对真正隔离的工作并行，最后为不同失败设计不同回路。读者掌握的应是这套考虑方式，而不是记住“测试—实现—接口—证据”四个名称。

12.11 本章小结

Orchestrate 的目标不是让更多 Agent 同时工作，而是把人类责任节点、专业 Agent、工具和依赖组织成可计算的能力网络，让合适的任务在明确上下文、权限、启动条件和停止条件下安全流动。

本章最小实践：为 T-001 创建 Work Graph，并完成微检查点。

完成证据：DAG、节点上下文、依赖和停止条件。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，把角色、工具和交接关系组织成可验证的 Work Graph。

第十三章 | Prove：先证明，再实现

13.1 AC 不是事后检查清单

在 Agentic Agile 中，AC 是驱动实现的可执行规约。Prove 不是等代码完成后再“看看是否符合”，而是先把签署后的 AC 转成能够否定错误实现的挑战，再让实现围绕这些挑战收敛。

```
签署 AC → 生成测试与断言 → 运行并确认可信 Red
      → Green：最小实现使测试通过
      → Refactor：不改变行为地改善结构
```

以“严格大于 5000 元进入总监审批”为例，Prove 至少应在实现前固定 5000.00 不触发、5000.01 触发两个相邻边界。若只从已有代码生成一个“6000 元审批成功”的测试，Agent 即使误写成“大于等于”也能通过，测试数量增加了，业务歧义却没有被消除。

测试先行也不代表一次性写全部测试。可以先选择当前风险最高、能够区分正确与错误实现的最小 AC 集合。关键是测试来源于契约和风险，而不是从实现结构反向猜测；签署后的关键断言要受版本保护，AS 不能为了 Green 自行修改。

13.2 编码门：先证明 Red 可信

编码门的目标不是确认“有一个红色结果”，而是确认测试已经成为可信挑战。它至少检查：

- 测试文件真实存在并已执行；
- 测试对应当前契约版本和明确 AC；
- 失败发生在业务断言，而不是语法、依赖或环境初始化；
- 当前代码如果缺少目标行为，测试确实会失败；
- Red 记录包含命令、退出码、时间、提交和原始输出；
- 关键测试与断言摘要已锁定，后续变更会触发漂移检查。

```
python scripts/gate_check.py --gate coding --task T-001 --project-dir .
```

可以把 Red 分为三类：expected_red 才允许进入实现；environment_red 应先修复环境；spec_red 表示 AC 互相冲突或无法判定，必须返回 Specify。把后两类失败误当作 TDD 起点，只会让实现 Agent 围绕错误信号反复试错。

编码门通过后，应输出一张小型“Red 凭证”：AC ID、测试路径、失败断言、契约摘要和允许进入的实现节点。没有这张凭证，Work Graph 不应释放编码节点。

13.3 最小实现：只改变足以让证据收敛的部分

最小实现不是追求最少代码行，而是完成当前签署意图所需的最小语义变化和最小影响半径。AS 应先说明当前 Red 的原因、计划修改的最小位置和预期影响，再开始 Green。

实现过程中要守住五条边界：

- 不顺手重构无关模块或升级无关依赖；
- 不添加尚未签署的“顺便功能”；
- 不扩大数据、网络、目录和工具权限；
- 不修改契约、AC 或关键测试来适配错误代码；

- 不用 Mock、skip、降低阈值等方式绕过真实依赖和失败。

例如金额边界错误来自领域策略中的比较条件，最小实现应修改该规则并运行相关回归，而不是同时迁移历史草稿、重写审批引擎或调整接口返回结构。若 Green 必须跨出允许文件范围，应暂停并回到影响分析或契约裁决，不能把范围扩大伪装成实现细节。

Green 后的 Refactor 也要与新增行为分开观察：测试保持全绿、对外行为不变、差异可解释。否则团队无法判断新的失败来自业务实现还是结构调整。

13.4 AI Coding 时代的多层验证工程

接口测试、UI 测试、SIT、UAT、性能测试都是软件工程中的成熟实践。AI Coding 到来以后，它们不会消失，也不只是“交给 AI 自动生成”这么简单。真正发生的变化是：测试从开发完成后的质量检查，转变为约束 Agent 行为、驱动 $P \Rightarrow E$ 收敛、并为上线裁决提供依据的多层证据系统。

传统测试讨论常围绕“谁来测试”“开发后第几阶段测试”“自动化率是多少”。在 Agent 可以分钟级生成代码、测试和报告之后，更关键的问题变成：

- 每条业务意图需要什么证据才能被相信？
- 测试是否来自签署后的 AC，还是从现有实现反推？
- 哪些层由 AI 自动执行，哪些需要独立 Agent 复核？
- 哪些结果属于事实验证，哪些仍需要业务价值判断？
- 当 AI 同时生成实现和测试时，如何防止同一个误解自证正确？
- 测试结果怎样进入 Evidence Bundle，而不是停留在流水线的绿色图标里？

这一整套问题，就是 AI Coding 时代的多层验证工程。

13.4.1 最大风险不是“没有测试”，而是同源误判

假设 Agent 把“超过 5000 元需要总监审批”错误理解为“达到 5000 元就需要总监审批”。如果它先按这个理解编写实现，再根据自己的实现补测试，就可能生成“5000.00 元必须升级审批”的断言。实现和测试完全一致，覆盖率很高，流水线全部绿色，但交付结果仍然违背真实意图。

错误理解意图

↓

生成错误实现

↓

根据错误实现生成一致的测试

↓

测试全部通过

↓

错误地宣布可以上线

这就是同源误判：需求解释、实现、测试和结论来自同一个认知源，错误假设贯穿整条链路，没有任何异质证据能够将其打断。AI 生成更多测试只能提高执行量，不能自动提高结论可信度。

3-4-3 用三种方式降低这种风险：

1. 测试首先从 IO 签署的 Intent Contract 和 AC 生成，而不是从实现代码反推；
2. 关键规则使用不同测试层、不同技术路径或独立 Agent 交叉证明；
3. P 和 E 可以由 Agent 高频循环，但最终 V 审查证据来源、覆盖、独立性和新鲜度，不能由执行 Agent 仅凭“全绿”自我放行。

独立性不是要求每个测试都换一个人写。它要求关键结论不能只依赖同一个未经校准的假设。对于低风险任务，从签署 AC 生成测试并保留可信 Red 可能已经足够；对于资金、权限、安全和不可逆操作，则应增加独立环境、第二

验证主体、真实样本或人工审查。

13.4.2 从测试金字塔转向验证证据组合

测试金字塔仍然有意义：底层测试通常执行快、定位准、成本低，端到端测试数量应更少。但仅用“单元测试多、UI 测试少”已经无法回答一个具体任务能否上线。它没有说明某条 AC 由谁证明，也没有说明接口兼容、跨系统协作、性能约束和业务接受是否得到覆盖。

因此，本书引入 **Verification Portfolio（验证证据组合）**：针对一份 Intent Contract，依据 AC、约束、风险和影响范围，预先选择一组足以支撑裁决的验证方式。

验证证据组合不是要求每个任务都执行所有测试，而是为每条关键意图回答七个问题：

- 1. 这条 AC 最适合由哪一层测试证明？
- 2. 一个测试层是否足够，是否需要交叉证明？
- 3. 哪些步骤可以由 AI 自动生成和执行？
- 4. 哪些结果需要独立 Agent、独立环境或人工复核？
- 5. 哪些问题属于业务价值判断，不能交给 AI 决定？
- 6. 证据的环境、版本、时间和有效期是什么？
- 7. 上线以后由什么遥测继续验证？

一份最小组合可以写进 Intent Contract 或 Work Graph：

AC/约束	风险	验证层	执行主体	独立性要求	通过判据	证据位置
AC-01 金额边界	高	单元 + 接口	测试 Agent	从签署 AC 生成	5000.00 通过，5000.01 拒绝	evidence/ac-01/
AC-02 审批链路	高	SIT	集成 Agent	独立测试环境	总监节点仅在超额时出现	evidence/ac-02/
C-PERF-01 响应时间	中	性能	性能 Agent	固定数据和负载模型	200 并发下 p95≤300ms	evidence/perf/
AC-03 提示可理解	中	UI + UAT	AI 执行、财务确认	业务代表裁决	用户能理解且完成处理	evidence/uat/

这张表把“应该测什么”从测试人员的隐性经验，转化为任务开始前可审查、执行后可追溯的证明设计。

13.4.3 单元与组件测试：证明局部规则和不变量

单元测试验证函数、类或领域规则在隔离条件下是否满足预期。组件测试则把若干内部对象组合起来，在不连接全部外部系统的情况下验证一个较完整的业务能力。它们运行快、失败定位清晰，是 $P \Rightarrow E$ 快内环的主要反馈来源。

AI 很适合根据 AC 生成正常、边界、异常和组合样例，也适合在失败后定位最小修复范围。但不能只把当前实现翻译成测试。测试设计应优先读取契约中的业务样例、不变量和 Non-goals。

费用规则至少应验证：

- 4999.99、5000.00、5000.01 三个边界；
- 零金额、负数、空值和非法精度；
- 人民币与外币换算发生在比较之前还是之后；

- 金额超限时不得静默截断；
- 历史草稿是否属于本次 Non-goals。

除了示例测试，还可以使用 Property-Based Testing 生成大量输入验证不变量，例如“任何超过阈值的合法金额都不能进入普通审批路径”。它能发现人类没有手工列出的边界，但属性本身仍须来自业务意图；如果属性写错，生成再多数据也会证明错误规则。

13.4.4 接口测试：证明服务契约，而不只是 HTTP 200

接口测试验证系统边界上的请求、响应、鉴权、错误、幂等和兼容性。AI 可以读取 OpenAPI、Schema、消费者契约和历史流量样本，生成用例、构造数据、执行回归并解释差异。这比人工逐条编写正常路径高效得多。

但接口通过不能只看状态码。一个可信的接口证据至少应覆盖：

- 请求字段、类型、必填项和取值边界；
- 响应结构、业务状态和错误语义；
- 身份认证、租户隔离和数据权限；
- 重复请求的幂等行为；
- 超时、重试、限流和降级；
- 版本兼容以及已有消费者是否被破坏；
- 数据写入后的状态，而不只是返回报文。

例如报销提交接口对 5000.01 元返回 200，但后台仍创建了普通审批单，仅在响应中附加一条警告，这在网络层“成功”，在业务层却失败。接口测试必须把 AC 连接到数据库状态或下游事件，才能证明真实行为。

AI 还可以做 Contract Testing：由消费方声明自己依赖的字段和语义，提供方在变更时自动验证。对于多 Agent 并行修改前后端的场景，这种可执行接口契约尤其重要，因为它能在集成前发现两个 Agent 各自合理、合在一起却不兼容的问题。

13.4.5 UI 测试：证明关键用户旅程，而不只是页面能打开

UI 测试可以包含三个不同目标：功能性端到端测试证明用户能够完成任务；视觉回归证明布局和样式没有异常变化；可访问性检查证明键盘、读屏和对比度等基本要求得到满足。三者不能用一张截图互相替代。

AI 在 UI 测试中的优势包括：根据用户旅程生成脚本、为页面元素选择更稳定的语义定位、自动准备数据、分析失败截图、比较视觉差异，并将自然语言场景转换为浏览器操作。多模态模型还可以发现遮挡、溢出、按钮不可见等传统 DOM 断言不易描述的问题。

与此同时，UI 自动化非常容易产生脆弱测试：坐标、动态文案、随机等待和复杂 CSS 选择器都会使脚本在没有真实缺陷时失败。AI 如果面对失败只会不断增加等待时间或改写选择器，就会制造“自动修复成功”的假象。因此 UI Harness 应要求：

- 优先使用角色、标签、测试 ID 等稳定语义定位；
- 等待业务状态而不是固定休眠；
- 测试关键旅程，不把所有组合都堆到 UI 层；
- 将视觉差异阈值、浏览器和屏幕尺寸固定；
- 失败时保存截图、视频、控制台和网络证据；
- 不允许 Agent 在没有审查的情况下自动更新 Golden Screenshot。

费用案例的 UI 测试不只是确认输入框存在，而应证明员工可以提交 5000.00 元、5000.01 元被明确引导到正确审批路径、错误提示可理解，并且系统没有产生重复申请。

13.4.6 SIT：证明多个系统在真实契约下共同工作

SIT 通常指 System Integration Testing，即系统集成测试。它验证的不是某个接口孤立可用，而是多个服务、数据库、消息队列、身份系统和第三方能力连接以后，端到端协作是否成立。

这正是 AI Coding 容易暴露问题的地方。不同 Agent 可以分别把报销服务、审批服务和通知服务实现得“局部正确”，却可能在字段含义、事件顺序、超时策略或幂等键上不一致。单元测试和 Mock 接口全部通过，也不能证明真实系统能够协作。

AI 可以帮助 SIT：

- 从 Work Graph、跨模块契约和调用链生成集成场景；
- 建立测试数据和环境依赖清单；
- 为暂不可用的外部系统生成 Stub 或服务虚拟化；
- 对日志、Trace、消息和数据库状态进行链路归因；
- 在失败后识别是提供方、消费方、数据还是环境问题；
- 根据影响图选择需要回归的跨系统路径。

但 Stub 只能证明双方符合约定，不能永久替代真实集成。关键链路在发布前仍应进入受控的真实或高保真环境。SIT 证据必须记录参与系统版本、接口 Schema、环境配置、测试数据、Trace ID 和外部依赖状态，否则“昨天 SIT 通过”无法证明今天这一批构建仍可集成。

费用案例的 SIT 至少要证明：报销服务产生正确审批请求；审批系统按金额路由；身份系统返回的角色被正确解释；审批结果能够回写；通知失败不会导致重复提交；同一业务单据重放时保持幂等。

13.4.7 性能测试：证明在明确负载下满足 NFR

“接口很快”“页面没有卡顿”“测试套件三秒跑完”都不是性能证据。性能测试必须同时说明环境、数据规模、负载模型、持续时间、指标和阈值。例如：在指定硬件、十万条历史报销数据和 200 个并发用户下，审批接口持续 15 分钟的 p95 响应时间不超过 300ms，错误率低于 0.1%。

AI 可以根据系统拓扑生成负载脚本，分析慢查询、调用链和资源瓶颈，比较基线与当前版本，并建议下一项诊断实验。它也可以根据代码影响图判断某次修改是否触发局部基准、容量测试、压力测试、稳定性测试或完整性能回归。

但这里有三条硬边界：

1. 普通单元测试或 `pytest` 总耗时不能替代专用性能基准；
2. Agent 不得为了获得全绿而擅自降低并发量、缩短持续时间或放宽阈值；
3. 性能结论必须带基线和环境，不能把共享测试环境的一次波动直接归因于代码。

性能测试的目的也不只是在发布前拦截。趋势遥测可以发现每次变化只有 3% 的退化，但连续十次后系统已经明显变慢。单契约 Dashboard 应保存本任务的基准差异，全局 Dashboard 则观察跨任务的性能债务。

13.4.8 安全、可靠性与数据验证：不能藏在“其他测试”里

一些团队把接口、UI 和 SIT 全部通过后就视为质量完成，把安全、可靠性和数据正确性留给最后一次扫描。这在 Agent 能够调用工具、修改配置和操作数据以后尤其危险，因为风险不只来自业务代码，还来自 Agent 的行动方式。

安全验证应覆盖依赖漏洞、密钥泄露、鉴权、越权、注入、租户隔离和工具调用审计；可靠性验证应覆盖超时、重试、熔断、降级、恢复和故障注入；数据验证应覆盖 Schema、迁移、精度、完整性、隐私和回滚后的状态一致性。

AI 能够生成攻击样例、扫描代码、设计故障实验并分析异常，但高风险安全结论不应只依赖同一个 LLM 的判断。确定性扫描、独立工具、受控攻击验证和人工安全审查应共同构成证据。对于生产凭证、删除数据和权限提升等不可逆行为，Harness 应在测试之外直接阻止，而不是等安全测试发现。

SAST 与 DAST 分别证明什么

SAST 是 Static Application Security Testing，即静态应用安全测试。它在程序不运行时分析源代码、字节码或构建产物，寻找不安全 API、污点传播、注入路径、硬编码凭证和危险编码模式。它适合在 IDE、提交和 CI 早期高频运行，反馈快、定位通常接近代码，但可能产生误报，也无法证明真实部署后的认证、配置和运行行为。

DAST 是 Dynamic Application Security Testing，即动态应用安全测试。它从外部向正在运行的应用发送请求和攻击载荷，根据响应发现注入、认证、授权、会话、输入输出校验和服务器配置等问题。它不需要读取源码，更接近攻击者视角，但必须具备可运行环境、有效身份、足够的攻击面覆盖和受控测试数据；扫描结果也未必能直接定位到代码。[OWASP 对 SAST、DAST 在 DevSecOps 流水线中位置的说明](#)；[OWASP DAST 说明](#)

二者不是二选一，也不能互相证明完全安全：

SAST：不运行程序，从内部寻找可疑实现
DAST：运行程序，从外部挑战真实行为
SCA：检查第三方依赖与已知漏洞
Secret Scan：检查凭证和敏感信息泄漏
IaC / Image Scan：检查基础设施配置与容器制品
IAST：在运行和测试过程中结合内部执行信息定位风险

SAST/DAST 怎样进入 P⇌E 快内环

最典型的循环是：

P：SAST 发现未过滤输入进入 SQL 执行
→ E：在授权文件范围内采用参数化查询
→ P：重跑同一规则、单元安全测试与回归

P：DAST 在测试环境证明普通用户可以读取他人报销单
→ E：修复对象级授权并补充审计日志
→ P：使用原身份、原对象和原攻击请求重新验证

但“扫描发现”不应直接等于“代码一定有漏洞”，扫描全绿也不等于安全完成。进入 E 前需要对发现去重、确认适用性和风险；修复后必须重跑原检查，不能通过加入全局忽略、降低严重级别或缩小扫描范围获得绿灯。误报可以被抑制，但抑制需要规则 ID、理由、责任人、范围和到期时间，并进入 Evidence Bundle。

并不是所有 DAST 都适合秒级内环。局部 API 安全用例和轻量扫描可以跟随每次候选构建；全站爬取、认证复杂的 DAST、性能敏感攻击、渗透测试和生产前安全评估通常属于较慢的 P⇌E 循环，需要专用环境、数据复位、速率限制与测试授权。高破坏性载荷不得由 Agent 因“需要更多证据”自行对生产运行。

红队、蓝队、紫队与高频对抗是什么关系

“高频对抗”是更广义的控制机制，红蓝对抗是其中一种安全实现，但不能把两个概念完全画等号。

- **红队（Red Team）** 站在攻击者视角，尝试绕过预期控制、串联弱点并实现攻击目标；它关注的不只是单个漏洞，还包括身份、流程、人员、Agent 工具和检测盲区；
- **蓝队（Blue Team）** 负责预防、检测、阻断、调查、恢复和修正规则；在 SCOPE-V 中，蓝队响应主要进入 E，但其修复结果必须回到 P 接受重新攻击；
- **紫队（Purple Team）** 不是简单的第三支队伍，而是一种协作方式：把红队攻击路径转化为蓝队能够复现的检测、测试、约束、遥测和恢复资产。

应用安全红队与 Agent 红队也应区分。前者攻击业务应用、API、身份和基础设施；后者还会测试 Prompt Injection、越权工具调用、数据外泄、Memory Poisoning、目标劫持、停止条件和人类接管。一次成熟演练可以同时覆盖两层：恶意 Issue 中的间接 Prompt Injection 是否能诱导编码 Agent 读取凭证并调用外部工具，以及 Harness 是否会发现、阻断并保存证据。

红蓝循环可以表达为：

P / 红队：提出攻击假设并保存攻击证据
↓
E / 蓝队：修复代码、配置、检测、权限或恢复机制
↓
P / 红队：沿原攻击路径重新验证
↓
紫队：把攻击与防御沉淀为测试、约束、告警和演练剧本
↓
V：独立审查覆盖、残余风险和是否允许放行

NIST SP 800-115 将安全测试视为需要设计、实施、分析发现并形成缓解措施的完整评估活动，而不是只运行扫描器。这与 P 产生攻击事实、E 修复、P 重验、V 独立裁决的职责划分一致。[NIST SP 800-115：信息安全测试与评估技术指南](#)

哪些安全结果不能留在自动快内环里

以下情况必须停止自动修复并进入 OA、安全负责人或 V：

- 发现可能已被真实利用、涉及数据泄漏或生产凭证；
- 修复需要改变认证模型、信任边界、加密方案或网络分区；
- 需要接受高危漏洞、关闭安全规则或扩大扫描豁免；
- DAST、渗透测试或红队攻击可能破坏数据、账户或服务可用性；
- 扫描工具之间结论矛盾，或关键攻击面无法覆盖；
- 修复会改变已签署业务规则、兼容性或监管义务；
- 红队与蓝队由同一 Agent、同一 Prompt 和同一上下文产生，缺少必要独立性。

因此，正确关系不是“安全扫描属于 Verify”，也不是“SAST/DAST 全部由 Agent 自动修到绿”。更准确的分工是：**C** 定义安全门槛和禁止动作，**O** 编排扫描与攻击节点，**P** 产生独立安全事实，**E** 完成受控修复，**V** 判断证据覆盖与残余风险，**Telemetry** 观察漏洞复发、误报、修复时长和攻击检测能力。

13.4.9 UAT：AI 可以执行场景，但不能代替业务接受

UAT 是 User Acceptance Testing，即用户验收测试。它回答的不是“软件是否按照技术规格运行”，而是“交付结果是否真正满足业务意图，业务是否愿意接受并承担上线后果”。因此 UAT 是 AI Coding 时代最不应该被错误全自动化的环节。

AI 可以承担大量 UAT Execution：

- 从意图契约生成业务语言的验收场景；
- 准备账号、订单、审批单等测试数据；
- 自动执行确定性操作并保存过程；
- 汇总接口、UI、SIT 和性能证据；
- 标记规则冲突、开放问题和需要业务判断的差异；
- 生成供业务代表快速审阅的演示与摘要。

但 UAT Acceptance 仍应由业务责任人完成。AI 不能代替财务负责人决定提示是否容易理解、审批政策是否公平、操作摩擦是否可接受，也不能替 IO 承担资金规则上线后的责任。

UAT Execution：AI 可以大量准备和执行
UAT Acceptance：业务负责人作价值裁决

业务人员也不应只是点击“同意”。UAT 记录至少包含场景、预期业务结果、实际结果、未解决问题、接受范围、限制条件、签署人和时间。如果业务代表发现 5000.01 元确实进入总监审批，但提示语会让员工误以为报销被拒绝，那么技术测试可能全绿，UAT 仍然有充分理由不接受。

13.4.10 生产验证：上线不是证据生命周期的终点

测试环境只能证明在已知条件下的行为。真实用户、真实数据分布、并发模式和第三方波动会暴露新的问题。因此验证证据组合还应声明上线后观察什么，以及什么信号触发回退、降级或 Bug 回溯。

AI 可以持续分析错误率、延迟、业务转化、异常审批比例和用户反馈，将异常关联到具体契约和发布版本。生产验证可以采用灰度、影子流量、金丝雀、Feature Flag 和合成监控等方式，在控制影响范围的同时取得真实证据。

但遥测不能被用来为未经验证的高风险发布开脱。上线后观察是对发布前证据的延续，不是“先上线再说”的替代方案。一旦发现逃逸缺陷，必须回到对应契约：修正首次成功状态、追加 Evidence Bundle、重新采集遥测并把教训回写 Intent Graph。

13.4.11 不同测试层在 SCOPE-V 中怎样分工

一个常见误区，是把单元、接口、UI、SIT、性能和 UAT 全部放到 Verify 最后执行。这样不仅反馈慢，也会把 V 重新变成传统测试阶段。SCOPE-V 将它们分布在不同控制层：

S 与 C：先定义证明责任

Specify 把业务意图写成可判定 AC；Constrain 把性能、安全、数据和恢复要求写成可执行边界。此时就应为关键 AC 选择验证层、执行主体和通过判据，而不是等代码写完再问“怎么测”。

O：把验证节点编入 Work Graph

Orchestrate 决定哪些测试先行，哪些可以并行，哪些等待真实环境，哪些结果是实现节点或发布节点的前置条件。接口契约测试可以在服务实现前产生 Red；性能环境准备可以与功能开发并行；UAT 必须等待关键业务链路具备可演示状态。

P⇌E：高频产生事实并驱动收敛

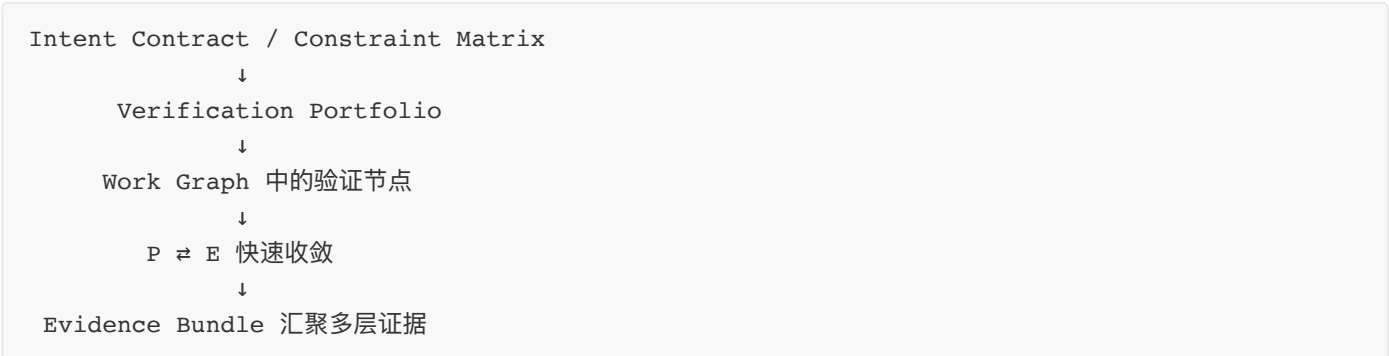
单元、组件、接口和局部 UI 测试通常进入高频快内环。SIT、性能、安全和可靠性测试也可以在条件具备后进入 P，失败则由 E 分类、修复并使用同一判据复验。每轮都要保存原始结果、修复动作和退出原因。

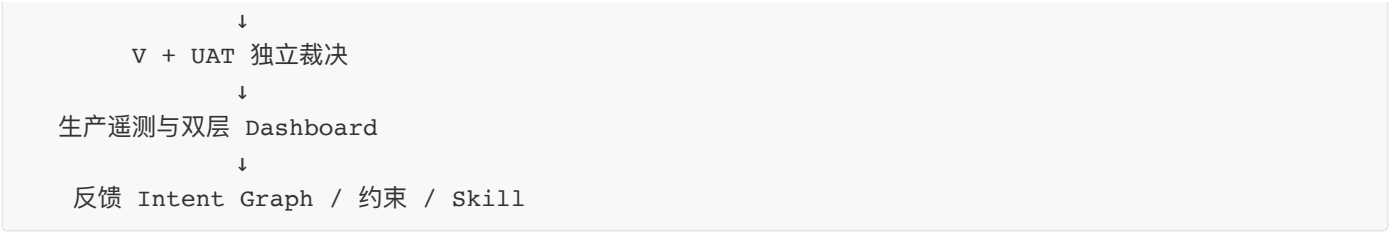
V：独立审查，而不是重新生产全部证据

Verify 判断各层证据是否覆盖签署意图和 MUST 约束，来源是否独立，环境和版本是否一致，证据是否新鲜，UAT 是否由正确责任人接受，以及回退条件是否成立。V 可以触发补测，但不能替 P 制造缺失证据，也不能因为时间紧而降低判据。

Telemetry：验证真实价值和长期趋势

上线后的质量、性能、成本、HITL 和业务结果进入契约级 Dashboard，再聚合到全局 Dashboard。慢外环将重复缺陷、脆弱测试、性能退化和高成本验证反馈给下一轮 S/C/O。





13.4.12 一个完整案例：5000 元规则需要哪些证据

假设签署意图是：“单笔人民币报销金额不超过 5000 元时进入普通审批；超过 5000 元时进入总监审批；不得修改历史草稿。”其验证证据组合可以这样设计：

验证层	要证明的事情	代表场景	主要执行者	输出证据
单元/组件	金额比较和币种规则正确	4999.99、5000.00、5000.01	测试 Agent	Red/Green、断言结果
接口	提交契约、鉴权和持久化正确	5000.01 返回正确审批级别	接口 Agent	请求响应、数据库断言
UI	员工能完成提交并理解结果	边界金额提示与审批展示	UI Agent	视频、截图、网络日志
SIT	报销、身份、审批、通知协作正确	总监节点、回写、幂等重放	集成 Agent	Trace、事件、系统状态
性能	规则变化未破坏容量目标	200 并发、十万历史记录	性能 Agent	基线、p95、错误率
安全/数据	无越权和历史数据副作用	普通员工越权、历史草稿检查	安全/数据 Agent	扫描、审计、数据差异
UAT	规则和交互得到业务接受	财务代表完成关键旅程	AI 辅助 + 财务负责人	UAT 记录与签署
生产验证	真实审批分布与体验正常	异常审批率、错误率、延迟	遥测系统 + OA	契约 Dashboard

这不意味着每个低风险修改都要运行八层验证。如果变更只修正一处无业务影响的展示错别字，Context 和影响图可能将组合裁剪为 UI 快照、可访问性检查和人工浏览；如果变更触及金额计算、权限或数据库迁移，即使代码只有三行，组合也必须升级。

验证成本应服从风险，而不是服从代码行数。

13.4.13 AI 参与测试的四级成熟度

团队可以用四级模型判断自己是否真正完成融合：

1. **AI 生成测试**：AI 根据代码补用例，主要改善编写效率；
2. **AI 按契约设计测试**：测试从 AC 和约束生成，开始建立意图追溯；
3. **AI 按风险编排验证**：系统依据影响图、历史缺陷和约束动态选择验证证据组合；
4. **AI 治理证据闭环**：多层结果进入 Evidence Bundle 和 Dashboard，V 独立裁决，生产反馈能够反哺图谱、约束和测试策略。

第一层容易演示，却可能只是“更快地补测试”；第四层才意味着测试成为 AI 研发治理系统的一部分。

13.4.14 多层验证的常见失败

- 同一个 Agent 解释需求、写实现、补测试并宣布完成；
- 测试从当前代码反推，与 Intent Contract 没有映射；
- 接口测试只断言 HTTP 200，不检查业务状态和数据副作用；
- UI 自动化大量依赖坐标、固定等待和脆弱选择器；
- SIT 长期使用 Stub，从未验证真实跨系统契约；
- 将普通测试套件耗时冒充性能基准；
- AI 自动更新 Golden Set 或截图，以消除失败；
- 把 UAT 执行自动化误认为业务已经接受；
- 所有测试全绿，却无法证明来自哪个提交和环境；
- Evidence Bundle 只附报告，没有建立 AC—约束—证据映射；
- 为了追求自动化率，把价值判断和不可逆批准也交给 AI；
- 上线后发现 Bug，没有回到原契约修正证据和遥测。

13.4.15 多层验证方法工具箱

除了按系统层次划分验证，还可以根据问题性质选择不同测试方法：

- **Example-Based Testing** 用具体样例验证典型与边界路径；
- **Property-Based Testing** 用大量生成输入验证业务不变量；
- **Differential Testing** 比较新旧版本、两个实现或两个模型的差异；
- **Metamorphic Testing** 在缺少标准答案时验证输入变化与输出变化的关系；
- **Mutation Testing** 主动制造代码变异，检查现有测试能否识别错误；
- **Golden Set** 使用经过人工校准的稳定样本验证模型或规则输出；
- **Contract Testing** 验证服务提供方与消费方的接口约定；
- **Chaos Testing** 注入故障，验证降级、恢复和可观测性；
- **Exploratory Testing** 由人或 AI 基于风险线索探索未预先列出的异常；
- **A/B、灰度与影子验证** 在受控生产条件下观察真实行为。

这些方法不是越多越好。团队应先识别本任务最可能出错、出错后损失最大、又最难由单一路径发现的部分，再选择证明力最强且成本合理的组合。

最终需要记住的不是测试类型清单，而是一条治理原则：

AI Coding 与测试融合的终点，不是让 AI 生成更多测试，而是让每项业务意图都拥有一组可信、独立、可追溯、与风险匹配的上线证据。

13.5 LLM-as-Judge 的边界

LLM-as-Judge 适合评价难以用单一断言表达的内容，例如解释是否完整、文案是否符合语气、方案是否覆盖给定维度、开放性回答是否与材料相关。它不适合替代金额计算、权限判断、状态转换、性能阈值和安全规则等可以确定性验证的事实。

问题	首选验证者
<code>5000.00</code> 是否触发审批	运行时断言或数据库检查
未授权用户是否被拒绝	接口与权限测试
页面说明是否清晰、完整	Judge 初评加人工校准
是否可以接受残余业务风险	有责任的人类

使用 Judge 时至少要固定评分 Rubric、输入材料、模型与提示版本，并准备一组人工已判定的校准样本。对同一输入可重复运行或使用第二评审路径，观察结论稳定性；只给分数不提供命中依据的结果，不应进入高风险发布裁决。

Judge 的输出应被视为一种有不确定性的证据，而不是事实本身。高风险结论必须由确定性检查、独立环境或人工责任人兜底；模型不能因为自信度很高就获得价值裁决权。

13.6 Proof-Carrying Delivery

Proof-Carrying Delivery 可以译为“携证交付”：交付物到达下一责任主体时，同时携带支持其结论的证据。朴素地说，**不要只说完成了，要把别人能够复查的依据一起交出来。**

一份合格证据需要具备四个属性：

- **可追溯**：能够从 Goal、AC 和约束找到对应证据；
- **真实**：保存原始命令、退出码和输出，而不是 Agent 的转述；
- **新鲜**：证据对应当前契约、当前提交和指定环境；
- **可复现**：接收者知道使用什么数据和步骤重新验证。

最小携证交付至少说明契约版本、代码与配置差异、每项 MUST AC 的检查、约束结果、环境与时间、未验证事项、残余风险、回退方式和例外批准。证据需要建立映射：`AC-01` 对应哪条测试，`C-SEC-02` 对应哪项扫描，回退结论对应哪次演练。

以报销规则为例，代码差异只是交付的一部分。Evidence Bundle 还要证明 `5000.00` 与 `5000.01` 的边界、异常人工队列、权限限制和功能开关关闭路径。若外市场景未验证，应明确标为 `unproven`，不能让“全部测试通过”掩盖验证范围。

携证也不等于附上一大包日志。接收者应能先看到结论与缺口，再沿 ID 下钻到原始依据。完成不再是执行者认为工作结束，而是结果和证据已经达到下一责任主体可裁决的状态。

13.7 验证门：不是只有通过和失败

验证门至少检查：测试全绿且测试数大于 0；AC 逐条有结果；编译或类型检查通过；Evidence Bundle 对应当前提交；关键约束没有失败；关键测试未被删除、skip 或削弱。

```
python scripts/gate_check.py --gate prove --task T-001 --project-dir .
```

门禁结果不应只有布尔值，至少区分四种状态：

- `passed`：判据满足且证据完整、新鲜；
- `failed`：已有证据明确证明不满足；
- `unproven`：缺少证据、环境不可用或映射不完整；
- `exception`：未满足但存在有效的人类批准、补偿控制和到期时间。

unproven 尤其重要。没有性能环境不能被解释为“性能默认通过”，测试报告来自旧提交也不能进入当前裁决。验证门只决定结果是否具备进入 Verify 的资格，不代替 IO 对价值、残余风险和上线作最终决定。

13.8 测试的独立性

让同一个模型根据自己的实现生成测试，可能把同一误解复制到代码和断言中。独立性不是简单换一个 Agent 名称，而是让验证至少在**判据来源、执行上下文或验证技术**上与实现拉开距离。

可以按风险选择三档：低风险任务由同一 Agent 执行，但测试必须来自签署 AC 并保留 Red；中风险任务由独立上下文或独立 Agent 生成关键反例；高风险任务再增加人工领域审查、Golden Set、不同技术路径或独立环境复核。

常用方法包括：

- 测试从签署 AC 和不变量生成，而不是阅读实现后反推；
- 关键测试文件对实现节点只读；
- 使用脱敏生产样本或历史事故建立 Golden Set；
- 用属性测试、数据库断言和外部接口观察交叉验证；
- 保留 Red、Green 及测试差异，检查断言是否被移动；
- 对高风险行为使用第二模型或人工设计反例。

独立性不要求所有工作都由不同的人完成。它要求关键结论不能只依赖产生该结果的同一条认知路径。

13.9 什么是无效全绿

绿色只能证明“被运行的检查没有报告失败”，不能自动证明意图已经实现。以下情况都属于无效或可疑全绿：

- 测试从未出现可信 Red，或者 Red 来自环境损坏；
- 测试没有有效断言，关键用例被 skip、only 或条件排除；
- 为适应实现修改了期望值、边界或 Mock；
- Mock 掉了真正需要验证的数据库、权限或外部契约；
- 只检查字符串、函数名或文件存在，没有验证运行行为；
- 测试数据绕开了关键边界和失败路径；
- 测试环境、制品或配置与发布目标不同；
- 编译、测试和证据来自不同提交。

验证门可以做一次“绿色审计”：比较 Red 到 Green 之间测试文件与断言差异，统计 skip/only，核对测试数量是否异常下降，检查 Evidence Bundle 的提交摘要和环境指纹，并随机重跑一条关键 AC。若绿色无法回答“证明了哪条 AC、使用了什么输入、在什么版本上运行”，它只是一个界面颜色。

13.10 Prove 常见失败

Prove 的失败大致来自四类：判据失真、执行失真、证据失真和责任失真。

- **判据失真**：先写完实现再补测试；AC 从代码反推；只覆盖快乐路径；
- **执行失真**：Red 来自环境损坏；用 Mock 绕过真实依赖；性能需求只看普通测试总耗时；
- **证据失真**：只保存最终绿色结果；测试与制品不是同一提交；Judge 没有记录版本与 Rubric；
- **责任失真**：实现 Agent 修改关键断言；为赶时间删除失败测试；机器替业务责任人接受风险。

遇到这些问题，正确动作不是继续增加测试数量。应先判断错误发生在哪一层：AC 不清回到 Specify，约束缺失回到 Constrain，节点权限或职责不清回到 Orchestrate，环境损坏先修复验证基础设施，证据不一致则重新取证。Prove 只有在判据和验证环境可信时才有意义。

13.11 从测试数量转向证明强度

一百个与风险无关的单元测试，可能不如三个直击关键边界的测试有价值。证明强度至少取决于五个维度：

1. **相关性**：是否直接对应已签署 AC、约束或关键风险；
2. **区分力**：是否能够让一个常见错误实现失败；
3. **独立性**：是否避免与实现共享同一错误假设；
4. **真实性**：环境、数据、依赖和制品是否接近裁决对象；
5. **可追溯性**：结果是否可回到命令、版本、输入和责任人。

覆盖率只能说明哪些代码被执行，不能说明业务语义正确。对于“严格大于 5000 元”的规则，4999.99、5000.00、5000.01 三个边界测试，比大量随机正常值更有区分力；再加一条真实接口路径和一条数据库状态断言，证明强度又高于只在函数层重复同一判断。

团队可以在 Evidence Bundle 中为关键 AC 简单标记上述五维的强、中、弱，用于暴露证据短板，而不是计算一个看似精确的总分。改进目标是补最薄弱的证据维度，不是机械增加用例数。

13.12 谁来验证验证者

当同一个 Agent 同时解释需求、编写实现、生成测试并总结结果时，偏差可能贯穿整条链。验证者也必须受到约束和抽查，但不能因此陷入“无限再找一个验证者”的递归。

可以建立三层终止结构：第一层由确定性工具验证事实，如编译器、断言、数据库检查和安全扫描；第二层由独立 Agent 或独立上下文检查 AC—测试—证据映射、寻找反例和识别证据矛盾；第三层只把价值判断、例外授权和不可逆风险交给有责任的人类。三层职责不同，不是简单重复执行同一测试。

对验证者本身，应检查：它使用的契约和提交是否正确，是否拥有修改被验证对象的权限，是否保存原始输出，是否存在跳过项，以及结论能否由另一条路径抽查。高风险规则可以定期注入已知错误或使用 Mutation Testing，确认验证系统真的能够抓住失败。

独立性的目标不是制造更多流程，而是打断“同一个错误假设自我证明”的闭环。当确定性证据充分、独立映射检查通过且责任边界明确时，验证链就可以终止；不需要让 Agent 无休止地互相评审。

13.13 本章小结

Prove 让“写代码”从中心动作变成验证循环中的一个步骤。Agent 不只是生成实现，还必须产生能够否定错误实现的测试和证据。

本章最小实践：为 T-001 从 AC 生成测试，记录 Red，再实现 Green。

完成证据：Red 与 Green 两次真实运行结果。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，先定义证明路径，再让 Agent 进入实现，避免把生成结果当成完成。

第十四章 | Evolve：让失败进入系统能力，而不是只进入复盘

14.1 失败不是例外，而是反馈

概率性模型、复杂系统和不完整上下文决定了失败不可避免。模型可能误解上下文，工具可能暂时不可用，多个 Agent 可能产生冲突，原本看似完整的规格也可能在真实数据面前暴露遗漏。

这并不意味着降低质量要求。既然失败不可完全避免，就要把发现失败、限制影响、恢复状态和保留证据设计成系统能力。一个从未报告失败的 Agent 系统未必更成熟，也可能只是测试过宽、日志缺失或失败被自动重试掩盖。

Evolve 关注两个层次：当前任务怎样从失败中恢复；组织怎样避免同类失败反复发生。前者可能是一处代码修复或任务重调度，后者则可能需要增加 AC、更新约束、修正图谱、改进工具权限或改变 Work Graph。

因此，对失败的第一反应不应是立即重试，而是先问：这是什么类型的失败，影响是否仍在授权范围内，我们是否拥有可信证据，继续尝试会不会扩大风险？成熟系统的差异不在于“从不失败”，而在于失败是否被正确分类、是否在边界内修复、是否留下可复用教训。

14.2 失败分类与回流路由

失败发生后，第一步不是修代码，而是形成一张最小诊断卡：**哪项检查失败、首次出现在哪个节点、原始信号是什么、影响范围多大、是否仍在授权边界内**。没有这些信息，“自动修复”很容易把规格、环境和权限问题都误诊为实现缺陷。

失败类型	识别信号	应回到哪里
目标或规则不清	AC 无法判定、示例相互矛盾	Specify，重新确认并签署
约束缺失或冲突	两条 MUST 无法同时满足、风险无人负责	Constrain，由 OA/责任人决策
依赖或编排错误	输入过期、写冲突、错误节点提前启动	Orchestrate，调整 Work Graph
实现缺陷	规格稳定、测试可信、候选行为错误	$P \Leftrightarrow E$ ，最小修复并重跑原检查
测试或环境故障	语法、依赖、夹具或环境导致假 Red	Prove，先修复验证基础设施
证据不足或不一致	缺少原始输出、提交与报告不匹配	补取证后再进入 Verify
权限、法律、伦理风险	需要越权、不可逆动作或责任裁决	立即停止，升级 HITL

路由应由失败分类器或 OA 根据证据决定，不能由当前实现 Agent 按“最容易继续”的路径选择。分类不确定时，状态应是 `needs_diagnosis`，而不是默认进入重试。正确返回上游看似变慢，却能避免在错误目标或坏环境上高速迭代。

14.3 可自动修复的条件

AS 只有在失败落入预先定义的“修复包络”时才适合自治修复：根因可定位，修改范围仍在授权内，不改变 Goal、Non-goals、Rules 和关键 AC，不降低 MUST，不需要新权限，并且修复后可以重跑同一项检查。

修复包络至少包含：允许修改的文件或资源、允许的失败类型、最大轮次、时间与 Token 预算、必须重跑的检查，以及立即停止的信号。例如：

```
auto_repair:
  allowed_failures: [implementation_defect, transient_tool_error]
  write_scope: [src/domain/expense/**]
  max_rounds: 2
  must_rerun: [AC-EXP-001, AC-EXP-002, domain-regression]
  stop_on: [rule_conflict, permission_expansion, new_must_failure]
```

网络抖动可以退避重试；比较符号错误可以在限定文件内修复；数据语义冲突、权限不足和不可逆操作则不能靠“再试一次”解决。每轮修复后如果失败类型变化、影响范围扩大或新增错误多于被消除错误，都应退出自动循环并升级。

14.4 修复证据：证明问题真的被消除

修复完成不能只保存最后一次 Green。每轮至少形成一条可比较记录：失败的 AC 或约束、原始错误、根因分类、修改差异、是否扩大范围、重跑命令、结果和下一动作。

一条可信修复链应当能够回答：

原检查为什么失败

- 哪个最小变化针对该根因
- 同一检查是否从失败变为通过
- 相邻回归和约束是否仍然成立
- 是否引入新风险、例外或人工决定

“修改后改跑另一套更容易通过的测试”不能证明恢复；只保留最终日志也无法判断 Agent 是否移动过判据。Evidence Bundle 应保存修复前后的命令、提交摘要和关键输出，并记录测试或 AC 是否发生变化。若判据确需变化，应先走规格漂移，而不是把它混进修复差异。

对报销边界错误，证据不仅要显示 5000.01 已通过，还应重跑 5000.00 和领域回归，证明修复没有把另一侧边界或既有审批路径破坏。修复证明的是“原失败已消除且影响受控”，不只是“当前屏幕变绿”。

14.5 反思与 Failure Mining

Failure Mining 可以译为“失败挖掘”：从多次失败记录中寻找可复用模式，而不是只修复眼前的一处代码。它与普通复盘的区别在于，输入必须来自真实证据，输出必须能够改变后续系统行为。

反思不应只是模型写一段“下次会更努力”。有效反思要从证据中提取：

- 哪个假设错误；
- 哪条约束缺失；
- 哪类上下文造成误导；
- 哪个检查最早发现问题；
- 哪个模式值得复用；
- 哪些知识需要进入图谱；
- 哪些记忆需要失效或衰减。

例如三个任务都因“业务阈值等号边界未确认”返工，Failure Mining 得到的不是三个独立 Bug，而是一个通用模式：含“超过、不少于、以内”等自然语言阈值时，Specify 必须生成等于、略低、略高三个边界例子。这个模式可以进入 Grill-Me 提示、Example Mapping 清单或约束检查。

一次失败适合修复局部实现；重复失败才可能说明流程、约束或知识资产有系统缺口。团队应保留失败类型、发现阶段、修复成本和影响范围，再判断是否值得提升为全局规则。否则每个偶然故障都新增门禁，会快速积累规则债务。

Failure Mining 的产出至少要回答“以后什么会不同”：下次会多问哪个问题、注入哪条上下文、运行哪项检查、限制哪个工具，或淘汰哪条过期记忆。没有改变治理系统的反思，只是文字总结。

14.6 规则债务与约束疲劳

失败不断转化为新规则，会形成另一种技术债：同义规则重复、旧规则与新架构冲突、门禁越来越慢、上下文被规则淹没，最终团队习惯性申请例外或直接关闭检查。规则多不等于治理成熟。

每条长期约束至少要有 Owner、来源事故或风险、创建版本、最近命中时间、误报率、执行成本和复核日期。团队可以定期把规则分成四类：保留、合并、降级为告警、删除。经常命中且真正阻止高损失风险的规则应保留；长期零命中但风险仍然存在的安全底线不能仅凭数据删除；频繁误报或持续申请例外的规则则必须重新设计。

清理时也要防止“为提高通过率而删门禁”。任何删除或降级都应说明风险是否已被其他控制覆盖，并在有限范围验证。Dashboard 可观察门禁耗时、误报、例外率和重复规则数，让规则债务像代码债务一样可见，而不是等开发者集体绕过时才发现。

14.7 Champion-Challenger 与有界演进

Champion-Challenger 是一种“现行方案与候选方案并行比较”的方法。Champion 是当前经过验证、正在使用的稳定方案；Challenger 是希望表现更好的候选方案。它解决的问题是：系统需要学习，却不能每出现一个新想法就直接替换生产规则。

例如当前上下文裁剪策略平均向 Agent 传入 30 个文件，这是 Champion。团队提出一种基于图谱关系的新策略，只传入 12 个文件，这是 Challenger。不能仅因为 Token 更少就立即替换，还需要比较目标准确率、依赖遗漏、首次成功率、人工介入和安全风险。

受控比较通常经过六步：

1. 明确 Challenger 想改善的指标和不能退化的底线；
2. 选择沙箱、历史回放、影子流量或低风险任务；
3. 保证两种方案面对可比较输入；
4. 保存结果、成本、失败和异常证据；
5. 达到预设门槛后由责任人决定晋升、继续观察或淘汰；
6. 晋升后仍保留回退到旧 Champion 的能力。

“有界演进”强调候选方案只能在批准范围内学习。Agent 不得自行修改全局约束、业务目标或发布阈值来提高分数，也不能用单一指标取胜。Challenger 即使降低了 Token，只要关键依赖遗漏率上升，就不能宣布成功。

Champion-Challenger 不只适用于模型，也适用于 Prompt、上下文裁剪、路由、测试生成和恢复策略。它让组织在保持稳定基线的同时持续试验，是“可以进化”与“不能失控”之间的桥梁。系统可以演进，但不能自主修改它用来证明自己正确的标准。

14.8 意图图谱回写：只沉淀经过验证的变化

任务结束前，应把新增能力、已确认规则、失败教训、依赖变化和后续方向回写 Intent Graph。但回写不是把任务总结整段粘贴进去，而是更新能够影响未来检索和决策的节点、关系与属性。

每条回写至少应带来源任务、契约版本、证据链接、适用范围、置信状态和责任人。例如“金额阈值必须检查等号边界”可以标记为来自 T-001 的已验证模式；“外币可能复用同一阈值”若尚未确认，只能作为待验证假设，不能升级为组织事实。

回写前要经过 Memory Write Gate：事实是否有证据，规则是否已获授权，是否与现有节点冲突，是否需要替代旧版本。回写后还要验证图谱引用能够解析、旧关系是否失效，以及下一任务的 Context 裁剪是否会取得新信息。

跳过回写，下一任务可能重复同一错误；无筛选地回写，则会把偶然结论和过期经验放大为系统偏见。Evolve 追求的是高质量更新，不是图谱节点数量增长。

14.9 停止条件：自治系统也必须知道何时认输

没有停止条件的自我修复会持续消耗 Token、污染更多文件，并把一个局部错误演化成系统性风险。停止条件应在执行前写入节点契约，而不是由 Agent 失败后自行判断“还值得再试一次”。

常见停止信号包括：达到最大轮次、Token 或时间预算；同一失败重复出现；新增失败持续增加；根因从实现问题变成规格或约束冲突；需要扩大权限或修改关键 AC；证据可信度下降；回滚安全无法证明；出现不可逆数据风险。

触发停止后，系统要执行四个动作：冻结受影响节点和输出、保留现场证据、撤销临时权限或锁、生成面向 OA/HITL 的裁决包。状态应明确区分 `stopped_by_budget`、`blocked_by_spec`、`blocked_by_constraint` 和 `unsafe_to_continue`，方便正确路由。

停止不是把任务标成失败后结束。它是系统主动防止风险继续扩散，并把“机器为什么无法继续”转换为人类可以裁决的信息。

14.10 记忆的来源与衰减

只有经过验证的事实、明确的人类决定和重复出现的稳定模式，才适合进入长期记忆。模型猜测、一次性 Workaround、未批准方案和过期架构信息必须带状态，不能作为事实自动注入下一任务。

记忆至少要记录来源、证据、适用范围、创建版本、Owner、置信状态和复核或失效条件。可以按稳定性设置不同生命周期：组织原则长期保留但定期复核；项目模式绑定架构版本；任务教训绑定模块与契约；临时例外到期自动失效；未验证假设默认不参与自动决策。

衰减不只是按时间删除。接口替换、规则改版、约束失效、反例出现和责任人否决，都应触发记忆降权或失效。检索时优先使用来源清楚、与当前版本匹配、近期被验证的记忆，并向 Agent 显示其状态，而不是把所有历史内容作为同等可信上下文。

Memory Write Gate 控制什么可以写入，Memory Read Policy 控制什么可以取出。两者缺一，组织记忆就会从经验资产变成高置信度的旧信息污染源。

14.11 Evolve 常见失败

Evolve 的失败通常不是“没有写复盘”，而是学习没有真正改变系统：

- **错误路由**：所有失败都自动重试，规格、环境和权限问题也被当作代码缺陷；
- **虚假恢复**：修复后换了一项检查，或只保存最终绿色结果；
- **空洞反思**：只写“下次更仔细”，没有新增问题、规则、测试或路由；
- **污染记忆**：把未经验证的猜测写入图谱，记忆只增加不衰减；
- **失控演进**：系统自行修改 MUST、AC 或评价标准；
- **伪实验**：Champion 与 Challenger 使用不同数据集，或只比较对候选方案有利的指标；
- **数据美化**：事后 Bug 被当成新任务，原任务首次成功率仍保持 100%。

检查一次 Evolve 是否有效，可以只问一句：**下一次相似任务具体会在哪个环节表现得不同？** 如果回答不出新增的上下文、测试、约束、路由、恢复动作或失效记忆，这次演进仍停留在文字层面。

14.12 组织记忆不是聊天记录仓库

保存全部对话解决的是归档问题，不是记忆问题。聊天记录包含试探、误解、已否决方案和过期状态；若被整体注入下一任务，Agent 很难区分最终决定与讨论过程，还会消耗大量上下文。

同一次失败应按性质进入不同载体：实现缺陷进入代码和回归测试；当前任务状态与局部教训进入 Loop Memory；重复出现的红线进入 Constraint Matrix；稳定业务关系进入 Intent Graph；工具风险进入 Tools Manifest；可复用处置步骤进入 Playbook。载体决定它由谁维护、何时检索和怎样失效。

高质量组织记忆应当“小而可用”：有明确结论、来源证据、适用范围、检索标签、Owner 和失效条件。原始聊天可以作为审计材料保留，但默认不进入 Agent 工作上下文；需要追溯时再沿记忆条目的来源链接查看。

组织记忆的价值不在于“什么都没忘”，而在于未来任务能在正确时机取到正确经验，同时不会把过去的错误猜测当成今天的事实。

14.13 演进也需要变更治理

Agent 从真实失败中提出的新规则，也不能自动成为全局政策。一次局部事故可能源于特殊环境，若直接推广，可能给所有任务增加误报和成本。演进项应像代码一样经过提出、验证、批准、发布和回退。

最小流程是：记录失败证据 → 形成候选规则或策略 → 分析适用范围及与现有约束的冲突 → 由对应 Owner 批准进入 Challenger → 在历史回放、影子或低风险任务中观察 → 达标后晋升并保留旧版本回退。

涉及 Goal、业务规则和风险承担的变化由 IO 或领域责任人批准；安全和数据底线由相应责任人批准；OA 负责组织证据与影响分析，不能代替他们签署。晋升时还要更新版本、受影响图谱、约束、测试和 Context 策略，避免只有规则文本变化，执行系统仍使用旧版本。

这使 Evolve 区别于无限自我修改：系统可以提出并验证学习，但不能自行修改它用来评价自己的标准。

14.14 Bug 回溯门：完成之后发现错误也必须回流

任务关闭后发现 Bug，不能只创建一个新故事并保持原任务“全绿”。Bug 是对原完成结论的新反证，必须回到原契约、原 Evidence Bundle 和原遥测，修正组织对真实交付质量的认知。

最小回溯动作包括：确认 Bug 归属的契约与 AC；保存复现输入和影响范围；在原 Evidence Bundle 追加事后记录；把首次成功状态修正为失败或自愈；重新采集测试、修复轮次和成本；把漏测原因回写测试、约束、图谱或记忆。

```
python scripts/gate_check.py --gate bug --task T-001 --project-dir .
```

例如上线后发现 5000.00 元在移动端接口仍进入总监审批，不能只给移动端开一个修复任务。团队还要回到 T-001，标记接口证据组合存在缺口，补充跨入口契约测试，并修正“首次通过”的 Dashboard 数据。否则组织会不断制造看似成功的旧任务和彼此无关的新 Bug，永远学不到验证体系哪里失效。

如果 Bug 暴露的是普遍模式，例如所有入口都可能各自实现阈值规则，还应启动 Failure Mining 和影响扫描。Bug 回溯门的目的是不是追责某个人，而是让完成事实、度量和组织学习重新与现实一致。

14.15 Agent Chaos Engineering：主动验证系统能否发现、停止与恢复

Agent Chaos Engineering 可以译为“智能体混沌工程”。它借鉴分布式系统混沌工程的思想：与其等真实事故证明系统脆弱，不如在受控环境中主动制造小故障，检查 Agent、门禁、恢复和人类接管是否真的有效。

这里测试的对象不只是应用代码，还包括整条 Agent 治理链。例如工具权限写着“禁止生产写入”，但可能从未有人验证阻断是否生效；Work Graph 写了超时升级，却可能在真实超时时无限等待。混沌实验把这些纸面承诺变成可观察事实。

成熟团队可以注入：工具不可用、上下文过期、依赖超时、权限拒绝、测试环境异常、遥测丢失或模型输出矛盾。一次合格实验需要先写“稳态假设”，例如“生产写入请求必须被阻断，任务在 30 秒内进入人工审批状态”。随后限定实验环境和影响半径，注入单一故障，观察真实行为并保留证据，最后恢复环境并修正规则。

以“删除必要上下文”为例，预期不是 Agent 仍然神奇完成，而是它能识别信息不足，停止生成高置信度结论并向 OA 请求缺失材料。如果 Agent 静默猜测并通过了宽松测试，说明评估与停止条件存在缺口。

混沌工程不能直接在高风险生产环境随意尝试。应从沙箱、模拟工具和低风险任务开始，明确中止条件、负责人和恢复动作。目标不是制造更多失败，而是验证四件事：能否发现、能否停止、能否恢复、能否把教训写回治理资产。

混沌实验的输出不是一次演示，而是一项演进输入：暴露出的检测盲区进入测试，越权路径进入约束，错误重试进入 Work Graph，恢复失败进入 Harness，反复出现的模式再经过 Memory Write Gate 沉淀。它因此服务于 Evolve 的系统级学习，而不是发布前临时增加的一项检查。

14.16 本章小结

Evolve 把修复、反思和知识更新连接起来。自治不是永不找人，而是知道哪些失败可以自己修、哪些必须停止，以及如何用证据证明已经恢复。未经 V 确认的反思只能作为候选记忆，不能直接晋升为长期规则或组织事实。

本章最小实践：故意注入一个边界错误，完成失败分类、修复、重跑和图谱回写。

完成证据：先红后绿的运行记录和一条可复用教训。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，把失败、修复和知识回写变成下一轮执行可以使用的资产。

第十五章 | Verify：从全绿到基于证据的责任裁决

Prove 与 Evolve 已经让候选结果经历测试、失败、修复和重新取证，但“内环已经收敛”仍不等于“组织应当上线”。Verify 必须站在执行和自愈之外，检查目标、边界、证据、时效、残余风险与恢复条件，再由拥有责任的人作出行动裁决。

15.1 测试通过不等于上线批准

测试通过只能证明“在给定环境、给定输入和给定断言下，已经覆盖的行为符合预期”。它不能自动证明需求本身正确，不能证明没有遗漏场景，也不能替组织接受发布风险。

上线还涉及测试之外的问题：目标是否仍有价值，生产环境是否准备好，数据迁移能否回退，监控和告警是否存在，权限是否经过批准，未验证事项是否可接受，事故发生时谁负责接管。Verify 的任务就是把分散证据组织成可裁决结论。

读者可以把 Verify 理解为法庭上的“证据审理”，而不是流水线最后一个绿色按钮。测试报告是一组证物，约束结果、代码差异、回退演练、运行记录与人工审查也是证物。Evidence Bundle 负责组织本次任务事实，OA 检查证据链，IO 或授权责任人作出发布、灰度、补证、拒绝等裁决；Telemetry 在这些任务事实进入事件账本并聚合后，才用于跨任务趋势判断。

因此，Verify 允许出现“测试全绿但暂缓上线”，也允许在明确残余风险、灰度范围和回退条件后作出有边界的批准。关键是决定由证据支持，并且责任主体清楚。

15.2 六维验证

“六维验证”不是把测试重复六遍，而是从六种不同失败来源检查同一交付。单元测试只能覆盖其中一部分，任何一个维度缺失都可能产生一种特定的假成功。

验证体系包括：

- 1. **契约 AC 验证**：证明本次承诺的行为是否实现；
- 2. **Evidence Bundle 完整性审计**：检查关键结论是否都有证据；
- 3. **图谱、契约、约束一致性**：防止高质量完成了错误或过期目标；
- 4. **NFR 验证**：检查安全、可靠性、性能和可观测性；
- 5. **回滚安全验证**：确认失败时能够恢复，且不会破坏下游状态；
- 6. **工件时效与版本一致性**：确认测试、证据和签署对应当前代码与规则。

未知项必须标记为 UNKNOWN，不能被默认当作通过。

费用报销任务可能在第一维全部通过，却在第三维发现测试依据旧契约；也可能功能正确，却在第五维发现数据迁移不可回滚。六维验证让团队看见“功能正确”之外的发布条件。

不同风险可以调整六维强度，但不能用“暂不检查”伪装成通过。低风险文案变更的回滚验证可能只是版本恢复；高风险资金规则则需要受控环境演练。未知项必须标记为 UNKNOWN，进入裁决，而不能因缺少数据默认当作 PASS。

15.3 Evidence Bundle 的证据映射

理想证据包能够回答：

每条 AC → 哪项测试或运行证据
每条 MUST → 哪个约束检查
每个变更 → 属于哪个授权范围
每次失败 → 如何修复并重新验证
每项风险 → 由谁接受、怎样回退

证据来源优先级通常是：真实运行与机器结果高于模型自述，经过校准的人工判断高于未经解释的自动评分。

15.4 收尾门：没有完整证据就不能宣告完成

收尾门机械检查：

- Evidence Bundle 存在；
- 单任务遥测存在；
- 项目遥测包含本任务；
- 仪表板已刷新；
- 意图图谱包含本任务和教训。

```
python scripts/gate_check.py --gate closing --task T-001 --project-dir .
```

未通过收尾门，任务不能标记完成。

收尾门要求遥测文件与 Dashboard 已生成，不是因为 Verify 负责分析长期趋势，而是因为一次裁决必须确认本次任务的真实信号已经进入外环。Verify 检查“遥测是否完整可信”，下一章再讨论“这些信号怎样被度量和解释”。

15.5 人类发布裁决

人类发布裁决不是让负责人在几十页日志末尾点击“同意”。有效裁决必须把技术事实翻译成可理解的选择：已经证明什么、尚未证明什么、残余风险是什么、影响范围多大、继续与停止分别有什么后果。

常见裁决至少包括六类：

- **批准**：证据充分，残余风险在授权范围内；
- **条件批准**：只允许灰度、限定时间或限定用户，并绑定监控与回退条件；
- **补充证据**：目标不变，但关键验证缺失；
- **拒绝**：当前结果不满足目标或风险不可接受；
- **撤回**：先前批准所依据的证据已经失效；
- **升级**：超出当前责任人的授权，需要安全、法务、财务或更高层级决定。

OA 可以把 Evidence Bundle 转成决策摘要，并给出推荐方案及理由，但不能替代责任人签署。IO 也不能只看 Agent 的自然语言总结，应能访问真正影响决定的原始证据。

裁决还应写明生效范围、有效期、责任人和撤回条件。例如“批准两个部门灰度 24 小时；错误率超过 0.5% 自动关闭功能开关；次日上午由财务负责人复核”。这样的决定比单一的“同意上线”更可执行，也更容易审计。

15.6 发布裁决示例

T-001 的 Evidence Bundle 显示：

- 两条核心 AC 通过；
- 全量回归通过；
- 未触碰生产数据；
- 发生一次边界值失败并已修复；
- 外币报销未验证，已列为 Non-goal；
- 回退开关可用；
- 单任务遥测已采集。

合理裁决不是“功能完全没有风险”，而是：批准在境内人民币差旅范围发布，保持外币路径不变，并在发布后观察审批路由异常率。

15.7 从“全绿”到上线裁决

发布裁决不是把所有检查结果做一次逻辑与。一个任务可能测试全绿，却仍因数据迁移不可逆、监控尚未接入或业务窗口不合适而暂缓；也可能存在低风险已知缺陷，在 IO 明确接受、具备回退和监控的情况下限量发布。

Evidence Bundle 应帮助决策者区分三类事实：已经证明的、尚未证明的、明确接受的风险。OA 对证据完整性负责，IO 对价值与残余风险选择负责。任何角色都不能用“AI 判断可以上线”替代责任签署。

15.8 Verify 常见失败

- 把单元测试全绿等同发布批准；
- UNKNOWN 被默认视为通过；
- 证据与当前提交、制品或契约版本不一致；
- 只展示最终结果，不展示失败与修复历史；
- 人类批准没有范围、条件、有效期和撤回条件；
- 由产生实现的同一 Agent 生成证据并自行宣布通过；
- 缺少回退、监控或责任人，却因为时间压力直接上线。

15.9 本章小结

Verify 不重复执行者的自检，也不替 Evolve 修复结果。它以独立视角审查契约、约束、证据、版本、UNKNOWN、回退与责任链，并把“批准、条件批准、补证、拒绝、撤回或升级”变成明确裁决。测试全绿只是重要证据之一，不是自动上线权。

本章最小实践：为 T-001 汇聚六维证据，运行收尾门，并形成一份带范围、期限、监控、回退和撤回条件的发布裁决。

完成证据：Evidence Bundle、收尾门输出与人类签署的裁决记录。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，用 Evidence Bundle 和人类裁决判断什么是真正完成。

第十六章 | 价值与效能遥测：证明 AI 转型真正兑现了提效

Verify 解决一次交付能否被批准，Telemetry 解决人机协同研发系统在多次交付中是否真的变得更准确、更自主、更经济、更会学习。二者共享契约 ID 与 Evidence 引用，但时间尺度和责任不同：前者面向单次裁决，后者面向趋势、异常与组织改进。四层九维 Dashboard 因此不是“AI 使用量看板”，而是一套 AI 研发价值与效能管理视图，用来证明提效是否真实、健康并可持续。

16.0 遥测的三层聚合边界

四层九维模型要与三层人机协同架构对齐。任务运行层采集执行、验证、Token、上下文和人工接管事实；价值网络层观察价值流吞吐、等待、返工、协作和能力变化；战略意图层只讨论跨周期的业务结果、风险暴露和是否继续投资。任何一层都必须保留来源、口径和失效状态，不能把任务层的 output 直接抬升为组织层的 outcome。

因此，Dashboard 的正确用法是下钻和裁决：先从组织趋势发现异常，再回到价值流和任务 Evidence 查明原因；不以 Agent 数量、工具激活率或 Token 消耗给个人和团队排名。提效只有在相同任务类型与风险边界下，连续改善有效交付、等待和返工，同时没有恶化质量、责任和恢复能力时，才可被称为转型结果。

16.1 从度量到遥测：Agentic Agile 的反馈控制系统

传统研发效能领域早已积累大量周期、吞吐、质量、稳定性和组织健康指标。AI Coding 并没有让这些度量失效，反而增加了一组过去很少被系统观察的问题：Agent 是否理解了正确目标？它经历多少轮尝试才完成？哪些错误被自主修复，哪些风险升级给人？消耗的 Token 对应什么价值？一次失败是否改善了下一次执行？

因此，Agentic Agile 需要的不是再造一套孤立指标，而是把传统研发效能与 AI 原生运行信号连接起来。前者观察研发系统交付得怎么样，后者解释 Agent 为什么得到这个结果、付出多少算力、触发什么边界，以及自治范围是否值得扩大。

16.1.1 先区分度量、指标、遥测、证据与 Dashboard

这五个概念经常在管理讨论中被混成“数据”，但它们承担完全不同的治理责任。

概念	核心问题	例子
Measurement（度量）	我们想判断什么变化？	判断 Agent 是否在相同风险下提高有效交付能力
Metric（指标）	用什么口径表达这种变化？	首次成功率、逃逸缺陷率、每有效任务 Token
Telemetry（遥测）	如何持续、自动、低干扰地获得信号？	从测试、门禁、Agent 日志和客户端用量自动采集事件
Evidence（证据）	哪些事实足以支持本次裁决？	T-101 的测试、约束、回退、人工审查和生产结果
Dashboard（仪表板）	如何让人发现异常、下钻原因并采取行动？	单契约体检页和项目价值效能趋势视图

可以简化为两条共享任务事实、但服务不同时间尺度的链：

单任务链：Intent Contract → 执行事实 → Evidence Bundle → Verify 裁决

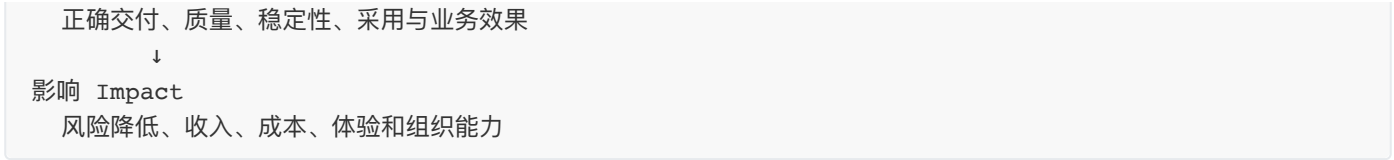
慢外环链：治理问题 → Measurement Contract → 遥测事件
→ 跨任务聚合与 Dashboard → 治理动作 → 下一轮 S / C / O

两条链通过 Contract ID、Evidence ID 和 Release ID 相互追溯，却不能互相替代。度量定义判断框架，遥测提供连续信号，Evidence 支持单次裁决，Dashboard 支持趋势判断。缺少任何一环，数据都可能沦为装饰。

16.1.2 从“产出多少”转向“有效交付能力”

代码行数、提交数、故事点、模型调用次数和生成代码占比都容易获得，却未必代表价值。AI 可以在几分钟内生成大量代码，也可以同时制造更多测试负担、架构耦合与维护成本。衡量 AI 研发不能只看产出量，而应沿着一条因果链观察：





Input 和 Output 容易测，Outcome 和 Impact 才接近价值。好的度量体系不会跳过中间因果关系，也不会把“生成更多”直接解释为“创造更多”。

16.1.3 传统研发效能与 AI 原生度量如何结合

传统研发效能通常关注流动、质量、稳定性、体验和价值。Agentic Agile 在此基础上增加意图、自治、算力、证据与学习信号：

观察面	传统研发效能问题	AI 原生补充问题
流动	交付周期、等待、吞吐如何	Agent 重试、Graph 阻塞、Handoff 是否制造新等待
质量	缺陷、返工、稳定性如何	首次是否真正成功，Bug 是否回溯原契约
价值	是否交付客户和业务结果	Agent 是否正确理解了签署意图
成本	人力、平台和机会成本如何	Token、模型、验证 Agent 与治理成本如何归因
风险	安全、合规、发布风险如何	工具越权、MUST 失败和自治边界是否被发现
体验	工程师能否顺畅完成工作	人类是否被低价值 HITL、核对和提示词返工淹没
学习	团队是否减少重复错误	记忆、知识、约束、测试和 Skill 是否有效复用

3-4-3 不替代现有研发效能体系。例如，DORA 主要从部署频率、变更前置时间、变更失败和恢复能力观察软件交付表现；SPACE 从满意度、绩效、活动、沟通协作和效率流动等多个维度理解开发者生产力。3-4-3 提供的是一层契约级解释坐标：把这些结果连接到具体意图、Agent 行为、约束、证据和算力，而不是用一个新的“AI 总分”覆盖原有体系。

16.1.4 古德哈特法则：指标一旦成为目标，就可能失去意义

古德哈特法则常被概括为：**当一项度量成为目标，它就不再是一项好的度量。**人和系统会围绕指标优化，直到数字与原本想观察的现实脱钩。

与之相关的坎贝尔定律提醒我们：定量指标越被用于重要奖惩和决策，它越容易受到操纵压力，也越可能反过来腐蚀原本想改善的过程。

AI 研发尤其容易发生指标游戏：

- 为了降低 Token，过度裁剪上下文，导致返工和遗漏增加；
- 为了提高首次成功率，隐藏上线后的 Bug，或另开任务不回溯原契约；
- 为了降低 HITL，Agent 遇到价值冲突和不可逆操作也不再上报；
- 为了提高自愈率，降低约束、删除失败测试或无限重试；
- 为了提高知识沉淀率，把每次偶然问题都写成规则；
- 为了缩短交付周期，把验证、UAT 和运维成本推给下游；
- 为了证明 AI 覆盖率，优先挑选大量简单任务，回避真正有价值的复杂问题。

所以，指标的第一用途不是奖励，而是学习；第一评价对象是治理系统，而不是个人。只要指标直接绑定个人排名、奖金或强制目标，就必须预先分析它会诱发什么行为。

16.1.5 每个主指标都需要反指标

单一指标只展示一个切面。主指标描述希望改善的方向，反指标负责暴露这种改善是否通过转移成本或隐藏风险获得。

主指标	可能被怎样游戏	必须联读的反指标
首次成功率	隐藏 Bug、只选简单任务	逃逸缺陷、Bug 回溯率、任务复杂度
Token 效率	过度裁剪上下文	目标准确率、返工、执行轮次
自愈率	弱化门禁、无限重试	MUST 通过率、修复轮次、约束变更记录
HITL 升级率	风险不再上报	越权事件、事后人工返工、残余风险
交付周期	验证成本后移	UAT 否决、回滚、QA 和业务人工小时
知识沉淀率	制造规则垃圾	有效复用率、冲突率、过期率
复合 ROI	夸大节省工时	真实人工投入、失败折现、治理建设成本

当一个指标改善而反指标恶化时，不能宣布成功，而应检查系统是否发生局部最优。

16.1.6 为重要指标建立 Measurement Contract

意图需要契约，重要指标同样需要。Measurement Contract（度量契约）不是第五个动态工件，而是指标目录中的结构化定义，用来防止不同团队用同一个名称表达不同含义。

```
metric_id: METRIC-FIRST-PASS-01
name: first_pass_success_rate
purpose: 判断任务是否第一次完整执行就达到签署契约
formula: 首次正确完成任务数 / 完成任务总数
unit: percentage
scope:
  task_types: [feature, bugfix]
  risk_levels: [low, medium]
window: 最近10个同类任务或30天
sources:
  - Intent Contract
  - Evidence Bundle
  - Bug Backtrace
owner: OA
counter_metrics:
  - escaped_defect_rate
  - task_complexity
gaming_risks:
  - 删除失败任务
  - 将事后Bug另开任务
  - 只选择简单样本
data_quality:
  freshness: 24h
  estimated_allowed: false
action_rules:
  warning: 低于历史基线15个百分点
  response: 检查契约、上下文、测试和任务组合
```

一份合格度量契约至少说明：目的、定义、公式、分母、单位、范围、窗口、来源、责任人、反指标、数据质量、操纵风险和触发动作。没有这些字段，同一个“成功率”可能在不同会议中被解释成完全不同的东西。

16.1.7 基线、分群与分布比平均值更重要

“Token 降低 30%”只有在比较对象一致时才有意义。度量至少要控制：

- 任务类型与复杂度；
- 风险等级和验证强度；
- 项目、模块与技术栈；
- 使用的模型、工具和价格口径；
- 新项目还是既有大型系统；
- 时间窗口、样本量和是否包含失败任务。

不要只展示平均数。十个任务平均消耗 20 万 Token，可能是每个都接近 20 万，也可能是九个只用 5 万、一个失控到 155 万。中位数、分位数、最大值、离群任务和趋势往往比单一均值更有治理价值。

没有历史基线时，第一阶段的目标不是宣称提升，而是建立可信基线。没有分母时不谈比例，没有任务分群时不做横向排名，没有样本量和置信说明时不把偶然波动解释成趋势。

16.1.8 领先、滞后与护栏指标要形成组合

- **领先指标**提示过程可能走向哪里，例如契约歧义、上下文缺失、门禁失败和重试轮次；
- **滞后指标**说明最终发生了什么，例如逃逸缺陷、回滚、业务命中和实际成本；
- **护栏指标**规定即使主目标改善也不能突破的底线，例如高危安全事件、MUST 失败和 UAT 否决。

只看滞后指标，反馈太慢；只看领先指标，可能优化了过程代理而没有改善结果；没有护栏指标，速度和成本目标会吞噬质量与安全。三类指标需要共同进入遥测和裁决。

16.1.9 为什么 Evidence Bundle 之后还需要 Telemetry

Evidence Bundle 回答的是“这一次交付是否有足够证据支持某个行动”；Telemetry 回答的是“这套交付系统是否在多个任务中变得更准确、更自主、更经济、更会学习”。前者主要面向一次裁决，后者主要面向趋势判断和治理改进。

如果只有 Evidence Bundle，团队能判断 T-001 是否应该发布，却很难回答：最近十个契约的首次成功率是否下降，人工介入是否越来越多，Token 成本为什么突然翻倍，知识回写是否真实减少了重复错误。Telemetry 把单点证据连接成时间序列。

在 3-4-3 中，遥测不是项目结束后的统计，而是每个意图契约完成 Verify 后的强制动作。每份契约形成一次独立运行记录，再聚合到项目级数据中。这使度量与真实任务、真实契约、真实证据一一对应，而不是月底由管理者追问团队“这个月 AI 大概节省了多少时间”。

16.1.10 四层九个核心维度

4 层模型从价值、能力、效率和进化四个角度观察 Agentic 交付。九个核心维度之外，Dashboard 还展示 HITL 升级率、MUST 通过率、失败折现率、门禁状态、测试和性能等支撑指标。这样既保持核心模型稳定，又不丢失任务裁决所需的工程细节。

Layer 1 价值层：AI 是否完成了值得完成的事情？

1. 目标准确率

2. 首次成功率

3. 复合 ROI

Layer 2 能力层：Agent 能在多大范围内可靠自治？

4. 约束自愈率

5. 自主性综合评分

支撑：HITL 升级率、MUST 约束通过率

Layer 3 效率层：人、上下文、模型与编排是否高效？

6. 上下文压缩比

7. Token 效率

8. 执行效率

Layer 4 进化层：系统是否把经验变成下一次能力？

9. 知识沉淀率

16.1.11 Layer 1：价值层——AI 创造了多少真实价值

1. 目标准确率

目标准确率衡量被正确完成的任务占总分配任务的比例：

$$\text{目标准确率} = \text{正确完成任务数} \div \text{总分配任务数} \times 100\%$$

它不同于测试通过率。测试可能全部通过，但如果实现偏离签署目标，目标准确率仍应判低。这个指标要求每个“正完成”都能回到 Intent Contract 和 Evidence Bundle，而不是由 Agent 自报。

2. 首次成功率

首次成功率衡量任务是否在第一次完整执行中就满足契约，而不是经过返工、Bug 回溯或多轮人工修正后才通过：

$$\text{首次成功率} = \text{首次即正确完成任务数} \div \text{总完成任务数} \times 100\%$$

它是连接质量与成本的重要指标。两个团队最终都完成十个任务，一个团队八个任务一次通过，另一个团队每个任务平均修复三轮，两者的全绿结果相同，算力、人力和交付风险却完全不同。

任务完成后发现 Bug，必须把该契约的首次成功状态修正为失败，并重新采集遥测。否则 Dashboard 会把发布后返工隐藏掉，变成宣传工具。

3. 复合 ROI

ROI 不能只用“节省工时 × 人力单价”减去模型账单。失败、返工、人工复核和治理建设都属于成本。3-4-3 引入失败折现，强调只有有效完成才能产生价值：

$$\begin{aligned} \text{复合 ROI} = & \\ & (\text{节省人力价值} \times (1 - \text{失败率}) - \text{AI 与治理成本}) \\ & \div \text{AI 与治理成本} \times 100\% \end{aligned}$$

对于无法可靠估算节省工时的任务，应显示 N/A，而不是编一个漂亮 ROI。ROI 适合观察相同口径下的趋势，不适合跨团队直接排名。

16.1.12 Layer 2：能力层——Agent 有多自主，而不是看起来有多自动

4. 约束自愈率

约束自愈率衡量发生约束失败后，Agent 在授权范围内自主修复并通过原检查的比例：

$$\text{约束自愈率} = \text{自主修复的约束失败数} \div \text{约束失败总数} \times 100\%$$

自愈率高不必然更好。如果 Agent 为了提高自愈率降低门禁、删除测试或无限重试，指标已经失真。只有保留原约束、原检查和修复证据的自愈才计入。

5. 自主性综合评分

自主性不是“人类出现得越少越好”。当前评分综合自愈率、MUST 通过率与 HITL 情况：

$$\begin{aligned} \text{自主性评分} = & \text{自愈率} \times 40\% \\ & + \text{MUST 通过率} \times 35\% \\ & + (1 - \text{HITL 升级率}) \times 25\% \end{aligned}$$

这个分数用于观察系统在既定边界内能否独立完成，不用于惩罚必要的人类判断。资金支付、生产发布或价值冲突触发 HITL，可能恰恰说明系统正确识别了责任边界。

Dashboard 因此同时展示 HITL 升级率和 MUST 通过率，让读者看到评分背后的结构，而不是只看一个总分。若自主性低是因为大量重复性 HITL，可以优化规则；若来自不可逆操作的必要批准，则不应盲目追求降低。

16.1.13 Layer 3：效率层——人机协作与算力使用是否合理

6. 上下文压缩比

上下文压缩比衡量从候选上下文到实际注入上下文的裁剪效果：

$$\text{上下文压缩比} = \text{裁剪前 Token} \div \text{裁剪后 Token}$$

压缩比越大不一定越好。极端裁剪可能遗漏关键依赖。它必须与目标准确率、首次成功率共同阅读：Token 下降而返工上升，说明裁剪过度；Token 下降且准确率稳定，才说明 Context Engineering 真正有效。

7. Token 效率

Token 效率通常表示每个完成任务消耗的 Token：

$$\text{Token 效率} = \text{Token 总消耗} \div \text{完成任务数}$$

它可以进一步拆分输入、输出、缓存、工具调用阶段或不同 Agent。单看总 Token 没有意义：复杂任务理应比文案修改消耗更多；增加独立验证也可能提高 Token，却显著降低风险。正确做法是在相似任务类型、相同质量门槛下比较趋势。

8. 执行效率

执行效率衡量每轮 Agent 执行完成多少有效任务：

$$\text{执行效率} = \text{正确完成任务数} \div \text{执行轮次}$$

它用来发现重复重试、编排阻塞和 Handoff 摩擦。执行轮次上升但完成任务不变，可能意味着上下文不清、测试反馈太晚、Graph 拆分不合理，或工具环境不稳定。

16.1.14 Layer 4：进化层——系统是否真的学会了

9. 知识沉淀率

知识沉淀率观察本周期新增并经验证的模式在累积知识中的占比：

$$\text{知识沉淀率} = \text{本周期新增有效模式数} \div \text{累积有效模式数} \times 100\%$$

“新增模式”不是模型自动写的一段反思，而是被回写到 Intent Graph、Constraint Matrix、Loop Memory、Tools Manifest 或测试资产，并且会改变后续任务行为的知识。

知识沉淀率持续为零，可能说明 Evolve 没有发生；长期异常高，也可能说明团队把每个偶然问题都写成规则，制造知识膨胀。进化既包括增加，也包括合并、修正和淘汰过期知识。

16.1.15 九维指标必须联动阅读

遥测最危险的用法是把某个指标变成单一目标。追求最低 Token 可能造成上下文不足；追求零 HITL 可能让 Agent 越过人类责任；追求最高知识沉淀率可能制造规则垃圾；追求首次成功率可能诱导团队隐藏 Bug。

真正有价值的是指标之间的关系：

组合信号	可能解释	下一步检查
Token 下降，首次成功率稳定	上下文裁剪或编排改善	确认任务复杂度可比
Token 上升，HITL 与返工同时上升	意图、上下文或工具环境恶化	检查最近契约和失败路由
自愈率上升，MUST 通过率下降	可能通过弱化约束获得“自愈”	审查测试和约束变更
准确率稳定，知识沉淀率为零	当前完成但组织没有学习	检查图谱回写与 Failure Mining
HITL 上升，事故下降	可能是系统更早识别高风险	区分必要与重复介入

这些指标不是用来评价个人，而是判断治理系统是否越来越准确、稳定、经济和会学习。

16.2 一份契约一张 Dashboard：任务级治理快照

3-4-3 遥测最有辨识度的设计，是每个意图契约都拥有自己的独立 Dashboard。T-001 完成后，不只生成 Evidence Bundle，还生成该契约的遥测数据与静态页面：

```
governance/evidence/EB-T-001.md
governance/telemetry/runs/telemetry-T-001.json
governance/dashboard-T-001.html
```

单契约 Dashboard 可以理解为一张“任务治理体检报告”。它围绕同一个任务 ID 展示：

- 契约目标与采集时间；
- 目标准确率和首次成功情况；
- 约束自愈、HITL 与 MUST 通过率；
- 上下文、Token 与执行轮次；
- 知识回写与进化结果；
- 测试、门禁、性能与成本；
- Evidence Bundle 和裁决状态。

Evidence Bundle 与单契约 Dashboard 不是重复。Evidence Bundle 保留细粒度证据和裁决依据；Dashboard 把关键指标与状态可视化，让 IO、OA 和管理者在几分钟内理解任务的价值、代价和治理质量，并能继续下钻到证据。

例如 T-002 显示目标准确率 100%、首次成功率 100%，但自主性只有 35/100、HITL 升级率 100%、每任务 Token 超过 80 万。结论不应只是“成功”，而是：结果正确，但自治能力和算力效率仍处于基线，需要检查为什么两轮执行都需要人工介入，以及上下文是否过大。

16.3 全局 Dashboard：从任务快照到项目价值与效能视图

项目级 Dashboard 聚合所有契约运行记录：

```
governance/telemetry.json
governance/dashboard.html
```

它不是把所有任务数字简单相加，而是提供三个层次的管理视图：

- 1. **项目全局状态**：当前治理成熟度、总体准确率、首次成功率、自主性、门禁与成本；
- 2. **历史契约列表**：每个 T-XXX 的采集时间和核心指标，可点击下钻；
- 3. **趋势与异常**：准确率、首次成功率、Token、HITL、知识沉淀等随时间如何变化。

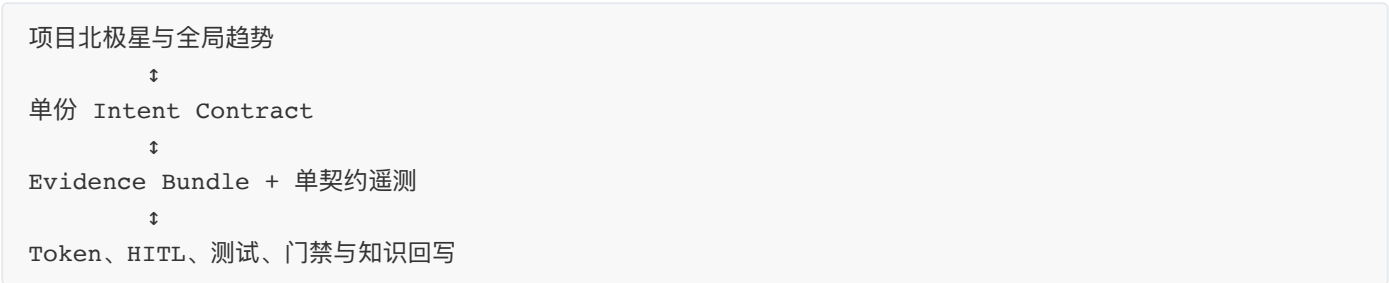
总览页中的历史契约表链接到 `dashboard-T-XXX.html`；单契约页顶部又可以返回 `dashboard.html`。双向导航让管理者从趋势进入现场，也让执行者从单次任务回到项目基线。

Dashboard 是纯静态 HTML，数据内嵌，不需要部署服务器。这样既降低了落地门槛，也便于在本地或受控环境查看。静态不等于手工：每次遥测采集会更新单任务 JSON、项目累计数据、单契约页面和总览页面。

为什么双层 Dashboard 是重要突破

传统研发看板往往以 Sprint、团队或项目为中心，任务完成后只留下状态和工时；AI 工具自身的用量页面又通常只展示 Token、请求次数和费用。两类看板彼此分离：一边不知道 AI 怎样工作，另一边不知道工作产生什么价值。

双层 Dashboard 通过契约 ID 把价值、行为、约束、算力、证据和学习连接起来：



这使组织第一次能够回答：“哪个业务意图消耗了多少算力，经过多少轮修复，触发哪些人工决策，最终产生了什么证据和价值？”

16.4 Token、算力成本与价值归因

Token 为什么是治理指标

Token 是模型处理文本和部分多模态信息时的基本计量单位。输入上下文、模型输出、缓存读写和不同模型的单价共同决定调用成本。对个人偶发使用，Token 可能只是账单数字；对多 Agent、长上下文和高频验证系统，它会成为真实的生产资源。

Agentic Agile 不主张一味压低 Token，而是要求 Token 可测、可归因、可解释。每次契约消耗多少 Token，应当能够关联到具体目标、执行轮次、失败修复和最终价值。

Token 成本怎样计算

最基本的成本模型是：

任务模型成本 =
输入 Token × 输入单价
+ 输出 Token × 输出单价
+ 缓存读写、工具或其他计费成本

实际计算必须使用对应模型和计费时点的价格。不同模型、输入输出、缓存和供应商价格差异很大，因此 Dashboard 应保存模型、Token 来源和价格口径，不能只保存一个美元结果。

实测优先，估算必须标记

Token 应优先从本地 AI 客户端的可验证记录实测。343 的工具自举可以把 `ocusage` 一次性安装到用户私有目录，后续由不同 AI 宿主复用；宿主工具与 Token 数据客户端必须独立选择，不能因为从某个 AI 工具启动 Skill，就默认只读取该工具或依赖某个厂商目录。成功采集时应保留客户端、来源与测量状态，Dashboard 明确显示实测标识。

只有无法获得客户端数据时才允许估算，并明确标记 `estimated`。实测和估算不能混在同一趋势中而不加区分，否则团队会把采集口径变化误判为效率变化。

怎样读每任务 Token

“每任务 10 万 Token”本身既不代表好，也不代表坏。需要同时看：

- 任务复杂度和风险等级；
- 输入、输出与缓存 Token 构成；
- 执行轮次和失败修复次数；
- 上下文压缩比；
- 是否使用多个 Agent 或独立验证；
- 目标准确率、首次成功率和残余风险；
- 相似任务的历史基线。

高风险支付任务为了独立验证多消耗 Token 可能合理；低风险文案任务因反复读取整个仓库而消耗大量 Token，则暴露上下文和编排问题。

从 Token 成本走向价值归因

真正的管理问题不是“本月花了多少 Token”，而是“这些算力支持了哪些契约，产生了多少有效结果”。可以从三个层次观察：

1. **任务级**：T-001 的 Token、轮次、HITL、成功与证据；
2. **能力级**：哪类任务或模块的单位有效交付成本最高；
3. **项目级**：算力投入变化是否带来准确率、速度、质量或业务价值改善。

这也是“算力与价值遥测”的意义：算力不是孤立成本，价值也不能脱离失败和治理成本被夸大。两者必须在同一契约坐标系中比较。

16.5 遥测真实性与采集闭环

遥测不能靠手填“漂亮数字”。

- 测试数来自真实测试运行；
- Token 用量优先使用本地客户端数据实测；
- 无法实测时必须标记为 `estimated`；
- 单任务遥测文件必须存在；

- 项目累计 run_count 必须更新；
- 仪表板必须刷新；
- 已知缺口必须进入 Evidence Bundle。

推荐由跨平台 Python 原生入口完成 Evidence 与遥测收口，Shell 只作为可选包装器：

```
PYTHON_BIN="$(bash scripts/ensure_py_env.sh)"
"$PYTHON_BIN" scripts/cli.py change verify --task T-001 --project-dir .
```

后续命令应复用同一个 PYTHON_BIN，避免解释器命令和依赖环境漂移。

16.6 遥测如何驱动动作

信号	可能原因	治理动作
目标准确率下降	意图或上下文漂移	检查图谱与契约质量
首次成功率下降	AC、架构或技术债问题	加强前置规格和关键模块治理
MUST 失败增加	约束不清或工具越权	检查权限和约束注入
HITL 激增	自治范围过大或规则冲突	收缩权限、改善决策入口
Token 上升	重试、上下文或编排低效	裁剪上下文、优化 Graph
知识沉淀停滞	反思没有回写	检查 Evolve 和 closing gate

16.7 Telemetry 常见失败

- 遥测手工估算却未标记，或测试、Token 和耗时没有真实来源；
- 只有项目总览，没有单契约遥测，无法定位异常来源；
- 只有 Token 账单，没有对应的契约、结果、证据和价值；
- 实测与估算 Token 混用，却没有标记口径和生效时间；
- Dashboard 为了汇报被手工美化，与原始遥测文件不一致；
- 把 HITL 越少、Token 越低直接等同治理越成熟；
- 完成后 Bug 未回溯原契约，导致首次成功率长期虚高；
- 指标只有红绿颜色，没有 Owner、阈值、诊断路径和治理动作。

16.8 度量治理与遥测数字的解释纪律

Token、时长、首次成功率、返工率和人工介入次数都需要上下文。同样的 Token 增长，可能意味着任务更复杂、验证更充分，也可能意味着上下文污染；人工介入减少，可能代表边界成熟，也可能代表团队放弃监督。

因此每个指标至少要有定义、采集来源、比较基线、解释责任人和可能的反指标。仪表板的任务不是制造确定感，而是让异常更早暴露，让改进假设可以被验证。

16.8.1 度量首先服务学习，而不是考核

把系统指标直接用于个人排名，会迅速改变数据的含义。工程师可能拆小提交提高吞吐、延迟上报缺陷保护首次成功率，或减少必要 HITL 以获得更高自治评分。最终 Dashboard 更漂亮，真实系统却更脆弱。

Agentic Agile 的默认评价对象应是：契约质量、上下文、约束、编排、验证、工具和组织环境。只有在充分理解系统条件后，数据才能用于能力辅导；不应用 Token、代码量、Agent 使用率或单一效率指标给个人贴标签。

16.8.2 指标也有生命周期

指标不是一旦建立就永久有效。它需要经历：

提出治理问题

→ 定义度量契约

→ 试采集与校准

→ 正式发布

→ 观察使用与操纵

→ 修订口径

→ 合并、停用或归档

当业务目标、模型、任务组合或工具链变化时，指标口径也可能失效。修改公式、数据源或价格口径必须留下版本和生效时间，旧数据不能在没有说明的情况下与新数据拼成一条趋势。

16.8.3 数据质量是度量系统的约束矩阵

遥测至少要检查：

- **完整性**：失败、取消、回退和人工接管是否同样进入数据；
- **准确性**：测试、Token 和时间是否来自真实系统，而非回忆；
- **一致性**：单契约、全局数据与 Evidence Bundle 是否采用同一口径；
- **新鲜度**：数据是否对应当前 Commit、模型、环境和规则版本；
- **可追溯性**：指标能否下钻到 Contract ID、Evidence ID 和 Release ID；
- **权限与隐私**：采集内容是否超出必要范围，是否被用于未授权的个人监控。

数据质量不足时，Dashboard 应明确显示 UNKNOWN、STALE 或 ESTIMATED，而不是用默认值补齐成绿色。

16.8.4 从信号到行动必须有责任闭环

一个无人负责、没有阈值、不会触发行动的指标只是装饰。关键指标应预先绑定：

要素	要回答的问题
Owner	谁解释这项指标并维护口径？
Threshold	什么变化值得调查，而非正常波动？
Diagnosis	首先下钻哪些契约、证据和分群？
Action	调整意图、约束、Context、Graph、测试还是模型？
Review	多久后验证动作是否有效？
Rollback	改动恶化时如何恢复？

例如 Token 上升不是直接要求“所有任务降低 20%”，而是先按任务类型和执行轮次下钻。如果上升来自更充分的安全验证，可以接受；如果来自重复读取无关仓库，则优化 Context；如果来自某个 Agent 无限重试，则修改 Graph 和退出条件。

16.8.5 度量评审不是读数字会议

有效的度量评审只聚焦少量异常和改进假设：发生了什么，数据可靠吗，可能原因是什么，要改变哪个治理部件，怎样知道改变有效。评审输出应该是一个有责任人、有验证窗口的改进行动，而不是要求所有数字下个月都变绿。

治理委员会关注跨任务趋势、重大例外和自治范围；OA 关注契约、编排、验证与成本；IO 关注价值命中和风险取舍；AS 执行被批准的调整。不同角色看同一事实，但不应各自维护一套互相矛盾的数字。

16.8.6 度量系统本身也要接受遥测

组织还应观察度量体系是否健康：多少指标长期无人查看，多少告警没有行动，多少数据无法追溯，多少指标被频繁手工修正，多少决策真正引用了证据。若 Dashboard 指标不断增加而改进行动没有增加，说明组织建设的是数据仓库，不是反馈控制系统。

16.9 本章小结

度量定义组织想判断什么，Telemetry 持续采集真实信号，双层 Dashboard 让管理者从项目趋势下钻到单份契约、证据与算力现场。古德哈特法则提醒团队：核心指标必须具有度量契约、比较基线、任务分群、反指标、护栏、数据质量和行动责任。遥测的价值不是证明 AI 很成功，而是让治理假设能够被数据证伪并持续修正。

本章最小实践：为一个核心指标建立 Measurement Contract，采集 T-001 单契约遥测并更新全局 Dashboard，再基于一个异常信号提出可验证的治理动作。

完成证据：度量契约、单任务遥测、全局仪表板、解释记录和带验证窗口的改进行动。

延伸实践：推荐学习《Agentic Agile 组织转型进阶：构建人机协同研发操作系统》，学习如何区分 Token、活动量、产出与真实结果。

第十七章 | AI 原生交付流水线：让每次发布携带意图、证据与回滚能力

前面的章节已经回答了如何签署意图、建立约束、编排 Agent、生成测试、汇聚 Evidence Bundle 和作出发布裁决。但企业真正落地时还会遇到一组无法绕开的工程问题：门禁在哪里执行？多个 Agent 如何隔离修改？测试通过的提交与最终发布的制品是不是同一个？契约、代码、证据和版本如何对应？Agent 是否可以直接合并主干或发布生产？

这些问题属于 CI/CD、DevOps、分支和版本管理的领域。本书不重复介绍 Git 命令、Jenkins 安装或 Kubernetes 操作，而关注它们在 AI Coding 时代承担的新治理职责。

核心判断是：

CI/CD 负责把代码安全地送到生产；3-4-3 负责证明这份代码为什么有资格被送到生产。

17.1 3-4-3 不取代 DevOps，而是补上意图与证据

DevOps 解决开发、交付、运行与反馈之间的协作和自动化；CI 持续集成代码并执行构建与验证；CD 让合格制品持续交付或部署；Harness 把权限、约束、门禁、观测和恢复变成 Agent 无法随意绕过的运行环境；SCOPE-V 则控制一项意图如何从规格走向证据和裁决。

它们处在不同层次，不能互相替代：

Intent Contract：为什么做、做到什么程度

Constraint Matrix：什么边界不可突破

Work Graph：任务、测试与集成怎样编排

CI Pipeline：构建、测试、检查并采集证据

Evidence Bundle：为什么可以作出当前裁决

CD Pipeline：制品怎样晋级、灰度、部署和回退

Telemetry：上线后是否产生真实价值和异常

没有 DevOps 底座，3-4-3 的可执行门禁很难规模化；只有 DevOps 而没有 3-4-3，流水线可以高效发布代码，却未必知道代码对应哪项业务意图、风险由谁接受、证据是否足以支持上线。

因此，Agentic Agile 不是在现有流水线外再搭建一条平行流程，而是把契约、约束、证据和遥测嵌入企业已有的 Git、CI/CD、制品库、发布平台和可观测系统。

17.2 为什么 AI 流水线必须前移到 Commit 之前

传统流水线通常从代码提交开始：

```
Commit → Build → Test → Scan → Deploy
```

但 AI Coding 的许多风险发生在 Commit 之前：Agent 是否读取了正确版本的契约，是否获得了过大的数据和工具权限，是否修改了未授权文件，是否擅自增加依赖，测试是否在实现前产生，多个 Agent 是否正在覆盖彼此的工作，以及 Token、时间与重试是否已经超过预算。

因此，AI 原生交付流水线要向前延伸：

意图签署

→ 上下文、权限与基线校验

→ 分支或 Worktree 隔离

→ TDD Red

→ Agent 实现与增量约束检查

→ 多层验证与集成

→ Evidence Bundle 与独立 Verify

→ 人类发布裁决

→ 不可变制品晋级与部署

→ 生产遥测和 Bug 回溯

这里的“流水线”不再只是 CI 服务器中的几个 Job，而是从 Agent 获得任务到生产反馈回流的完整受控通道。Commit 只是其中一个事件，不是治理起点。

17.3 分支管理首先解决隔离，而不是命名格式

传统团队讨论分支策略时，常在 GitFlow、Trunk-Based Development 和 Feature Branch 之间选择。AI Coding 时代仍需这些策略，但首先要解决一个更基础的问题：高速并行的 Agent 是否拥有清晰、隔离、可追溯的写入空间。

如果多个 Agent 直接共享一个工作目录或分支，可能出现：

- 一个 Agent 覆盖另一个 Agent 尚未提交的修改；
- 测试基于不同代码基线运行，结果却被混在一起；
- 某 Agent 顺手提交了其他节点的未完成文件；
- 合并冲突被 AI 语法正确地解决，却改变了业务语义；
- Evidence Bundle 无法说明某项证据究竟对应哪一组代码；
- 代码验证完成后又有新提交进入，最终发布内容发生漂移。

所以，每个可独立执行的 Work Graph 节点至少要绑定：

- Task ID、Contract ID 和节点 ID；
- 起始 Commit 或制品基线；
- 分支、Worktree 或受控沙箱；
- 文件所有权与允许修改范围；

- 上游依赖版本；
- 允许使用的工具和凭证；
- 合并目标和汇合节点；
- 提交、合并和废弃条件；
- 对应证据与遥测位置。

分支名称是否使用 `agent/T-101-api` 并不是重点。重点是系统能否从分支追溯契约，从节点看见允许范围，从合并结果反查参与的 Agent 和证据。

17.4 默认建议：受保护主干、短分支或 Worktree、Merge Queue

对于多数 AI Coding 团队，一个合理默认是：

受保护主干 + 短生命周期任务分支或 Worktree + 自动门禁 + Merge Queue。

```
main (受保护)
├─ agent/T-101-test
├─ agent/T-101-api
└─ agent/T-101-ui
    ↓
    按 Work Graph 汇合
    ↓
    集成验证与证据聚合
    ↓
    Verify / HITL
    ↓
    Merge Queue
    ↓
    main
```

短生命周期分支减少长期漂移，Worktree 允许同一仓库为多个 Agent 提供物理隔离的工作目录，受保护主干阻止未验证结果直接进入稳定基线，Merge Queue 则在最新主干上重新验证候选合并，解决“每个分支单独都绿，组合以后失败”的问题。

这不是唯一正确策略。高监管行业、需要长期维护多个客户版本或存在固定发布列车的产品，仍可能需要 Release Branch；极小、低风险、单 Agent 项目可以直接在主干进行受控的小提交；紧急修复也可能使用 Hotfix Branch。书中不把某种流派绝对化，判断标准是：

- 是否降低并行冲突和基线漂移；
- 是否保持提交、证据和制品可追溯；
- 是否让风险匹配适当门禁；
- 是否支持快速集成而不是制造长期分叉；
- 是否能够安全废弃失败的 Agent 结果。

17.5 文件所有权与语义冲突

Git 能发现同一行文本冲突，却无法发现语义冲突。一个 Agent 修改金额规则，另一个 Agent 修改审批路由，两边可能没有任何文本冲突，合并后却形成互相矛盾的行为。Graph Engineering 因此需要把文件所有权进一步提升为能力和契约所有权。

对于高耦合节点，可以采用：

- 预先声明写集合，Graph Engine 检测重叠；

- 同一文件只允许一个节点写，其他节点通过 Handoff 提建议；
- 接口由跨模块契约管理，提供方和消费方分别验证；
- 汇合后重新运行组合级 AC、SIT 和 NFR；
- 对关键业务冲突设置模块 OA 或全局 OA 审查；
- 不允许 Agent 仅因合并无文本冲突就宣布集成成功。

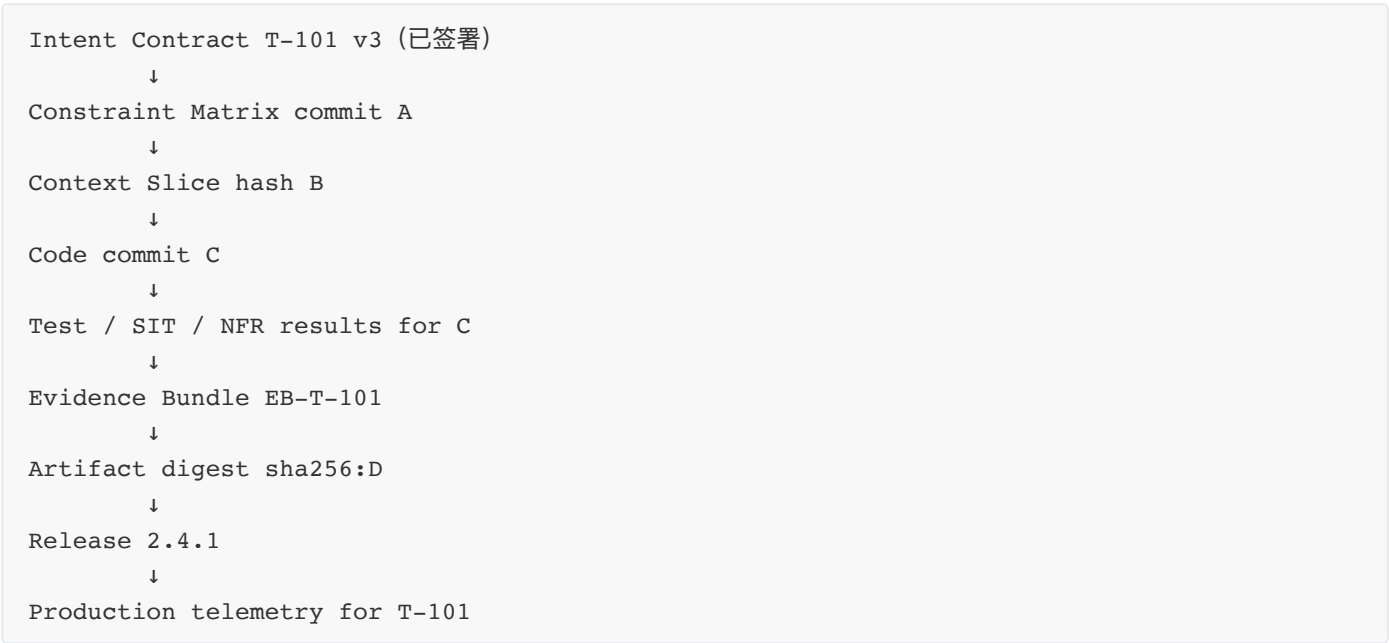
以报销案例为例，测试 Agent 可以独立写边界测试，领域 Agent 修改金额判断，接口 Agent 适配响应格式，UI Agent 修改提示与审批展示。但它们汇合后必须重新证明：5000.00 元进入普通审批、5000.01 元进入总监审批、接口和页面表达一致、历史草稿没有被迁移。

17.6 从代码版本升级为交付谱系

传统版本管理主要关注 Commit、Tag、软件版本号和发布制品。AI Coding 时代还要管理影响结果的认知与治理资产：

- Intent Graph 的版本；
- Intent Contract 的版本和签署状态；
- Constraint Matrix 与 Tools Manifest 的版本；
- Work Graph、Context Slice 和 Handoff 版本；
- Agent、模型、Prompt 和 Skill 版本；
- 测试、Golden Set、数据集和 Schema 版本；
- Evidence Bundle 与发布裁决版本；
- Artifact Digest、配置和部署环境版本；
- 契约级遥测及生产事件。

这些版本共同形成 **Delivery Lineage（交付谱系）**：



交付谱系回答的不是“当前版本号是多少”，而是：生产环境中的这份制品，依据哪份意图，由哪些 Agent 在什么约束下生成，经过哪些真实验证，由谁接受风险，并在线上后表现如何。

如果测试报告对应 Commit C，Evidence Bundle 却没有记录版本；如果发布平台从最新主干重新构建了 Commit D；或者契约签署后约束矩阵已经发生重大变化，证据链就已经断裂。

17.7 建立 Release Manifest

Evidence Bundle 面向人类解释“为什么可以裁决”，还可以增加一份机器可读的 Release Manifest，固定发布对象的谱系关系：

```
release_id: R-2.4.1
task_id: T-101
contract:
  path: governance/contracts/Intent_Contract_T-101.md
  version: 3
  signed_at: 2026-08-07T10:30:00+08:00
governance:
  constraints_commit: a1b2c3d
  work_graph_hash: sha256:...
source:
  repository: expense-service
  commit: c4d5e6f
artifact:
  image: registry.example.com/expense:2.4.1
  digest: sha256:...
evidence:
  bundle: governance/evidence/EB-T-101.md
  verdict: CONDITIONALLY_APPROVED
  approved_by: Finance-IO
deployment:
  strategy: canary
  initial_traffic: 10%
  rollback_to: sha256:previous...
```

这份清单不一定采用相同字段，但至少应绑定契约、源码、治理基线、证据裁决、不可变制品、部署策略和回退对象。CI 可以自动生成大部分字段，人类只负责不能自动推导的风险接受和发布条件。

17.8 CI 如何承载五道机械门

五道门不是要求所有检查都放在同一个 CI Job，而是说明不同时间点必须有什么事实。它们可以映射到本地 Harness、Pull Request、Merge Queue、发布流水线 and 事后回溯：

门	典型执行位置	CI/CD 要证明什么
Pre 前置门	Agent 启动前、本地或任务编排器	契约真实签署、约束与 AC 已准备
Coding 编码门	分支首次实现前	测试先写并产生可信 Red
Prove 验证门	PR 与候选合并流水线	测试、AC、构建、NFR 与证据通过
Closing 收尾门	发布候选生成前	Evidence Bundle、遥测、Dashboard 和图谱回写完整
Bug 回溯门	生产缺陷处理流水线	原契约、证据、首次成功率和知识已如实修正

Pre 与 Coding 发生在 Commit 之前，因此不能只依赖远程 CI；它们可以由本地 Hook、Agent Harness 或任务平台机械执行。Prove 适合在 PR 和 Merge Queue 中运行。Closing 连接验证结果、发布裁决与制品。Bug 门则由事故或缺陷事件触发。

门禁应满足三条工程要求：

- 1. **不可静默绕过**：跳过、允许失败或管理员强制合并必须形成例外记录；

2. **结果可消费**：使用结构化输出、稳定退出码和原始日志，而不是只显示一个绿色图标；
3. **结论可追溯**：每项结果绑定 Task ID、Commit、环境和时间，并进入 Evidence Bundle。

17.9 Build Once, Verify Once, Promote Many

传统 DevOps 强调 Build Once, Deploy Many：代码构建一次，产生不可变制品，同一制品逐级部署，避免不同环境重复构建出不同结果。在 3-4-3 中还要再强调一步：被验证和裁决的必须是最终晋级的那一份制品。

```
Source Commit C
    ↓ 一次构建
Artifact Digest D
    ↓
测试 / 扫描 / SIT / 性能
    ↓
Evidence Bundle 绑定 D
    ↓
Verify 与发布裁决
    ↓
D 晋级 UAT / 预生产 / 生产
```

不能出现“Commit C 通过测试，Agent 随后又提交 D，发布平台从最新代码重新构建”的情况；也不能在 UAT 环境测试镜像 A，却在生产重新构建镜像 B。版本号或镜像标签可能被覆盖，Artifact Digest 才是更可靠的不可变身份。

因此应遵循：

- CI 只从明确 Commit 构建；
- 制品进入受控 Registry，禁止覆盖同一 Digest；
- Evidence Bundle 和 Release Manifest 记录 Digest；
- 环境晋级使用同一制品，只改变经过治理的环境配置；
- 部署后再次校验运行中的 Digest；
- 若源码、依赖或制品变化，原裁决自动失效并重新验证。

这条规则可以概括为：**验证过的就是发布的，发布的就是被裁决的。**

17.10 版本号、Tag 和配置也要受治理

语义化版本可以表达兼容性意图，但版本号本身不是证据。Agent 可以建议 Major、Minor 或 Patch，OA 和模块负责人仍需确认变更性质。破坏性接口变更不能因为 Agent 自动把版本号加一就获得许可，还需要消费方影响分析、跨模块契约更新和迁移策略。

Git Tag 应指向已经形成 Release Manifest 的 Commit；发布说明应从真实契约、代码差异和已知风险生成，而不是由模型凭聊天记录总结。配置、Feature Flag、数据库 Schema 和基础设施同样属于版本化交付物，因为同一代码在不同配置下可能产生完全不同的行为。

对于模型型或 Agent 型产品，还要记录模型、Prompt、Skill、工具和 Golden Set 的版本。生产行为变化不一定来自代码，如果这些非代码资产没有版本，事故回溯将找不到真实原因。

17.11 CD 中的人机权限边界

AI 可以在交付流水线中承担大量工作：创建任务分支和 Worktree，提交代码，创建 Pull Request，运行 CI，分析失败，在预算内自动修复，生成发布说明，部署测试环境，准备灰度计划，观察指标并汇总证据。

但自动化能力不能等同于授权。以下行为通常需要明确批准：

- 合并到受保护主干；

- 使用或扩大生产凭证；
- 修改生产配置和安全策略；
- 执行破坏性数据库迁移；
- 跳过或降低 MUST 门禁；
- 扩大灰度流量或全量发布；
- 删除生产数据或执行不可逆补偿；
- 接受高风险例外；
- 撤回或覆盖已有发布裁决。

一个实用原则是：

AI 可以准备发布
AI 可以证明发布条件
AI 可以执行已授权、可恢复的步骤
人类负责价值判断、风险接受和不可逆裁决

并非每次部署都要让高管点击按钮。团队可以通过风险分级进行预授权：低风险、可逆、证据完整的变更由 CD 自动晋级；资金、隐私、权限和不可逆数据变更触发 HITL；任何 MUST 失败都不能进入自动发布。

17.12 灰度、Feature Flag 与条件批准

发布不是只有“上线”和“不上线”两个选项。Evidence Bundle 可能支持条件批准：允许指定用户、地区、部门、时间或流量范围内发布，并绑定观察指标和撤回条件。

Feature Flag、金丝雀、蓝绿和影子流量是实现这种治理决定的工程手段：

- Feature Flag 将代码部署与功能启用分离；
- 金丝雀逐步扩大真实流量；
- 蓝绿发布保留可快速切换的旧环境；
- 影子流量在不影响用户的情况下比较新旧行为。

这些手段并不会自动降低风险。Feature Flag 如果没有负责人和失效日期，会变成长期配置债务；金丝雀如果没有明确成功指标，只是把事故延迟发生；蓝绿如果数据库迁移不可逆，也无法真正回切。

条件批准至少要写明：范围、持续时间、观察指标、阈值、负责人、扩大条件、停止条件和回退目标。CD 把这些条件转成自动步骤，Dashboard 显示实时状态，人类不必在每个正常步骤中介入，只在边界被触发时裁决。

17.13 回滚不只是重新部署旧版本

应用制品回滚通常容易，数据、消息、外部副作用和跨模块状态却可能已经向前推进。一个可靠回退计划至少的分析：

- 下游节点是否已经消费新事件；
- 数据库迁移是否可逆或需要补偿；
- 新旧版本能否同时读取当前 Schema；
- 缓存、队列和搜索索引是否需要恢复；
- Feature Flag 关闭后是否停止新行为；
- 已发生的资金、通知和外部调用如何处理；
- 回退是否影响同一并行组的其他任务。

因此，CD 的回滚动作应受 Work Graph 和 `verify_rollback_safety` 一类检查约束。Agent 可以分析依赖、生成命令和在沙箱演练，但高风险生产回退仍需要授权。Evidence Bundle 中的“可回滚”应来自真实演练或可执行验证，不应只是一句计划。

17.14 从发布版本回到契约级遥测

发布完成后，生产观测必须能够从 Artifact Digest 或 Release ID 回到 Contract ID。否则错误率上升时，团队只能看到“2.4.1 有问题”，却不知道它改变了哪项业务意图、由谁裁决、当时接受了什么风险。

理想链路是：

生产事件 / 指标

→ Release ID 与 Artifact Digest

→ Source Commit

→ Contract ID 与 Evidence Bundle

→ 对应 AC、约束和发布条件

→ Bug 回溯、遥测修正和图谱学习

单契约 Dashboard 可以显示该任务发布后的错误率、性能、业务命中和异常审批比例；全局 Dashboard 则观察不同版本、团队、模型或分支策略对首次成功率、HITL、Token、部署失败和恢复时间的影响。

这里仍需避免传统 DevOps 指标崇拜。部署频率更高不一定代表价值更高，变更前置时间更短也不代表风险更低。DORA 类工程指标可以与 3-4-3 的目标准确率、首次成功率、证据完整度、Token 效率和知识沉淀率联动阅读，而不是单独追求某个数字。

17.15 一个端到端的携证发布案例

以 T-101 报销金额规则为例：

1. IO 签署 Intent Contract v3，OA 固定约束和当前主干 Commit；
2. Graph 为测试、领域、接口和 UI 节点创建独立 Worktree，声明写集合；
3. 测试节点先产生可信 Red，Coding 门保留结果；
4. 各 Agent 在授权范围内实现，Harness 监视文件、依赖和工具调用；
5. 分支分别通过单元和接口检查，再按 Work Graph 汇合；
6. Merge Queue 在最新主干候选上运行组合 AC、SIT、安全和性能测试；
7. CI 从候选 Commit 只构建一次制品，记录 Artifact Digest；
8. Evidence Bundle 将每条 AC、MUST、测试结果和 Digest 建立映射；
9. 财务负责人完成 UAT Acceptance，IO 条件批准两个部门灰度 24 小时；
10. CD 将同一 Digest 部署到灰度环境，Feature Flag 限定范围；
11. Dashboard 观察异常审批率、错误率、p95 和用户反馈；
12. 指标正常后按已批准策略扩大流量；若超过阈值，自动关闭开关并升级人工；
13. 生产版本、遥测和教训回写 T-101，而不是成为一条无归属的运维记录。

这条链说明，分支、CI/CD 和版本管理并不是 3-4-3 之外的技术细节，而是把意图、约束和证据送达生产的物理通道。

17.16 不同规模团队的最小配置

治理不应要求所有团队一开始建设同样复杂的平台。

场景	分支与隔离	CI/CD 最小要求	版本与证据
单人、低风险	短分支或单 Worktree	测试、Harness、Closing 门	Commit + 契约 + Evidence Bundle
小团队、多 Agent	受保护主干 + 任务 Worktree	PR 门禁 + Merge Queue + 不可变制品	Release Manifest + Digest
多模块系统	模块分支 + 集成 DAG	跨模块契约、SIT、证据聚合	模块证据 + 发布级证据包
强监管、高风险	职责分离 + Release Branch	独立环境、签名制品、多级批准	完整谱系、审计、SBOM、长期留存

不论规模多小，仍要守住三个底线：最终代码必须对应签署意图，验证结果必须对应唯一版本，发布制品必须与被裁决制品一致。

17.17 常见失败

- Agent 直接在共享主干写入，多个任务的修改和证据混在一起；
- 分支存在很久，契约和主干已变化却没有重新裁剪上下文；
- PR 单独全绿，合并后不在最新基线重新验证；
- AI 自动解决语义冲突，没有重新运行组合 AC；
- 测试报告没有 Commit、环境和时间；
- Evidence Bundle 记录源码版本，却没有记录 Artifact Digest；
- UAT 测试的是制品 A，生产重新构建并部署制品 B；
- 使用可覆盖的 `latest` 标签作为发布身份；
- Agent 可以关闭分支保护或将门禁设为允许失败；
- Release Note 由模型生成，却与真实契约和变更不一致；
- Feature Flag 没有负责人、到期时间和清理计划；
- 把“重新部署旧镜像”误认为完整数据回滚；
- 发布指标无法关联 Contract ID，生产缺陷成为无归属事件；
- 为追求部署频率，把必要证据和 HITL 视为效率障碍。

17.18 本专题的工程验收

团队可以选择一项真实变更，验证以下问题：

1. 能否从任务分支反查签署契约、起始基线和允许修改范围？
2. 多个 Agent 的写入是否隔离，汇合顺序是否进入 Work Graph？
3. CI 的测试和门禁是否绑定唯一 Commit，并保存原始结果？
4. Evidence Bundle 是否绑定最终 Artifact Digest？
5. UAT、Verify 和发布裁决面对的是不是同一份不可变制品？
6. Agent 是否无法自行绕过受保护主干、MUST 门禁和生产权限？
7. 灰度、回退、负责人和触发阈值是否可执行并经过验证？
8. 生产指标能否回到 Release ID、Contract ID 和 Evidence Bundle？

如果这些问题只能靠某位工程师回忆，组织拥有的是自动化工具；如果都能由系统和制品回答，组织才拥有一条可验证的 AI 原生交付流水线。

17.19 本章小结

AI 原生交付流水线把契约、分支、Commit、测试、Evidence Bundle、Artifact Digest、发布裁决和生产遥测连接成完整交付谱系。CI/CD 负责把合格制品安全送向生产，3-4-3 负责证明该制品为什么有资格被交付。受保护主干、隔离工作区、机械门、Build Once、条件批准、灰度和可验证回退，共同确保 Agent 不能绕过责任与证据直接发布。

本章最小实践：把一个真实契约接入现有 CI/CD，生成绑定 Commit 与 Artifact Digest 的发布证据。

完成证据：Release Manifest、Evidence Bundle、门禁记录、发布裁决、灰度与回退验证。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，把意图、证据和回滚能力带进 AI 原生交付流水线。

B. 3-4-3 工程化落地与扩展

前面的 A 组说明 3-4-3 怎样在一次任务中运行。B 组继续回答三个工程问题：新项目 and 既有大型系统从哪里开始，知识与记忆怎样成为可信基础设施，多人、多工具、多模块怎样从局部闭环扩展为可组合交付。

这三章仍属于“验证工程与 3-4-3 落地”，重点是工程承载和规模化接口，不提前讨论岗位体系、组织激励或完整转型路线。人员、组织、流程和工具怎样共同成为企业运行方式，将在第四部分集中展开。

第十八章 | 跑通第一个可信闭环：新项目与既有大型系统

18.1 两条并行落地路径

3-4-3 面对新项目和既有大型系统时，使用的是同一套治理原则，却不能采用同一条进入路径。

新项目通常缺少历史包袱，团队可以从意图、约束和测试开始塑造系统；既有大型系统已经包含大量真实行为、隐性规则和历史责任，团队必须先认识并保护现实，再改变其中一小部分。前者是 **Governance by Design**（治理先行），后者是 **Governance by Change**（变更驱动治理）。

维度	新项目路径	既有大型系统路径
起点	新的业务意图	下一次真实变更
主要不确定性	目标与方案尚未成形	现状、依赖与历史规则不完整
首要动作	澄清契约、建立约束和测试骨架	Recon、现状基线和影响分析
主要风险	过度设计、需求漂移、治理过重	隐性耦合、规格漂移、破坏未知行为
边界工具	Intent Contract、Constraint Matrix	Baseline、Preserve/Unknown、Change Envelope
验证重点	新行为是否符合签署意图	新行为成立且关键旧行为未被破坏
扩展方式	从 T-001 复用到后续任务	从第一座治理岛扩展邻接保护区

两条路径是并行选择，不是成熟度先后。新项目不必先经历遗留系统治理，既有系统也不必等到“完全重构成新项目”才开始使用 3-4-3。

18.2 新项目：从零建立第一个可验证治理闭环

新项目路径适合边界较清晰的新能力、内部工具、独立服务或可隔离模块。这里的“新”不是指整个企业从零开始，而是指本次交付可以在没有大量未知历史行为的前提下建立新的契约、约束和验证基线。

18.2.1 不从平台建设开始

组织第一次实践 3-4-3 时，最容易犯的错误是先规划一个庞大的 Agent 平台、治理门户和指标中心。更可靠的方式是选择一个真实、边界清楚、可自动验证、可安全回退的小任务，跑通完整闭环。

第一个闭环的目标不是证明 AI 无所不能，而是证明：

- 模糊需求能够被校准；
- 人类责任能够被明确；
- Agent 能在约束下执行；
- 失败能够被发现和修复；
- 完成能够被证据证明；
- 数据能够反哺下一次任务。

18.2.2 选择第一个任务

推荐特征：

- 对业务有真实价值；
- 规则边界可以说清；
- 影响范围可控；
- 有现成测试环境；
- 可以回退；
- 能在较短周期完成；
- IO 能够及时作出决策。

不推荐把生产数据删除、核心资金链路重构、跨十几个团队的大型迁移作为第一个任务。

18.2.3 初始化治理目录

```
governance/
├─ Intent_Graph.md
├─ Constraint_Matrix.md
├─ constraints.yaml
├─ AI_Coding_Guide.md
├─ contracts/
│   └─ Intent_Contract_T-001.yaml
├─ evidence/
│   └─ EB-T-001.md
└─ telemetry/
    └─ runs/
```

从开源 Skill 复制模板，而不是从零发明格式。

18.2.4 T-001：费用报销规则

Specify

IO 提出：“超过 5000 元的报销，增加总监审批。”

经过 Grill-Me 后确认：

- 按报销单含税总额计算；
- 严格大于 5000 元触发；
- 本任务只处理境内人民币差旅；
- 5000 元及以下保持现有路径；
- IO 审阅并签署完整契约。

Constrain

设置：

- 禁止读取生产数据；
- 只允许修改审批规则模块和对应测试；
- 现有 5000 元及以下路径必须回归通过；
- 必须先写失败测试；
- 任何生产发布需人类批准。

Orchestrate

工作图包含：影响分析、测试生成、实现、回归、证据和遥测。测试与影响分析可以并行，实现必须等待测试骨架完成。

Prove

先运行 5000.01 元和 5000.00 元边界测试，看到预期 Red；再完成最小实现，使测试 Green；最后重构重复条件。

Evolve

故意注入一个 `>= 5000` 的错误，确认 AC-02 能拦截；修复为 `> 5000`，重跑同一测试并记录教训。

Verify

运行 AC、Harness、全量测试、构建和回归，生成 EB-T-001。

Telemetry

采集单任务遥测，更新项目累计数据和仪表盘，回写意图图谱。

18.2.5 五道机械门

```
# 1. 前置门：契约、签署、约束与 AC
python scripts/gate_check.py --gate pre --task T-001 --project-dir .

# 2. 编码门：TDD Red
python scripts/gate_check.py --gate coding --task T-001 --project-dir .

# 3. 验证门：TDD Green、AC、构建与证据
python scripts/gate_check.py --gate prove --task T-001 --project-dir .

# 4. 收尾门：证据、遥测、仪表盘与图谱
python scripts/gate_check.py --gate closing --task T-001 --project-dir .

# 5. Bug 回溯门：完成后发现问题时使用
python scripts/gate_check.py --gate bug --task T-001 --project-dir .
```

门禁的价值是把“应该做”变成“没有做到就无法通过”。

18.2.6 第一个闭环的验收标准

- IO 真实签署契约；
- 至少一项运行时 AC；
- 有真实 Red 和 Green；
- MUST 约束全部通过；
- Evidence Bundle 完整；
- 单任务遥测存在；
- 图谱完成回写；
- 人类裁决有范围、条件和依据。

18.2.7 第一次实践的角色配置

最小团队不需要新增正式岗位，但必须明确三种责任：

- IO：由真正拥有报销规则解释权的人承担；
- OA：由能理解业务、代码、测试和治理工具的人承担；
- AS：可以是一个主 Agent，也可以是测试与实现两个 Agent。

财务、安全或运维专家按风险参加，不需要全程参与所有执行。

18.2.8 第一个闭环的工作节奏

启动

选择任务、确认 IO/OA、检查测试环境和回退条件。时间可以很短，但不能跳过。

契约形成

通过 Grill-Me 解决真正会改变结果的歧义。OA 起草，IO 独立签署。

受控执行

AS 按 Work Graph 完成 Red、Green、Refactor 和约束检查。OA 只在微检查点和异常时介入。

证据裁决

Evidence Bundle 先由 OA 审查完整性，再由 IO 作价值和风险裁决。

复盘反哺

采集遥测，识别下次最值得改善的一个环节。不要第一次就追求全面平台化。

18.2.9 失败也可以是有有效结果

第一个任务可能没有通过发布门。只要系统成功发现了不清晰规则、越权工具、无效测试或不可恢复风险，这次实践仍然产生价值。

评估首个闭环时，应问：

- 是否更早发现了问题？

- 是否限制了影响范围？
- 是否明确了责任？
- 是否保留了真实证据？
- 是否形成了下一次可复用资产？

不要为了证明试点成功，降低门禁获得一次虚假的全绿。

18.2.10 首次实践的常见陷阱

- 选择过大或不可逆任务；
- IO 没有时间参与，实际由 OA 代替决策；
- 直接使用完整模板，产生大量空字段；
- 没有真实测试环境；
- 只演示正常路径；
- 把“Agent 自动修复了很多次”当作成熟；
- 为制作汇报手工美化遥测；
- 试点结束后治理资产无人维护。

18.2.11 从 T-001 到 T-002

第二个任务不应简单重复。它应使用 T-001 沉淀的图谱、约束和模式，验证系统是否真的学习：

- Grill-Me 问题是否更少、更准确；
- 上下文是否更精简；
- 首次成功率是否改善；
- 约束是否更早拦截问题；
- Evidence Bundle 是否更容易生成；
- 人类介入是否更集中在真正的价值判断。

只有跨任务改善，才能说明组织开始形成能力。

18.2.12 一次完整的 T-001 决策记录

假设业务提出：“员工报销超过 5000 元需要总监审批，明天上线。”团队不直接把这句话交给 Agent，而是依次形成可判定事实。

Specify 阶段澄清金额口径、币种、税费、等于 5000 元的处理、历史草稿和紧急回退；Constrain 阶段写入禁止修改历史已审批单、禁止访问生产凭证、数据库迁移必须人工批准；Orchestrate 阶段把测试、领域规则、接口和证据节点组成 DAG；Prove 阶段先用三个边界例子看到 Red，再完成最小实现；Evolve 阶段处理一次旧接口语义冲突并保留修复证据；Verify 阶段汇总构建、测试、约束、差异和残余风险。

最终裁决可能不是简单的“上线”，而是：“新建报销单规则通过；历史草稿迁移不在本次范围；先对财务测试账号启用，观察两小时；异常率超过阈值自动关闭开关。”这就是从一句话需求到可控制发布的完整转化。

18.2.13 试点复盘应该讨论什么

第一次复盘不要只问 AI 节省了多少时间。更值得讨论的是：

- 哪个歧义如果未被追问，会造成错误交付？
- 哪条约束真正阻止了越界？
- 哪份证据改变了上线决定？

- 哪次人工介入不可替代，哪次可以自动化？
- 哪个工件下次可以复用，谁负责维护？
- 为了治理增加的成本，是否换来了更早的风险发现？

团队应保留原始数据和失败过程。一次看起来不够顺畅、却真实揭示问题的试点，比精心排练的自动化演示更有价值。

18.3 既有大型系统：从真实变更建立可验证治理岛

前面的 T-001 案例接近一项边界清楚的新能力：团队可以先写意图契约，再建立测试和实现。但企业真实环境中，大量 AI Coding 工作发生在运行多年、几十万甚至上百万行代码的既有系统里。文档不完整，模块相互耦合，现有测试薄弱，业务规则藏在代码、数据库、配置和人员记忆中；某些看似多余的逻辑，可能承担历史兼容或监管责任。

这时如果要求团队先治理整个系统、重建全部文档、补齐全部测试，再允许 Agent 开始修改，3-4-3 会变成一个没有终点的大工程。反过来，如果把整座遗留系统直接交给 Agent，让它“先理解再修改”，高速生成又会把未知依赖迅速放大。

因此，既有大型系统需要采用不同于新项目的落地策略：

新项目适合治理先行；既有系统适合变更驱动、增量接管。不是先治理全部代码，而是先治理下一次变化。

18.3.1 不追求先理解整个系统

百万行系统不可能在一次会话、一次扫描或一次架构梳理中被完整理解。代码也不等于真实业务意图：文档可能过期，配置可能改变行为，线上数据可能触发隐藏分支，实际流程可能依赖人工补偿，生产运行结果还可能与设计不一致。

以下做法看似系统，实际风险很高：

- 一次性把整个仓库塞给模型；
- 自动生成全局架构图并当作事实；
- 为所有历史模块补写意图契约；
- 先重构遗留代码，再处理当前需求；
- 给全部业务路径补齐测试后才开始试点；
- 建设统一 Agent 平台以后才允许真实交付。

正确目标不是“Agent 已经理解全部代码”，而是：

在当前变更的影响半径内，团队已经获得足以安全行动的理解，并明确标出了仍然不知道什么。

这意味着治理深度由变化和风险驱动，而不是由代码库体积驱动。一项只涉及一个报表字段的低风险修改，不需要先解释全部支付系统；一项涉及资金计算和历史数据迁移的三行修改，却可能需要跨模块、数据和运行时证据。

18.3.2 从项目初始化转向 Change-Driven Governance

新项目通常从治理资产向实现展开：

意图图谱

- 全局约束
- 第一份意图契约
- Work Graph
- 新实现与证据

既有系统则从一次真实变化进入：

```
真实变更请求
→ 找到业务与代码入口
→ 探测影响半径
→ 固化当前行为
→ 签署局部意图与兼容责任
→ 建立变更围栏
→ 在局部运行 SCOPE-V
→ 用前后证据证明结果
→ 将新知识回写局部图谱
```

这可以称为 **Change-Driven Governance（变更驱动治理）**。治理对象不是百万行代码本身，而是每一次进入系统的变化：为什么改、允许影响什么、必须保持什么、哪些区域仍未知、用什么证据证明没有破坏，以及本次发现怎样降低下一次变化的风险。

变更驱动不等于被动打补丁。每次任务都会增加一小块经过验证的系统知识，多个任务逐步把局部治理岛连接起来，最终形成真实而非想象出来的系统治理地图。

18.3.3 Recon：进入 Specify 前的认知侦察

新项目可以直接从 Specify 开始；既有系统通常需要在 S 之前增加一个轻量的 Reconnaissance，即**认知侦察**。Recon 不是 SCOPE-V 的第七个控制面，也不是无限期架构研究，而是为了让本次 Specify 有足够事实基础。

```
Recon
→ Specify
→ Constrain
→ Orchestrate
→ Prove ≠ Evolve
→ Verify
→ Telemetry
```

Recon 至少调查六类信息：

1. **代码结构**：入口、模块、共享库、关键类和静态依赖；
2. **运行关系**：API、消息、数据库、批处理、配置和外部系统；
3. **现有行为**：生产日志、脱敏样本、测试、监控、缺陷和人工补偿；
4. **变更历史**：相关 Commit、事故、回滚和反复修改区域；
5. **知识所有权**：谁真正理解规则，谁维护模块，谁能批准变化；
6. **未知项**：哪些关系尚未证明，错误假设会产生什么影响。

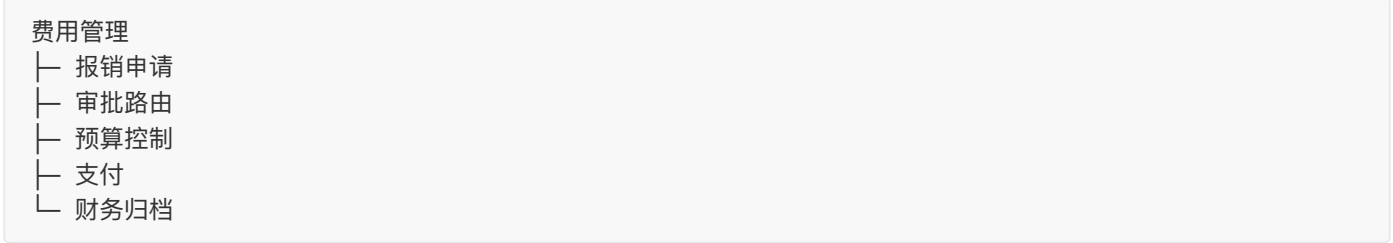
AI 可以承担代码搜索、依赖抽取、接口枚举、调用链摘要和 Git 历史聚类，但其输出只能作为候选知识。候选关系应尽量用静态分析、运行时 Trace、真实样本、现有测试和领域专家交叉验证。

Recon 应有时间盒。低风险局部变更可能只需两小时；高风险跨系统任务可能需要数天和专门验证。如果时间盒结束仍无法确认关键依赖，正确结果是缩小范围、增加保护或停止，而不是让 Agent 自动补齐缺失事实。

18.3.4 三层地图：业务能力、系统结构和变更影响

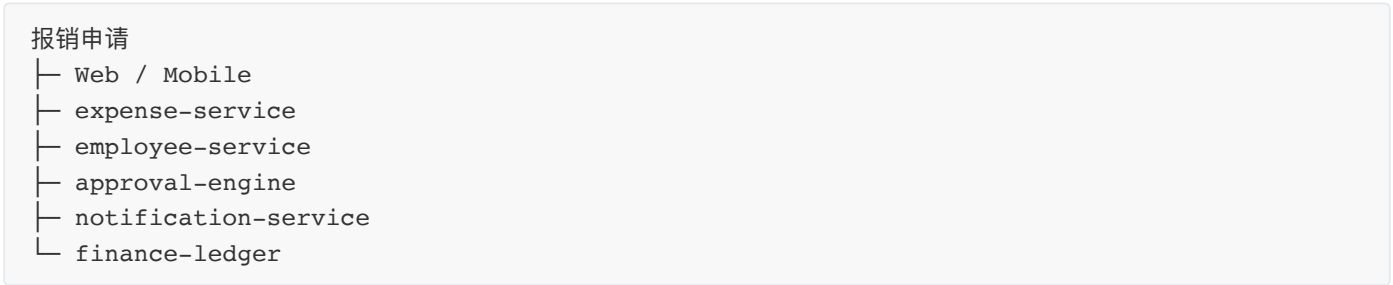
既有系统需要地图，但不需要在第一次任务中绘制一张声称完整的企业架构全景图。建议按三个层次逐步建立。

第一层是业务能力图，回答系统对外提供什么价值：

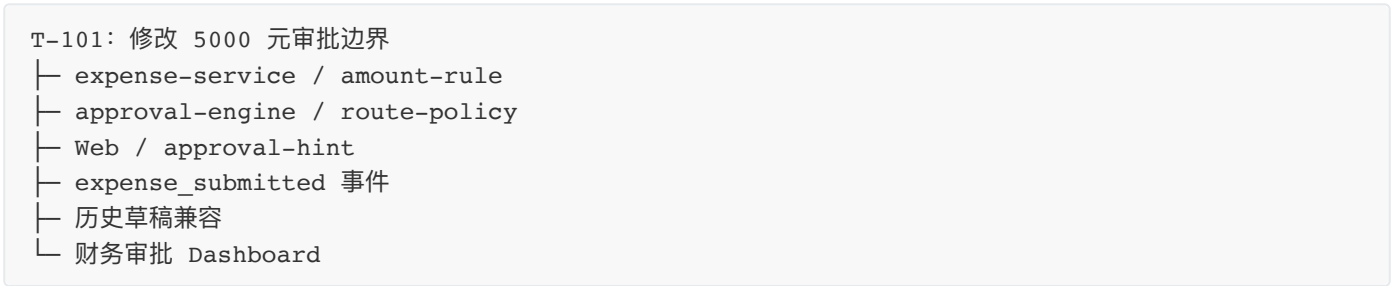


这一层主要来自业务负责人、用户流程、制度和真实运营，不应只从包名或数据库表推导。

第二层是系统与模块图，回答能力由什么承载：



第三层是当前任务的变更影响图，只展开本次需要理解的关系：



业务能力图保持方向，系统图提供粗粒度导航，影响图才是 Agent 的主要 L3 Context。三张图都应标注来源、版本、责任人和可信度，避免“图画出来了”被误解为“关系已经证明”。

关系	主要来源	状态
报销服务调用审批引擎	运行时 Trace	VERIFIED
历史草稿可能复用当前规则	老员工访谈	HYPOTHESIS
通知失败不影响审批提交	静态代码分析	PARTIALLY_VERIFIED
外币使用独立规则	生产样本 + 测试	VERIFIED

18.3.5 四类行为必须分开

遗留系统最危险的假设，是把代码里的行为、业务想要的行为和生产真实行为当成一回事。至少要区分四种状态：

- **Intended Behavior**：业务和制度希望系统怎样运行；
- **Observed Behavior**：在生产、测试或样本中实际观察到什么；
- **Implemented Behavior**：静态代码和配置看起来实现了什么；
- **Verified Behavior**：当前证据能够可靠证明什么。

例如制度规定“严格超过 5000 元进入总监审批”，这是 Intended；生产样本显示 5000.00 元有时进入总监审批，这是 Observed；代码中某个模块使用 `>=`、另一个模块使用 `>`，这是 Implemented；经过指定版本、数据和链路测试后能够确认的新建人民币报销行为，才是 Verified。

四者不一致时，Agent 不能自行选一个看起来更合理的答案。OA 负责把差异整理成可决策问题，IO 或领域责任人决定要保持现状、修复行为、兼容一段时间，还是修改制度和契约。

这也是既有系统中 Intent Graph 的特殊之处：一部分从业务意图向下展开，另一部分通过真实变更、运行证据和专家确认逐步恢复。恢复出来的不是“代码的故事”，而是带来源和可信度的系统知识。

18.3.6 遗留契约要增加 Baseline、Preserve 和 Unknown

新项目契约主要说明目标、Non-goals、规则和 AC。既有项目还要承担兼容责任，因此建议增加三组内容。

Current Behavior Baseline 记录已经观察到的事实：

BASE-01：5000.00 元当前进入普通审批
 BASE-02：历史草稿打开后不会自动重算审批链
 BASE-03：旧版移动端仍调用 v1 提交接口

Baseline 不是对现状的价值认可，只是说明修改前系统怎样运行。

Preserve List 说明本次必须保持的行为：

PRESERVE-01：不得改变历史草稿已有审批结果
 PRESERVE-02：v1 消费方继续获得兼容响应
 PRESERVE-03：现有财务导出字段与口径不变

Unknown List 明确尚未证明的区域：

UNKNOWN-01：外币离线批处理是否复用同一金额规则
 UNKNOWN-02：某地区定制模块是否依赖旧错误码

UNKNOWN 不能默认等于安全。若变更可能触及它，应补充侦察、缩小范围、增加灰度和监控，或进入 HITL。把未知显式写入契约，比让 Agent 用高置信度语言填补空白更可靠。

18.3.7 Characterization Test：先冻结现实，再改变现实

对于新功能，TDD 的 Red 表示新能力还没有实现；对于遗留系统，团队经常连“现有行为究竟是什么”都不确定。这时需要在新 AC 测试前建立 Characterization Test，通常译为**特征测试或现状刻画测试**。

它不证明现有行为正确，而是把今天观察到的行为固定下来，防止 Agent 修改一处逻辑时无意改变其他历史行为：

修改前现状
 └─ 4999.99 → 普通审批
 └─ 5000.00 → 普通审批
 └─ 5000.01 → 总监审批
 └─ 历史草稿 → 保留原审批链
 └─ v1 接口 → 返回旧字段结构

随后建立四类证据：

Characterization Test：证明哪些旧行为保持不变
 Contract Test：证明提供方与消费方仍兼容
 New AC Test：证明新意图已经实现
 Regression / SIT：证明影响半径内没有产生组合破坏

现状刻画可以来自确定性测试、脱敏 Golden Master、接口响应、数据库快照差异或生产流量回放。Golden Master 只能用于发现变化，不能让 AI 自动更新期望值消除失败。每一项变化仍需判断是预期修正、兼容风险还是

测试噪声。

18.3.8 Change Envelope：为 Agent 建立变更围栏

百万行系统不应把整个仓库默认为 Agent 的可写空间。Constrain 应围绕本任务建立 Change Envelope：

允许修改

└ expense-service/rules/**

└ approval-adapter/**

└ web/expense/approval-hint.tsx

只读参考

└ employee-service/**

└ finance-export/**

└ shared-money/**

禁止触碰

└ production-migrations/**

└ legacy-settlement/**

└ region-custom/**

└ archived-contracts/**

围栏还应定义是否允许新增依赖、改变公共接口、修改数据库、运行生产工具、自动解决冲突，以及最大文件数、Token、时长和重试次数。路径只是第一层，更重要的是数据、接口、行为和发布权限边界。

一个实用默认是将功能变化与大规模重构分开。如果 Agent 声称为了三行规则必须先重构二十七个类，OA 应把它视为风险信号。可以先通过适配器、防腐层或 Feature Flag 局部实现；结构性重构另建契约，单独说明价值、影响和回退。

18.3.9 治理岛、邻接保护区与未知遗留区

既有系统可以被划分为三个动态区域：

已治理岛	
契约、约束、测试、证据和责任已建立	
邻接保护区	
依赖已识别，只读或受严格限制	
未知遗留区	
尚未充分理解，默认不允许自动修改	

第一座治理岛可能只包含一个接口、一个规则模块、三条 AC、五个现状测试、一条跨系统契约和一份 Evidence Bundle。下一次相关变化到来时，在已有知识上扩展：

T-101 金额规则

↓

T-117 外币换算

↓

T-136 审批委托

↓

T-152 历史草稿兼容

这与遗留系统改造中的 **Strangler Fig Pattern（绞杀榕模式，也常简称 Strangler Pattern）** 相似。这个名字来自热带雨林中的绞杀榕：它先在一棵宿主树上生长，逐渐向下扎根、向上获得阳光；当自身能够独立生存后，原来的宿主可能逐步退出。Martin Fowler 用这个自然现象比喻遗留系统现代化：不进行一次高风险的“大爆炸替换”，而是在旧系统周围建立新的能力，将一小部分行为逐步导向新实现，让新旧系统在过渡期共存，最后再安全退出已被替代的部分。[Martin Fowler 对 Strangler Fig 的解释](#)

它通常包含四个关键思想：

- 1. **先找接缝（Seam）**：找到能够隔离流量、接口、数据或业务能力的边界；
- 2. **小块替换**：先迁移一个可独立验证的能力，而不是复制整个旧系统；
- 3. **新旧共存**：通过路由、适配器、防腐层或代理，让旧实现和新实现同时运行；这里的防腐层是一个翻译边界，用来把旧系统的数据和语义转换成新系统能够接受的模型，避免旧设计继续渗透到新能力内部；
- 4. **逐步切换与退役**：新能力经过真实流量和业务验证后，再扩大范围并关闭相应旧路径。

例如，团队需要改造遗留结算系统中的“退款审核”。大爆炸方案是重写整个结算平台；Strangler Fig 方案则可能先在旧系统入口前增加路由层，只把一个地区、一个退款类型或少量灰度流量导向新的审核服务。新服务稳定后，再逐步迁移其他退款类型，最终让旧审核模块退出，而总账、对账和其他尚未迁移的能力继续留在旧系统中。

治理岛借用了这种“从小边界开始、允许共存、用证据逐步扩大”的思想，但替换对象不完全相同：

Strangler Fig Pattern	3-4-3 治理岛
逐步替换旧代码或旧服务	逐步替换依赖个人记忆、口头约定和偶然测试的治理方式
用路由和适配层隔离新旧实现	用 Change Envelope、权限和 Work Graph 隔离可改与不可改区域
用运行结果证明新实现可以接管	用契约、现状测试、Evidence Bundle 和遥测证明治理范围可信
最终目标通常是退役旧实现	目标是扩大可信治理范围，旧代码可以继续存在

因此，治理岛并不要求每次都把旧模块重写成新服务。一个旧模块即使暂时不重构，只要关键行为已被识别、约束、测试、证据和责任覆盖，也可以进入已治理岛。治理岛的边界由证据决定，不由团队主观宣布。

18.3.10 两级或三级 OA：让知识靠近系统

百万行项目不可能由一个全局 OA 理解所有代码和业务。大型既有系统可以采用三级责任结构：

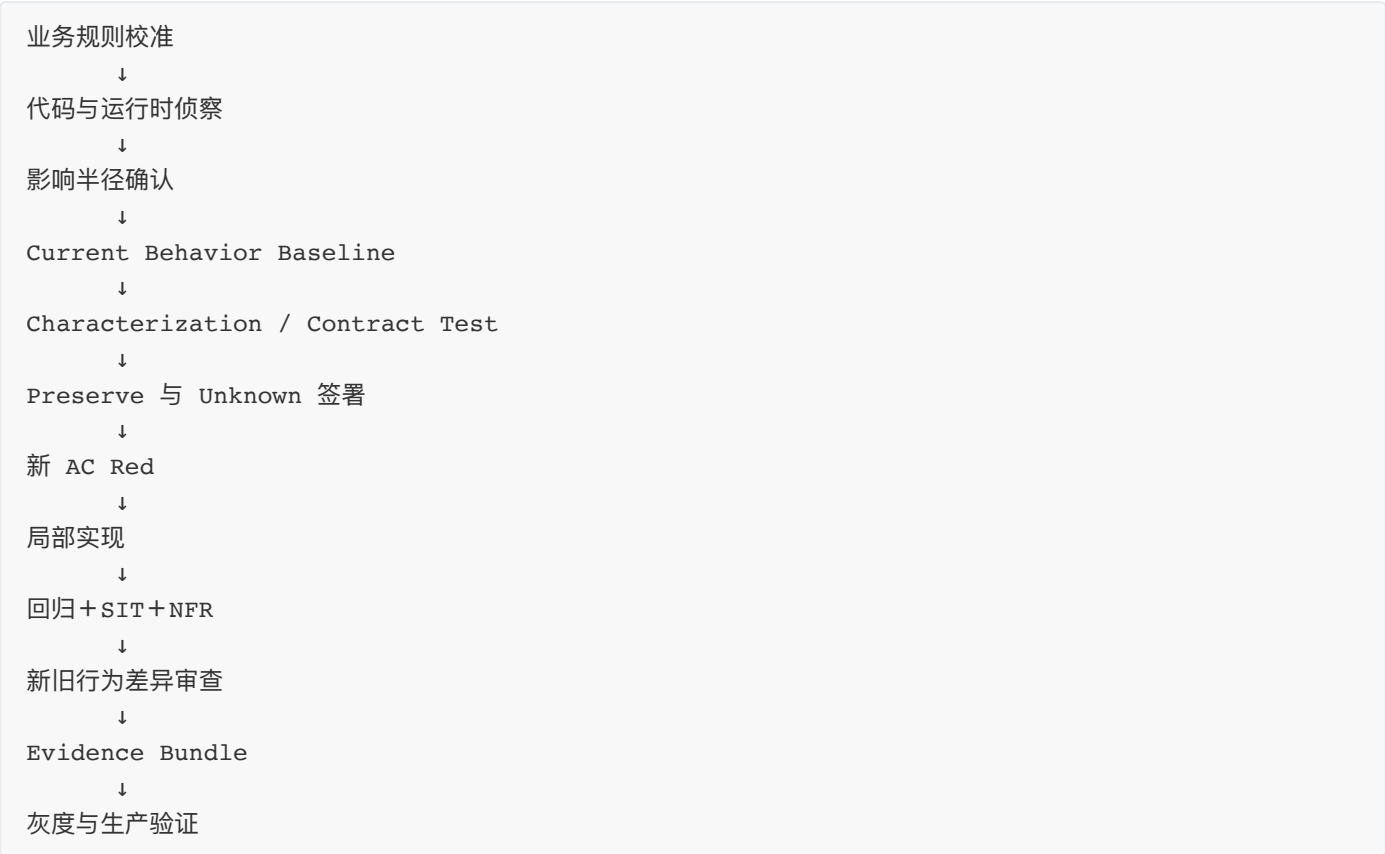


全局 OA 不是百万行代码的总审核员。它治理跨域关系、全局底线和发布裁决；领域 OA 对业务能力与规则负责；模块 OA 让局部知识、代码和验证靠近真正维护者。一个人在小型组织中可以承担多层职责，但关键变更必须知道当前由哪一层作决定。

当知识所有者已经离职时，不能用 Agent 自动总结替代所有权。组织需要指定临时责任人，以运行证据、历史缺陷和受控实验逐步恢复知识，并将不确定性保留在工件中。

18.3.11 遗留 Work Graph 要包含探索和基线节点

新项目的 Work Graph 可以较快进入测试和实现；既有系统的工作图应显式安排侦察、基线和兼容审查：



探索节点必须有产物和停止条件，例如“输出已验证调用方列表、两项未知和一张影响图”；不能写成“让 Agent 研究代码直到理解”。基线节点则要说明哪些现状已经被运行证据固定，哪些只来自静态推断。

若实现过程中出现新依赖，Work Graph 应回流到影响确认，而不是让 Agent 静默扩大范围。既有系统的 O 不只是安排执行顺序，更承担控制认知扩张和变化半径的责任。

18.3.12 遗留 Evidence Bundle 要证明“改变了什么，也保住了什么”

新项目证据包主要证明新能力满足 AC；遗留系统还需要证明现有责任没有被意外破坏。建议增加：

- 变更前行为基线；
- 静态依赖和运行时调用链；
- 受影响模块、数据和消费方；
- 修改文件、只读区域和禁止区域；
- Characterization Test 前后结果；
- 新增 AC、接口契约、回归和 SIT 结果；
- Schema、数据迁移和配置差异；
- 新旧版本行为差异；
- 未验证区域与残余 UNKNOWN；
- 灰度、监控、回退和领域专家确认。

证据映射可以写成：

裁决结论	主要证据
新规则已经实现	New AC Test
历史行为没有被意外改变	Characterization Test
现有消费方仍兼容	Contract Test + SIT
影响范围没有静默扩大	依赖图 + 文件差异 + 工具审计
未触碰未知高风险区域	Change Envelope 检查
可以有限发布	灰度条件 + 回退演练 + 人类裁决

“全量回归通过”如果无法说明覆盖哪些历史责任，证明力仍然有限。Evidence Bundle 应同时显示已证明、未证明和明确接受的风险。

18.3.13 不测全量代码覆盖率，测治理变化覆盖率

百万行系统不可能短期全面治理，也没有必要用全项目代码覆盖率衡量 3-4-3 成熟度。更有意义的是观察从现在开始的变化是否被治理：

变更治理覆盖率 =

纳入契约、约束和证据的 AI 参与生产变更数

÷ AI 参与生产变更总数

还可以观察：

- 高风险变更的契约覆盖率；
- 关键业务链路的治理岛覆盖率；
- 高频变更区域的 Characterization Test 覆盖；
- 新增代码和接口的治理覆盖率；
- 跨模块契约覆盖率；
- UNKNOWN 的新增、关闭与逾期趋势；
- Agent 越界修改和静默扩域次数；
- 逃逸缺陷能否归属原契约；
- 经过验证并复用的遗留知识数量。

首要目标不是“一年治理完一百万行”，而是：

从某个时间点开始，所有 AI 参与的重要变化不再制造新的治理盲区，并持续减少最危险的旧盲区。

这可以理解为先遏制治理债务增长，再通过真实任务偿还高价值、高风险区域的存量债务。

18.3.14 如何选择第一座治理岛

第一个遗留系统试点不应选择核心结算重构、跨十几个系统的架构升级、全局权限替换、不可回退的数据迁移，或业务部门尚未形成共识的历史规则。

更适合的任务通常具备：

- 有真实业务价值和近期交付需求；
- 变更入口能够定位；
- 影响约一到三个模块；

- 有领域专家或真实证据可用于确认；
- 能够构造当前行为样本；
- 主要 AC 可以自动验证；
- 可以灰度、关闭或快速回退；
- 所在区域未来还可能继续变化，治理资产能被复用。

典型任务包括：已有接口增加校验、高频缺陷修复、审批边界调整、旧接口兼容、关键报表字段修正或局部性能瓶颈治理。它们足够真实，又不会把第一次实践拖入全系统改造。

18.3.15 客户现场可直接使用的七步试点

第一步，选择一项正在排期的真实变化，而不是为演示虚构需求。

第二步，用两小时到半天完成第一轮 Recon。让 AI、维护人员、代码、Trace、样本和历史缺陷共同形成局部影响图，并明确 UNKNOWN。

第三步，建立现状基线。补少量高价值 Characterization Test、接口样本和生产指标，不追求覆盖整个模块。

第四步，形成增量 Intent Contract。除目标和 AC 外，签署 Baseline、Preserve List、Unknown List、Non-goals 和回退要求。

第五步，建立 Change Envelope。限制 Agent 可写模块、文件、接口、数据、工具、依赖和重试预算。

第六步，运行完整 SCOPE-V。完成新 AC Red、局部实现、回归、SIT/NFR、差异审查、Evidence Bundle 和独立 V。

第七步，把本次确认的业务关系、接口、约束、测试、未知与教训回写，形成第一座治理岛，并观察下一次相关任务是否真正复用。

第一次成功的标准不是开发速度最快，而是：

团队能够证明 Agent 在没有理解整个百万行系统的情况下，仍然安全完成了一次真实变化。

18.3.16 一个大型既有系统案例

假设客户拥有运行十年的费用平台，约八十万行代码，包含 Web、移动端、六个服务、两套审批引擎和多个地区定制模块。业务要求将人民币报销的总监审批阈值从 5000 元调整为 8000 元。

如果只搜索 5000 并替换，可能触及展示文案、旧数据校验、财务报表和地区特殊规则；如果要求先重构全部审批系统，任务又会失控。

团队先进行 Recon：静态搜索发现三个阈值定义，运行时 Trace 证明新建报销走 approval-engine-v2，生产样本显示历史草稿保留旧路由；但外币夜间批处理是否复用公共金额库仍未知。领域专家确认本次只改变境内人民币新建报销，历史草稿、外币和地区定制全部保持。

于是契约写入：

目标：人民币新建报销严格超过 8000 元进入总监审批
Baseline：当前阈值为 5000 元
Preserve：历史草稿保持原审批链；v1 接口结构不变
Unknown：外币批处理与公共金额库关系尚未证明
Non-goals：外币、历史数据迁移、地区定制、审批引擎重构

Change Envelope 只允许修改 v2 金额策略、对应接口适配和 UI 提示；公共金额库只读，外币批处理和迁移目录禁止修改。Work Graph 先建立 4999.99、5000、5000.01 等现状测试，再为 7999.99、8000、8000.01 建立新 AC Red。Contract Test 固定旧移动端响应，SIT 验证审批引擎与通知链路，数据检查证明历史草稿未重算。

实现完成后，Evidence Bundle 不只写“8000 元测试通过”，还展示现状基线、影响图、文件范围、接口兼容、UNKNOWN 未触碰证明、灰度计划和回退开关。IO 最终批准只对两个部门灰度；生产遥测观察审批分布、错误率

和历史草稿异常。任务完成后，已验证的 v2 调用关系、历史草稿规则和外币 UNKNOWN 被写入领域图谱。

这次实践没有治理八十万行代码，却在一个高频业务区域建立了第一座可信治理岛，为下一次外币或委托审批变化提供了可复用起点。

18.3.17 既有系统落地的常见失败

- 把 AI 自动生成的架构图直接当作系统事实；
- 为了“上下文完整”向 Agent 注入整个仓库和生产数据；
- 没有 Current Behavior Baseline 就开始修改；
- Characterization Test 将历史行为一律视为正确；
- 只写新 AC，不写 Preserve 和 Unknown；
- 允许 Agent 以顺手重构为由扩大二十多个模块；
- 只看静态依赖，不验证运行时、配置和数据关系；
- UNKNOWN 在汇报中被隐藏或默认视为 PASS；
- 全量回归全绿，却无法说明关键历史责任是否被覆盖；
- 以全项目代码覆盖率衡量试点，导致治理工程无限膨胀；
- 全局 OA 试图审核所有模块，成为新的瓶颈；
- 第一座治理岛完成后没有回写图谱和资产，下一次仍从零开始；
- 为证明 AI 成功，选择精心搭建的新模块而回避真实遗留变化。

18.3.18 既有系统专题的验收问题

团队可以用一项真实变更检查：

1. 是否明确区分 Intended、Observed、Implemented 和 Verified Behavior？
2. 是否有时间盒 Recon、影响图、证据来源和 UNKNOWN？
3. 契约是否包含 Baseline、Preserve List 与 Unknown List？
4. 是否在新 AC 前建立了必要的 Characterization Test？
5. Change Envelope 是否能机械限制 Agent 的写入、工具和依赖范围？
6. Work Graph 是否包含探索、基线、差异审查和回流节点？
7. Evidence Bundle 是否同时证明新行为成立和关键旧行为未被破坏？
8. 生产遥测是否能关联本次契约、版本和灰度条件？
9. 新确认的知识是否回写图谱、约束、测试或模块指南？
10. 下一次相关变化是否能够复用这座治理岛，而不是重新扫描全部系统？

如果这些问题能够由真实工件和机器结果回答，3-4-3 就已经进入既有系统；不需要等到百万行代码全部被重新解释以后才宣布开始。

18.4 两条路径的共同底座与选择原则

新项目与既有系统的起点不同，但进入执行后都必须回到同一套治理底座：

可判定意图

→ 明确约束与权限

→ 有边界的 Work Graph

→ 真实测试与机器证据

→ Evidence Bundle 支持独立裁决

→ Telemetry、知识与记忆反哺

两条路径都不允许跳过 IO 签署、MUST 门禁、Evidence Bundle 和遥测；差异主要发生在 Specify 之前和 Prove 的证据结构中。新项目需要证明“设计出来的新行为正确”，既有系统还要证明“被保留的现实没有遭到破坏”。

团队可以用下面四个问题选择入口：

1. 当前能力是否已经在生产环境中运行并承担真实责任？
2. 现有行为能否由可靠契约、测试和文档完整解释？
3. 本次变化是否可能触碰未知依赖、历史数据或兼容责任？
4. 如果取消本次变化，是否仍需要先恢复大量系统认知？

如果多数答案指向“否”，可以采用新项目路径；如果多数答案指向“是”或 UNKNOWN，应采用既有系统路径。对于大型系统中新建的隔离服务，可以局部采用新项目路径，但它与旧系统的接口、数据迁移和发布仍要接受既有系统路径的保护。

共同检查点	新项目的证据	既有系统的附加证据
Specify	目标、非目标、AC、IO 签署	Intended/Observed/Implemented/Verified、UNKNOWN
Constrain	权限、质量、安全、成本和回退	Preserve List、历史责任、Change Envelope
Orchestrate	实现、测试、证据和发布 DAG	Recon、基线、影响分析和回流节点
Prove/Evolve	TDD、边界测试、自愈记录	Characterization Test、差异审查、邻接保护
Verify	契约与约束证据完整	新旧行为双重证明、灰度与生产观察

两条路径最终都通过真实任务积累治理资产。区别不是哪条更先进，而是哪条更诚实地面对当前系统的未知。

18.5 本章小结

第一个可信闭环不需要宏大平台。新项目可以从意图与约束向实现展开；既有大型系统则从下一次真实变化开始，通过 Recon、现状基线、变更围栏和治理岛逐步恢复可见性、限制影响并积累证据。一个最小 Intent Graph、一份签署契约、一套可执行约束、一份 Evidence Bundle，以及进入慢外环的真实遥测，足以让团队从“会用 AI”迈向“会运行人机协同研发系统”。

本章最小实践：完整运行 T-001，或在既有系统中完成一个有变更围栏的真实任务。

完成证据：通过 closing gate 的治理目录、影响范围与可回退交付。

延伸实践：推荐学习《Agentic Agile FDE 进阶：从客户现场到企业级生产交付》，在新项目与既有系统中练习 Recon、变更边界、保留行为和分层验证。

第十九章 | Knowledge & Memory Engineering：让上下文和经验成为组织资产

19.1 Agentic Knowledge Engineering：用双地图连接需求与代码

既有大型系统的增量治理还有一个基础问题：Agent 怎样找到当前任务所需的代码、规则、历史、接口和运行证据？当知识来自代码、Wiki、工单、聊天、日志和人员记忆时，哪些内容可以相信，哪些已经过期，哪些只能作为线索，哪些又不能被当前 Agent 访问？

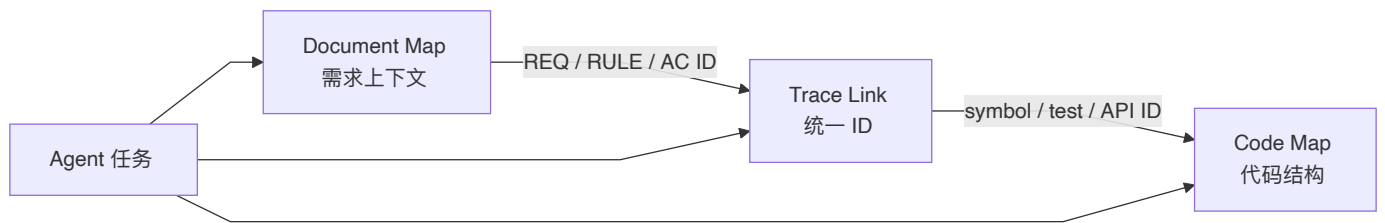
企业常把答案归结为“建设一个 AI 知识库”：收集文档、切片、Embedding、写入向量数据库，再让大模型检索。这样的系统可以提高文本可发现性，却无法自动保证知识正确、新鲜、适用、授权和可验证。对 AI Coding 而言，一段被高相似度召回的过期架构说明，可能比完全没有知识更危险。

因此，Agentic Agile 3-4-3 需要的不是单一文档库或 RAG 平台，而是一套 **Knowledge & Memory Engineering（知识与记忆工程）**：让知识能够被结构化发现、按任务裁剪、带来源使用、经过证据晋升、在失效时撤回，并通过交付结果持续校验。它是 3-4-3 的知识与状态基础设施，不是第五个动态工件，也不能绕过契约、约束和 Verify 直接获得组织事实地位。

从工程落地看，这套体系首先要把两个经常混在一起的知识世界分开：

- **Document Map 管需求上下文**：需求、业务规则、验收标准、领域术语、约束、决策与历史原因；
- **Code Map 管代码结构**：仓库、模块、文件、符号、调用关系、接口、数据流与测试；
- **Trace Link 用统一 ID 串联两张地图**：回答一项需求由哪里实现、由什么测试证明、发生变更时影响什么。

两张地图不是两座信息孤岛，而是面向同一任务的两种视角。Document Map 回答“为什么这样做”，Code Map 回答“在哪里实现”，Trace Link 回答“需求—代码—测试是否闭环”。



知识面	核心对象	主要问题	推荐开源工具
Document Map	PRD、需求、规则、AC、术语、ADR、约束	为什么做、什么才算完成、哪些决定仍有效	IWE
Code Map	模块、文件、符号、调用、接口、测试	在哪里实现、谁依赖它、改动影响哪里	codebase-memory-mcp
Trace Link	Requirement ID、Rule ID、AC ID、Symbol ID、Test ID	需求是否落实到实现并得到验证	项目内可版本化的关系清单或图索引

AI Coding 时代真正稀缺的不是知识存量，而是能够被正确检索、经过验证、带有责任边界并持续演进的可信知识。

19.1.1 双地图的工具落地：IWE 与 codebase-memory-mcp

Document Map 与 Code Map 的共同理念是：**原始资产留在本地并保持可检查，机器生成的索引可以重建，通过 MCP 为分布在不同工具和任务中的 Agent 提供统一查询能力**。两者都可以采用 local-first 架构，但处理的对象和判断逻辑不同。

Document Map 推荐使用 IWE。 IWE 以本地普通 Markdown 文件为事实源，通过文档层级、交叉引用和反向链接形成知识图谱，并同时提供 CLI、MCP 和编辑器 LSP。它不要求把需求迁入专用数据库；需求、业务规则、验收标准、术语、ADR 和会议结论仍然是人可以直接审阅、Git 可以版本化的文件。项目可以用 YAML Front Matter 固

定 `req_id`、状态、责任人、适用范围和版本，用显式链接表达 `depends_on`、`supersedes`、`constrained_by` 与 `verified_by`。

IWE 的优势不只在安装轻量。它能够对 Front Matter 和文档结构执行 Schema 校验，为 Agent 写入声明影响范围，并在重命名或重构文档时维护链接关系。这与需求治理的核心要求一致：知识必须可检查、可追踪、可验证、可回退，而不只是“语义相似”。IWE 使用 Rust 实现，可通过 Homebrew、npm 或 Cargo 安装；采用 Apache-2.0 许可证。项目说明：<https://github.com/iwe-org/iwe>。

IWE 是 Document Map 的推荐结构底座，不是需求治理本身。它能够建立、查询和维护关系，却不能替代需求签署、责任人裁决、有效期管理和 Evidence。AI 写入的新结论仍应先是候选知识，不能因为已经进入图谱就自动成为正式业务规则。

如果组织还需要大规模语义召回，可以把 QMD 作为 IWE 之上的可选检索增强；如果更关注跨会话对话记忆，也可以接入 Basic Memory 等可选 Provider。但语义索引和 Agent Memory 不应取代 IWE 中可审阅、可版本化的需求图。QMD 项目说明：<https://github.com/tobi/qmd>。

Code Map 推荐使用 codebase-memory-mcp。它通过 Tree-sitter AST 与语义解析，为函数、类、调用链、HTTP 路由和跨服务关系建立持久化代码知识图谱，并通过 MCP 支持结构查询、调用链追踪和影响分析。它直接把代码结构图的持续构建、跨会话保存和查询作为产品能力，更适合作为企业 Code Map 的工程底座。项目说明：<https://github.com/DeusData/codebase-memory-mcp>。

Code Map 至少应遵守四条原则：

1. 代码知识按符号、定义、引用、调用和依赖建模，不能只按固定字符数切片；
2. 地图用于导航和影响分析，不需要把全部源代码放入模型上下文；
3. 上下文应随任务动态选择，而不是所有 Agent 共用同一份静态摘要；
4. 代码变化后，结构地图应自动增量刷新，并绑定仓库、分支或 Commit 版本。

两种工具之间不能只靠模糊文本相似度连接。建议把统一 ID 写入双方都能查询的关系中，例如：

```
requirement_id: REQ-PAY-017
acceptance_criteria: [AC-PAY-017-01, AC-PAY-017-02]
implemented_by:
  - symbol: payments.refund_service.create_refund
    repository: payment-platform
verified_by:
  - test: tests/refund/test_refund_policy.py::test_refund_window
evidence: governance/evidence/EB-T-137.md
```

Document Map 不应复制整份代码结构，Code Map 也不应猜测业务意图。Trace Link 保存稳定标识和关系，查询时再分别下钻到原始需求、源代码、测试与证据。

19.1.2 知识库不等于 RAG，也不等于向量数据库

典型 RAG 流程是：

```
文档收集
→ 文本切片
→ Embedding
→ 向量检索
→ 将相似内容注入 Prompt
```

它解决的是语义相似内容的发现，不天然解决知识治理。向量距离不能证明某段内容是最新版本，也不能证明它来自权威来源、适用于当前模块、未被后续决策废弃，或者当前 Agent 有权读取。

对 AI Coding 而言，知识至少同时具有六个维度：

- **内容**：这条知识说了什么；
- **来源**：来自代码、运行数据、制度、专家还是模型推断；
- **范围**：适用于哪个领域、模块、版本和环境；
- **状态**：候选、已验证、已批准、冲突、过期或撤回；
- **权限**：谁可以检索、引用和修改；
- **证据**：什么事实支持它，什么条件会推翻它。

没有这些维度，知识库只是可搜索的材料集合。它可能帮助 Agent 更快回答，却不能保证 Agent 更可靠地行动。

19.1.3 五层知识体系

建议把企业知识分为五层，而不是全部塞进同一个向量索引。

L0：事实源层

这一层保存真正的 Source of Truth：

- Git 中的代码、配置和 Commit；
- API、数据库 Schema 和消息定义；
- CI/CD 原始运行结果；
- 生产日志、Trace、指标和事件；
- 已签署 Intent Contract；
- 已批准 Constraint Matrix 与例外；
- Artifact Digest、发布裁决和审计记录；
- 正式制度、法规和责任决定。

模型摘要不能替代原始事实。摘要可以帮助导航，但高风险结论必须能够下钻到 L0。

L1：结构索引层

这一层对应 Code Map，主要由机器自动生成：

- 文件、目录、类、函数、接口和类型；
- 定义、引用、调用和依赖；
- API 提供方与消费方；
- 数据表与读写关系；
- 测试与被测代码映射；
- 代码所有者、变更热点和历史 Commit；
- 部署单元与运行时调用链。

L1 的使命是帮助 Agent 找路，不负责解释全部业务意义。

L2：语义知识层

这一层解释系统为什么这样工作：

- 业务能力地图和领域术语；
- 业务规则与 Intent Graph；
- 架构决策记录 ADR；
- 跨模块契约和兼容策略；

- Preserve List、已知例外和历史原因；
- 运维手册、故障模式和恢复策略。

AI 可以从代码和资料中生成草稿，但需要 OA、IO、领域专家或正式证据确认。

L3：任务上下文层

它不是永久知识库，而是针对当前任务动态组装的 Context Slice：

```
当前目标与 Non-goals
+ 适用约束
+ 相关代码符号与完整文件
+ 上下游接口
+ 历史缺陷与决策
+ 现状测试与运行证据
- 无关模块
- 过期版本
- 未授权数据
```

L3 决定当前 Agent 实际看到什么。信息进入知识系统，不代表每次任务都应该注入。

L4：验证后的组织记忆层

这一层保存经过真实任务验证、未来能够复用的知识：

- 已确认的失败模式；
- 被证据支持的业务规则；
- 可复用测试和恢复模式；
- 工具风险与权限边界；
- 常见误解与反例；
- 已关闭或仍存在的 UNKNOWN；
- 对特定任务有效的上下文与编排策略。

模型反思不能自动进入 L4。只有经过 Evidence Bundle、独立验证、责任人裁决或生产遥测确认的内容，才有资格成为组织记忆。

19.1.4 每条知识都要有“身份证”

一段没有来源和范围的文字不应被称为正式知识。最小知识对象可以表示为：

```
id: K-EXPENSE-017
title: 历史草稿不重新计算审批链
type: business_rule
scope:
  domain: expense
  modules: [expense-service, approval-engine]
  environments: [test, production]
statement: 历史草稿再次打开时保留创建时的审批路径
source:
  type: production_trace
  reference: trace-set-2026-08-07
status: VERIFIED
confidence: high
owner: Expense-Domain-OA
version: 3
```

```
valid_from: 2026-08-07
valid_until: null
supersedes: K-EXPENSE-009
access:
  classification: internal
  contains_sensitive_data: false
evidence:
  - governance/evidence/EB-T-101.md
```

字段可以按组织裁剪，但至少要回答：它是什么类型、适用哪里、来自哪里、现在处于什么状态、谁负责、何时生效、替代了什么、谁能访问以及证据在哪里。

知识的引用也应携带 ID 和版本。只把一段文字复制进 Prompt，会丢失其身份；后续无法判断它是否被更新、撤回或误用。

19.1.5 知识状态机：存在不等于可信

知识至少可以有以下状态：

状态	含义	Agent 使用规则
CANDIDATE	AI 或人员提出，尚未验证	只能作为侦察线索
HYPOTHESIS	有部分依据，仍需验证	不得支撑高风险结论
VERIFIED	已被独立证据确认	可在声明范围内使用
APPROVED	已由有权责任人批准	可作为正式规则或边界
CONFLICTED	与其他来源产生冲突	停止并升级 OA/HITL
DEPRECATED	被新知识替代	默认不再召回
EXPIRED	超过有效期或环境已变化	重新验证后使用
REVOKED	因错误或风险被撤回	禁止注入和引用

状态必须影响检索和行动。一个系统如果把 DEPRECATED 内容仍以高相似度排在第一位，即使页面上显示了灰色标签，治理仍然失败。

知识状态与遗留系统的四类行为可以衔接：Observed 和 Implemented 可以产生候选事实，Intended 提供价值方向，只有经过证据确认的 Verified Behavior 才能晋升为可信知识；需要组织承担责任的规则还要进入 APPROVED。

19.1.6 知识晋升不能由模型自我证明

知识生命周期应包含：

- 发现
- 标注来源、范围和敏感级别
 - CANDIDATE / HYPOTHESIS
 - 用代码、测试、运行数据或专家交叉验证
 - VERIFIED
 - 必要时由责任人 APPROVED
 - 进入可召回知识
 - 观察实际使用效果
 - 修正、合并、过期或撤回

例如 Agent 在一次失败中发现“审批服务要求金额保留两位小数”，不能直接写成全局规则。它需要确认：要求是否来自正式 Schema；只适用于人民币还是全部币种；生产是否存在三位小数历史数据；哪些消费方依赖当前行为；谁拥有解释权。

还需要一条防止知识污染的硬规则：

模型生成内容不能仅凭被模型重复引用而获得更高可信度。可信度必须来自独立事实、运行证据或有权责任人的裁决。

否则会形成自我强化回路：模型生成错误总结，知识库收录，下一次模型召回并再次引用，重复次数被误当成共识。

19.1.7 五种检索必须协同

大型代码库不能只依赖向量相似度，至少需要五类检索。

词法检索用于精确定位错误码、方法名、配置键、表名、API 和 Contract ID。

符号与代码图检索用于回答定义在哪里、谁调用、实现了什么接口、有哪些测试和依赖。这是 Code Map 的优势区域。

语义检索用于寻找“历史报销兼容”“审批超时恢复”等概念相近内容。

运行时关系检索使用 Trace、日志、消息链路、数据库访问和生产指标，区分“代码可能调用”与“当前版本在这个场景确实调用”。

治理关系检索根据当前任务找到对应 Intent Graph 节点、有效契约、适用 MUST、历史 Evidence Bundle、UNKNOWN、发布和 Bug 回溯记录。

最终结果应进行混合排序：

综合相关度 =
任务相关性
× 来源可信度
× 版本新鲜度
× 范围匹配度
× 权限允许度
× 风险权重

具体公式可以因平台而异，但语义相似度不能成为唯一排序依据。对于高风险任务，一条低相似但明确适用的 MUST 约束，应排在一段高相似的旧设计讨论之前。

19.1.8 从检索结果到任务 Context

检索只是发现候选内容，Context Engineering 还要完成选择、冲突处理、权限过滤、压缩和注入。可以采用：

任务与契约解析
→ 多路召回
→ 版本、状态和 ACL 过滤
→ 冲突检测
→ 依据 Work Graph 节点重排
→ Token 预算内裁剪
→ 生成带引用的 Context Slice
→ 注入前隔离检查

同一个任务中，不同 Agent 获得的知识也不同。测试 Agent 需要业务规则、反例和历史缺陷；实现 Agent 需要可信 Red、相关代码、接口和变更围栏；安全 Agent 需要数据流、权限和威胁知识；V 需要原始证据、版本和裁决规则。

Context Slice 应保留知识 ID、版本、状态和来源，使 Agent 可以说明“我根据什么行动”。发生 CONFLICTED 或关键知识缺失时，系统应停止或请求补充，而不是自动选择篇幅最长的答案。

19.1.9 自动知识与治理知识的双通道

知识更新可以分成快通道和慢通道。

快通道：机器可重建知识

- Code Map 与符号索引；
- 静态依赖和 API 清单；
- Schema、测试映射和代码所有者；
- Git 热点和变更历史；
- 部署拓扑与运行时 Trace 摘要。

这些内容随代码和部署自动刷新，绑定版本即可，一般不需要逐条人工批准。

慢通道：需要责任确认的知识

- 业务目标、规则和领域术语；
- Preserve、例外和兼容责任；
- 架构决策与风险接受；
- 安全、合规和生产边界；
- 失败教训与组织策略。

AI 可以起草，OA 负责证据和范围审查，IO 或相应责任人批准。快通道不能自动覆盖慢通道：代码扫描发现某个阈值是 5000，不代表业务政策就是 5000；它只说明当前某个版本的实现包含这个数字。

19.1.10 权限隔离：可检索不等于可见

知识库可能包含生产日志、客户数据、安全漏洞、密钥痕迹、商业策略和跨租户信息。把文档切片以后写入统一向量库，并不会消除原始数据权限。

知识治理至少要落实：

- 源数据接入前分类与脱敏；
- 租户、项目、领域、角色和环境级 ACL；
- 向量、全文索引、缓存和摘要继承源权限；
- Agent 检索使用最小权限身份；
- Context Slice 再次做敏感信息隔离检查；
- 检索、引用、导出和拒绝都保留审计；
- 删除或撤权能够传播到派生索引与缓存。

工具权限与知识权限也要分开。Agent 可以知道生产表结构，不代表可以查询生产数据；可以读取脱敏事故报告，不代表可以获得其中的凭证或客户标识。

19.1.11 知识与四大动态工件的关系

知识系统不能取代四大动态工件：

工件	与知识系统的关系
Intent Graph	组织稳定的业务方向、能力和关系，不是全文仓库
Intent Contract	对当前任务需要的目标、规则和知识形成签署快照
Constraint Matrix	把重要知识升级为机器可执行边界
Evidence Bundle	证明本次知识和结论为什么值得相信

知识库可以保存候选和已验证知识；Intent Graph 提供宏观导航；Intent Contract 将当前适用知识冻结成执行基准；Constraint Matrix 规定 Agent 不能如何使用知识；Evidence Bundle 则为知识晋升、冲突解决和结果裁决提供证据。

不能因为知识库里存在一条规则就自动把它当成契约；不能因为检索到了某份旧 Evidence Bundle 就认为当前版本仍然通过；也不能把所有经验都升级成 MUST，制造约束疲劳。

19.1.12 知识如何进入 SCOPE-V

知识治理贯穿控制回路，而不是任务结束后的文档整理。

Specify 检索历史意图、术语、规则、缺陷和用户反馈，同时检查有效期和冲突。

Constrain 加载适用的权限、安全、合规和兼容知识；敏感内容在这里完成访问控制。

Orchestrate 按 Work Graph 为节点组装不同 Context，并安排需要补证的知识任务。

Prove 验证知识主张。例如“旧移动端不依赖该字段”必须通过 Contract Test、Trace 或消费方证据证明。

Evolve 将失败、反例和新发现形成 CANDIDATE，而不是直接写入正式记忆。

Verify 审查来源、范围、新鲜度、独立性和冲突，决定晋升、保持候选、标记过期或撤回。

Telemetry 观察哪些知识被召回、是否减少返工、是否造成误导、消耗多少 Token，以及有没有真正改善首次成功率。

因此知识回写属于 LOOP-3，不是“Agent 自动总结一下”。回写内容必须改变未来任务行为，并且能够被治理和撤销。

19.1.13 治理岛知识包

百万行项目不应先导入全部代码、Wiki、工单、聊天和日志再建设企业大知识库。知识建设应沿治理岛扩张：

选择真实变更

→ 建立局部 Code Map

→ 找到相关业务知识和历史证据

→ 标记冲突、权限与 UNKNOWN

→ 组装任务 Context

→ 完成验证与裁决

→ 将确认知识沉淀为治理岛知识包

第一座费用治理岛的知识包可能是：

```
knowledge/expense/  
├─ capability-map.yaml  
├─ document-map/  
│  └─ requirements/  
│  └─ business-rules.md  
│  └─ glossary.md
```

```
| └─ decisions/
└─ code-map/
    | └─ symbols.index
    └─ interfaces.yaml
└─ trace-links.yaml
└─ known-unknowns.yaml
└─ owners.yaml
└─ access-policy.yaml
└─ characterization-tests/
└─ evidence-index.yaml
```

这些文件不一定采用相同格式。关键是自动地图、人工语义、未知项、所有权、访问策略、测试和证据索引彼此分开又能通过 ID 连接。T-101、T-117 等任务完成后，知识包逐步扩张，而不是一次生成一部“费用系统百科全书”。

19.1.14 Knowledge Release: 知识变化也要携证发布

批量导入、规则晋升、权限变化、索引算法升级和知识撤回都会改变 Agent 行为，因此重要知识变化也需要版本与证据，可以形成 Knowledge Release。

一份知识发布应说明：

- 新增、修改、合并、废弃或撤回哪些知识；
- 来源、状态、范围、责任人和批准记录；
- 影响哪些 Agent、模块、检索策略和 Context；
- 使用哪些基准任务验证召回准确性；
- 是否发生敏感信息越界；
- 新旧版本对 Token、准确率和首次成功率的影响；
- 失败时如何回退知识版本和清理缓存。

高风险知识不能直接在生产检索链路上原地覆盖。可以采用 Champion-Challenger：旧版继续服务，候选知识或新排序策略先在回放任务、影子流量或有限团队中验证，证明效果后再晋升。

Knowledge Release 与软件 Release 类似：发布的不是代码制品，而是一套会影响 Agent 认知和行动的知识基线。

19.1.15 知识质量遥测

知识治理不能只统计文档数、切片数、向量数和访问量。更有价值的指标包括：

- 知识来源标注率；
- 有明确责任人的知识比例；
- VERIFIED/APPROVED 比例；
- 过期、撤回知识的误召回率；
- CONFLICTED 知识发现与解决时间；
- 敏感信息误召回和越权次数；
- 关键任务的有效召回率；
- 因知识缺失导致的返工；
- 因错误知识导致的失败；
- UNKNOWN 的新增、关闭和逾期趋势；
- 知识复用后首次成功率的变化；
- 每个成功任务的检索与上下文 Token 成本；

- 知识修正和撤回的平均时间。

知识指标必须与交付结果联动。如果文档和向量增长一倍，目标准确率、首次成功率和执行效率没有改善，知识系统可能只是更昂贵的文档仓库。如果 Token 降低但错误知识导致返工增加，也不能称为 Context 优化。

19.1.16 六类典型失败

文档坟场：大量材料被导入，却没有责任人、版本、有效期和使用反馈。

知识污染：Agent 的推测和通用反思被直接写入长期记忆，后续任务不断引用。

过期知识高置信召回：旧接口、旧制度和旧架构因语义相似度高而排在当前规则之前。

权限泄露：生产日志、客户数据或跨租户资料进入共享索引，派生摘要没有继承原权限。

上下文污染：系统召回大量相关但冲突的内容，却不展示来源、版本和状态，让 Agent 自行猜测。

反馈回路自我强化：模型生成错误总结，知识库收录；下一次模型引用它并生成更多相似内容，使错误看起来越来越像共识。

这些失败说明，知识工程的主要风险不是“搜不到”，还包括“搜到错误内容并让 Agent 相信”。高风险系统宁可明确返回 UNKNOWN，也不能用不可靠知识制造虚假确定性。

19.1.17 一个大型系统知识召回案例

继续使用八十万行费用平台的 T-101。用户要求把人民币审批阈值从 5000 调整为 8000。Document Map 先找到当前需求、阈值规则、验收标准和地区例外；Code Map 再用词法检索定位三个 5000，并显示它们分别属于 v1 接口、v2 规则和地区定制。符号图找到调用者和相关测试，运行时 Trace 证明当前新建人民币报销走 v2，同时 Document Map 还召回一份两年前的“统一金额规则”设计稿和最近一次地区例外裁决。

如果只用向量相似度，旧设计稿可能因为描述完整而排名最高。知识治理层发现它已经 DEPRECATED，适用版本止于 v1；地区例外是 APPROVED，但范围只覆盖特定租户；v2 Trace 和当前接口契约是 VERIFIED。系统因此为领域 Agent 组装：当前契约、v2 规则符号、Trace、现状测试和地区禁止修改约束，同时把外币批处理关系标记为 UNKNOWN。

实现和验证完成后，Evidence Bundle 证明新阈值、历史兼容和地区隔离。Evolve 提议“新建人民币报销走 v2”进入组织记忆；V 检查 Trace、版本和适用范围后将其晋升为 VERIFIED。旧设计稿仍可用于历史解释，但不会进入默认任务 Context。

这个案例体现了完整链路：Document Map 负责找到需求上下文，Code Map 负责找到代码结构，Trace Link 用统一 ID 连接需求、实现、测试与证据；混合检索汇聚多种来源，知识状态和范围决定能否使用，Context Engineering 完成裁剪，Evidence Bundle 支持晋升，Telemetry 检查知识是否真正帮助任务成功。

19.1.18 本专题的工程验收

团队可以选择一次真实变更，检查：

1. 是否区分事实源、结构索引、语义知识、任务 Context 和组织记忆？
2. Document Map 与 Code Map 是否职责分离，并能通过统一 ID 下钻到需求、完整文件、测试和证据？
3. 每条关键知识是否有来源、范围、状态、责任人、版本和证据？
4. CANDIDATE、CONFLICTED、DEPRECATED、EXPIRED 和 REVOKED 是否真正影响检索？
5. 是否同时使用词法、符号图、语义、运行时和治理关系检索？
6. Context Slice 是否经过权限、冲突、新鲜度和 Token 预算检查？
7. 自动结构知识与需要责任人批准的业务知识是否走不同更新通道？
8. 知识索引、缓存和摘要是否继承源数据 ACL 与删除要求？
9. 新知识是否通过 Evidence Bundle 或生产证据晋升，而非由模型自报？

- 10. Knowledge Release 是否可以验证、灰度和回退？
- 11. Dashboard 是否能看见错误召回、过期知识、权限事件、Token 与交付效果？
- 12. 下一次相关任务是否能够复用知识，又不会把过期或越权内容带入 Context？

如果这些问题能够由真实工件、审计记录和任务结果回答，企业建设的才是面向 Agent 的可信知识系统，而不只是一套向量检索服务。

19.2 Agent Memory Engineering：从任务状态到可治理组织记忆

Agent 可以在一次会话里表现得很聪明，却在下一次任务中重新犯下昨天的错误。它也可能记住了某次临时讨论，却把已经撤销的结论当成长期规则。前者是“没有记忆”，后者是“记忆失控”。

因此，记忆管理不是给模型接一块无限大的硬盘，更不是把所有聊天记录塞回上下文。**Agent Memory Engineering 是一套对经验进行捕获、筛选、验证、召回、衰减、晋升和遗忘的工程系统。** 它要解决的不是“模型能不能记住”，而是以下治理问题：

- 什么内容值得记住，什么内容必须忘记？
- 一条记忆适用于本次任务、当前模块，还是整个组织？
- 谁提供了这条记忆，它是否仍然有效？
- 记忆与新事实冲突时，Agent 应该相信哪一个？
- 哪些经验可以晋升为约束、测试、知识或 Skill？
- 一条被撤销的敏感信息，能否从索引、摘要和缓存中同步删除？

19.2.1 先区分 State、Context、Memory 与 Knowledge

这四个概念经常被混用，但它们在治理上承担不同责任。

对象	回答的问题	典型内容	生命周期
State（状态）	任务现在进行到哪里	当前节点、失败次数、待办项、锁定文件	秒到天
Context（上下文）	Agent 此刻看见什么	当前契约、相关代码、约束、检索片段	一次推理或一次任务
Memory（记忆）	过去哪些状态与经验值得再次使用	上轮失败原因、有效修复策略、未决风险、交接信息	天到月，按价值衰减
Knowledge（知识）	哪些结论已经被验证，可以稳定复用	业务规则、架构事实、接口语义、组织标准	月到年，版本化治理

例如，“当前正在执行 SIT-03”是状态；SIT-03 的契约、接口定义和相关代码是上下文；“上次在预生产环境因时区配置导致该用例失败”是情景记忆；“所有结算时间统一采用 UTC 存储、业务时区展示”经过验证和批准后，才成为组织知识或工程约束。

记忆记录经验，知识表达经过治理的结论。记忆可以是知识的候选来源，但不能因为被模型记住，就自动拥有规则效力。

19.2.2 为什么“保存全部聊天”不是记忆管理

聊天记录最多是一种原始材料或审计记录。直接将全部聊天作为长期记忆，会同时放大五类风险：

1. **噪声污染**：探索过程、重复提问和无效尝试挤占 Context Window。

- 2. **事实冲突**：早期假设与后续结论并存，Agent 不知道哪一个有效。
- 3. **敏感泄露**：密钥、客户数据和个人信息可能被长期保留并越权召回。
- 4. **版本漂移**：代码和规则已经变化，旧讨论仍被当成当前事实。
- 5. **成本失控**：每次任务重复注入大量历史文本，Token 成本上升，准确率反而下降。

尤其不能把模型的隐式推理过程当作组织资产保存。组织真正需要保留的是可审计的输入、关键决策、验证结果、失败模式、责任人和证据，而不是不可验证的思维草稿。

19.2.3 五类 Agent 记忆

为了避免“一种存储解决所有问题”，可以将 Agent 记忆分成五类：

记忆类型	作用	例子	主要治理要求
工作记忆 Working Memory	支撑当前一步推理与执行	当前文件、最近一次命令结果	小容量、短寿命、及时清理
任务记忆 Task Memory	维持一个契约内的连续性	当前节点、已完成 AC、未决问题	与任务 ID 绑定、可交接、任务结束后归档
情景记忆 Episodic Memory	保存发生过的事件及其结果	某次发布失败、某种修复成功	必须带时间、环境、版本和证据
语义记忆 Semantic Memory	保存被验证的事实和规则	业务术语、接口含义、架构事实	来源、版本、作用域、审批和冲突管理
程序记忆 Procedural Memory	保存“如何做”的可复用方法	排障步骤、测试策略、回滚剧本	可执行、可验证、可版本化，适合晋升为 Skill

五类记忆不是五套孤立数据库，而是五种不同治理语义。一次失败首先进入任务记忆；经过复盘形成情景记忆；多次复现并被证据支持后，可能沉淀为语义规则或程序化 Skill。

19.2.4 六级记忆作用域

记忆必须回答“对谁有效”。同一句话在不同作用域下，治理风险完全不同。

- 1. **节点级**：仅服务于当前 SCOPE-V 节点，例如本轮 Prove 的失败现场。
- 2. **任务级**：服务于一个 Intent Contract，例如 T-101 的未决问题和验证进度。
- 3. **模块级**：适用于支付、订单或结算等特定模块。
- 4. **领域级**：适用于某一业务领域，例如财务合规或客户身份治理。
- 5. **项目级**：适用于整个代码库和交付体系，例如分支、发布和环境规则。
- 6. **组织级**：可跨项目使用的标准、模式、Skill 和治理政策。

作用域越大，写入门槛越高。任务中的一次偶然经验可以自动进入候选记忆，但不能由 Agent 自行升级为组织规则。跨级晋升必须经过证据、责任人和治理流程。

19.2.5 哪些内容值得记，哪些内容必须忘

值得保存的通常是能够降低未来风险或重复劳动的信息：

- 已作出的关键决策、理由、责任人和适用版本；
- 已验证的成功策略与失败模式；
- 可复现的异常、环境差异和恢复动作；

- 未关闭风险、未决问题和 Handoff 状态；
- 对验收标准、约束或接口语义的澄清；
- 多次任务中稳定出现、可能晋升为知识或 Skill 的模式。

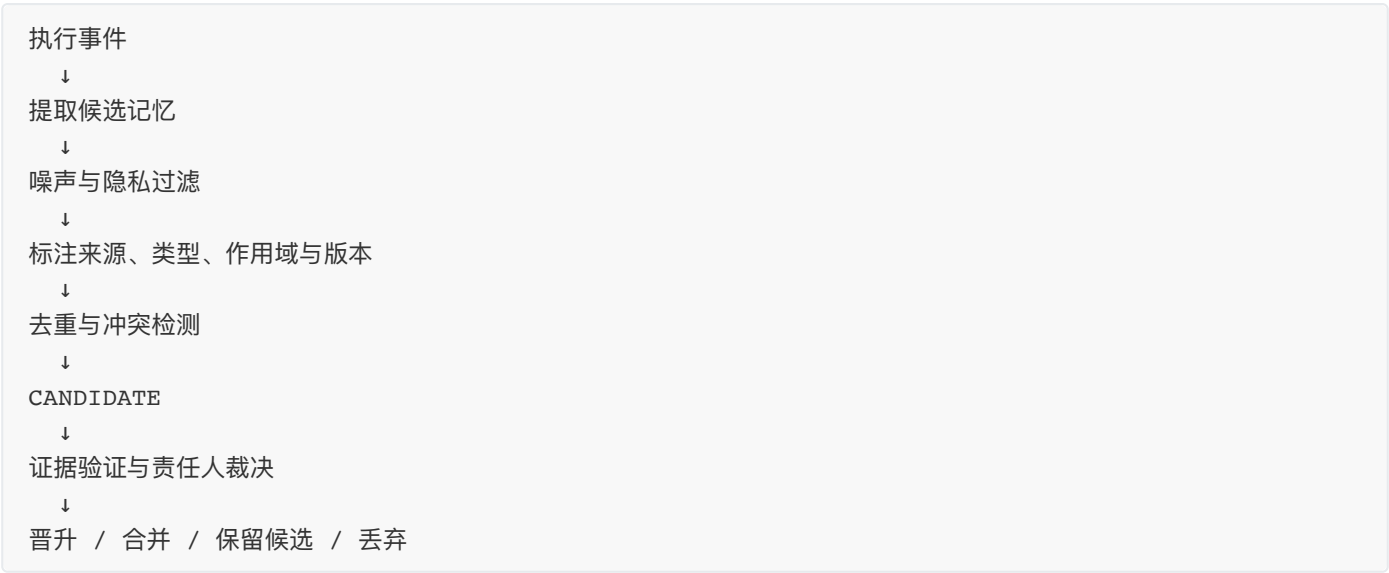
不应进入长期记忆的包括：

- 完整聊天记录和模型的隐式推理过程；
- 未经验证的猜测、临时脑暴和一次性措辞；
- 已撤销或被新版本取代的规则；
- 密钥、访问令牌、无必要的个人信息和客户敏感数据；
- 对未来任务没有解释力的实现碎片；
- 无法追溯来源、适用范围和时间的信息。

一个简单判断是：**如果未来的 Agent 无法知道这条信息为何可信、何时适用、由谁负责，它就不应被当作治理记忆召回。**

19.2.6 Memory Write Gate：记忆不能由 Agent 随意写入

记忆写入应经过一道类似代码合并的门禁：



自动化可以完成提取、分类、去重和风险提示，但不能代替规则所有者完成价值与责任判断。特别是会影响业务行为、安全边界和上线条件的记忆，必须由 IO、OA 或领域责任人批准。

19.2.7 结构化记忆对象

一条可治理记忆不能只有一句自然语言。它至少需要身份、来源、范围、状态、证据和生命周期信息。

```
memory_id: M-T101-004
type: episodic
scope:
  level: module
  project: settlement-platform
  module: refund
statement: "预生产环境未配置 UTC 时，跨日退款校验会出现假失败"
source:
  task_id: T-101
  event_id: EVT-20260807-17
evidence_refs:
```

```
- evidence/T-101/sit-03-result.json
status: VERIFIED
confidence: 0.96
owner: settlement-tech-lead
created_at: 2026-08-07T16:20:00+08:00
last_used_at: 2026-08-07T16:20:00+08:00
validity:
  environment: pre-production
  code_version: v4.8.x
  valid_until: 2026-11-07
access:
  classification: internal
  allowed_roles: [developer, tester, tech-lead]
promotion:
  candidate_for: constraint
  status: pending
supersedes: null
```

这不是为了把 YAML 写得复杂，而是为了让检索、冲突处理、过期判断、权限控制和审计能够自动化。

19.2.8 记忆状态机

记忆状态必须真正改变系统行为，而不是只作为标签展示。

状态	含义	默认行为
CANDIDATE	新提取、尚未验证	可供复盘，不得驱动关键决策
VERIFIED	已有证据支持	可在对应作用域内召回
CONFLICTED	与其他记忆或当前事实冲突	退出默认 Context，等待裁决
EXPIRED	超出有效期或版本范围	不再自动召回，可保留审计
REVOKED	被责任人明确撤销	禁止使用，并传播撤销状态
ARCHIVED	已归档，仅用于历史分析	不进入执行上下文
PROMOTED	已沉淀为知识、约束、测试或 Skill	召回正式工件，而非旧记忆文本

当一条记忆被晋升后，系统应优先引用正式工件。例如“退款阈值必须校验审批”晋升为 Constraint 与自动化测试后，Agent 应加载约束和测试，而不是继续依赖最初那段复盘文字。

19.2.9 完整记忆生命周期

Agent Memory Engineering 的闭环可以概括为九个动作：

1. **Capture**：从执行、验证、遥测和人工裁决中捕获事件。
2. **Filter**：去除噪声、重复信息、敏感数据和无长期价值内容。
3. **Verify**：用测试、日志、代码、契约和人工裁决确认真实性。
4. **Consolidate**：合并同类经验，识别稳定模式与反例。
5. **Retrieve**：根据任务、范围、版本、权限和预算检索。
6. **Apply**：将少量高相关记忆注入 Context，并记录使用情况。
7. **Evaluate**：判断记忆是否帮助了任务，是否造成误导。

8. **Decay**: 降低过期、低复用或被新事实削弱的记忆权重。
9. **Archive or Forget**: 归档审计记录，或按政策彻底删除。

执行事件 → 工作/任务状态 → 情景记忆候选 → 证据验证
↓
知识 / 约束 / 测试 / Skill ← 语义与程序记忆 ← 复用与晋升

这个闭环的价值不在于记得更多，而在于每一次召回都能够提高下一次执行的确定性。

19.2.10 衰减不能只依靠“保存 90 天”

当前 Agentic Agile 3-4-3 的 Loop Memory 已提供跨周期进度、经验、风险、模式、开放问题和决策记录，并以 `max_age_days: 90` 作为轻量衰减配置。这对单项目起步很实用，但企业级系统不能只按时间决定记忆是否有效。

一条记忆的有效性至少受以下信号影响：

- 来源证据是否可靠；
- 当前任务与原作用域是否匹配；
- 代码、接口、模型、环境和业务规则是否已经换版；
- 最近是否成功复用；
- 是否出现新的反例或冲突证据；
- 责任人、政策或安全等级是否变化。

可以用一个概念模型理解：

记忆有效度
= 来源可信度 × 作用域匹配度 × 新鲜度 × 版本一致性 × 成功复用度
- 冲突证据 - 反例惩罚

MUST 级约束不能因为 90 天没有使用就自动失效；业务规则应通过版本和责任人撤销；敏感信息则必须服从更严格的保留与删除政策。时间只是衰减信号之一，不是最终裁决者。

19.2.11 召回、注入与 Context 污染

“存得好”不等于“用得好”。记忆检索至少应结合：

- Intent Contract 与 AC；
- 项目、模块、文件和符号；
- 失败类型、环境和版本；
- 记忆类型、状态、作用域和新鲜度；
- 当前操作者权限；
- Context Window 与 Token 预算。

系统不应把所有相关记忆一次性塞给 Agent，而应形成一个最小 Context Slice：先召回少量 VERIFIED 记忆，在需要时再沿来源和证据展开。召回之后还要记录“是否被采用、是否提高一次通过率、是否造成误导”，否则系统无法判断一条记忆究竟是资产还是噪声。

19.2.12 记忆冲突如何处理

假设旧记忆写着“退款金额超过 5000 元必须人工审核”，而新契约经业务负责人批准后将阈值调整为 8000 元。系统不能让两条记忆随机竞争注意力。

建议采用以下冲突流程：

- 1. 检索或写入时识别范围重叠和数值冲突；
- 2. 将相关记忆标记为 CONFLICTED，移出默认执行上下文；
- 3. OA 比较契约、代码、测试、上线记录和来源时间；
- 4. IO 或领域责任人对业务含义作出裁决；
- 5. 新版本记忆通过 `supersedes` 指向旧版本；
- 6. 旧记忆进入 REVOKED 或 ARCHIVED，并将撤销传播到索引和缓存。

“最新”并不天然等于“正确”，“出现次数多”也不等于“权威”。冲突裁决依据应是来源等级、证据质量、责任归属和适用范围。

19.2.13 三层 LOOP 如何管理不同记忆

三层 LOOP 与五类记忆天然对应：

LOOP	主要记忆	典型动作	人类责任
LOOP-1 自洽环	工作记忆	在一次执行中检查输入、输出和局部一致性	预设规则与工具边界
LOOP-2 自愈环	任务记忆、情景记忆	根据失败诊断、修复、重试并保留现场	规定重试上限和退出条件
LOOP-3 自进化环	语义记忆、程序记忆	跨任务识别模式，提出约束、测试、知识或 Skill 候选	审批晋升，承担组织级后果

LOOP-2 可以自主记录“本次如何恢复”，但 LOOP-3 不应自主宣布“以后所有项目都这样做”。从经验到制度的跃迁，就是记忆治理最重要的人类控制点。

19.2.14 记忆如何贯穿 SCOPE-V

SCOPE-V 不是六步瀑布流程，记忆也不是在最后统一归档。它参与每一个持续运行的控制面：

- **S—Specification**：召回历史歧义、类似契约和已确认术语，帮助把意图写得可判定。
- **C—Constraints**：召回风险与反例；只有经过批准的内容才能转为 MUST/SHOULD/MAY。
- **O—Orchestration**：加载任务状态、文件锁、Handoff 和上轮开放问题，保持跨 Agent 连续性。
- **P—Proof**：利用既往失败模式设计反例、边界和测试，但必须匹配当前版本。
- **E—Evolution**：从失败与修复中提取候选记忆，在快内环中诊断和自愈。
- **V—Verification**：独立核验证据，裁决哪些记忆可信、哪些经验可以晋升。
- **Telemetry**：观察记忆的命中、误导、成本和复用效果，反向调整下一轮 S/C/O。

这使 SCOPE-V 不只是完成一次交付，还能在不牺牲治理边界的前提下形成组织学习。

19.2.15 Evidence Bundle 支撑记忆晋升

记忆晋升必须由证据支撑。例如三个退款任务都出现阈值误判，团队想把“阈值规则必须进入自动化验证”晋升为项目级程序记忆。Evidence Bundle 至少应包含：

- 三次事件的任务 ID、环境、版本和日志；
- 对应 Intent Contract 与业务规则来源；
- 失败可复现结果和修复后的自动化测试；

- 是否存在反例、适用范围和非适用范围；
- IO、OA 或领域责任人的审批记录；
- 晋升目标：Constraint、测试、知识条目、Playbook 或 Skill；
- 上线后遥测与回退方式。

Evidence Bundle 证明的不是“Agent 说它学会了”，而是“这条经验已经具备成为组织能力的条件”。

19.2.16 隐私、权限与删除传播

记忆存储往往比代码库更容易聚合敏感信息，因此必须独立治理：

- 每条记忆继承源数据的分类、ACL 和保留期限；
- 检索先做权限过滤，再做相关性排序；
- 敏感字段在写入前脱敏，密钥和令牌禁止进入长期记忆；
- 用户或源数据被删除时，派生摘要、向量索引、缓存和备份按政策同步处理；
- 被 REVOKED 的记忆不再出现在新的 Context Slice 中；
- 所有读取、修改、晋升和删除动作留下审计记录。

模型内部不可见、不可审计的“记住了”不能作为企业治理基础。真正可治理的组织记忆应位于外部、显式、版本化、可授权和可删除的存储中。

19.2.17 记忆遥测与 Loop Memory 的增强方向

Memory Dashboard 不应只显示“保存了多少条”，还要回答记忆是否真正改善交付：

指标	治理含义
候选记忆数与验证率	是否产生大量未经治理的经验碎片
召回命中率与有效复用率	记忆是否与真实任务相关并被采用
错误召回与过期召回率	是否把错误或旧事实注入 Context
冲突发现与裁决时长	组织能否及时处理规则分歧
晋升率	经验是否转化为约束、测试、知识和 Skill
长期未使用率	存储中是否积累大量无价值记忆
记忆 Token 成本	召回成本是否换来了质量收益
一次通过率变化	记忆是否减少返工和重试
撤销与删除完成时间	敏感或错误信息能否及时退出系统

Agentic Agile 3-4-3 的 Loop Memory 提供了一个轻量起点：它记录跨周期进度、经验、风险、模式、开放问题与决策，并支持时间衰减。面向企业级应用，可在此基础上逐步增强：

1. 为每条记忆增加唯一 ID、类型、作用域、来源和适用版本；
2. 引入 CANDIDATE、VERIFIED、CONFLICTED、REVOKED 等状态；
3. 绑定 Evidence Bundle、责任人和访问控制；
4. 用多信号衰减替代单一时间过期；
5. 增加去重、冲突检测、晋升路由与删除传播；

6. 将召回效果、Token 成本和交付结果纳入全局 Dashboard。

这不是推翻现有 Loop Memory，而是把它从“任务跨周期状态文件”演进为“可治理的组织记忆控制面”。

19.2.18 大型既有系统中的记忆闭环案例

以拥有百万行代码的结算平台为例。团队执行 T-101：将退款人工审核阈值从 5000 元调整为 8000 元。

1. **任务开始**：O 从现状基线、Code Map 和 Task Memory 中加载相关模块、上次变更记录与开放风险。
2. **执行失败**：接口测试通过，但 UAT 发现一个历史批处理仍按 5000 元拦截。LOOP-2 保存失败现场、版本、调用链和修复结果。
3. **形成候选**：E 提取情景记忆——“退款阈值同时存在于在线服务与历史批处理配置中”。状态为 CANDIDATE。
4. **独立验证**：V 通过代码检索、配置比对、SIT/UAT 结果和 Evidence Bundle 确认该事实，将其标记为 VERIFIED。
5. **跨任务复现**：后续两次金额规则变更出现相同风险，LOOP-3 提出晋升建议。
6. **组织裁决**：领域责任人批准建立统一阈值配置、架构约束和契约测试；程序记忆进一步沉淀为变更 Playbook。
7. **遥测反馈**：Dashboard 跟踪相似变更的一次通过率、遗漏率、召回 Token 和人工审核时间。

这个过程中，原始失败没有被粗暴保存为“永远正确的答案”，而是经过任务记忆、情景记忆、证据验证、跨任务复现和正式工件晋升，最终成为可治理的组织能力。

19.2.19 Agent Memory Engineering 验收清单

建设或评审一套 Agent 记忆系统时，可以追问：

1. State、Context、Memory 与 Knowledge 是否被明确区分？
2. 是否定义工作、任务、情景、语义和程序五类记忆？
3. 每条记忆是否具有来源、类型、作用域、版本、状态和责任人？
4. 写入前是否经过隐私过滤、去重、冲突检测和 Memory Write Gate？
5. CANDIDATE 是否被禁止直接驱动高风险行为？
6. 召回是否同时考虑契约、版本、权限、新鲜度和 Token 预算？
7. CONFLICTED、EXPIRED 与 REVOKED 是否真正退出默认 Context？
8. 衰减是否结合时间、版本、复用、反例和责任人裁决？
9. 经验晋升为约束、测试、知识或 Skill 时，是否有 Evidence Bundle？
10. 是否禁止保存模型隐式推理、密钥和无必要的个人数据？
11. 源数据删除后，摘要、索引、缓存和派生记忆能否同步处理？
12. Dashboard 是否能够证明记忆降低了返工、风险或 Token 成本？

如果这些问题只能由“我们用了向量数据库”来回答，组织拥有的仍然只是存储能力；只有当记忆的产生、使用、晋升和遗忘都可被控制、证明和追责时，才真正具备 Agent Memory Engineering。

19.3 本章小结

Knowledge Engineering 解决 Agent 如何找到、判断和复用可信组织知识；Memory Engineering 解决任务状态、失败经验和跨周期决策如何被捕获、验证、召回、衰减、晋升与遗忘。Document Map、Code Map、混合检索和 Context Slice 帮助 Agent 找到信息，Trace Link、来源、状态、权限、Evidence Bundle 与 Memory Write Gate 决定信息能否被相信并影响行动。知识与记忆只有进入可审计生命周期，才能从个人经验升级为组织能力。

本章最小实践：为一个真实模块建立 Document Map、Code Map 与最小 Trace Link，并将一次已验证失败沉淀为结构化候选记忆。

完成证据：可追溯知识条目、Context Slice、记忆状态与一次证据支持的晋升或撤销记录。

延伸实践：推荐学习《Agentic Agile 组织转型进阶：构建人机协同研发操作系统》，把上下文、知识和失败经验沉淀为可复用的组织资产。

第二十章 | 多人、多工具、多模块：从单任务闭环到规模化协作

单个 Agent 完成一个边界清晰的任务时，意图、上下文、写入范围和验收责任相对容易对齐。一旦多人、多工具或多模块同时进入，复杂度不再只来自任务本身，而来自它们之间的关系：谁听谁的，谁可以改什么，哪个版本才是事实，局部完成能否组成系统完成，失败后又该由谁停止和恢复。

20.0 组织级扩展判断

规模化不是把任务运行层的 Agent 数量扩大，而是把可验证的责任单元、协议和证据聚合到价值网络层，再由战略意图层决定是否扩展。企业级扩展至少要经过三道判断：任务闭环是否连续成立，价值流是否在相同口径下获得净收益，组织是否具备跨单元的责任、遥测和恢复能力。任何一层不成立，都应停止扩展并回到较小责任域。

这就是白皮书所说的组织级 AI First 转型：分形扩展，是从单任务自治走向组织级可信自治的扩展路径。它要求先证明可重复，再证明可迁移，最后才讨论跨价值流复制；规模越大，越不能用“更多 Agent”替代协议、证据和责任。

因此，本章不是教组织如何堆叠更多 Agent，而是讨论如何减少不必要的协调，并为无法消除的依赖建立共同控制面。

本章只处理规模化协作的工程接口：所有权、Protocol、跨模块契约、集成 DAG、Handoff、证据聚合和跨工具语义。岗位、组织单元、激励与转型治理留到下一部分，避免把工程协作协议误当成完整组织设计。

最好的规模化是去规模化。

去规模化不是拒绝大型系统，也不是把团队退回个人作业，而是尽可能把大协作拆成小责任域：减少共享状态、并发写入者、共同决策者和全局上下文，让每个单元拥有明确意图、边界、责任和独立证据。只有那些不能通过拆分消除的依赖，才进入 Protocol、XC、集成 DAG 和证据聚合。

规模化 ≠ 更多人 × 更多 Agent × 更多协调流程

规模化 = 更小责任单元

- × 更少耦合
- × 可独立验证
- × 必要时再集成

规模化风险也不是人数或模块数的简单相加，而会被协作关系放大。可以用下面的概念模型帮助团队识别危险信号：

规模化风险

= 并行任务数

- × 交接次数
- × 接口数量
- × 工具差异
- × 状态变化速度
- × 验证不一致程度

这不是用于计算精确分数的数学公式，而是一张诊断图。任一因子持续上升，都会放大其他因子的影响：更多并行任务会产生更多交接；更多接口会加快状态变化；工具差异越大，验证结果越难比较。去规模化的意义，就是优先

降低这些乘数，而不是在风险已经放大后继续增加协调层。

20.1 先去规模化，再升级治理复杂度

面对复杂项目，OA 的第一反应不应是增加全局会议、共享群聊和统一大模型，而应先追问：任务能否缩小，责任域能否拆开，写入边界能否隔离，接口能否提前稳定，验证能否在模块内完成？

去规模化至少包含四项原则：

- 1. **减少共享状态**：能通过独立模块、工作树、稳定接口和版本化工件隔离的，不依赖所有人实时同步。
- 2. **减少并发写入者**：同一责任域在同一时间窗口内默认只有一个活跃写入者，其他人或 Agent 可以并行审查、设计测试和核验证据。
- 3. **减少共同决策者**：一个任务可以有多个参与者，但必须有一个明确 IO；参与讨论不等于拥有最终执行权。
- 4. **减少全局上下文**：模块 Agent 只获得完成任务所需的 Context Slice，不把全部聊天、代码和历史无差别注入每个 Agent。

只有去耦后仍满足下列任一条件，才应考虑升级治理复杂度：

- 项目包含两个以上独立模块；
- 两人以上并行工作；
- 同时使用两种以上 AI 工具；
- 存在跨服务接口和发布依赖；
- 模块使用不同语言或测试体系；
- 单模块证据需要汇聚为统一发布结论。

这些条件只是触发进一步判断，不表示必须启用全套治理。两个互不依赖的模块可以各自形成小闭环；两个修改同一核心模块的人，即使人数很少，也可能需要强冲突控制。治理重量应由共享状态、依赖强度、失败传播和责任冲突决定，而不是由团队人数机械决定。

单人单模块项目不需要 Protocol 和两级 OA。能够通过串行、拆分或明确所有权消除的协调，也不应为了“完整”引入额外层级。

20.2 三条复杂度轴

多人、多工具和多模块不是同一个问题：

复杂度轴	主要风险	首选治理动作
多人	目标解释冲突、责任不清、并发写入、验收权争夺	明确 IO、所有权、Change Envelope 和交接规则
多工具	权限不同、上下文不一致、身份与证据来源丢失	统一 Task ID、契约版本、Tools Manifest 和证据语义
多模块	接口漂移、依赖顺序、数据一致性、系统级回滚	Protocol、XC、集成 DAG 和 Release Evidence Bundle

三条轴可以单独出现，也会相互叠加。一个人用多个 Agent 修改同一模块，首先是多工具和并发写入问题；多人分别维护互不依赖的模块，治理反而可以很轻；多人通过多个工具修改多个强耦合模块，则需要完整的跨模块控制面。

OA 应先识别复杂度来自哪条轴，再选择最小充分控制。不要因为项目用了多个 Agent，就自动复制所有多模块工件。

20.2.1 九类必须解决的协同问题

多人、多工具、多模块治理需要回答九类问题。后续 Protocol、XC、DAG、Handoff 和证据聚合，都是对这些问题的具体回应。

问题	失控表现	最小控制	本章落点
1. 意图分裂	各模块局部正确，端到端价值失败；后来的口头要求覆盖已签署目标	全局意图、模块契约、唯一 IO、变更版本	18.4、18.14
2. 并发写入冲突	多人或多批 AS 修改同一责任域，Git 可合并但业务语义互相覆盖	所有权、Change Envelope、单写入者、合并验证	18.3、18.18
3. 上下文不一致	不同人和工具使用不同契约、接口、记忆或代码版本	仓库事实源、Context Slice、版本标识、UNKNOWN	18.13—18.15
4. 模块边界漂移	职责外溢、共享模型被多方修改、接口语法一致但含义不同	模块所有权、Protocol、XC、跨模块不变量	18.5、18.6
5. 依赖与顺序失控	下游基于推测开工、迁移先于兼容版本、失败后无法判断阻塞范围	集成 DAG、检查点、超时、失败路由和系统级回滚	18.7、18.17
6. 多工具治理失配	权限、能力和输出不同；工具升级后行为变化；证据来源丢失	Tools Manifest、统一 Task ID、Handoff 和证据协议	18.8、18.13、18.15
7. 局部证据无法组成整体	模块全部 GREEN，系统仍不能发布；UNKNOWN 被汇总页面隐藏	XC 验证、集成测试、Release Evidence Bundle	18.9—18.11
8. 责任悬空	多人都能发指令，却无人拥有最终意图、接口或发布责任	两级责任、唯一签署人、接口所有者、异常裁决人	18.4、18.14、18.18
9. 规模化没有净收益	Agent 越多，等待、返工、验证和监督成本越高	协同成本遥测、去规模化、动态选择自治级别	18.19

九类问题并不要求九套独立流程。最好的控制通常同时解决多个问题：稳定 XC 可以减少边界漂移、上下文争议和重复交接；单写入者可以同时降低写入冲突、状态变化和验证不一致；唯一 IO 可以同时解决意图分裂与责任悬空。

20.2.2 协同场景矩阵

治理应从真实协同拓扑出发，而不是看到“多人”或“多 Agent”就启用相同流程。

协同场景	主要矛盾	推荐默认	关键治理资产
不同项目、没有依赖	几乎没有共享状态	完全独立闭环	各自的 Intent Contract 与 Evidence
多个项目共享平台、治理资产或组织目标	指标口径、共享依赖和改进政策可能跨项目传播	项目独立执行，组织层只汇聚可比信号	度量契约、可比群组、组织 Dashboard、证据下钻
同一项目、不同模块、无依赖	并行工作但不存在系统冲突	独立并行，最后做基础集成	Task ID、独立 Change Envelope
同一项目、不同模块、有接口依赖	文件不冲突，但语义、版本和顺序冲突	受约束并行	XC、集成 DAG、端到端验证
同一项目、不同模块、共享数据或配置	状态一致性和迁移顺序	分阶段执行	数据所有权、迁移计划、补偿与系统级回滚
同一项目、相同模块	并发写入和共享不变量冲突	默认单写入者，其他人并行验证	所有权、写入边界、合并验证
同一任务、多种 AI 工具接力	上下文、权限和证据来源不一致	统一任务身份后 Handoff	Tools Manifest、Task ID、契约版本、证据协议
多语言、多测试体系	测试框架不同，结果难以聚合	不统一工具，统一证据语义	checks、AC/Constraint ID、Evidence Bundle
大型既有系统跨模块变更	隐性依赖、历史行为和未知模块	先 Recon，再限定变更围栏	Baseline、Preserve、UNKNOWN、Change Envelope、Characterization Test
客户、FDE、总部和供应商共同交付	上下文可见性、商业边界和外部证据可信度	按组织边界分层授权	Protocol、责任矩阵、证据来源、商业与数据边界
多人共享聊天、单一 IO	多人参与，但执行权可明确	协作平面与执行平面分离	消息分类、唯一 IO、Task ID、Context Slice
多人共享聊天、多个 IO	多个意图主体争夺同一批 AS	默认停止，先裁决责任	意图冲突记录、HITL、独立任务或 Amendment
跨模块紧急事故	正常 DAG 失效，各模块自行恢复可能扩大事故	单一事故指挥，冻结非必要写入	Incident Work Graph、权限撤销、证据保全、系统级恢复

其中三个场景需要特别强调：

- **大型既有系统**不能从理想模块图出发，必须先 Recon。未发现的依赖应保留为 UNKNOWN，不能因为 Code Map 没显示就断言“不受影响”。
- **跨组织和 FDE 交付**不仅是模块协作，还涉及谁能看到什么、谁有权接受外部证据、现场例外如何回流产品治理，以及商业责任在哪里停止。
- **跨模块事故**不能继续沿用正常并行节奏。应指定单一事故指挥者，暂停无关 Work Graph，冻结非必要写入，统一判断隔离、回滚和恢复顺序，防止各模块分别“恢复成功”却造成系统级二次故障。

20.3 团队内多人协同的三种拓扑

“多人共同开发同一个项目”至少包含三种完全不同的场景。

20.3.1 不同模块、没有实质冲突

例如 A 修改前端页面，B 修改独立报表服务，C 补充文档；三者不共享接口、配置、数据迁移和发布顺序。这是最适合并行的场景。

每项工作只需要独立 Task ID、Intent Contract、Change Envelope 和本地证据。最终进行基础集成检查即可，不必让所有参与者共享完整上下文，也不必为不存在的依赖建立 XC。

20.3.2 不同模块、存在语义或依赖冲突

例如 A 修改服务端接口，B 修改前端消费者，C 修改数据库结构。三者可能没有修改同一个文件，但字段语义、数据状态、兼容窗口和发布顺序互相影响。这类问题可以概括为：**文件不冲突，系统仍然会冲突。**

治理必须从文件边界上升到系统边界：先签署 XC，明确接口语义、幂等性、错误处理和版本策略；再用集成 DAG 规定实现、迁移、验证和发布顺序；最后通过端到端证据证明局部结果可以组成系统结果。

20.3.3 相同模块、存在并发写入冲突

例如 A 修复支付模块，B 同时重构支付模块，C 又要求调整相同异常逻辑。Git 没有文本冲突，并不代表没有语义冲突：一个修改可能删除另一个修改依赖的不变量，两组测试可能分别通过，合并后却不再满足任何一份完整意图。

默认策略应是：

相同模块、相同责任域、相同时间窗口，只允许一个活跃写入者。

其他参与者可以并行进行测试设计、安全审查、反例生成和 Evidence 核验。如果确实要并行写入，必须先把模块进一步拆成不重叠的 Change Envelope，并证明它们不存在共享不变量。不能完成这种拆分时，串行执行比增加协调工具更可靠。

团队内场景	默认执行方式	最小治理
不同模块、无依赖	并行独立闭环	Task ID + 独立 Change Envelope
不同模块、有接口依赖	受约束并行	XC + 集成 DAG + 端到端验证
不同模块、共享数据或配置	分阶段并行	所有权 + 迁移顺序 + 系统级回滚
相同模块、相同责任域	默认串行写入	单写入者 + 并行验证
相同模块、可进一步拆分	隔离并行	不重叠 Change Envelope + 合并验证

20.4 全局 OA 与模块 OA

全局 OA 负责：

- 全局意图和约束；
- 模块边界；
- Protocol；
- 跨模块契约；
- 集成 DAG；
- 证据聚合和发布裁决。

模块 OA 负责：

- 模块任务契约；
- 模块上下文和工具；
- 本地约束与测试；
- 模块 Evidence Bundle；
- 向全局 OA 报告真实状态和未知项。

两级 OA 不是天然必须存在的管理层级。只有跨模块依赖无法通过去耦消除时，才需要全局 OA；模块能够独立闭环时，全局 OA 只管理共同边界和集成事实，不介入模块日常执行。

20.5 Protocol：跨主体治理协议

Protocol 至少定义：

- 模块清单与所有者；
- 共享术语；
- 跨模块接口；
- 版本兼容；
- 集成顺序；
- 每种语言的 checks；
- Handoff 格式；
- 失败升级路径；
- 发布证据要求。

它不是替代所有模块文档，而是建立跨模块最小共同语言。

20.6 跨模块契约 XC

模块内部测试通过，不代表集成成立。XC 应定义：

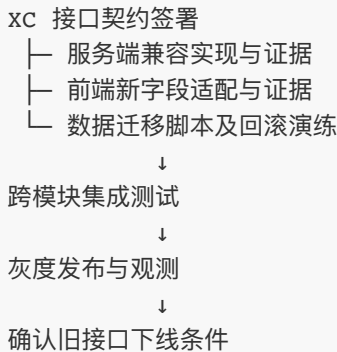
- 请求和响应模式；
- 必填字段；
- 错误语义；
- SLA；
- 兼容策略；
- 破坏性变更规则；
- 上下游验证证据。

XC 还应说明业务语义、幂等性、事件顺序、数据所有权、迁移责任、补偿动作和旧版本退出条件。字段结构一致只能证明语法兼容，不能证明两个模块对“审批完成”“付款成功”或“允许重试”的理解一致。

20.7 集成 DAG 与发布顺序

集成 DAG 是一张描述跨模块交付依赖的有向无环图。每个节点代表能够判断完成与否的工作单元，箭头表示“后一个节点必须等待前一个节点的有效产物”。“无环”意味着依赖不能形成 A 等 B、B 又等 A 的死锁。

任务清单能说明有什么工作，却不能说明顺序、并行机会和失败影响。一个接口升级可能同时涉及服务端兼容、前端适配、数据迁移、集成测试和灰度发布。若迁移先于兼容版本上线，或者消费者尚未准备就删除旧字段，所有单项任务即使都完成，系统仍会失败。



每个节点需要输入、输出、责任人、AC、超时、失败路由和证据位置。发布节点不能只检查上游状态为“完成”，还要验证上游证据属于同一契约版本，并且所有 MUST 门禁通过。

发布顺序本身就是风险控制。常见策略包括“先兼容生产者，再升级消费者，最后移除旧接口”；数据迁移则可能采用“先双写、再回填、后切读、最后停旧写”。具体顺序取决于系统，但必须显式化并可回滚。

当某个节点失败时，DAG 帮助 OA 判断哪些并行节点仍可继续、哪些下游必须阻塞、回滚会不会破坏已经完成的工作。回滚前还需要分析下游依赖和数据兼容，避免一个模块安全回滚却破坏整个系统状态。

20.8 多语言 checks

不同模块可以使用不同工具，但必须把结果转换为统一证据语义：

```
command
exit_code
result
artifact
timestamp
constraint_or_ac_id
```

统一的不是测试框架，而是证据协议。

20.9 证据聚合

每个模块生成自己的 Evidence Bundle。全局 OA 聚合：

- 模块契约版本；
- 跨模块 XC 结果；
- 集成测试；
- 全局 MUST 约束；
- 模块未知项；
- 合并遥测；
- 统一回退方案。

最终产生 Release Evidence Bundle，供发布裁决使用。

聚合不能把模块状态简单相加。Release Evidence Bundle 至少要证明：各模块满足局部责任，模块之间满足 XC，端到端系统满足全局意图和 MUST，系统级回退能够成立。任何模块的 UNKNOWN、证据过期或契约版本不一致，都不能在汇总页面中被绿色平均值掩盖。

20.10 跨项目价值与效能视图

这个视图不等于另一个项目管理大屏，更不是对个人工作量的监控。它面向人机协同研发系统，展示：

- 当前意图和任务状态；
- 五道门通过情况；
- MUST 失败；
- HITL 等待项；
- 模块和跨模块证据；
- 单任务与累计遥测；
- 风险、例外和失效时间；
- 发布建议和依据。

它的目标是让价值流、系统能力、治理状态和残余风险透明，使管理者能够判断哪里值得扩大自治、哪里需要补强工程系统、哪里必须由人承担决策责任。

20.11 一个跨模块失败案例

报销前端、规则服务和财务接口分别通过了本地测试，但集成时发现规则服务返回字段 `approverRole`，财务接口仍读取 `approver_role`。三个模块都声称完成，系统却无法运行。

根因不是测试不足，而是缺少跨模块契约和统一发布证据：

- 模块契约只描述内部行为；
- 没有 XC 定义字段和兼容策略；
- 集成 DAG 没有安排契约验证；
- 模块 Evidence Bundle 没有汇聚；
- 下游相信了上游的完成声明。

加入 XC 后，字段、版本、错误语义和破坏性变更规则成为可验证对象。模块仍然可以自主开发，但接口不能自主漂移。

20.12 两级约束

多模块项目通常同时存在：

全局约束：安全、数据、发布、审计、公共技术标准。

模块约束：语言、框架、局部性能、模块目录和测试方式。

模块 OA 可以维护局部约束，但不能降低全局 MUST。发现冲突时必须升级给全局 OA 和责任人。

20.13 多工具治理

团队可能同时使用 Codex、Claude Code、IDE Agent、CI Bot 和内部平台。治理不能依赖某个工具的对话记忆，而应落在仓库资产中：

- 同一意图契约；
- 同一约束矩阵；
- 同一 Tools Manifest；
- 同一 Evidence Bundle 语义；
- 不同工具各自适配执行接口。

工具可以替换，治理责任和证据链不能随工具消失。

多工具接力同一任务时，每一次 Handoff 都应携带 Task ID、契约与约束版本、已修改范围、测试状态、开放 UNKNOWN、下一步动作和证据位置。新工具不能把上一工具的聊天摘要当作唯一事实源，也不能因为自己具备更

高权限就自动扩大执行范围。

20.14 多人共享一个聊天窗口：多主控制问题

一种看似自然的协同产品，是让两人或多人共同面对一个聊天窗口：A 提出需求，等待 AS 完成；B 接着要求修改；C 又提出新需求；随后 A 认为方向不对，再要求改回。底层始终是同一批 AS、同一上下文和同一个工作目录。

这不是普通的多人协同，而是“多意图主体共享同一执行会话”的**多主控制问题**。它最先冲突的不是代码，而是意图和授权：

- A、B、C 谁是当前任务的 IO；
- 后续消息是澄清、正式变更、新任务，还是未经确认的建议；
- 谁能修改非目标、扩大 Change Envelope 或改变优先级；
- 已完成的测试和 Evidence 对应哪一版要求；
- 多人意见冲突时，AS 应停止还是自行折中；
- 最终由谁验收并承担结果。

“多人能够参与同一对话”是真实需求；“多人可以直接向同一批 AS 连续发出执行指令”通常是伪需求。团队真正需要的往往是共同可见、共同讨论、专业意见输入和决策透明，而不是所有人同时握住执行方向盘。

共享会话应被拆成两个逻辑平面：

平面	参与者可以做什么	不能自动产生什么
协作平面	提建议、补充上下文、提出风险与反例、评论证据	不直接改变已签署意图和执行范围
执行平面	由当前任务授权 IO/OA 创建任务、确认变更、启动 AS、批准不可逆动作	不接受无责任身份的最后一句话覆盖当前契约

20.14.1 多主会话的最小规则

如果产品确实要支持多人共享一个 Agent 会话，至少需要遵守以下规则：

1. **保留身份与角色**：每条消息必须记录发送人、时间和角色，不能把多人输入压缩成无来源的统一 Prompt。
2. **每个任务只有一个 IO**：可以有多名参与者、专家和审核人，但当前任务必须有唯一的意图责任人。
3. **普通消息默认是建议**：非当前 IO/OA 的消息默认进入协作平面，不能直接触发代码修改、工具调用或范围变化。
4. **执行前先做消息分类**：系统必须区分建议、澄清、正式变更、新任务和讨论，不能把每一句话都解释为新命令。
5. **变更已签署意图时暂停执行**：涉及目标、非目标、AC、优先级或 Change Envelope 的变化，必须停止当前 AS，等待 IO 确认并形成新版本或 Amendment。
6. **每批 AS 绑定明确执行快照**：执行必须绑定 Task ID、契约版本、约束版本、Change Envelope 和 Context Slice，不能跟随聊天窗口持续漂移。
7. **执行上下文与完整聊天隔离**：AS 只接收裁剪、确认后的执行上下文；争论、猜测、玩笑、撤回意见和未确认方案不进入执行 Prompt。
8. **新任务不得混入运行中的任务**：新需求应获得独立 Task ID、契约和证据链，不能悄悄追加到正在执行的任务中。
9. **冲突时停止，不由模型折中**：多人意见冲突、责任不清或批准边界为 UNKNOWN 时，AS 必须暂停并触发 HITL，不能依据最后发言、职位高低或表达强度自行选择。
10. **验收权必须明确**：任务完成、发布和不可逆动作只能由预先定义的责任人裁决；群聊中的沉默、点赞或无人

反对不构成签署。

11. **撤回和替代必须传播**：被撤回、拒绝或 superseded 的要求必须退出 Context、Memory、任务队列和后续 Handoff，不能继续暗中影响 AS。
12. **全链路可以追溯**：最终产物必须能够回答谁在何时提出、谁确认、基于哪份契约执行、哪些 AS 参与、产生了什么证据，以及谁作出最终裁决。

这 12 条规则的目标不是让共享聊天变成审批系统，而是确保“多人可以参与”不会退化成“任何人都可以随时改变机器正在执行的意图”。如果产品无法提供身份、唯一 IO、版本快照、暂停机制和可追溯性，就不应把共享会话直接连接到具有写入权限的 AS。

所有消息进入执行前，应先分类为“建议、澄清、变更、新任务或讨论”。建议进入候选区；澄清由 IO 确认后写回当前契约；正式变更产生新版本或 Amendment；新任务获得独立 Task ID；讨论内容不进入 AS 的执行 Context Slice。

多人发言

- 消息分类：建议 / 澄清 / 变更 / 新任务
- IO 确认
- 形成稳定契约版本或独立任务
- 绑定一批 AS 与 Change Envelope
- 执行、验证、验收
- 再切换下一意图

多人共享窗口在结对编程、Mob Programming、方案评审、跨专业咨询和教学工作坊中可以有价值，但必须有一个明确 Driver 或 IO。多人意见发生冲突时，系统应暂停执行并请求责任裁决，不能让模型依据消息顺序、表达强度或“综合判断”选择方向。

每批 AS 必须绑定明确 Task ID、契约版本和上下文快照。AS 不直接消费未经裁剪的全部聊天历史，已撤回意见也必须退出后续上下文。否则共享聊天只是在用界面便利掩盖责任和状态混乱。

20.15 Handoff 摩擦

多 Agent、多模块和多工具会产生交接摩擦。可以观察：

- 上游输出被下游退回次数；
- 因字段或版本不一致造成的失败；
- 重复读取和重复验证；
- 等待人工澄清时间；
- 交接后新发现的未知项；
- 跨模块返工比例。

这些数据帮助团队判断问题来自执行能力，还是协议和上下文设计。

有效 Handoff 不是“我做完了”的自然语言声明，而是一个可验证的状态转移。下游应能确认上游产物属于正确契约版本、相关 MUST 已通过、未知项未被隐藏、证据仍在有效期内。条件不满足时，Handoff 应被拒绝，而不是由下游默默补齐上游责任。

20.16 跨项目视图反模式

- 把每个 Agent 的 Token 排名作为绩效；
- 只显示绿色状态，不显示 UNKNOWN；
- 模块自己上传“已通过”文本；
- 跨项目视图与真实仓库工件不同步；

- 例外到期后仍显示通过；
- 大屏很漂亮，却不能追溯到原始证据；
- 管理层用自动化率替代价值和风险判断。
- 把更多并行 Agent 数量当作规模化成功；
- 允许多人在共享会话中以最后一条消息覆盖当前意图；
- 同一模块存在多个活跃写入者，却只依赖 Git 解决语义冲突。

20.17 跨模块变更的治理链

以“报销审批结果同步到财务付款”为例，变更可能同时影响报销服务、审批服务、财务接口和审计日志。全局契约定义端到端结果与不可妥协边界，各模块契约定义局部责任，XC 定义接口字段、时序、幂等性和错误语义。

某个模块的单测全绿，只能证明局部实现满足局部假设。集成门还必须验证：审批完成事件不会重复付款，失败重试不会丢失审计信息，旧版本消费者能够处理新增字段，任何模块失败时端到端状态可恢复。最终 Evidence Bundle 必须保留模块证据来源，不能只给出“集成成功”的汇总结论。

20.18 所有权比工具统一更重要

组织可能同时使用 Codex、Claude Code、GitHub Copilot、内部 Agent 和传统脚本。治理目标不应是强迫所有人使用同一个界面，而是要求不同工具服从共同协议：读取同一份签署契约，遵守同一约束来源，产生可识别 Handoff，接受统一门禁，并把证据写回任务目录。

真正需要统一的是任务身份、责任、协议和证据语义。工具可以替换，治理链不能因为换了模型就断裂。

20.19 规模化的经济边界

多 Agent 不总是更便宜。组织应同时计算执行成本、上下文准备成本、冲突与重试成本、验证成本以及人类监督成本。只有当任务可分解、接口稳定、验证可自动化并且失败影响可控时，自治规模化才会产生净收益。

对于高度探索、意图频繁变化或责任无法拆分的任务，小团队与单一 Agent 可能更有效。成熟度不是永远增加自治，而是能够根据风险动态选择自治级别。

组织评估规模化收益时，不应只统计 Agent 产出或并行任务数，还应计算上下文准备、协议维护、等待、冲突、重复验证、证据聚合和人工监督成本。如果增加一个 Agent 节省了十分钟编码，却带来一小时协调和返工，它不是扩展了能力，只是扩大了队列。

真正成熟的规模化表现往往不是中央大屏上运行着最多 Agent，而是大量任务已经被拆成彼此独立的小闭环，只有少数真实依赖需要全局协调。组织级视图的成功，应体现为协调需求下降，而不是协调活动增加。

20.20 跨项目遥测：可以汇聚，但不能直接排名

第十四章建立了“单契约 → 单项目”的双层遥测。进入多人、多工具、多模块和多项目环境后，组织还需要第三层视角：项目组合与组织级遥测。它用于识别跨项目风险、共同瓶颈和治理资产复用机会，但不能把不同项目的数字直接放在一起排名。

单契约遥测：这次任务发生了什么

↓

单项目遥测：这一类任务是否正在改善

↓

跨项目遥测：哪些模式具有组织意义

跨项目汇聚最难的不是把 JSON 合并，而是保证比较有意义。一个高风险金融项目和一个内部演示工具，即使都显示“首次成功率 80%”，其失败后果、验证成本和 HITL 责任也完全不同；一个遗留系统的改动可能只涉及几十行代码，却需要大量 Recon 和兼容验证；一个新项目的高自动化率，也可能只是因为没有历史约束。

因此，跨项目比较必须先建立可比群组。至少按以下维度分群：

- 任务类型：探索、功能交付、缺陷修复、迁移、事故恢复；
- 治理 Profile：explore、delivery、high-risk、legacy、multi-module；
- 风险与影响：内部可逆、用户影响、资金或敏感数据、安全关键；
- 工程基础：新项目、既有小项目、大型遗留系统；
- 技术与执行环境：语言、测试体系、工具、模型和运行环境；
- 观察窗口：相同周期、相似样本量和同一指标版本。

没有分群时，不做横向排名；没有共同口径时，不计算组织平均值；没有来源和样本量时，不把趋势解释成事实。

20.20.1 跨项目度量契约

每个准备进入组织级 Dashboard 的项目，都应声明一份最小度量契约：

字段	作用
project_id / owner	标识项目和解释责任人
task taxonomy	说明任务怎样分类，避免混合不同工作
governance profile / risk class	说明治理重量和失败后果
metric definition version	标识公式、分母和阈值版本
source / evidence link	能从汇总数字下钻到任务与原始证据
baseline / comparison cohort	说明与谁比较、为什么可比
sample size / time window	防止小样本和时间错位造成误判
tool / model / environment	解释工具和环境变化带来的影响
data quality / UNKNOWN	暴露缺失、估算、过期和采集失败
access / retention	约束谁能看到、保留多久、怎样删除

度量契约发生变化时，组织应保留版本边界。不能把不同公式、不同任务分类或不同风险口径的数据无标记地画成一条连续趋势线。

20.20.2 哪些信号值得跨项目观察

跨项目遥测的目标不是寻找“最强团队”，而是识别系统性问题和可复用改进机会。适合观察的信号包括：

- 目标准确率与首次成功率在同类任务中的分布；
- 发布后缺陷、回滚和事故率；
- 验证成本是否随风险合理增长；
- HITL 是否发生在价值、异常和不可逆边界，而不是被当作补锅；
- UNKNOWN 被发现、裁决和关闭的时间；
- Handoff 退回率、跨模块返工和集成等待；
- 相同约束失败是否在多个项目重复出现；
- 共同 Skill、模板、测试模式和治理政策是否真正提高复用效果；
- Token、时间和人工投入是否换来了质量或风险改善。

Token 总量、测试数量、覆盖率、自动化率和 Agent 数量不能脱离任务价值与风险单独排名。低 Token 可能来自上下文不足，高覆盖率可能没有覆盖关键行为，高自动化率也可能意味着必要 HITL 被绕过。

20.20.3 汇聚必须能够下钻和反向传播

组织级 Dashboard 上的每个结论，都应能够沿下面的链路下钻：

组织趋势

→ 可比项目群组

→ 单个项目

→ 单份 Intent Contract

→ Evidence Bundle

→ 原始测试、约束与遥测来源

汇聚也必须支持反向传播。如果多个项目反复出现同一类接口漂移、权限误用或验证缺口，治理委员会可以提出组织级约束、共享测试、Protocol 或 Skill 候选；但从“共同现象”晋升为“统一政策”，仍需验证适用范围、反例和副作用，不能由 Dashboard 自动修改所有项目。

跨项目遥测应帮助组织判断哪些复杂度可以被消除。例如多个项目都因同一种 Handoff 失败而返工，最佳行动可能不是增加一个跨项目协调角色，而是稳定接口、统一证据字段或进一步拆小责任域。遥测最终应服务于去规模化。

20.20.4 跨项目遥测的反模式

- 用首次成功率、Token 或自动化率直接给项目和团队排名；
- 把不同风险、任务类型和工程基础的数据计算成一个平均值；
- 只展示百分比，不展示分母、分布、时间窗口和 UNKNOWN；
- 指标公式升级后，仍把新旧数据画成连续趋势；
- 汇总数字无法下钻到契约和 Evidence Bundle；
- 为追求统一口径，删除项目真正需要的行业与风险差异；
- 把治理成本转移给 QA、运维、客户或供应商，却仍宣称研发效率提升；
- 组织级 Dashboard 收集了敏感代码、客户数据或个人绩效信息，却没有访问和删除边界；
- 发现跨项目问题后，第一反应是增加审批层，而不是减少耦合和共享状态。

跨项目遥测的正确问题不是“哪个项目数字最好”，而是：在相同风险和任务类型下，哪些控制真正减少了失败、返工和决策成本；哪些做法只是在本地看起来高效，却把风险转移给了其他模块、项目或人。

20.21 本章小结

多人、多工具、多模块治理的目标，不是让更多人和 Agent 同时进入同一个上下文，而是通过去规模化减少必须协调的部分。不同模块无依赖时各自闭环；存在接口和数据依赖时使用 XC 与集成 DAG；相同责任域默认保持单写入者；多人共享聊天时把协作权与执行权分开，并为每批 AS 绑定唯一意图和责任主体。

Protocol、XC、集成 DAG、Handoff 和证据聚合，是为无法消除的依赖建立共同控制面。它们最终要证明的不是“所有模块都说自己完成了”，而是局部结果能够组成端到端价值，失败能够被阻断和恢复，发布结论能够追溯到真实责任与独立证据。

跨项目遥测把单契约与单项目信号提升为组织学习，但必须先分群、统一度量契约并保留证据下钻。它的目的不是排名团队，而是发现系统性风险、验证共同治理资产，并判断哪些协调和耦合可以继续被消除。

本章最小实践：把报销案例拆成前端、规则服务和财务接口三个模块，先判断哪些协作可以通过去规模化消除，再为剩余依赖设计所有权、XC、集成 DAG 和多人消息分类规则。

完成证据：协同拓扑与单写入边界、Protocol、两个 XC、Handoff 记录、共享会话执行权规则、Release Evidence Bundle，以及一份带可比群组 and 证据下钻的跨项目度量契约。

延伸实践：推荐学习《Agentic Agile 组织转型进阶：构建人机协同研发操作系统》，判断什么时候可以从单任务闭环扩展到价值网络，什么时候必须停止扩张。

第四部分 | 四维组织转型：把工程能力变成企业运行方式

第三部分证明了 3-4-3 能够在真实任务、既有系统和规模化协作中运行。第四部分转向企业转型本身：人员责任如何迁移，组织决策权怎样重构，流程怎样从仪式与交接转向持续控制，工具怎样从单点助手形成最小人机协同研发系统。

这一部分不再重复 SCOPE-V、Harness 和 Evidence 的工程实现，而是把它们放回同一条价值流，用四维转型画布、成熟度和阶段证据回答：哪些提效可以重复，哪些能力可以跨场景复用，组织应当扩大、收缩还是停止自治。

第二十一章 | 人员与组织重构：从岗位团队到人机责任网络

AI 转型首先改变的不是组织架构图，而是价值流中的任务分配、判断权和责任归属。把 Copilot 发给所有人，仍按原岗位、原交接和原审批运行，只会得到局部提速；真正的重构要回答：哪些执行交给 Agent，哪些判断必须由人承担，谁可以授权，谁必须停止系统，谁对最终结果负责。

21.1 重构单位是任务组合，不是岗位名称

岗位由多种任务组成：信息收集、方案生成、编码、验证、协调、价值判断、风险承担和对外承诺。AI 对这些任务的适配度不同，因此“某岗位是否会消失”不是一个可执行问题。组织应逐项判断：

任务类型	默认责任安排
高频、可逆、可自动验证的执行	优先交给 AS，在 Harness 内自治
规则明确但影响较高的执行	Agent 执行，独立证据与人类裁决
目标、优先级和价值取舍	IO 负责，Agent 只能提供分析与选项
权限例外、不可逆操作、法律伦理判断	具备资格的人类责任人批准
失败分类、编排和控制系统设计	OA 负责，专业人员与 Agent 共同参与

提效来自任务重新组合：机器承担更多高速执行，人类集中在意图、边界、异常和责任判断。若只是要求同一批人在原流程中多使用一个工具，组织没有更换操作系统，只是增加了一个输入设备。

21.2 IO、OA、AS 不是三个新职位

IO、OA、AS 是三种不可混淆的责任，而不是强制新增三个岗位：

- **IO** 对价值意图、非目标、关键 AC、风险偏好和最终签署负责；
- **OA** 对 Context、Graph、Harness、验证策略、异常路由和证据完整性负责；
- **AS** 由专业 Agent、自动化工具和参与执行的工程人员共同构成，在授权边界内交付可验证结果。

小团队可以由业务负责人兼任 IO、技术负责人兼任 OA；高风险场景则应拆开签署、执行和验证责任。判断角色设计是否有效，不看头衔是否齐全，而看价值责任、治理责任和执行责任能否被独立指认。

21.3 人类不可转移的五类责任

无论模型能力多强，以下责任不能因为“AI 建议如此”而消失：

- 1. 决定什么价值值得追求，以及明确不做什么；
- 2. 在速度、成本、公平、安全和体验冲突时作出取舍；
- 3. 批准权限扩大、约束例外和不可逆操作；
- 4. 对证据不足、UNKNOWN 和残余风险作出行动裁决；
- 5. 对外部客户、监管者、员工和社会后果承担责任。

Agent 可以生成方案、模拟影响和整理证据，但不能成为责任逃逸装置。组织授权给 AI 的是执行权，不是责任主体资格。

21.4 从固定团队到意图单元和动态能力网络

传统项目常围绕岗位和部门组队，再通过会议与交接协调。Agentic Agile 以一个签署意图为中心形成最小责任单元：IO 提供价值方向，OA 装配所需能力，AS 按 Work Graph 执行，领域、安全、架构和运营专家在关键节点加入。



这种结构不是取消长期团队。稳定团队仍保存领域知识、工程资产和共同责任；动态意图单元则减少跨部门排队，让能力围绕真实价值流临时组合，任务结束后把证据和知识带回组织资产。

21.5 决策权必须与信息 and 风险一起前移

如果 Agent 可以在几分钟内执行，而所有例外仍等待周会、层层汇报或委员会逐项审批，自动化只会制造更长队列。组织需要预先定义授权矩阵：什么由规则自动放行，什么由 IO/OA 当场裁决，什么必须升级到安全、合规或治理委员会。

RACI-D 在传统 RACI 上增加 Decision Owner：谁执行、谁负责、咨询谁、通知谁，以及谁拥有最终决定权。每个高风险节点只能有一个清晰的 Decision Owner；多人“共同负责”通常等于无人能及时决定。

21.6 能力成长：从亲自执行到设计人机系统

传统角色	需要增强的能力
产品与业务负责人	意图建模、Non-goals、AC、价值证据、风险签署
架构与技术负责人	Context、Graph、Harness、恢复和跨模块协议
开发工程师	可执行规格、测试、Agent 工作流和工具设计
测试工程师	Eval、属性测试、证据独立性和门禁完整性
安全与合规	Policy as Code、工具权限、例外与审计证据
工程经理与研发效能	人机系统运营、遥测解释、能力建设和资产产品化

专业经验不会因 AI 失效，而会从“亲自完成每一步”迁移到“把判断转成上下文、约束、验证和恢复能力”。

21.7 激励与反监控边界

如果组织用代码量、AI 使用率、最低 Token、最高自动化率或最低 HITL 排名个人，人和 Agent 都会学会隐藏失败、弱化门禁和挑选简单任务。更健康的评价对象是系统：目标是否准确、风险是否被及时暴露、证据是否真实、同类失败是否减少、总成本是否下降、经验是否形成可复用资产。

遥测默认用于改进契约、上下文、约束、编排和工具，不用于简单监控个人。个人能力辅导必须结合任务风险、角色责任和系统条件；未经授权，不应采集与价值流无关的私人提示词、敏感代码或行为细节。

21.8 本章小结

人员与组织重构不是岗位消亡计划，而是任务、执行权、决策权与责任的重新分配。IO/OA/AS 建立最小责任网络，动态意图单元减少无效交接，授权矩阵让决策跟上机器速度，人类则继续承担价值、例外、不可逆操作和最后果。

本章最小实践：选择一条真实价值流，把十项关键任务标记为 Agent 执行、人机共决或人类专属，并为每个高风险节点指定 Decision Owner。

完成证据：人类责任节点与 Agent 能力网络图、RACI-D、授权与升级矩阵。

延伸实践：推荐学习《Agentic Agile 组织转型进阶：构建人机协同研发操作系统》，把岗位分工重构为人机责任网络和可签署的授权关系。

第二十二章 | 流程与工具重构：从旧流程加 AI 到 AI 原生价值流

人员和组织责任改变后，若需求、开发、测试、审批和发布仍按原交接链运行，提效会被等待时间吞噬；若工具仍停留在个人 Copilot，又无法让意图、权限、证据和反馈跨步骤连续。流程与工具必须共同重构：流程定义控制怎样流动，工具保证关键状态不能被静默跳过。

22.1 旧流程加 AI 为什么只会局部提速

典型做法是在需求完成后让 AI 写代码，在测试阶段让 AI 补用例，在发布前继续人工汇总证据。编码时间缩短了，但需求等待、环境准备、跨团队交接、重复审批和证据整理没有减少，甚至因生成速度提高而增加返工。

判断流程是否真正重构，可以看三个信号：决策是否前移到执行前，反馈是否进入执行内环，完成是否由证据而非状态汇报决定。

22.1.1 先测价值流，不要先画未来蓝图

流程重构应从一条真实价值流的事实开始。选择一种重复出现的任务，例如“从业务规则变化到生产灰度”，沿途记录：

观察项	要采集的事实	常见问题
触达时间	从提出意图到产生业务结果的总时长	只统计编码时间，忽略排队和上线等待
活动时间	真正分析、实现、验证和裁决的时间	把会议时长全部当作价值活动
等待时间	等需求、权限、环境、评审、测试和发布窗口	AI 加速后瓶颈转移到审批和环境
交接次数	任务、上下文和责任主体发生多少次切换	每次交接都重新解释目标并丢失约束
返工与失败	因规格、实现、环境、权限和证据不足造成的重做	最终全绿掩盖多轮失败
决策时延	异常出现到有权者作出决定的时间	Agent 数分钟执行，例外等待数天
证明成本	为确认完成投入的测试、人工核对和证据整理	编码节省被 QA、业务和运维成本抵消

流程图只显示“经过哪些部门”还不够。必须把工作、等待、决策、证据和返工放在同一张价值流图上。否则组织容易优化最显眼的编码节点，却看不见真正限制端到端提效的约束。

22.1.2 识别三类被 AI 放大的流程债

第一类是**批量债**：需求、测试和发布按周或按 Sprint 批量处理，机器速度无法变成更短反馈。第二类是**交接债**：每个岗位只交付文档或状态，下游必须重新理解意图。第三类是**审批债**：低风险事项也依赖人工逐项放行，而高风险事项又缺少真正有资格的 Decision Owner。

AI 不会自动消除这些债，反而会让它们更明显：上游几分钟生成更多工作，下游队列更长；更多 Agent 并行产生更多交接；为了控制失速，组织继续增加审批。流程重构必须减少批量、消除不必要交接，并把审批改造成“预授权规则 + 事件触发例外”。

22.2 哪些敏捷与工程活动保留，哪些应被替代

Agentic Agile 不以“删除所有会议”为先进。活动是否保留，取决于它是否承担不可自动化的价值和责任：

传统活动	处理原则
目标、优先级和风险取舍	保留人类决策，用 Intent Contract 固化结果
站会中的纯状态播报	由 Work Graph、事件和 Dashboard 自动替代
需求澄清	从大批量评审转为批判性检查、Grill-Me 和 Example Mapping
开发后集中测试	前移为 AC、TDD Red 与 P⇌E 快内环
每一步人工审批	改为约束、门禁和事件触发 HITL
发布前手工拼报告	由 Evidence Bundle 和 Release Manifest 汇聚
复盘	保留价值与系统判断，输入来自真实失败和 Telemetry 趋势

流程重构的目标不是少开会，而是让机器处理可机械判断的状态，让人的时间集中在目标冲突、异常和责任裁决。

22.2.1 用事件替代状态汇报，用证据替代完成声明

旧流程围绕日历运行：到时间开会、汇报状态、请求批准。AI 原生流程更多围绕事件运行：契约已签署、可信 RED 已形成、MUST 被阻断、重试预算耗尽、Evidence 已具备裁决资格、生产事实推翻原结论。事件发生时，系统自动推进、停止或通知正确责任人。

这并不取消人的节奏。战略优先级、跨团队冲突、重大例外和遥测趋势仍需要定期讨论；但会议输入应是事件、证据和选项，而不是逐人询问“做得怎么样”。

22.2.2 一项活动是否保留的四问

- 1. 它是否作出机器不能承担的价值或风险判断？
- 2. 它是否产生下游真正使用的决策或证据？
- 3. 它是否能被事件触发，而不是固定等待日历？
- 4. 若删除它，哪个门禁、责任人或恢复机制接替其控制功能？

不能回答第四问时，直接删流程会造成控制真空；前两问都答“否”时，继续保留通常只是组织惯性。

22.3 AI 原生价值流：从任务传递到持续控制

Intent Graph

→ Intent Contract + Constraint Matrix

→ S / C / O

→ P ⇌ E 快内环

→ Evidence Bundle

→ V 独立裁决

→ Telemetry 慢外环

→ 下一轮人员 / 组织 / 流程 / 工具调整

这条价值流不是新的阶段瀑布。S、C、O 持续控制方向、边界和协作，P⇌E 快速收敛，V 保持独立，Telemetry 跨任务反馈。五道机械门只裁决状态转换，不增加五层审批。

22.3.1 从文档流转变成控制状态转换

控制状态	进入条件	机器可以做什么	必须停止的情况
意图探索	目标尚未签署	检索、分析、提问、生成选项	不得创建业务实现或扩大权限
可执行准备	契约、约束和 AC 已成立	裁剪 Context、规划 Work Graph、准备测试	签署缺失、UNKNOWN 影响目标或边界
受控执行	前置门和编码门满足	在授权文件、工具和预算内实现与验证	越界、MUST 失败、不可逆操作、预算耗尽
证据收敛	P⇌E 达到退出条件	汇聚 Evidence、检查新鲜度和一致性	证据矛盾、关键 AC 未证明、回退不可用
责任裁决	V 具备独立审查条件	提出批准、条件批准、补证或拒绝建议	机器不得替有权者接受残余风险
反馈捕获	裁决与任务事实已记录	生成单任务遥测并进入跨任务聚合	不得用一次任务宣称组织趋势

状态转换比岗位交接更适合高速 Agent：它说明当前事实允许什么行动，而不是规定某份文件必须在某个部门停留多久。岗位仍承担责任，但流程由可验证状态驱动。

22.3.2 HITL 是异常路由，不是默认收费站

人类介入应绑定事件、责任资格和决策时限。例如规格冲突路由给 IO，权限例外路由给安全或数据责任人，跨模块责任冲突路由给相应 Decision Owner，证据不足路由给独立验证者。通知必须同时携带问题、证据、影响、选项和推荐理由，不能只发一句“请审批”。

组织还要观察 HITL 的队列时间和重复类型。大量相同低风险问题反复找人，说明规则或工具能力没有产品化；重大风险从不触发 HITL，则更可能是漏报而不是成熟。

22.4 工具从 Copilot 孤岛走向最小工程系统

AI 原生工具体系至少要形成五种相互连接的能力：

- 1. **Context**：从图谱、契约、约束、代码和证据裁剪可信上下文；
- 2. **Orchestration**：用 Work Graph 管理节点、依赖、并行、Handoff 和停止；
- 3. **Harness**：落实权限、沙箱、门禁、预算、观测和恢复；
- 4. **Evidence**：按任务和 AC 聚合真实、当前、可追溯的完成事实；
- 5. **Telemetry**：把多个任务信号聚合为价值与效能慢外环。

它们不要求采购一套庞大平台。仓库中的 Markdown/YAML、现有 Git 和 CI/CD、结构化脚本与静态 Dashboard 就能跑通最小系统；只有当真实使用暴露复用、权限、规模或运营瓶颈时，才把能力产品化。

22.4.1 六层工具架构：能力分层，事实贯通

工具层	主要职责	不应承担的责任
AI 交互与宿主层	对话、搜索、编辑、Agent 执行与人工交互	自行定义签署、约束和完成语义
组织语义层	Intent Graph、Contract、Constraint、Evidence 的稳定 ID、状态与 Schema	绑定某个模型的专有 Prompt 格式
运行控制层	Context、Work Graph、Tools Manifest、Harness、五道门和恢复	替人作价值取舍和例外批准
验证交付层	测试、Verification Plan、CI/CD、Release Manifest、制品与回滚	用流水线绿色代替责任裁决
事件与遥测层	追加式事实、Measurement Contract、单任务与聚合 Dashboard	把 UNKNOWN 填成 0 或直接排名个人
企业集成层	身份、权限、代码托管、工单、知识、制品、安全与审计系统	建设另一套无法追溯的平行事实源

分层的价值不是增加平台数量，而是防止职责混乱。AI 宿主可以替换，组织语义不能随之丢失；CI 可以执行检查，不能签署业务风险；Dashboard 可以暴露趋势，不能自动扩大自治范围。

22.4.2 先连接现有系统，再决定是否建平台

大多数企业已经拥有代码仓库、CI/CD、身份权限、测试平台、制品库、工单和监控。第一步应把 Contract ID、Evidence ID、Commit、Artifact Digest 和 Release ID 穿过这些系统，而不是复制一套“AI 专用研发平台”。

只有出现以下瓶颈时，平台化才有明确理由：多个团队无法复用约束和验证器；权限与审计无法统一；跨工具状态频繁丢失；Evidence 聚合成本过高；指标口径无法治理；治理资产缺少 Owner、版本和灰度。平台建设必须对应已观测问题，并有停用旧路径的迁移计划。

22.4.3 Build、Buy 与复用的决策原则

- **复用现有系统**：身份、Git、CI/CD、制品、安全扫描和监控通常优先复用；
- **购买通用能力**：模型访问、IDE Agent、可观测和安全产品可按企业要求采购，但必须接入稳定协议；
- **自建差异能力**：组织特有的意图语义、约束策略、证据映射和度量契约往往需要内部所有权；
- **开源与可替换**：对关键运行路径保留数据导出、Schema 和替换能力，避免治理事实被供应商锁定。

决策标准不是功能清单最长，而是能否在目标风险下提供可验证控制、可持续运营和合理总成本。

22.5 跨 AI 工具是架构要求，不是兼容性加分项

企业不会永久只使用一个模型、IDE 或 Agent 宿主。组织资产必须独立于具体工具：契约 ID、状态、约束语义、Evidence Schema、事件账本和门禁退出码保持稳定，Codex、Claude、WorkBuddy、IDE Agent 或内部平台只负责适配这些协议。

跨工具不等于最低公分母。不同宿主可以发挥各自搜索、执行、并行和交互能力，但不能拥有不同的签署规则、成功定义或证据真相。宿主工具与 Token 数据客户端也应解耦，避免从哪个工具启动就只能度量哪个工具。

22.5.1 稳定协议至少包含什么

跨工具交接至少要保留：任务与契约版本、签署状态、Change Envelope、允许工具、节点输入输出、AC 与约束结果、Evidence 引用、UNKNOWN、裁决状态和追加式事件。自然语言总结可以附加，不能成为唯一交接载体。

22.5.2 能力探测与降级必须诚实

不同工具拥有不同的终端、浏览器、多 Agent、MCP、上下文窗口和用量数据能力。适配器应先探测能力，再选择路径；缺失功能时保留 UNKNOWN、改用受控替代或升级人工，不能为了展示“全工具兼容”而静默跳过验证。跨工具的最低要求不是功能完全相同，而是责任、边界和证据不降级。

22.6 流程与工具共同裁剪

轻量任务可以使用最小图谱索引、单契约、核心约束和一份 Evidence；高风险跨系统任务需要 Verification Plan、职责分离、多层验证、Release Manifest 和生产观察。裁剪依据是影响、可逆性、数据敏感度和验证能力，而不是团队是否愿意填写更多文件。

任何轻量化都不能删除四件事：目标有人签署、边界可以执行、完成能够证明、异常有人裁决。任何平台化也不能把 UNKNOWN 自动补成绿色。

风险 Profile	流程形态	最小工具控制	人类责任点
轻量、可逆	单责任单元，短闭环，少量门禁	最小契约、核心约束、真实测试、Evidence	意图签署与异常裁决
标准业务变更	Work Graph、P⇌E、独立 V、灰度	四大动态工件、权限围栏、多层验证、Telemetry	IO、OA、发布责任人
高风险或不可逆	职责分离、双人复核、独立环境和演练	Verification Plan、强约束、完整谱系、Release Manifest、恢复验证	领域、安全、合规及有资格批准者

裁剪必须同时改变流程与工具。流程要求双人复核，工具却允许单一 Agent 持有生产凭证，控制并不存在；工具设置大量门禁，流程却没有异常责任人，失败只会堆积成队列。

22.7 谁运营这套流程和工具

AI 原生价值流不是安装后自动运行的系统，需要清晰的产品与运营责任：

运营对象	主要 Owner	持续职责
意图与任务类型模板	IO 社群 / 产品负责人	校准业务语义、Non-goals 和价值证据
Context 与知识来源	OA / 领域知识 Owner	来源、新鲜度、权限、失效和回写
约束、工具权限与门禁	OA + 安全/架构/数据责任人	规则版本、例外、误报、恢复与审计
Work Graph 与 Agent 路由	OA / 平台工程	节点能力、并发、预算、Handoff 和停止条件
验证器与 Evidence Schema	测试/质量工程	独立性、覆盖、证据新鲜度和兼容性
Telemetry 与 Dashboard	研发效能 + 数据 Owner	Measurement Contract、数据质量、访问和行动闭环
AI 宿主与模型组合	平台/采购/安全	能力、成本、数据边界、替换和退出策略

Owner 不是审批所有任务，而是维护默认路径，使大多数低风险任务无需临时协调。治理资产应像内部产品一样拥有版本、使用数据、问题反馈、灰度和淘汰机制。

22.8 费用报销规则：一条价值流怎样被真正重构

旧流程中，“超过 5000 元增加总监审批”可能经历产品写需求、架构评审、开发排期、编码、QA 补测试、安全检查、发布审批和人工汇报。每个环节都完成自己的文档，但金额口径、外币、代理总监和回退条件可能直到测试或生产才暴露。

重构后的流程不是把八个岗位替换成八个 Agent：

1. IO 与 OA 在意图进入队列时完成 Grill-Me 和 Example Mapping，关键决定进入完整契约并由 IO 签署；
2. Constraint Matrix 和 Tools Manifest 预先限定数据、文件、权限、预算和生产禁区；
3. Work Graph 释放边界测试、影响分析和回退准备，只有可信 RED 后才释放实现节点；
4. AS 在 $P \rightleftharpoons E$ 中完成有限修复，代理总监规则冲突触发领域 HITL，而不是继续猜测；
5. Evidence Bundle 按 AC、约束、提交和回退聚合事实，V 独立给出灰度、补证或拒绝建议；
6. 有权者决定两个部门灰度，Release Manifest 绑定制品和恢复路径；
7. 单任务信号进入 Telemetry，多个同类任务后再决定是否调整 Context、约束、编排或自治范围。

流程变化的价值体现在：关键决策前移，测试和恢复准备并行，普通偏差在快内环收敛，人工只处理真正的业务冲突，完成与发布都能追溯到同一意图。工具变化的价值则是让这些规则跨 AI 宿主、会话和流水线持续成立。

22.9 常见反模式

- **旧流程自动化**：把原来的工单、评审和审批全部做成机器人，却不减少批量与交接；
- **Agent 数量崇拜**：用更多 Agent 模拟更多部门，协调成本反而上升；
- **平台先行**：没有真实价值流和基线，先建设统一门户、知识库和指标中心；

- **宿主锁定**：契约、记忆、证据和用量只能存在某个 AI 工具的私有空间；
- **工具即治理**：购买安全或观测产品后，仍没有 Decision Owner 和异常路由；
- **门禁堆叠**：所有任务使用同样重的检查，导致团队绕过正式路径；
- **数据假闭环**：Dashboard 指标丰富，却不能下钻到契约和 Evidence，也不触发行动；
- **影子流程并存**：新旧流程长期双轨，正式状态与实际执行逐渐分离；
- **自动化例外**：为了不中断 Agent，把权限扩大、UNKNOWN 压平或让模型自行批准风险。

这些反模式的共同点是保留旧系统的责任与等待结构，只在表面增加 AI 和自动化。

22.10 如何判断流程工具重构是否成功

不要只统计 AI 生成代码占比。更有意义的信号包括：端到端等待是否减少，首次成功是否提高，证据整理是否自动化，HITL 是否集中在真正需要人的事项，跨工具交接是否保持语义，失败是否可停止和恢复，以及单位有效交付的总成本是否下降。

结果维度	建议信号	必须联读的护栏
流动	端到端周期、等待占比、交接数、决策时延	逃逸缺陷与下游人工不能恶化
质量	目标准确率、首次成功率、证据完整度	不得通过弱化 AC 和测试获得改善
自治	自动收敛轮次、HITL 类型、停止与恢复成功率	必要 HITL 不应被压低
工具	跨宿主成功率、上下文新鲜度、门禁可用性	失败必须显式，不得静默降级
成本	人工时间、Token、平台和证明成本	观察总成本，不只看编码成本
学习	同类失败复发率、资产复用率、规则淘汰率	防止知识和约束无限膨胀

改造前后必须比较相同任务类型和风险等级。一个低风险演示任务不能证明核心交易流程提效；一个成功样本也不能证明组织能力。流程与工具重构只有在连续任务中减少等待和返工，同时保持质量、责任与恢复能力，才算完成第一轮验证。

22.11 本章小结

流程与工具重构不是把 AI 插入旧节点，也不是把旧流程逐项机器人化，而是把价值流改造成意图驱动、边界可执行、事件触发、反馈高频、证据裁决和遥测学习的持续控制系统。流程决定责任、状态和回流，工具把这些决定变成跨宿主、跨会话、跨流水线仍不能被静默跳过的运行事实。

本章最小实践：选择一条真实研发价值流，测量活动、等待、交接、决策和证明成本；删除一项纯状态交接，把一项审批改为“预授权规则 + 事件触发 HITL”，并画出六层最小工具架构。

完成证据：当前/目标价值流图、控制状态与事件清单、Owner 矩阵、六层工具架构、风险 Profile，以及带护栏的提效假设。

延伸实践：推荐学习《Agentic Agile 组织转型进阶：构建人机协同研发操作系统》，把旧流程、工具和审批链重构为 AI 原生价值流。

第二十三章 | 四维协同落地：转型画布、成熟度与 90 天首轮验证

第四部分前两章分别展开人员/组织与流程/工具，但企业落地不能把四个维度拆成四项孤立计划。四维转型画布把它们绑定到同一条价值流、同一个业务结果和同一组证据；成熟度与 90 天路线则用于判断改变能否重复、能否扩展，

以及是否值得继续投入。

23.0 三层转型画布

四维画布还需要有纵向结构：战略意图层说明为什么转型、价值网络层说明哪条价值流和哪些责任单元被重构、任务运行层说明 AI 如何在边界内完成真实工作。横向四维回答“改什么”，纵向三层回答“在哪一层发生、用什么证据判断”。

90 天不是等待组织“慢慢适应”的长周期承诺，而是三个连续决策窗口：首周跑通一个真实任务闭环，30 天在同一价值流重复并修正，60 天迁移到相邻责任单元，90 天根据遥测和业务结果决定扩展、收缩或停止。若首周连任务证据都没有，后面的组织规模化没有依据。

23.1 四维转型画布：先绑定同一条价值流

画布层	必须回答的问题
业务结果与目标价值流	客户或经营结果是什么，价值从哪里开始、在哪里兑现？
基线与提效假设	当前周期、等待、返工和成本在哪里，AI 必须改善什么？
人员	哪些任务交给 Agent，哪些判断留给人，谁承担最终责任？
组织	哪些决策权前移，哪些交接消失，异常由谁裁决？
流程	哪些节点删除、自动化或改为 SCOPE-V 事件与门禁？
工具	最小 Context、Orchestration、Harness、Evidence、Telemetry 能力是什么？
阶段证据	课堂/首周、30 天、60 天和 90 天分别用什么事实决定继续或停止？

画布必须从真实价值流和基线开始，不能先讨论“采购什么 Agent 平台”。四个维度共享同一个提效假设：只要其中一维保持旧运行方式，其他维度的提升就可能被等待、返工、权限或责任真空抵消。

23.1.1 画布不是四张清单

第二十一章已经完成任务与责任迁移，第二十二章已经完成价值流与工具设计。画布在这里承担的是一致性检查：人员维度交给 Agent 的任务，必须在流程维度拥有运行位置，在工具维度拥有权限和证据，在组织维度拥有 Decision Owner；任何一项无法闭环，都应回到前两章重新设计，而不是在画布上写一句“加强协同”。

23.2 证据驱动的成熟度

成熟度不应只看“自动化率”或“人介入多少”。建议结合：

- 目标准确率；
- 首次成功率；
- MUST 通过率；
- 约束自愈率；
- HITL 是否发生在正确位置；
- Evidence Bundle 完整度；
- Bug 回溯真实性；
- 知识沉淀率；
- 多模块一致性；
- 价值和成本表现。

23.3 四级演进路径

没有企业能一键切换到完全自治。四级演进路径描述的不是采购了哪种工具，也不是有多少代码由 AI 生成，而是执行权在什么边界内交给 Agent、组织用什么证据证明这种授权仍然安全有效。

等级	名称	AI 的组织身份	主要治理方式	自治范围
L1	工具依附期（Copilot Agile）	个人外骨骼	人工目标、人工检查	单点辅助
L2	人机编队期（Hybrid Agentic Scrum）	可派发任务的虚拟同事	任务说明、检查清单、人工验收	边界清晰的低风险任务
L3	受控自治期（Constrained Autonomy）	在 Harness 内运行的自治执行单元	3-4-3、SCOPE-V、机械门禁、证据裁决	可验证、可恢复、风险受控的工作流
L4	硅基自治期（Silicon Autonomy）	意图驱动系统中的自治执行网络	遥测反馈、治理资产运营、持续审计	执行闭环默认自治，价值与不可逆风险仍由人负责

四级不是给整个企业贴一个标签。同一组织可能在测试生成上达到 L3，在生产发布上维持 L2，在业务价值判断上始终保留人类裁决。成熟度应按任务类型、风险等级和能力维度分别评估。

23.3.1 L1：工具依附期——AI 是个人外骨骼

L1 保持现有 Scrum、Kanban、瀑布或 DevOps 流程基本不变。AI 被用于代码补全、问答、文档草稿、测试草稿和局部重构，但工作仍以“人提出—人判断—人验收”为主。

典型运行画像：

- 提示词和个人经验决定输出质量；
- 同一个问题在不同工程师手中得到不同结果；
- AI 生成内容与任务、约束和验收证据之间缺少追溯；
- 安全、成本、质量数据散落在个人工具中；
- AI 出错后主要依靠人工 Review 和返工。

L1 并不是失败状态。对于低频、探索性或难以自动验证的工作，L1 可能就是合理终点。它的风险在于把个人效率提升误报成组织交付能力。

进入 L2 的证据不是“安装了 Agent 工具”，而是团队能够把一类边界清晰的任务显式派给 AI，定义期望结果，并稳定回收、验证和记录结果。

23.3.2 L2：人机编队期——AI 成为可管理的虚拟同事

L2 中，Agent 开始承担完整任务，而不只是生成片段。人类仍主导目标、优先级、风险判断和对外承诺，Agent 可以完成单元测试、简单 CRUD、报表、文档整理、代码迁移或重复性修复。

最小治理能力：

- 任务有明确目标、非目标和验收条件；
- 可说明哪些工具、数据和代码区域允许 Agent 使用；
- 结果进入现有 CI、测试和 Review 流程；
- 能记录任务耗时、失败、人工往返和实际 Token 成本；

- 发生越界、证据不足或目标冲突时，Agent 会停止并请求人类裁决。

此时可以弱化站会中的纯状态同步，但不能删除障碍暴露、价值判断和责任确认。工具能够汇总“做到了哪一步”，却不能替组织决定“这件事是否值得做、风险是否可以接受”。

L2 的常见假成熟是给 Agent 分配任务，却没有统一约束和独立证据；产出看似更快，QA、架构师和业务人员却承担了更多隐性验证成本。

进入 L3 的证据是连续一组真实任务已经把 IO/OA/AS 责任、四大动态工件、SCOPE-V 门禁和任务级遥测变成默认交付方式，而不是演示时才使用。

23.3.3 L3：受控自治期——3-4-3 接管可验证 workflows

L3 的核心变化不是“AI 写更多代码”，而是执行权从会议和个人提醒转移到**契约、约束、编排、证明与异常升级组成的控制系统**。

在这一阶段：

- IO 签署意图契约，对价值、非目标和风险取舍负责；
- OA 设计 Context、Work Graph、Harness、验证策略和恢复边界；
- AS 在最小权限下执行、测试、自检、自愈并生成 Handoff；
- 意图图谱、意图契约、约束矩阵和 Evidence Bundle 可相互追溯；
- CI/CD、接口测试、UI 测试、SIT、性能测试和 UAT 被纳入证据链；
- MUST 失败会机械阻断，无法收敛或不可逆操作会触发 HITL；
- 每份契约和全局 Dashboard 都能看见质量、成本、自治与价值趋势。

L3 不要求废弃 Scrum 或看板。只要关键执行和发布裁决已经由可验证治理闭环承担，原有节奏可以按需要保留或裁剪。

建议观察窗口：至少连续 10 个已完成任务，且包含正常任务、失败自愈、人工升级和回退样本。可参考但不可机械照搬的信号包括：

- 目标准确率和首次成功率趋势稳定，首次成功率可参考 $\geq 70\%$ ；
- MUST 约束无未闭环失败，任何例外都有签署和时效；
- Evidence Bundle 能覆盖关键 AC、约束、测试和发布结论；
- 运行自主性得分可参考 ≥ 60 ，且不是通过隐藏 HITL 获得；
- 无未关闭的高危安全项；
- 相较基线，验证总成本没有被转移给下游人员。

进入 L4 的证据不是“连续十次没有人介入”，而是组织已经能够根据真实遥测调整授权范围、路由、约束和治理资产，并能证明这些调整改善了结果。

23.3.4 L4：硅基自治期——执行闭环自治，价值与责任仍归人

L4 的重点是硅基自治：Agent 不再只是等待人类逐项派发任务，而是在已签署意图、可执行约束、授权范围和停止条件内，持续完成规划、执行、验证、恢复和经批准的进化。它向“意图即软件”演进，意图图谱与运行态、业务结果、发布证据、算力成本和风险信号持续关联；系统可以提出路由、约束、测试、知识或 Skill 的改进建议，并在批准后的范围内执行。

L4 的关键能力包括：

- 组织级价值与效能视图能够按契约、模块、项目和风险等级观察运行状态；
- 经验通过 Memory Write Gate 和 Evidence Bundle 晋升为知识、约束、测试或 Skill；
- 自治权限可以按遥测结果扩大，也可以在风险上升时自动收缩；

- 多团队共享治理资产具有所有者、版本、灰度、回退和淘汰机制；
- 人类 HITL 主要集中于商业策略、价值冲突、法律伦理和不可逆风险，而非默认逐任务返工；
- 发布和回滚路径均可证明，生产写权限仍由明确授权策略控制。

建议在包含生产相关或较高风险样本的连续窗口中观察：目标准确率是否接近或超过 90%、约束自愈率是否接近或超过 80%、自主性得分是否稳定在较高水平，以及低 HITL 连续段是否来自能力提升而非漏报。

L4 硅基自治不是“人类退出”，更不是系统可以自行修改目标和安全底线。它改变的是默认执行主体，而不是最终责任主体：Agent 可以自治执行，价值定义权、不可逆决策权、例外批准权和责任承担仍掌握在人类手中。

23.3.5 等级跃迁必须同时满足四类门槛

任何升级自治范围的决定，都应同时检查：

1. **能力门槛**：Agent 在目标任务上是否稳定完成，而非偶尔成功？
2. **验证门槛**：错误能否被独立测试、门禁和证据发现？
3. **恢复门槛**：失败能否停止、回退、重试或升级，不造成不可接受损失？
4. **责任门槛**：意图、约束、例外和发布分别由谁签署，是否可追责？

四者缺一，不应升级。能力强但无法验证，属于高风险黑箱；验证充分但无法回退，不适合扩大生产自治；责任模糊时，再好的技术指标也不能替代治理授权。

23.4 90 天首轮验证：不是等 90 天才跑通闭环

90 天不是交付一个项目所需的时间，更不是到第 60 天才跑通闭环。线下工作坊第二天就能完成一个真实仓库的可信增量；企业试点也应在首周完成同等闭环。90 天用于收集样本、验证重复性、迁移到第二场景并作出规模化决策。其目标是回答：**3-4-3 在本组织、此类任务和当前工程基础上，是否持续带来可证明的净收益？**

23.4.1 先写试点契约，而不是先安装工具

试点本身也要有一份 Intent Contract：

- **目标**：验证一种明确任务类型能否在不降低质量与安全的前提下，提高首次成功率、缩短交付时间或降低总验证成本；
- **非目标**：不证明所有项目都适合，不以裁员、代码生成量或“零 HITL”为成功标准；
- **范围**：一个团队、一个代码域、1—3 类重复出现且可验证的任务；
- **基线**：选取试点前 4—8 周同类任务，记录周期、缺陷、返工、人工投入、回滚和成本；
- **对照**：条件允许时保留同类常规交付样本；不能并行对照时，使用历史基线并标注差异；
- **假设**：例如“首次成功率提高 15 个百分点，同时逃逸缺陷不增加”；
- **护栏指标**：MUST 失败、高危安全事件、回滚率和 UAT 否决不得恶化；
- **退出条件**：出现重大越权、连续证据失真、总成本明显上升或业务 Owner 无法持续参与时暂停试点。

没有基线和退出条件，90 天结束时几乎任何结果都能被解释成“初步成功”。这不是实验，而是自我证明。

23.4.2 试点准入：选择能被证明的场景

首个试点宜同时满足：

- 任务边界相对清晰且有稳定业务 Owner；
- 至少具备基础 CI、自动化测试、静态检查、隔离环境和回滚路径；
- 仓库具备最基本的模块说明、依赖关系和权限边界；
- 能获得真实的质量、周期、人工投入和 Token 数据；

- 风险可以隔离，不直接从核心交易、生产数据库写入或高度敏感数据开始。

优先场景可以是内部工具、BFF、报表、自动化脚本、存量测试补强、可回退的小型重构。若选择既有大型系统，应从一个治理岛和下一次真实变更开始，而不是要求先理解百万行代码。

23.4.3 最小角色与工件

最小团队不必新设岗位，但三种责任必须可指认：IO 对目标和签署负责，OA 对控制系统和证据完整性负责，AS 负责受约束执行。

试点至少建立四大动态工件：

1. **意图图谱**：说明试点目标、任务关系、影响范围和回写结果；
2. **意图契约**：每个任务的目标、非目标、AC、风险与 IO 签署；
3. **约束矩阵**：权限、质量、安全、成本、环境和升级条件；
4. **Evidence Bundle**：汇聚 AC、约束、测试、风险、回退和裁决证据。

Work Graph、Execution Log、Loop Memory 和 Telemetry Dashboard 是支撑运行与反馈的基础设施。工件数量可以轻量，关键字段和责任链不能省略。

23.4.4 第1—7 天：完成真实闭环并在企业场景复现

线下课堂使用标准仓库或合格的企业自带仓库，在一天内完成角色重组、契约签署、可信 RED、真实实现、Evidence 与裁决。使用标准项目的小组必须在课后 7 天内把同一闭环迁移到企业场景；使用企业项目的小组则在 7 天内完成第二次真实任务。

关键动作：

- 锁定一条价值流、一个可回退切片和基线数据；
- 明确 IO/OA/AS、Decision Owner 与升级条件；
- 初始化四大动态工件和最小 Harness；
- 接入现有测试、CI 和沙箱，不重建一套平行流水线；
- 运行一次完整 SCOPE-V，形成真实代码、测试、Evidence、Telemetry 和 Dashboard；
- 主动设计至少一次失败注入，检查权限越界、MUST 失败和证据不足能否阻断；
- 在企业场景复现，记录与课堂标准项目的环境、数据和责任差异。

证据闸门：必须已有真实增量、真实测试、签署契约、Evidence Bundle 和 Telemetry Dashboard。无法形成这些事实时，不进入“稳定运行”阶段，而是先补工程基础。

23.4.5 第 8—30 天：稳定运行并验证可重复提效

第二阶段不是重新设计方法，而是在同一类价值流中连续运行，验证首周结果不是一次演示。样本量按任务频率决定，不机械追求数量，但必须覆盖正常成功、失败修复和至少一次 HITL。

关键动作：

- 稳定 Work Graph、Handoff、重试上限和 HITL 路由；
- 让接口、UI、SIT、性能和必要 UAT 证据回填同一契约；
- 记录首次成功、自愈、人工升级、返工、Token 与等待时间；
- 将试点组与历史基线或对照任务比较，识别成本是否被转移给 QA 和业务；
- 对高频失败做一次 LOOP-3 晋升，将稳定经验转为约束、测试、知识或 Skill；
- 审查至少一次回退、暂停或人工接管是否真实有效。

证据闸门：30 天时必须看到相同任务类型下可重复的提效，且质量、安全、下游人工和总成本没有恶化。速度提升不能抵消逃逸缺陷或成本转移。

23.4.6 第 31—60 天：跨场景扩展并验证方法可迁移

第三阶段把已经稳定的方法迁移到第二类任务、第二个模块或第二个责任单元，验证它能否跨场景复用。迁移不是复制全部模板，而是在保持责任、状态和证据语义的前提下重新裁剪参数。

关键动作：

- 选择风险或技术条件不同、但价值边界清晰的第二场景；
- 培养第二组 IO/OA/AS，避免成功依赖单一专家；
- 复用经过验证的约束、测试、Skill 和 Dashboard 口径，同时记录必要差异；
- 检查跨工具、跨模块 Handoff 与 Evidence 聚合是否仍可追溯；
- 比较两个场景的净收益、失败模式和治理维护成本；
- 对不能迁移的部分区分“场景差异”与“方法缺陷”。

证据闸门：60 天时必须证明方法至少在第二场景可迁移，或明确证明它只适用于原场景。不能迁移不是自动失败，但不能再宣称具有组织级可复用性。

23.4.7 第 61—90 天：形成组织化与规模化决策

最后阶段不继续堆功能，而是把已经验证的做法转成组织决策：哪些任务扩大自治，哪些保持受限，哪些先补基础，哪些停止。治理委员会查看原始证据和分群趋势，确定资产 Owner、授权边界、支持机制和下一轮投资。

建议以“效果指标 + 护栏指标 + 成本指标 + 学习指标”组成平衡记分卡：

类别	核心指标	要回答的问题
效果	目标准确率、首次成功率、端到端周期、吞吐	是否更快地交付了正确结果
护栏	MUST 失败、逃逸缺陷、回滚、高危安全事件、UAT 否决	是否把风险转移到了下游或生产
能力	自愈率、HITL 类型、验证覆盖、证据完整度	自治是否来自真实能力
成本	人工小时、Token、工具成本、等待和返工	总成本是否下降，而非只减少编码时间
学习	知识/约束/测试/Skill 晋升数及复用效果	一次经验能否改善下一次任务
价值	业务结果、采用率、风险损失避免	交付是否产生了值得支付的价值

每项指标都应同时保留任务类型、风险级别、样本数和时间窗。没有分母的“提升 80%”和没有风险分层的平均值都不适合作为自治升级依据。

证据闸门：90 天结束时决定的不是项目是否终于交付，而是哪些任务可以扩大到 L3、哪些仍需人类主导、哪些治理资产值得产品化，以及下一阶段必须补强哪项验证、恢复或组织能力。

23.4.8 四种试点结论

裁决	证据特征	下一步
扩大	净收益稳定，护栏未恶化，验证与恢复可靠	扩大同类任务，不立即跨风险域复制
受限继续	局部有效，但 HITL、成本或某项门禁不稳定	保持范围，针对短板再运行一个证据窗口
先补基础	AI 能执行，但测试、数据、知识或 CI/CD 无法支撑证明	暂停扩权，优先补工程基础设施
停止	风险、总成本或质量持续劣于基线	结束该场景试点，保留证据与反例

能够得出“此场景暂不适合”的结论，恰恰说明试点发挥了作用。真正危险的不是试点失败，而是在无法证明收益时仍凭热情扩大自治。

23.5 团队运行节奏

保留必要的人类节奏：

- 意图与优先级决策；
- 例外审查；
- Evidence Review；
- 遥测趋势评审；
- 重大事故复盘；
- 治理资产维护；
- 模型、工具和权限变更评审。

会议是否保留，不由“传统或先进”决定，而由它是否承担不可自动化的价值和责任判断决定。

23.6 Agentic Agile 治理委员会

治理委员会不审批每一行代码，而负责：

- 确认治理政策；
- 批准高风险例外；
- 审查跨团队证据；
- 处理职责冲突；
- 决定自治范围升级或收缩；
- 审查真实价值、质量、成本和风险趋势。

委员会的输入应该是证据和趋势，不是汇报型 PPT。

23.7 成熟度不是线性升级游戏

团队可能在测试自动化上达到 L3，在意图治理上仍处于 L1；也可能拥有完善约束，却没有真实价值遥测。成熟度应按能力维度呈现，而不是用一个总分掩盖短板。

建议至少评估：

维度	L1表现	更成熟表现
意图	口头需求	可签署契约和图谱追溯
边界	人工提醒	可执行约束和权限控制
编排	单 Agent 自由执行	Work Graph、Handoff、停止条件
验证	事后测试	TDD、多层验证和证据映射
恢复	人工救火	分类、回退、自愈和升级
遥测	手工汇报	单任务真实数据和趋势动作
组织	个人技巧	共享资产、Protocol 和价值与效能视图

23.8 不要追求零 HITL

HITL 既可能代表治理缺口，也可能代表系统正确识别了人类责任。要区分：

- 必要介入：价值取舍、不可逆操作、法律和伦理判断；
- 可减少介入：重复批准、信息缺失、工具配置问题；
- 异常介入：Agent 越权、证据矛盾、系统无法收敛。

成熟团队不是没有人类介入，而是把人类时间集中到机器不应决定的事项。

23.9 治理资产运营

治理资产需要产品化维护：

- 有所有者；
- 有版本和发布说明；
- 有使用数据和问题反馈；
- 有兼容与迁移策略；
- 有淘汰机制；
- 有贡献和审查流程。

如果模板、约束和 Skills 只是某次试点的文件副本，它们很快会过期。组织应把治理资产当作内部产品。

23.10 激励设计

错误激励会破坏真实遥测。如果团队以“自动化率最高”“HITL最少”“Token最低”作为单一目标，人们和 Agent 都可能绕过必要检查。

更健康的激励组合包括：

- 目标是否准确；
- 关键约束是否稳定；
- 失败是否如实记录；
- 是否减少重复缺陷；
- 是否沉淀可复用资产；
- 是否在相同风险下提高效率；
- 是否让价值决策更透明。

23.11 采纳阻力

团队可能担心流程变重、岗位被替代、数据被监控或专业判断失去价值。治理转型需要公开说明：

- 遥测评估系统，不用于简单排名个人；
- 人类价值判断不会被模型接管；
- 轻量任务使用轻量治理；
- 专业知识将转为更有影响力的约束和验证资产；
- 任何自治扩张都必须有证据支持。

23.12 治理委员会不是新的审批官僚

治理委员会的职责不是审阅每个任务，而是管理组织级规则：确定风险分级、批准高影响约束、处理跨团队冲突、复盘重大异常、决定哪些试点可以扩大。日常低风险任务应由预设规则和授权角色快速完成。

一个有效委员会看趋势和例外，不接管执行。它应减少重复争论，把一次裁决沉淀成可复用政策；如果所有任务都必须排队等待委员会，说明授权矩阵和机械门还没有建设好。

23.13 从一个英雄试点到第二责任单元

首个试点常依赖一位同时懂业务、技术和 3-4-3 的核心推动者。这可以加速起步，却不能证明组织能力。60 天跨场景阶段必须培养第二组 IO/OA/AS，让另一组人在不依赖原推动者逐步指挥的情况下，使用同一语义协议完成契约、执行、证据和裁决。

第二责任单元不是复制岗位，而是验证知识、约束、Skill、Dashboard 和授权矩阵能否被复用。若第二组只能照抄模板却无法处理异常，说明组织复制的是流程外观；若能根据新场景重新裁剪并保留责任与证据边界，能力才真正从个人经验进入组织系统。

23.14 变革沟通：从威胁叙事转向能力叙事

“AI 将彻底取代某个岗位”会制造防御，却不能帮助组织形成能力。更有效的沟通是说明责任如何变化：重复执行可以交给 Agent，人类把精力转向目标、边界、异常和判断；同时明确，任何高风险自治都需要更强证据和更清晰责任，而不是更少治理。

管理者应允许团队暴露失败，避免用单一速度指标奖励绕过门禁。否则成员会把治理工件当成汇报材料，把真实问题留在系统之外。

23.15 本章小结

四维转型不能拆成四个互不相干的项目。人员任务、组织决策权、流程控制和工具能力必须围绕同一条价值流共同改变，并接受同一组业务与护栏证据检验。课堂或首周跑通真实闭环，30 天验证可重复提效，60 天验证跨场景迁移，90 天形成规模化决策；90 天不是交付周期，而是首轮组织能力验证窗口。

本章最小实践：为一条真实价值流填写四维转型画布，评估 L1-L4 当前水平，并设计课堂/首周、30 天、60 天和 90 天四个证据闸门。

完成证据：四维转型画布、成熟度画像、基线与提效假设、护栏指标、阶段证据和最终四类裁决规则。

延伸实践：推荐学习《Agentic Agile 组织转型进阶：构建人机协同研发操作系统》，把四维重构、任务证据和 90 天行动放入同一张转型画布；线下工作坊进一步完成企业首轮扩展判断。

第二十四章 | 风险与行业裁剪：操作系统相同，参数与责任边界不同

24.1 不建立一套适用于所有行业的阈值

金融、车载、政企、互联网和内部工具的风险不同。图书不能替代法律、监管、安全和行业专家，也不应给出未经验证的统一留存年限、自治上限或发布阈值。

3-4-3 提供的是裁剪框架：识别风险，建立约束，定义证据，把不可自行决定的事项交给正确的人。

这里需要区分两个经常被混淆的概念：**治理原则可以复用，治理参数不能照抄**。无论在哪个行业，目标都要有人负责，权限都要有边界，关键行为都要留下证据，不可逆操作都不能由 Agent 自行批准；但什么数据属于敏感数据、什么变化需要双人复核、证据保存多久、灰度比例多大、谁有资格签署，必须由所在组织依据业务影响、适用法规和工程能力确定。

行业裁剪也不是简单地“删掉几份模板”。真正的裁剪，是回答四个问题：哪些风险在当前场景中真实存在，哪些控制可以机械执行，哪些证据足以支持下一步行动，哪些剩余风险必须交给具备责任资格的人。轻量化可以减少工件数量和审批层级，但不能删除责任、边界、验证和恢复。

因此，同一个“生成代码”任务在不同场景中可能对应完全不同的治理重量。内部演示工具可能只需要轻量契约、沙箱和自动测试；涉及客户资金的规则变更则可能需要职责分离、独立对账、合规审阅、灰度窗口与可追溯发布。差异不来自模型名称，而来自失败后果和组织承担责任的方式。

24.2 六个裁剪问题

- 1. 任务会接触什么数据？
- 2. Agent 可以使用哪些工具和权限？
- 3. 哪些行为不可逆？
- 4. 哪些失败会造成安全、资金或合规影响？
- 5. 哪些证据才能支持发布？
- 6. 哪些决策必须由具备责任资格的人批准？

六个风险问题确定治理重量，四维检查则保证裁剪能够进入企业运行方式：

转型维度	行业裁剪必须确认
人员	哪些判断需要执业资格、双人复核或领域专家，哪些执行可交给 Agent？
组织	最终责任主体、Decision Owner、例外批准和事故指挥权属于谁？
流程	哪些状态必须独立验证、监管报送、留痕、灰度或人工接管？
工具	数据驻留、模型路由、权限、证据保全、供应链和恢复能力需要什么等级？

只提高扫描强度而不改变责任与流程，不是行业裁剪；只增加审批而没有可执行工具控制，也不是。相同的 3-4-3 操作系统可以复用，但参数、证据门槛、自治范围和责任资格必须由具体场景决定。

这六个问题不应只在项目启动会上讨论一次，而应被转成可追踪的“风险裁剪卡”，分别写入治理工件：

裁剪问题	应落入的治理资产	最低可验证结果
数据是什么	数据分级、Tools Manifest、模型路由策略	未批准数据无法离开允许边界
工具与权限是什么	工具白名单、最小权限、审批策略	越权调用被阻断并留痕
哪些动作不可逆	Work Graph、停止条件、HITL 节点	不可逆步骤必须等待责任人批准
失败影响是什么	风险记录、威胁模型、故障模式	每个高影响失败都有预防、发现与恢复控制
什么证据足够	AC、验证计划、Evidence Bundle	发布结论可以回溯到独立证据
谁承担责任	RACI、签署矩阵、例外政策	Agent 不能替代具备资格的责任主体

裁剪过程可以按“场景事实 → 风险等级 → 治理 Profile → 增量约束 → 证据要求 → 人类责任人”展开。若信息不足，应保留 UNKNOWN 并提高治理重量，而不是为了快速启动把未知项默认解释为低风险。随着证据增加，治理可以升级；任何降级都应有明确理由、批准人和可撤销条件。

24.3 金融场景

可能需要强化：

- 数据最小化和脱敏；
- 资金与权限双重审批；
- 规则版本和审计链；
- 确定性计算与对账；
- 更严格的回退和发布窗口；
- 法务、风险和合规签署。

示例阈值必须由具体机构依据适用监管和风险政策确定。

例如，Agent 辅助修改授信规则时，真正的治理对象不是“生成了一段规则代码”，而是这段变化是否会影响客户资格、额度、定价或公平性。Intent Contract 应明确允许调整的规则范围和禁止触碰的政策；Constraint Matrix 应把金额边界、职责分离、数据用途和审批责任写成 MUST；验证至少包含历史样本回放、边界值、反例、差异分析与独立对账。Evidence Bundle 不能只展示单元测试通过，还应说明影响了哪些客群、是否出现异常分布、使用了哪个规则版本，以及谁批准进入下一阶段。

对于支付、清算和账务场景，系统还要回答“正确一次”之外的问题：重复执行是否幂等，部分失败如何补偿，账实不符怎样停止，重试会不会重复扣款，人工修复是否留下双向审计链。Agent 可以发现异常、生成候选修复和组织证据，但资金动作、关键参数和例外放行仍应由授权角色裁决。

24.4 车载与安全关键系统

可能需要强化：

- 安全等级与系统边界；
- 仿真、台架和实车分层验证；
- 时序、不变量和故障模式；
- 供应链和软件物料清单；
- OTA 灰度、熔断和恢复；

- 功能安全与网络安全责任人审批。

Agent 可以辅助生成和验证，但不能绕过安全生命周期要求。

在这类系统中，代码通过通常只是验证链的起点。需求、安全目标、系统边界、接口假设、故障模式、测试环境和发布对象必须可追溯。Agent 生成的测试也需要证明其覆盖了安全不变量，而不是只覆盖正常路径。仿真通过不能自动替代台架验证，台架通过也不能自动推导实车安全；每一层证据只能支持其适用范围内的结论。

一个典型的裁剪结果可能是：开发环境允许 Agent 自动修改并运行仿真；进入安全相关分支前必须锁定依赖和配置；台架、道路与 OTA 阶段设置独立 HITL；任何安全约束变化都触发重新评估影响面。发布证据还应包含软件物料清单、工具链版本、已知偏差、回滚条件和车辆批次范围，避免把一次实验成功扩张成全量安全结论。

24.5 政企与涉敏环境

可能需要强化：

- 网络和模型部署边界；
- 数据分级分类；
- 工具和外部连接白名单；
- 国产化环境兼容；
- 审批、留痕和证据保全；
- 涉密场景的人类责任链。

政企场景常见的误区，是只关注“模型是否私有化部署”，却忽略插件、MCP、日志、遥测、缓存、向量库和运维通道同样可能形成数据出口。真正的边界应覆盖完整数据流：数据从哪里进入，经过哪些 Agent 和工具，被哪些中间系统保存，谁能够检索，何时删除，发生异常时怎样证明没有继续扩散。

在隔离网络或涉敏环境中，Tools Manifest 应采用显式允许而不是默认开放；依赖、模型和 Skill 更新需要离线校验与版本固定；Evidence Bundle 应避免复制敏感正文，而用摘要、哈希、分级标识和受控引用证明结果。对于法定审批、公共资源分配和行政决定，Agent 可以提供检索、比对与建议，但最终决定必须保留清晰的人类责任链、理由和申诉入口。

24.6 互联网与高频业务

高频交付不等于低合规。互联网产品同样涉及隐私、账号、支付、内容和用户权益。

可以重点设计：

- 小流量灰度；
- 业务漏斗遥测；
- 自动回退；
- 用户影响边界；
- 大促资源预算；
- 实验伦理与隐私保护。

速度可以压缩范围和反馈周期，不能压穿底线。

互联网场景更适合把治理做成“快速反馈的控制回路”，而不是在发布前堆积一次性审批。团队可以用功能开关、用户分群、流量上限、错误预算和自动回退，把一个大风险拆成多个可观察、可停止的小步骤。但灰度本身不是免责机制：若实验涉及未成年人、敏感画像、价格歧视、内容安全或难以撤回的用户权益，即使只影响 1% 用户，也可能是高风险任务。

例如，推荐算法或运营文案由 Agent 高频迭代时，Evidence Bundle 除技术指标外，还应包含目标人群、实验假设、负向指标、投诉与伤害信号、停止阈值和实验后处置。只看点击率上升，可能会奖励夸张内容、成瘾设计或对弱势群体不公平的策略。价值遥测必须与护栏指标一起解释，不能把短期增长自动等同于业务正确。

24.7 医疗与生命科学

医疗场景需要先区分科研辅助、行政支持、临床决策支持和直接诊疗控制。四类任务看似都在“处理医疗信息”，但责任、证据和准入要求完全不同。越接近患者诊疗、剂量、诊断或生命支持，越不能用通用软件测试替代专业验证。

可能需要强化：患者数据最小化与用途限定；数据集代表性和偏差分析；模型、Prompt、知识库与指南版本追踪；临床专家复核；适用人群和禁用场景；错误建议的升级与纠正；对患者可理解的说明。Agent 可以汇总病历或提示潜在冲突，但不能掩盖不确定性，也不能把“模型高置信度”当作临床证据。

24.8 工业、能源与生产控制

工业场景需要区分办公建议、生产排程、设备参数优化和直接闭环控制。Agent 越接近物理设备，验证就越需要覆盖实时性、环境边界、故障安全、人工接管和网络隔离。

一个稳妥的渐进路径是“只读观察 → 离线建议 → 人工确认执行 → 受限自动执行”，每一级都用真实运行证据决定是否扩大自治。数字孪生和历史回放可以降低试验成本，但不能证明所有现场条件；进入生产控制前，还需要明确参数上下限、联锁保护、异常停机、手动接管和恢复演练。

24.9 风险分层与治理重量

风险	推荐治理
低风险、可逆、内部使用	轻量契约、核心约束、自动验证
中风险、影响用户、可灰度	完整 Evidence Bundle、灰度、遥测和回退
高风险、不可逆、资金或敏感数据	强约束、独立验证、人工批准和审计
安全关键或监管关键	行业生命周期、专家签署、隔离环境和正式合规证据

风险层级不是给整个项目贴一个永久标签。更准确的做法是按任务、数据、工具和动作分别判断，并取其中足以决定控制强度的高风险项。同一产品中，“生成测试数据”可能是低风险，“修改生产权限”是高风险，“自动执行资金或物理控制”则可能属于监管或安全关键。若只按项目平均风险治理，高风险动作很容易被低风险背景稀释。

治理 Profile 也不是成熟度荣誉。选择 high-risk、legacy 或 multi-module，并不代表团队落后，而是说明当前失败后果、未知项或协作复杂度需要更多控制。只有当历史证据证明特定控制可以被更可靠的自动机制替代，组织才应考虑减轻人工步骤；减少文档不是目标，降低剩余风险和决策成本才是目标。

24.10 Agentic 风险的七类来源

意图风险

目标模糊、激励错误、非目标缺失、规则冲突。主要通过批判性思维、Grill-Me、契约和签署治理。

上下文风险

信息过期、来源冲突、Prompt Injection、Context Poisoning 和不可信内容获得过高指令权。主要通过分层上文、来源优先级、内容隔离和指令信任治理。

数据与隐私风险

源代码、设计、客户数据或身份信息被发送到未批准模型、地区、日志、缓存、向量库、Memory、MCP 或第三方工具；供应商训练、留存、删除和子处理边界不清；Telemetry 与 Evidence 过度复制敏感正文。主要通过数据分级、目的限定、最小化、模型路由、DLP、合同条款、Tools Manifest、保留删除策略与负向验证治理。具体工程控制见第九章 9.7。

工具风险

权限过大、任意命令、生产访问、供应链工具不可信。主要通过 Tools Manifest、沙箱、批准和审计治理。

执行风险

死循环、资源争用、重试风暴、跨模块漂移。主要通过 Work Graph、预算、超时、熔断和 Protocol 治理。

验证风险

测试确认偏差、Judge 未校准、证据过期、UNKNOWN 被隐藏。主要通过独立验证、证据映射、一致性和时效检查治理。

组织风险

职责模糊、OA代签、指标游戏、例外泛滥、事故数据美化。主要通过角色责任、机械门、真实遥测和治理委员会治理。

七类风险往往不是独立发生，而是形成链式放大。一个过期的外部页面首先是上下文风险；其中隐藏的指令诱导 Agent 调用高权限工具，转化为工具风险；自动重试扩大影响，成为执行风险；测试只验证正常返回，又形成验证风险；团队以“无人干预率”奖励 Agent 不上报异常，最终演变为组织风险。行业裁剪因此不能只列单点控制，还要检查风险如何跨层传播。

建议为高风险任务记录“风险链”：触发源、传播路径、放大条件、发现信号、阻断点、恢复动作和责任人。它能帮助 OA 判断控制应放在哪里——不是在每一层重复审批，而是在最早、最可靠且最便于验证的位置切断传播。

24.11 威胁建模

每个高风险任务可以按以下路径分析：

资产是什么

→ 谁或什么 Agent 可以访问

→ 可能发生什么错误或攻击

→ 哪条约束负责预防

→ 哪项观测负责发现

→ 哪个动作负责阻断

→ 怎样恢复

→ 谁承担最终责任

威胁建模不只针对恶意攻击，也包括模型误解、错误工具调用和组织性绕过。

威胁模型至少应产出一张可执行映射，而不是停留在头脑风暴：

资产或目标	威胁/失效方式	预防约束	发现证据	阻断与恢复	责任人
客户敏感数据	被送往未批准模型或日志	模型路由、DLP、最小权限	出站审计、数据流检查	断开连接、撤销令牌、删除与通报	数据责任人
生产配置	Agent 误改或批量扩散	只读默认、审批、变更范围	配置差异、异常指标	熔断、回滚、隔离批次	系统责任人
发布结论	证据不完整却被判定通过	AC 映射、独立验证、时效门	Evidence 审计、UNKNOWN 列表	阻断发布、补证、重新裁决	IO / 验证责任人

威胁建模完成后，应把结果分别回写到 Constraint Matrix、Tools Manifest、Work Graph、验证计划和事故预案。只保留一份孤立的风险表，不能改变 Agent 的实际行为。

24.12 供应链风险

Agent 可能自动安装依赖、调用 MCP Server、下载脚本或使用外部 Skill。团队需要检查：

- 来源和许可证；
- 版本锁定；
- 权限需求；
- 网络访问；
- 是否读取敏感上下文；
- 更新是否会改变行为；
- 是否能够离线或降级运行；
- 产生的证据是否可信。

开源不自动等于安全，可运行也不等于可授权。

Agentic 供应链还包括 Prompt、Skill、Agent 配置、MCP Server、模型版本、Judge、数据集和自动下载脚本。它们未必进入传统软件物料清单，却可能直接改变系统决策。组织需要维护“Agentic BOM”：组件来源、版本摘要、能力、所需权限、数据访问、更新策略、验证状态和责任人。

升级不能只验证“是否还能运行”，还应进行行为差异测试：工具权限是否扩大，默认模型是否变化，Prompt 是否改变决策边界，Judge 是否产生系统性偏差，遥测是否新增数据出口。关键组件应支持固定版本、可信摘要、隔离验证和快速回退；无法说明来源或权限的组件，不应仅因效果演示良好就进入生产 Harness。

24.13 事故响应

Agentic 事故响应至少包括：

1. 停止相关 Work Graph；
2. 撤销工具和数据权限；
3. 隔离受影响产物；
4. 保存契约、上下文、工具调用和证据；
5. 判断是否需要回滚；
6. 通知责任人；
7. 修正 Evidence Bundle 和遥测；
8. 运行 Bug 回溯；
9. 更新约束、图谱和治理资产。

事故不能只归因于“模型幻觉”。需要沿治理链寻找为什么错误能够发生、扩散并被放行。

组织可以按影响范围和可逆性定义事故等级，但分级标准必须由本组织确定。低等级事件可以由预设 Runbook 自动隔离并通知；涉及敏感数据、资金、用户权益、安全控制或大范围生产影响的事件，应立即触发责任人、法务、安全或行业专家参与。事故期间首先保护人、数据和系统，其次保护证据，最后才是恢复交付速度。

恢复服务也不等于事故关闭。关闭前至少要回答：受影响范围是否确认，错误产物是否全部隔离，权限是否恢复到最小状态，客户或监管通知是否完成，原 Evidence Bundle 的哪些结论已失效，新的防复发控制是否经过真实测试。定期演练比只写预案更可靠，尤其要演练 Agent 无法停止、权限撤销失败、证据不完整和回滚本身造成二次影响等困难情形。

24.14 未来演进：作为假设，而不是承诺

更强模型、更长上下文、自适应路由、机器可读政策和动态软件可能继续改变研发。但以下原则不会因为技术进步自动消失：

- 目标需要责任主体；
- 权限需要边界；
- 不可逆操作需要批准；
- 证据需要独立性；
- 系统需要可观测、可停止、可恢复；
- 法律与伦理责任不能交给概率模型。

未来可以更自治，但必须先更可验证。

24.15 本章小结

行业不同，约束和证据强度不同；治理原则保持一致。真正可靠的行业适配不是套用一组神奇阈值，而是由了解业务、技术和监管的人共同签署一套可执行边界。

裁剪后的结果应当能够回答一条完整链路：当前场景有哪些事实与 UNKNOWN，为什么选择这一治理 Profile，新增了哪些行业约束，需要什么独立证据，哪些动作必须 HITL，失败后如何停止和恢复，最终由谁承担责任。如果这些问题仍只能靠会议口头解释，说明行业要求还没有真正进入 Harness。

本章最小实践：选择一个真实行业任务，完成六个风险问题和人员、组织、流程、工具四维裁剪，并沿“事实 → 风险 → Profile → 约束 → 证据 → HITL → 恢复”形成一页风险裁剪卡。

完成证据：行业约束增量、四维责任与能力差异、风险链、威胁模型映射、最低证据集、HITL 节点和恢复条件。

延伸实践：推荐学习《Agentic Agile FDE 进阶：从客户现场到企业级生产交付》，把本章的风险裁剪、行业边界和恢复机制迁移到复杂客户环境。

第五部分 | 附录与行动入口

前四部分完成了从问题、架构、控制回路到组织落地的完整论述，第五部分则把这些内容收束为可以随时查阅和立即使用的行动入口。它不是与正文脱节的资料堆积，而是本书的“操作台”：当读者需要回顾价值观、选择模板、定位课程、查找术语或启动第一个任务时，可以从这里快速进入。

本部分首先给出 Agentic Agile 宣言与开源治理资产速查，帮助读者从原则直接找到模板、脚本和 Dashboard；随后提供图书与四门新版 Portal 课程的学习地图，说明诊断、真实任务、组织重构和 FDE 现场实践之间的关系；术语表、五道门、贯穿案例和常见质疑，则分别承担概念对齐、机械检查、端到端示范与异议澄清的作用。

最后的四大动态工件实战指南面向真正准备动手的读者：可以从一个真实但边界可控的任务开始，建立意图图谱与意图契约，配置约束，运行 SCOPE-V，生成 Evidence Bundle，采集契约级遥测并观察全局 Dashboard。附录不求按顺序阅读，可以按照“理解概念—查找资产—完成闭环—进入课程或组织试点”的需要反复使用。

本书最终希望留下的，不是一套只能被复述的概念，而是一种可以被验证的行动方式：先让意图可判定，再让边界可执行；先让执行产生证据，再让人和系统基于证据作出裁决；最后用遥测和真实结果决定哪些方法值得保留、扩展或修正。

附录 A | Agentic Agile 宣言

我们正在通过构建、运行和治理智能体研发系统，探索更适合 AI 时代的软件交付方式。通过实践，我们更加重视：

1. 意图锚定，胜过详尽规约；
2. 多模态涌现协作，胜过仅依赖跨职能人类团队；
3. 算法规则与架构韧性，胜过仅依赖流程纪律与仪式；
4. 实时遥测与闭环对抗，胜过仅依赖阶段性复盘；

5. 基于验证的系统工程，胜过随性提示与氛围编程。

虽然右项仍有价值，我们更加重视左项。

附录 B | 开源治理资产速查

Agentic Agile 3-4-3 开源仓库提供模板、脚本、参考文档、统一 CLI、遥测仪表板和示例。资产会持续演进，本附录只说明稳定的能力分类，不把某个版本的文件数量写成长期承诺；实际文件、参数和兼容性以当前仓库为准。

核心模板

资产	用途
Template_Intent_Graph.md	全局意图与知识导航
Template_Intent_Contract.md / yaml	单任务契约
Template_Constraint_Matrix.md	人类可读约束
Template_Constraints.yaml	可执行约束
Template_Evidence_Bundle.md / yaml	证据包
Template_Verification_Plan.yaml	风险驱动验证计划
Template_Work_Graph.yaml	DAG 工作图
Template_Tools_Manifest.yaml	工具和权限
Template_Protocol.yaml	跨模块治理协议
Template_Loop_Memory.yaml	跨周期状态与教训
Template_Change_Envelope.yaml	本次变更允许触及的边界
Template_Release_Manifest.yaml	发布制品、证据和回滚事实绑定
Template_AI_Coding_Guide.md	AI 编码红线与规范

统一 CLI

```
PYTHON_BIN="$(bash scripts/ensure_py_env.sh)"
"$PYTHON_BIN" scripts/cli.py list
"$PYTHON_BIN" scripts/cli.py recon --project-dir .
"$PYTHON_BIN" scripts/cli.py change verify --task T-001 --project-dir .
"$PYTHON_BIN" scripts/cli.py evidence finalize --task T-001 --project-dir .
"$PYTHON_BIN" scripts/cli.py release --help
```

以上是 macOS/Linux 示例；Windows 可直接运行 `_bootstrap.py --print-python` 并使用它返回的解释器路径。第一次自举后应稳定复用项目专用 Python，不要假定所有机器都把解释器暴露为同一个命令名。CLI 是跨 AI 工具的稳定入口，不依赖某个 IDE、聊天产品或智能体平台。具体子命令和参数以 `scripts/cli.py --help` 及仓库当前版本为准。

附录 C | 图书与课程学习地图

本书的学习延伸分为线下工作坊和 Portal 在线课程两条路径。两者共享 Agentic Agile 的价值观与操作系统主干，但承担不同任务：线下强调现场共创、真实组织决策和工程突围；在线强调可重复的 Mission、学习证据和个人能力形成。章节引用只说明能力关系，不把某个历史模块号当成永久接口。

C.1 线下工作坊延伸

新版《Agentic Agile（智能体敏捷）：从碳基协作到硅基自治——AI 时代研发组织重构与转型沙盘》以 AI First / AI Native 和四维重构为主线：第一天建立转型事实、重构人员/组织/流程/工具，第二天在真实或明确标记为 SIMULATION 的工程任务中完成突围、证据裁决和组织扩展判断。工作坊的最终产物不是一份口号式转型规划，而是一张连接真实价值流、任务证据和首轮行动的转型画布。

C.1.1 书籍章节与线下工作坊映射

线下工作坊不是每章都重复出现的课程广告，而是本书方法的集中实践场。章节与工作坊的对应关系如下：

图书章节	线下工作坊对应实践
第 1-4 章	第一天：AI 转型事实、三大理论基石、Agentic Agile 宣言与 AI First / AI Native 讨论
第 5-8 章	第一天：操作系统总装图、四维转型设计、人员与组织重构；第二天：责任单元、动态工件和自治机制
第 9-15 章	第二天 Phase 2-4：SCOPE-V、意图契约、约束矩阵、Work Graph、TDD、故障修复与 Evidence 裁决
第 16-20 章	第一天：价值与效能遥测；第二天：Dashboard、发布裁决、既有系统 Recon、知识回写与跨模块扩展
第 21-24 章	第一天：四维转型画布；第二天 Phase 5：企业首轮转型判断、风险裁剪、回滚和组织扩展边界

因此，正文中的“延伸实践”只在章节确有必要时提示工作坊动作；完整的线下映射统一放在这里，避免在每章重复同一段说明，也避免读者误以为线下工作坊只是在线课程的实体版。

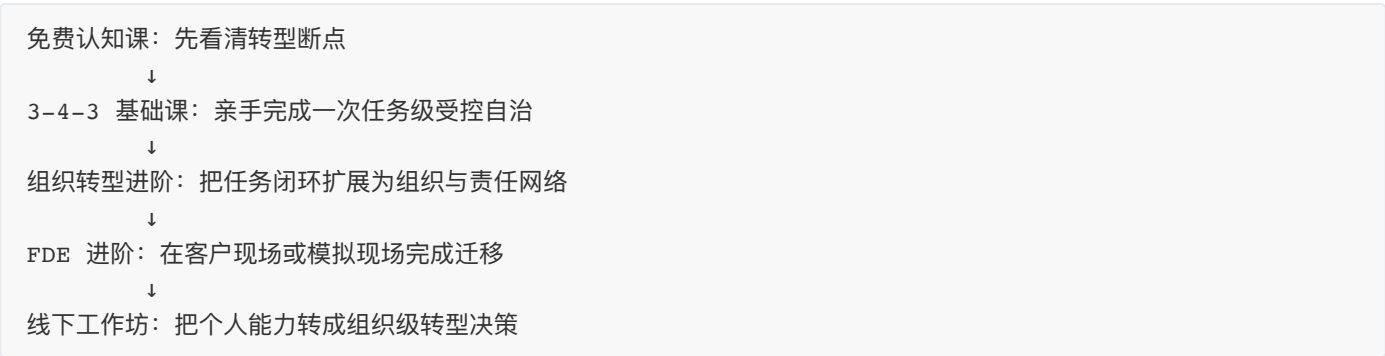
C.2 四门 Portal 在线课程

Portal 网址: <http://agentic.iloveagile.me>

Course	定位	核心产出
《Agentic Agile 免费认知课：AI 转型为什么不是工具升级》	免费认知	理解为什么局部工具提效不会自动转化为组织提效，并建立 AI First / AI Native 的判断框架
《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》	方法与工程入门	在本地 AI 工具与 343 Skill 中完成一次真实工程任务，留下契约、约束、执行证据和裁决结果
《Agentic Agile 组织转型进阶：构建人机协同研发操作系统》	组织级转型	把一次任务闭环扩展为责任单元、协作协议、授权边界和组织级行动
《Agentic Agile FDE 进阶：从客户现场到企业级生产交付》	客户现场与生产交付	在客户现场或明确标记的模拟环境中完成事实、场景、责任、价值流和交付证据闭环

四门课不是传统的“看完视频再做题”。每个 Mission 都应在本地 AI 工具中完成，按游标逐题交互，自动运行规定的动手实验，产出脱敏学习证据和可下载的阶段作品。Portal 负责门禁、进度、预览和显式上传；Learning Skill 负责本地引导、反馈、证据落盘；343 Skill 只在课程明确授权的工程任务中承担治理与验证。

C.3 推荐路径



这不是强制的商业购买顺序，而是能力形成顺序。读者也可以按职责进入：管理者从免费认知课和工作坊开始，工程人员从 3-4-3 基础课开始，组织与平台负责人重点看组织转型进阶，FDE 和交付团队重点看 FDE 进阶。无论从哪里进入，都必须回到真实任务、真实证据和真实责任，而不是停留在概念认同。

C.4 按目标选择阅读与实践

目标	建议阅读	对应实践
建立 AI First / AI Native 共识	前言、第一部分、第二十三章	免费认知课或线下工作坊第一天
跑通第一次真实人机协同	第五至十三章	3-4-3 基础课 + Agentic Agile 343 Skill
重构组织与责任网络	第六、二十一、二十三章	组织转型进阶 + 价值网络画布
进入客户现场和复杂系统	第十四至二十章、第二十四章	FDE 进阶 + 现场任务
形成企业级扩展判断	第十六、二十、二十三章	线下工作坊第二天 + 6.5 扩展门

C.5 书、Skill、Portal 的边界

图书：建立价值观、模型和判断框架

Learning Skill：在本地 AI 工具中陪伴学习、提问、反馈和生成证据

343 Skill：把真实工程任务变成可治理、可验证、可回滚的运行闭环

Portal：提供课程目录、Mission 门禁、学习进度、证据预览和显式同步

线下工作坊：把个人闭环带回组织，完成四维转型与扩展裁决

课程和 Skill 会持续演进，因此正文使用面向读者的课程名称，不固化旧模块编号或历史 URL。具体 Mission 版本、Skill 版本和资源下载入口以 Portal 与开源仓库当前发布信息为准。

附录 D | 术语表

术语	含义
Agentic Agile	面向智能体时代，对研发人员、组织、流程与工具进行系统重构的人机协同研发操作系统；治理是其运行机制，不是全部目的
四维重构	对人员、组织、流程与工具四个相互耦合的维度共同重构，而不是只引入 AI 编码工具

3-4-3	三类角色、四大动态工件、三条运行原则构成的最小治理内核
IO	意图主理人，负责价值、取舍和签署
OA	编排架构师，负责治理设计和异常升级
AS	在授权边界内执行的自治蜂群
Intent Graph	全局意图、依赖和知识导航
Intent Contract	单任务目标、非目标、规则、AC 和签署
Constraint Matrix	机器可执行的边界和门禁来源
Evidence Bundle	支持人类裁决的结构化证据
四大动态工件	Intent Graph、Intent Contract、Constraint Matrix、Evidence Bundle；它们有所有者、状态、版本和触发事件，并随真实运行持续变化
SCOPE-V	Specify、Constrain、Orchestrate、Prove、Evolve、Verify
Verify	依据证据判断当前状态支持发布、条件通过、阻断还是补充验证；它不是“测试通过”的同义词
Telemetry	从已验证任务事实中派生价值、能力、效率和进化信号的慢外环；它不是第五个动态工件
Measurement Contract	规定指标定义、数据源、口径、责任人和失效处理的度量契约；它约束遥测解释，不增加新的运行时工件
Harness	包裹概率性执行的约束、验证和恢复系统
Work Graph	描述任务依赖和执行关系的 DAG
HITL	需要人类承担判断或批准责任的介入点
Context Engineering	设计 Agent 可获得的上下文、边界、来源和新鲜度，而不只是堆叠提示词
Graph Engineering	用图结构表达意图、工作、证据和依赖，使并行执行与追溯成为可能
Loop Engineering	设计观察、行动、验证、恢复和升级循环，使自治可持续运行
Harness Engineering	把约束、工具权限、测试、门禁、证据与恢复机制工程化地包裹在 Agent 外部
Tools Manifest	声明可用工具、权限、来源、版本和风险边界的清单
Verification Plan	根据风险说明验证层次、方法、环境、通过标准与责任人的计划
Change Envelope	明确本次允许修改与禁止触及范围的变更边界
Release Manifest	将发布制品、版本、证据、审批、环境与回滚事实绑定的清单
Recon	在修改既有系统前重建代码、依赖、行为、测试和风险认知的侦察活动
Preserve	先用特征测试或基线证据固定现有行为，再实施有界变更
UNKNOWN	当前没有足够证据判断，必须保留未知状态，不能自动视为通过

NOT_APPLICABLE	经明确规则和责任判定后不适用，与“未执行”不同
快内环 / 慢外环	快内环用于单任务执行、验证和恢复；慢外环用跨任务遥测调整规则、能力和组织设计
RACI-D	在传统责任分工上显式增加 Decision Owner，确保关键裁决由有权承担后果的人作出
验证工程	关键行动由可复查证据支持，并允许失败、未知和残余风险被如实表达的工程方式

附录 E | 五道门速查

五道门是状态转换的机械裁决点，不是按顺序移交给不同部门的五个阶段。它们阻止系统在前置事实不足时宣称进入下一个状态。

门	标识	时机	核心问题
前置门	Pre	编码前	契约是否真实签署，约束、风险和 AC 是否准备好？
编码门	Coding	实现前	需要的测试是否先写，并真实证明过 Red？
验证门	Prove	Verify 前	测试、AC、构建、约束和证据是否支持当前结论？
收尾门	Closing	宣告完成前	Evidence Bundle、遥测入口、图谱回写和发布事实是否完整？
Bug 回溯门	Bug	完成后发现 Bug	是否如实修正证据、遥测、裁决和组织记忆？

附录 F | 参考资料与行动入口

开源项目

Agentic Agile 3-4-3：
<https://github.com/elegant1998/Agentic-Agile-343>

官网

<http://agentic.iloveagile.me/>

许可证

- 代码与模板：MIT；
- 白皮书：CC BY 4.0，转载和二次创作需保留署名。

建议的下一步

1. 获取开源仓库，完成 bootstrap，并用返回的专用 Python 运行 `scripts/cli.py list`，确认本地入口可用；
2. 选择一个低风险但真实的需求，建立最小 Intent Graph；
3. 由真实 IO 签署 Intent Contract，并明确非目标与决策边界；
4. 建立 Constraint Matrix，用 Verification Plan 设计与风险匹配的验证；

5. 在 Change Envelope 内完成变更，通过五道机械门并形成 Evidence Bundle；
6. 从已验证事实采集单任务遥测，检查哪些指标仍为 UNKNOWN；
7. 回写图谱、规则和教训，再决定扩大自治还是先补强工程系统。

附录 G | 贯穿案例：从一句话需求到可验证发布

本附录把全书方法压缩到一个连续案例中。它不是唯一正确答案，而是一份可以对照真实项目修改的治理样本。

G.1 原始需求

周四下午，业务负责人提出：

员工报销超过 5000 元，需要总监审批。这个需求很急，希望明天上线。

传统做法很容易立即进入估时和编码。Agent 也可能迅速找到审批逻辑，修改判断符号，补一两个测试，然后报告完成。但这句话至少隐藏着以下问题：

- “超过”是否严格大于，等于 5000 元怎么办？
- 金额按含税、未税还是本位币折算后计算？
- 总监是申请人的直属总监、成本中心总监还是财务总监？
- 历史草稿、审批中单据和已完成单据是否受影响？
- 总监缺席时是否允许代理？
- 规则是否适用于全部费用类型和全部组织？
- 明天上线是全量启用还是具备灰度与回退？

高速度不会消除这些问题，只会让某一种未经签署的解释更快进入系统。

G.2 Specify：把自然语言转成判定规则

OA 通过 Grill-Me 与 IO 校准，形成以下关键决定：

- 仅适用于中国区差旅报销；
- 金额按含税人民币总额判断；
- 5000.00 元不触发，5000.01 元触发；
- 审批人为申请人所属成本中心总监；
- 只影响规则启用后新提交的单据；
- 历史草稿迁移明确列入非目标；
- 若无法解析成本中心总监，禁止自动通过，转人工财务队列；
- 首日采用功能开关，仅对两个试点部门启用。

IO 阅读目标、非目标、规则、AC 和风险后完成独立签署。OA 可以起草，不能代签；Agent 可以帮助澄清，不能承担业务责任。

可执行 AC 至少包括：

1. 给定含税总额 5000.00 元，提交后不增加总监审批节点；
2. 给定含税总额 5000.01 元，提交后增加成本中心总监节点；
3. 给定无法找到成本中心总监，提交进入人工财务队列且不得自动通过；
4. 给定规则启用前已存在草稿，再次保存时审批链不被静默改写；

5. 关闭功能开关后，新提交单据恢复旧路径。

G.3 Constrain：把底线从文字变成门禁

Constraint Matrix 同时记录业务、数据、技术、工具和发布约束：

- 禁止修改已审批单据；
- 禁止读取或输出生产个人报销明细；
- 禁止 Agent 获取生产发布凭证；
- 数据库结构变更必须由 DBA 审批，本任务原则上不得引入迁移；
- 单次自动修复不超过三轮；
- 任何 AC 失败都不得生成“可发布”结论；
- 功能开关、回退步骤和负责人必须在发布前验证；
- Evidence Bundle 必须包含测试原始输出、代码差异、约束检查和残余风险。

其中能自动执行的规则进入 YAML 和 CI，不能完全自动判断的规则绑定责任人和批准记录。约束不是一份“注意事项”，而是执行环境的一部分。

G.4 Orchestrate：让工作有依赖、有交接、有停止条件

OA 创建 Work Graph：

```
N1 生成并运行规则测试，确认可信 Red
├─ N2 实现领域审批规则
└─ N3 准备功能开关与回退验证
N2 + N3 → N4 接口与集成测试
N4 → N5 约束检查与 Evidence Bundle
N5 → N6 IO 发布裁决
```

每个节点只获得所需上下文。测试节点读取契约和接口，不读取候选实现；实现节点读取签署契约、测试和编码约束；证据节点读取原始检查输出，不允许仅采用执行 Agent 的自然语言总结。

若发现旧系统对 5000 元使用 `>=`，节点必须停止并报告规则冲突。它不能以“保持兼容”为由自行修改新契约，也不能悄悄改变旧数据。

G.5 Prove：先证明测试会失败

测试 Agent 先提交边界和异常测试，并在未实现新规则时运行。预期 Red 不是形式动作：团队检查失败原因确实是缺少新行为，而不是环境损坏、断言错误或测试根本未执行。

实现 Agent 只完成使测试通过的最小改动。随后执行：

- 领域单元测试；
- 审批链集成测试；
- 功能开关关闭后的回退测试；
- 权限和敏感数据检查；
- 全量回归中与审批相关的测试；
- 构建与静态检查。

若测试与实现由同一模型生成，关键边界还需由 OA、第二模型或既有业务样本交叉检查，避免同一误解同时写进代码和测试。

G.6 Evolve：失败如何形成组织能力

第一次集成测试发现，代理总监配置会产生两个候选审批人。AS 没有权限决定业务优先级，因此停止并提交：触发输入、候选人列表、当前规则、测试结果和两个可选方案。

IO 决定“优先使用当前生效代理；同时存在多个代理时转人工财务队列”。契约和 AC 更新并重新签署，AS 再完成测试与实现。这个经验随后写回意图图谱和约束矩阵，下一次涉及组织代理关系的任务会主动提出同类问题。

系统学习的不是一句空泛的“注意代理人”，而是一条具有适用范围、证据来源、责任人和失效条件的治理资产。

G.7 Verify：证据支持哪一种行动

Evidence Bundle 最终显示：

- 5 项业务 AC 全部通过；
- 回归、构建和安全检查通过；
- 未使用生产数据和生产凭证；
- 功能开关关闭路径已验证；
- 历史草稿迁移未验证，且明确属于非目标；
- 多代理冲突规则已由 IO 重新签署；
- 首日监控负责人和异常阈值已登记。

IO 据此作出有边界的裁决：批准两个部门灰度启用；若人工财务队列异常增长或审批链错误率超过阈值，自动关闭功能开关；次日复核遥测后再决定是否扩大。

这里真正重要的不是“Agent 一天写完了”，而是团队能说明它理解了什么、被禁止做什么、如何证明结果、还有什么没有证明，以及为什么当前发布范围可接受。

G.8 发布后 Bug 回溯

灰度期间发现一张外币报销单被错误按原币数值判断。团队立即关闭开关，保留事件与日志，修正 Evidence Bundle 中原有的发布结论，并启动 Bug 回溯门。

根因分析发现，契约虽然写了人民币含税总额，但测试样本全部为人民币，接口字段中“本位币金额”与“原币金额”命名相近。改进不只是一行代码：

- 增加外币和汇率缺失 AC；
- 在约束中要求金额判断必须引用标准化本位币字段；
- 更新字段语义到意图图谱；
- 增加静态检查或类型封装，降低字段误用；
- 更新遥测和事故记录，不保留虚假的首次全绿。

Bug 没有证明治理无用，反而检验治理是否诚实：能否快速限制影响、纠正证据，并把失败转化为下一次更强的系统能力。

附录 H | 常见质疑与回答

H.1 “这不就是把敏捷流程重新命名吗？”

不是。Agentic Agile 继承快速反馈、增量交付和持续改进，但它处理了新的执行现实：概率性 Agent 能调用工具、修改多个文件、并行协作并快速放大错误。因此治理对象从人类协作流程扩展到意图、权限、上下文、机器门禁、证据和自治边界。它不是否定敏捷，而是为 Agent 参与交付补齐控制系统。

H.2 “小需求也要写这么多文档吗？”

不需要。治理重量应与风险匹配。低风险单模块任务可以只保留最小契约、核心约束、可执行 AC 和 Evidence Bundle；高风险、多模块、不可逆任务才需要更完整的 Protocol、Work Graph、跨模块契约和批准链。关键不是模板页数，而是目标、边界、证据和责任不能缺席。

H.3 “模型越来越强，以后还需要这些吗？”

更强模型会减少部分实现错误，却不会自动拥有组织授权，也不能替人类承担法律、伦理和商业责任。能力越强、作用范围越大，越需要知道它能做什么、何时停止、如何证明和如何恢复。治理方式会更自动化，但治理问题不会自动消失。

H.4 “TDD 会不会拖慢 AI Coding？”

在简单演示中，先写测试似乎增加步骤；在真实交付中，它减少的是错误方向上的高速实现和事后争论。可信 Red 证明测试能发现缺失行为，AC 建立业务语义，回归测试保护已有能力。真正需要优化的是测试分层和执行速度，而不是取消证明环节。

H.5 “Evidence Bundle 会不会变成新的形式主义？”

会，如果它由人手工拼接、只展示漂亮截图、与 AC 和原始命令无映射。有效 Evidence Bundle 应尽量自动采集，保留来源、时间、退出码和适用范围，并坦诚列出未验证事项。它服务于裁决，不服务于表演。

H.6 “为什么 IO 必须本人签署？”

因为目标、非目标和残余风险属于价值责任。OA 可以澄清和起草，AS 可以执行，二者都不应替业务责任人决定“做什么值得”和“哪些风险可以接受”。允许代理签署会让治理看似完整，实则责任空心化。

H.7 “多 Agent 一定比单 Agent 快吗？”

不一定。并行收益受任务独立性、共享资源、接口稳定性和验证成本限制。不可分解任务强行并行，会产生冲突与上下文开销。Orchestrate 的目标是优化端到端流动，而不是最大化同时在线的 Agent 数。

H.8 “发生 Bug 是否说明证据包失效？”

证据不是绝对正确保证，而是对特定范围、特定时间和特定环境的可复查证明。Bug 可能来自遗漏 AC、环境差异、测试盲区或证据失真。成熟系统会立即修正原证据、控制影响、追溯根因并更新治理资产，而不是用“当时测试通过”维持虚假结论。

H.9 “我们已经有 DevOps、CI/CD 和平台工程，还需要 3-4-3 吗？”

这些能力是重要基础，但它们通常不会自动回答业务意图由谁签署、Agent 的权限如何裁剪、跨 Agent 交接如何验证、残余风险由谁接受。3-4-3 可以复用现有流水线、策略引擎和观测平台，把它们连接到意图与责任，而不是推倒重建。

H.10 “怎样证明这套方法值得投入？”

先选一个低风险但真实的任务做基线对比。不要只测开发时长，同时记录歧义发现时点、返工、越权阻断、首次成功率、验证成本、发布后缺陷和人工介入质量。若治理成本没有换来更早风险发现、更少返工或更清晰决策，就应调整，而不是为了维护方法正确而美化数据。

附录 I | 四大动态工件实战指南

四大动态工件不是四份彼此独立的文档，而是同一条治理链的不同视图。Intent Graph 提供全局关系，Intent Contract 锁定单任务承诺，Constraint Matrix 定义不可突破的边界，Evidence Bundle 汇总事实并支持裁决。任何工件脱离其他三者，都容易退化为静态材料。

“动态”至少意味着四件事：有明确所有者、有可识别状态、有内容版本、有真实事件触发更新。文件格式只是实现手段，Markdown、YAML、数据库或平台对象都可以承载工件；只要语义、责任和追溯关系稳定，不同 AI 工具就能参与同一条运行链。

工件	主要所有者	典型状态变化	主要触发事件
Intent Graph	IO 对意图负责，OA 维护结构与新鲜度	假设 → 已验证；活跃 → 过期或替代	战略变化、任务完成、事故、依赖确认
Intent Contract	IO 签署，OA 检查可执行性	草拟 → 签署 → 关闭；实质变更后重新签署	目标澄清、边界变化、裁决完成
Constraint Matrix	规则责任人批准，OA 工程化	草拟 → 生效 → 修订或废止	风险变化、例外、误报漏报、事故教训
Evidence Bundle	执行者生成，OA 聚合，Decision Owner 裁决	持续积累 → 可裁决 → 归档或更正	验证运行、人工批准、发布、回退、Bug

Telemetry 不在这张表里，因为它不是第五个动态工件。四大工件承载任务运行中的意图、边界和事实；Telemetry 从已验证事实中派生跨任务信号，进入慢外环，用来调整规则、能力投资和组织设计。

I.1 Intent Graph：不要从画一张宏大地图开始

意图图谱的最小节点可以只有五类：业务目标、能力、模块、任务和证据。边描述“为什么依赖”“由谁负责”“依据什么判断”。团队应从一个真实任务向外扩展，而不是先把整个企业建模。

以报销规则为例：

降低违规付款风险（业务目标）

└ 审批路由能力（能力）

└ 报销领域服务（模块）

└ 组织关系服务（模块）

└ T-001 高额报销总监审批（任务）

└ IC-T001（契约）

└ AC-01..05（验收）

└ EB-T001（证据）

每次任务完成后，只回写经验证的新关系：字段语义、真实依赖、已确认风险和可复用规则。未经验证的推测可以记录，但必须标为假设，不能与事实混在一起。

图谱需要解决新鲜度问题。模块所有者、规则生效时间、最后验证任务和可能失效条件都比漂亮的可视化重要。一条两年前建立、无人负责的依赖关系，可能比没有图谱更危险，因为它会制造错误信心。

图谱质量检查

- 能否从任务追溯到业务目标？
- 能否找到约束与证据来源？
- 关键节点是否有责任人？
- 事实与假设是否可区分？
- 变更后是否有回写机制？

- 过期节点能否被发现和淘汰？

I.2 Intent Contract：不是把需求写得更长

契约应把会改变实现或裁决的内容写清楚。典型字段包括：

- 业务背景与目标；
- 可观测成功结果；
- 非目标；
- 业务规则与边界例子；
- MUST/SHOULD/MAY 验收标准；
- 已知假设和待决问题；
- 风险与回退要求；
- 数据、工具和权限边界；
- IO/OA 身份、版本和签署时间。

目标写成“优化审批体验”仍然不可判定。更好的写法是说明受影响对象、期望状态和观察方式，例如：“在不改变 5000 元及以下报销路径的前提下，对中国区新提交且含税本位币金额大于 5000 元的差旅报销增加成本中心总监审批节点。”

非目标不是附属字段。它防止 Agent 出于“顺手优化”修改历史数据、重构无关模块或扩大业务范围。对于高速执行系统，明确不做什么与明确做什么同样重要。

签署也不是在文件末尾写一个名字。有效签署至少绑定内容摘要、版本、身份和时间；契约发生实质变更后，旧签署应失效。OA 不能因为“IO 已经口头同意”而代替其完成签署。

契约质量检查

- 两名不同执行者是否会得到相近理解？
- 每个 MUST 是否有判定方法？
- 边界值、异常路径和回退是否明确？
- 非目标是否足以阻止范围蔓延？
- 假设是否有验证人或失效条件？
- 最终版本是否由真实 IO 签署？

I.3 Constraint Matrix：从原则到可执行边界

约束矩阵至少区分五类边界：业务规则、数据与隐私、架构与依赖、工具与权限、运行与发布。每条约束应说明强度、适用范围、检查方式、失败动作、例外批准人和证据位置。

“注意数据安全”无法执行；“Agent 不得读取生产客户明细，测试数据必须来自已脱敏数据集，检查失败立即停止并通知安全责任人”才开始具备治理意义。

MUST 代表违反后不能继续；SHOULD 允许在记录理由和责任人后例外；MAY 是可选优化。团队应谨慎使用 MUST：过多不可执行的硬规则会造成约束疲劳，成员最终学会绕过全部门禁。

约束冲突需要显式优先级。例如业务希望快速灰度，安全政策要求生产权限人工批准，项目预算限制额外环境。OA 不能选择最方便的一条，而应展示冲突、影响和备选路径，由具有相应责任的人裁决。

约束质量检查

- 是否说明适用对象和阶段？
- 能否由脚本、策略或明确人工动作检查？

- 失败后是阻断、降级、回退还是告警？
- 是否存在隐含的例外通道？
- 例外是否有时限、责任人和复核？
- 规则是否定期根据误报、漏报和事故更新？

I.4 Evidence Bundle：把事实组织成决策材料

Evidence Bundle 的读者不是只有工程师。IO 需要知道目标是否实现和哪些风险仍存在；OA 需要检查治理链是否完整；开发与测试需要复现结果；安全和审计需要追踪权限与批准。

一份有效证据包通常包括：

- 任务和契约版本；
- 代码与配置变更摘要；
- 每项 AC 对应的检查与结果；
- 原始命令、环境、退出码和时间；
- 构建、测试、安全、依赖和约束结果；
- 人类批准、例外和关键 Handoff；
- 未验证事项与残余风险；
- 回退方案及验证结果；
- 可追溯的遥测采集入口；
- 推荐裁决，但不替代责任人决定。

证据强度存在层级。Agent 声称“已经测试”是低强度陈述；保存命令和退出码更强；在隔离环境中由独立流水线复现更强；与真实但受控的业务样本、回退演练和运行监控结合，才能支持更高风险的发布。

证据包也必须允许得出“不发布”。如果模板只允许填写成功结果，团队会隐藏未知项。成熟的 Evidence Bundle 应把失败、跳过和不适用区分开，把“没有证据”与“证据证明失败”区分开。

证据质量检查

- 每个 MUST 是否能找到对应证据？
- 证据来自原始系统还是人工转述？
- 执行环境和时间是否可识别？
- 跳过项是否有理由和责任人？
- 残余风险是否能被非开发角色理解？
- 发布后发现 Bug 时，原证据能否被修正和追溯？

I.5 四大动态工件的一致性检查

最常见的问题不是某个工件缺失，而是彼此矛盾：图谱仍指向旧模块，契约已经变更，约束检查使用旧规则，Evidence Bundle 却把全部 AC 标为通过。

一致性检查至少覆盖：

目标一致: Contract 的目标来自 Graph 中有效意图
边界一致: Contract 的规则被 Matrix 覆盖
验证一致: 每项 MUST AC 都映射到 Evidence
版本一致: 签署、检查和证据针对同一版本
责任一致: 批准者与授权矩阵匹配
回写一致: 经验证的新知识返回 Graph 和规则资产

这也是五道机械门存在的原因。人类很难在高速交付中持续手工核对所有关系，必须让脚本和结构化标识承担重复一致性检查，把人的注意力留给价值冲突与异常判断。

跨 AI 工具协作时，真正需要稳定的不是某个产品的会话记录，而是任务 ID、契约版本、约束 ID、AC ID、证据引用、状态枚举和签署身份。工具可以更换，模型可以升级，运行事实不能因此失去语义。任何只在某个 Agent 私有记忆中存在、无法被下一工具读取和复查的信息，都还没有成为组织资产。

I.6 工件的轻量化与升级条件

低风险修正文案时，契约可以很短，约束可以继承仓库默认值，证据可能只包含快照测试和差异；涉及支付、权限、隐私或跨模块发布时，工件必须更完整。升级条件可以包括：

- 存在不可逆数据变更；
- 使用生产凭证或敏感数据；
- 影响两个以上模块或团队；
- 引入新工具、模型或外部依赖；
- 业务规则存在法律、财务或安全责任；
- 回退时间超过组织可接受窗口；
- 自动验证无法覆盖关键判断。

轻量化不是删除责任，而是减少与当前风险无关的字段；升级也不是增加文档页数，而是提高边界和证据强度。

附录 J | Agentic Agile 实践社区

J.1 宣言与社区入口

[Agentic Agile 宣言网站](#)提供宣言签署与社区入口。

J.2 配套资料与课程

- [Agentic Agile 白皮书与配套资料](#)
- [Agentic Agile 90 天首轮转型验证清单](#)
- [Agentic Agile 四维成熟度自测表](#)
- [《Agentic Agile 智能体敏捷：从碳基协作到硅基自治》](#)：构建 AI 时代的人机协同研发操作系统
- 《Agentic Agile（智能体敏捷）：AI 时代研发组织重构与转型沙盘》

J.3 联系方式

- 微信: iloveagile
 - Email: 3433839@qq.com
-

结语 | 从碳基协作到硅基自治

AI 正在改变软件研发的执行方式。它可以读取更多信息、调用更多工具、并行完成更多任务，也会以更快速度放大模糊目标、错误假设和薄弱边界。

Agentic Agile 不承诺所有需求都能自动完成，也不要求人类退出研发。它提供的是一套责任清楚的控制系统：

人类定义价值和承担责任

Agent 在授权范围内执行

Harness 守住边界

Evidence 支持裁决

Telemetry 驱动进化

真正成熟的 AI 研发，不是模型说“完成了”，也不是团队因为赶时间选择相信。

它应当能够回答：目标是什么、边界在哪里、验证做了什么、风险还剩什么、当前证据支持哪一种行动。

硅基自治不是这条责任链的终结，而是它足够成熟后的运行结果：Agent 成为默认执行主体，人类继续定义价值、批准不可逆决策并承担最终责任。

不要只让 AI 交代码。

让它交付可治理、可追溯、可验证的结果。

—— 让我们一起进入智能体敏捷 Agentic Agile 时代 ——

1. 参见 Paul A. David, *The Dynamo and the Computer: An Historical Perspective on the Modern Productivity Paradox*。该研究以电气化说明：通用技术的生产率收益，依赖互补性组织创新以及工厂布局、物料流和生产方式的重构。[↗](#)

2. 参见 The Henry Ford, *How did Henry Ford's Model T assembly line work and how did it evolve?*。其档案资料记载，福特于 1913 年形成移动装配线，到 1914 年初可每 93 分钟完成一辆 Model T，而固定装配方式约需 12.5 小时。[↗](#)

3. 参见 OECD, *Empirical Evidence for Integration and Disintegration of Maritime Shipping, Port and Logistics Activities*。集装箱化的影响来自海运、港口和陆路物流围绕多式联运形成的新组织模式，而不只是容器本身。[↗](#)